Preface

The Complete AI Engineer's Knowledge Map

Why This Book

AI engineering demands extraordinary breadth. You need to understand **why gradients in calculus can optimize a loss function**, and also know **how the Linux kernel's CFS scheduler and HugePages affect training throughput**; you must master classical machine learning from decision trees to XGBoost, while keeping pace with the frontier of Transformers, MoE, and FlashAttention; you need to work through the math behind RLHF and DPO, and also know how to deploy models to production with vLLM and LoRA.

There is no shortage of good books out there — *Linear Algebra Done Right* for math, the Deep Learning book for neural networks, CSAPP for systems. But each covers one domain in isolation, while real-world work demands all of this knowledge **at the same time**. A single inference optimization problem may simultaneously touch the Roofline model (numerical computation), KV Cache quantization (linear algebra + information theory), PagedAttention (OS memory management), and FP8 training (hardware architecture).

This book attempts to bring the core knowledge an AI engineer needs **into a single coherent system**, covering the complete chain from mathematical foundations to Agent applications with consistent depth and perspective. It is not a textbook — there are no exercises or lengthy proofs; nor is it a quick-reference handbook —

every topic is covered to the depth of "what an experienced colleague could explain to you in a 30-minute conversation."

Who It's For

This book is written for three types of readers:

Reader Type	Background	Reading Advice
AI Engineers	Familiar with deep learning, working on model training / inference / deployment	Skim the math chapters; focus on systems software and performance engineering; use Paper Deep Dives to fill gaps
Systems Engineers Transitioning to AI	Experienced with Linux / networking / databases, starting to work with models	Build confidence starting from Chapter 2 (Systems Software), then read Chapters 1, 3, and 4 in order
Students / Self-Learners	Solid programming background, systematically studying AI engineering	Read chapter by chapter from the beginning; return to Chapter 1 whenever unfamiliar math appears

Prerequisites: you know Python, understand basic data structures and algorithms, and have used at least one deep learning framework (PyTorch preferred). No graduate-level math background is assumed — Chapter 1 reviews everything from the ground up.

Book Structure

The book is organized into 5 chapters, progressing layer by layer from foundational mathematics to high-level applications:

Chapter 1 · Mathematical Foundations

A math review handbook for AI engineers. Covers calculus (the mathematical essence of gradient descent and backpropagation), linear algebra (from SVD to low-rank approximation in LoRA), probability and statistics (the connection between MLE and cross-entropy), stochastic processes (Markov modeling in Tree of Thoughts), information theory (entropy, KL divergence, and LLM training loss), and numerical computation (floating-point precision, the Adam optimizer, ODE solvers for diffusion models). Each topic includes an "In AI" section that anchors the mathematical concepts to engineering practice. 7 parts, 29 sections.

Chapter 2 · Systems Software

From CPU to DPDK/VPP — the systems perspective every AI engineer needs. Covers CPU and GPU/NPU architecture (the CUDA programming model, Tensor Cores), operating system internals (process scheduling, virtual memory, concurrency primitives, eBPF), programming languages and compilers (Rust ownership, LLVM/MLIR, AI compilers), databases (B+Tree vs. LSM-Tree, graph databases, time-series databases, vector databases and HNSW), message middleware, and high-performance networking (DPDK user-space drivers, VPP vectorized forwarding, TCP/QUIC, observability). 6 parts, 28 sections.

Chapter 3 · AI Model Technology

From classical machine learning to performance engineering — the longest chapter. Covers traditional ML (complete derivations from KMeans to XGBoost), fundamental neural network operators (GEMM, convolution, activation functions, normalization), classical deep learning (CNN / LSTM / GNN / GAN / VAE / diffusion models), data and feature engineering, training optimization (supervised / self-supervised / SFT / RLHF / DPO / RL / distillation / federated learning / AutoML), AI performance engineering (the Roofline model, quantization and pruning, FlashAttention, software stack optimization), and AI applications in networking (traffic classification, log anomaly detection, KPI time-series forecasting). 7 parts, 36 sections.

Chapter 4 · LLM Technology

From Transformer to Agent — the core technology stack for the large-model era. Covers the LLM landscape and Scaling Laws, a complete derivation of the Transformer architecture, positional encodings (RoPE and long-context extrapolation), attention optimizations (MQA / GQA / MLA / sparse attention / Mamba), tokenizers and embeddings, MoE principles and engineering, distributed training (DDP / ZeRO / 3D parallelism), inference engines (KV Cache / PagedAttention / continuous batching / speculative decoding), inference compression (AWQ / GPTQ / FP8), RAG (from basics to GraphRAG / Agentic RAG), Agents (ReAct / Function Calling / MCP / multi-agent), and multimodality (VLMs / video generation / unified multimodal models). 12 parts.

Chapter 5 · Paper Deep Dives

From ResNet to DeepSeek — in-depth analysis of a curated set of foundational papers. This is a distinctive feature of the book: while the first four chapters address "what it is" and "how to use it," the Paper Deep Dives chapter addresses "why it was designed this way" and "the complete chain of mathematical derivation." Each paper includes background, core methodology (with full formula derivations), code implementation, experimental results, and a critical perspective on impact and limitations. High-citation papers include complete code examples and architecture diagrams. Every mention of a paper in the preceding chapters links directly to the corresponding deep dive.

Reading Guide

This is not a book that needs to be read cover to cover. Based on your goals, here are a few recommended paths:

Path A: Fast Track to LLM Engineering

Chapter 4 (LLM Technology) → Chapter 3, Sections 3.5–3.6 (Training Optimization and Performance Engineering) → Chapter 5 relevant papers. Return to Chapter 1 whenever unfamiliar math comes up.

Path B: Systems Engineer Transitioning to AI

Chapter 2 (Systems Software, build confidence) → Chapter 1 (fill in the math) → Chapter 3 (AI Models, starting from classical ML) → Chapter 4 (LLMs).

Path C: Interview Preparation

Chapter 1 focusing on calculus + linear algebra + probability → Chapter 3, Sections 3.1–3.3 (classical ML + deep learning) → Chapter 4, Sections 4.2–4.4 (Transformer + attention) → Chapter 5 Paper Deep Dives for a quick pass.

Path D: Complete Study

Follow the chapter order: Chapter 1 → Chapter 2 → Chapter 3 → Chapter 4 → Chapter 5. After completing each chapter, read the corresponding paper deep dives.

> **TIP**
>
> The blue orientation bar at the start of each section describes where that section sits within the overall knowledge system and its relative importance, helping you decide whether to read carefully or skim.

About Paper Deep Dives

Chapter 5 is a defining feature of this book. From the hundreds of papers referenced across the first four chapters, we have selected approximately 26 for in-depth analysis, organized along several dimensions:

Category	Papers Covered	Depth
Classic Models (related to Chapter 3)	ResNet, LSTM, BatchNorm, Adam, XGBoost, GAN, VAE, DDPM, Word2Vec	1–3 pages; high-citation papers include code
Systems Foundations (related to Chapter 2)	HNSW, Raft	1–2 pages

Transformers & Pretraining (related to Chapter 4)	Attention Is All You Need, BERT/GPT, Scaling Laws, RoPE	2–3 pages, with complete implementations
Efficiency & Inference (related to Chapter 4)	FlashAttention, LoRA/QLoRA, PagedAttention, MoE	2–3 pages, with algorithmic pseudocode
Alignment & Applications (related to Chapter 4)	InstructGPT/RLHF, DPO, Chain-of-Thought, DeepSeek-V3	2–3 pages
LLM Supplements	Mamba, RAG, ReAct	1–2 pages

Paper Deep Dives are not paper translations — the focus is on: **what problem this paper solves, what the core idea is, the key steps in the mathematical derivation, and how it influenced subsequent work**. Each entry ends with a "Critical Perspective" section that identifies limitations and traces subsequent developments.

Conventions

Density and Style

The target density for each section is "what an experienced colleague could explain to you in a 30-minute conversation" — more compact than a textbook, more rigorous than a blog post. Formulas appear only when necessary, but when they do, they are derived in full. Code examples are primarily in Python/PyTorch, with occasional use of C, Rust, or SQL.

Cross-References

The connections between concepts matter. The orientation bar at the start of each section notes its relationship to other chapters. When a paper is first mentioned in the main text, a hyperlink points to its deep dive in Chapter 5. For example, when

the text mentions "FlashAttention's tiling strategy," you can jump directly to the paper deep dive for the complete algorithm.

"In AI" Sections

Every topic in Chapters 1 and 2 closes with an "In AI" section that anchors the foundational knowledge to AI engineering practice. This is a deliberate design choice — isolated knowledge is easily forgotten; it becomes genuinely internalized only when connected to real applications.

Updates & Errata

The AI field evolves at extraordinary speed. The content of this book reflects the state of the technology as of **April 2026**. We will continuously update the following:

- Significant newly released models and architectures (e.g., DeepSeek V4/R2, Qwen3.6-Plus, Claude Mythos, etc.)
- Ongoing expansion of Paper Deep Dives
- Errata corrections from community feedback
- Updates to outdated information (hardware specs, framework versions, ecosystem changes)

If you find errors or have content suggestions, please report them to the maintainers.

The book comprises 5 chapters, 130+ sections, covering five major modules: mathematics, systems, AI models, LLMs, and paper deep dives. Happy reading.

Chapter 1: Mathematical Foundations

A Review for AI Engineers

The goal of this chapter is not to produce mathematicians. The assumed reader has studied undergraduate mathematics at some point but has forgotten most of it. Day-to-day work involves reading AI papers or calling library APIs, and the aim is to look at a formula without flinching, diagnose numerical errors when they arise, and draw on theory for guidance when designing algorithms.

These requirements shape three writing principles. First, **every formula is preceded by the problem it solves**, followed by its formal statement, and then connected to a concrete AI application — no purely symbolic derivations. Second, **coverage is deliberately asymmetric**. Gradient descent, eigendecomposition, the law of large numbers, information entropy, and floating-point error — concepts that come up in AI almost daily — are treated in depth. Manifold optimization, queueing theory, and channel coding theory are touched on lightly. Third, **code is treated as a second language**. When a formula resists intuition, a runnable Python snippet often beats a page of proof.

One more note: the mathematical topics in this chapter are not equally important for AI. A rough priority ordering is *numerical computing* ≈ *linear algebra* > *calculus* > *probability and statistics* > *information theory* > *stochastic processes*. That does not mean the lower-ranked topics are unimportant — it means that if you want to

allocate your time first, start with numerical stability and matrix decompositions, where the return on investment is highest.

PART ONE

1.1 Calculus

Functions, differentiation, integration — the mathematical foundation of all "optimization" in AI lives here.

1.1.1 Overview of Multivariable Calculus

> *Orientation: from single-variable review to multiple variables, clarifying three core objects — the gradient, the Jacobian, and the Hessian.*

A Quick Review of Single-Variable Calculus

The derivative of a function $f: \mathbb{R} \to \mathbb{R}$ at a point x_0 is defined as the limiting slope:

> $$f'(x_0) = \lim_{h \to 0} \frac{f(x_0 + h) - f(x_0)}{h}$$

Intuition: zoom in on f over a sufficiently small neighborhood and it looks like a straight line; the slope of that line is the derivative. This idea of "locally looks like a line" generalizes to "locally looks like a hyperplane" in the multivariable setting. Integration is the inverse of differentiation — given a slope distribution, recover the change in the original function's value. The Fundamental Theorem of Calculus ties the two together: $\int_a^b f'(x)\,dx = f(b) - f(a)$.

Multiple Variables: From Scalar Fields to Vector Fields

Switching to $f: \mathbb{R}^n \to \mathbb{R}$, the input is an n-dimensional vector and the output is still a scalar. We can differentiate with respect to each dimension separately to obtain the **partial derivatives** $\frac{\partial f}{\partial x_i}$. Stacking all partial derivatives into a vector gives the **gradient**:

> $$\nabla f(\mathbf{x}) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots, \frac{\partial f}{\partial x_n}\right)^\top$$

The gradient has one crucial geometric meaning: **it points in the direction of steepest ascent of f, with magnitude equal to the rate of increase in that direction**. The negative gradient is the direction of steepest descent — and that is the entire rationale for gradient descent.

Going further, the derivative of $f: \mathbb{R}^n \to \mathbb{R}^m$ (input n-dimensional, output m-dimensional) is an $m \times n$ matrix called the **Jacobian matrix** $\mathbf{J}$, whose (i,j) entry is $\frac{\partial f_i}{\partial x_j}$. Backpropagation in neural networks is essentially a chain of Jacobian matrix products.

The $n \times n$ matrix formed by the second-order partial derivatives of f is called the **Hessian matrix** $\mathbf{H}$, with (i,j) entry $\frac{\partial^2 f}{\partial x_i \partial x_j}$. The Hessian describes the local curvature of a function: positive definite means it is a bowl (local minimum), negative definite means it is a hilltop (local maximum), and indefinite means it is a saddle point.

In AI

The loss function $L(\boldsymbol{\theta})$ of a neural network maps from an extremely high-dimensional parameter space to a scalar — modern LLMs range from billions to hundreds of billions of parameters. What we need is $\nabla_{\boldsymbol{\theta}} L$ — a vector of the same dimensionality as the parameters, pointing in the direction of fastest loss increase. Each step $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla L$ is gradient descent. The Hessian matrix is far too large to store ($10^{10} \times 10^{10}$), so optimizers like Adam *approximate* Hessian information using the second moment

of the gradient — this is the dividing line between first-order and second-order optimization.

1.1.2 Function Analysis Techniques

> *Orientation: a set of repeatedly encountered operators — convexity/concavity, gradient, divergence, curl, moments, convolution. Emphasis on convexity and convolution.*

Convexity and Concavity

A function f is **convex** on an interval if, for any x_1, x_2 and $t \in [0,1]$:

> $$f(t x_1 + (1-t) x_2) \le t f(x_1) + (1-t) f(x_2)$$

Geometric interpretation: the line segment connecting any two points does not fall below the function's graph. The greatest advantage of convex functions is that **every local minimum is a global minimum** — a paradise for optimization algorithms. The squared loss in linear regression and the cross-entropy loss in logistic regression are both convex, which is why these classical algorithms are guaranteed to converge to the global optimum.

Neural networks are not convex. The loss surface assembled from millions of parameters is highly non-convex, riddled with local minima and saddle points. An important finding in modern optimization theory: *in high-dimensional non-convex problems, what truly stalls algorithms is not local minima but saddle points*, because random initialization makes escaping saddle points likely. This has reshaped the conventional belief that "non-convex problems are hard to optimize."

Gradient, Divergence, and Curl

The gradient was covered above. Divergence $\nabla \cdot \mathbf{F}$ acts on a vector field and outputs a scalar, measuring the net outward flux; curl $\nabla \times \mathbf{F}$ also acts on a vector field but outputs a vector, measuring local rotation. These two appear rarely in everyday AI work — typically only in contexts

involving continuous normalizing flows (CNF) or the Fokker-Planck equation in diffusion models. A passing familiarity is sufficient.

Moments

The k-th moment of a probability density $p(x)$ is $\mathbb{E}[X^k] = \int x^k p(x)\,dx$. The first moment is the mean; the second central moment is the variance. The "m" and "v" in the Adam optimizer are the first-moment estimate and second-moment estimate of the gradient, respectively — the names come directly from statistics.

Convolution

This deserves emphasis, as it appears in CNNs, signal processing, and probability distribution convolution formulas. Continuous case:

> $(f * g)(t) = \int_{-\infty}^{+\infty} f(\tau) g(t - \tau)\,d\tau$

Discrete case: $(f * g)[n] = \sum_m f[m] g[n-m]$.

The intuition is "flip g, slide it, and at each position compute a weighted sum with f." The "convolution" in CNNs is strictly speaking *cross-correlation* (no flipping), but the term convolution has persisted in engineering usage. Convolution has two remarkable properties that underpin its importance in AI:

Property 1: The Convolution Theorem. Under the Fourier transform, $\mathcal{F}(f * g) = \mathcal{F}(f) \cdot \mathcal{F}(g)$ — convolution in the spatial domain equals pointwise multiplication in the frequency domain. This is the theoretical basis for FFT-accelerated convolution and explains why frequency-domain signal analysis is so natural.

Property 2: Translation Equivariance. Shifting the input by some amount shifts the output by the same amount. A CNN's ability to recognize a cat anywhere in an image rests on this property.

```
import numpy as np

def conv1d(x, k):
```

```
    """Naive 1D convolution (cross-correlation), no padding"""
    n, m = len(x), len(k)
    y = np.zeros(n - m + 1)
    for i in range(n - m + 1):
        y[i] = np.sum(x[i:i+m] * k)
    return y

x = np.array([1, 2, 3, 4, 5], dtype=float)
k = np.array([1, 0, -1], dtype=float)   # edge detection kernel
print(conv1d(x, k))   # [-2, -2, -2]
```

The kernel `[1, 0, -1]` is a first-order difference operator; each output value is the "left minus right" of the input — it detects signal transitions. Extending this observation from 1D to 2D yields Sobel edge detection; replacing the hand-crafted kernel weights with learned ones gives you a CNN.

1.1.3 Function Approximation Techniques

> *Orientation: Taylor expansion, trigonometric series, splines — three fundamental tools for approximating complex functions with simpler ones.*

Taylor Expansion

Any sufficiently smooth function f can be expanded around x_0 as:

> f(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2!}(x-x_0)^2 + \ldots + \frac{f^{(n)}(x_0)}{n!}(x-x_0)^n + R_n

Intuition: approximate f locally with a degree-n polynomial. Higher order means a closer fit, but the approximation is only valid near x_0 and may diverge farther away.

Two typical uses in AI. First, **deriving the convergence of optimization algorithms**. Expand the loss to second order around the current point: L(\boldsymbol{\theta} + \Delta) \approx L(\boldsymbol{\theta}) + \nabla L^\top \Delta + \frac{1}{2}\Delta^\top \mathbf{H} \Delta, then minimize over \Delta to

obtain $\Delta = -\mathbf{H}^{-1} \nabla L$ — this is Newton's method. First-order expansion corresponds to gradient descent, second-order to Newton's method, and all mainstream optimizers are variations of this formula.

Second, **explaining universal approximation theorems for neural networks**. The claim that "any continuous function can be approximated arbitrarily well by a sufficiently wide single-hidden-layer network" shares the same origin as Taylor expansion — both are different realizations of "approximate a complex function by a linear combination of simple basis functions."

Trigonometric Series (Fourier Series)

A periodic function can be expanded as an infinite sum of sines and cosines:

> $$f(x) = \frac{a_0}{2} + \sum_{n=1}^\infty \left[a_n \cos(nx) + b_n \sin(nx)\right]$$

This is a completely different basis from Taylor — Taylor uses polynomials $1, x, x^2, \ldots$ while Fourier uses trigonometric functions. Both bases are orthogonal, allowing coefficients to be computed by "projection."

In AI, the Fourier transform is the foundation of signal processing and also appears in **positional encodings** — the sinusoidal positional encoding in the original Transformer paper assigns each position an encoding using sines and cosines of different frequencies. RoPE also uses the rotational representation of trigonometric functions in the complex-number sense.

Spline Approximation

A spline is a piecewise polynomial where adjacent pieces are required to join smoothly (with continuous derivatives of successive orders). The cubic spline uses a degree-3 polynomial on each piece with second-order derivative continuity, making it the most widely used interpolation tool in engineering. The distinction from Taylor/Fourier: Taylor is accurate near a single point but diverges farther away; splines guarantee global smoothness through piecewise construction, keeping each piece simple.

Splines are not mainstream in deep learning, but in recent years **Kolmogorov-Arnold Networks (KAN)** have replaced the activation functions inside neural networks with learnable B-splines, offering an alternative to MLPs. Although scaling remains challenging, this is a signal that splines and neural networks can be meaningfully combined.

1.1.4 Computing Function Extrema: From Gradient Descent to Backpropagation

> *Orientation: the most critical section of this chapter — the mathematical skeleton of neural network training.*

Starting with One-Dimensional Gradient Descent

To find the minimum of f(x), the algorithm is:

> x_{k+1} = x_k - \eta f'(x_k)

\eta is the learning rate. The intuition is immediate: the derivative tells us which direction is uphill, so subtracting it moves downhill. Generalizing to multiple dimensions:

> \boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \nabla L(\boldsymbol{\theta}_k)

At this point nothing specific to AI is visible — the real challenge is: \boldsymbol{\theta} has hundreds of millions of dimensions, so how do we compute \nabla L?

The Chain Rule Is All of Backpropagation

The chain rule: \frac{d}{dx} f(g(x)) = f'(g(x)) \cdot g'(x). High-school mathematics. But the output of a neural network is a chain of nested functions y = f_L(f_{L-1}(\ldots f_1(x))), so computing the partial derivative with respect to

some intermediate parameter w_i means a long chain of multiplications. The key engineering observation is: **many intermediate quantities in this chain are shared** . If each parameter independently computes a full chain, the total work is O(L \cdot N) (N being the number of parameters); if the computation proceeds in "output-to-input" order and is done in a single pass, the total work is O(L + N). This orders-of-magnitude difference is the reason backpropagation exists.

Backpropagation Through a Two-Layer Network

Consider the simplest network:

> \mathbf{z} = \mathbf{W}_1 \mathbf{x}, \quad \mathbf{a} = \sigma(\mathbf{z}), \quad \hat{y} = \mathbf{w}_2^\top \mathbf{a}, \quad L = \frac{1}{2}(\hat{y} - y)^2

We want the gradient of L with respect to \mathbf{W}_1. Applying the chain rule from the outside in:

> \frac{\partial L}{\partial \hat{y}} = \hat{y} - y

> \frac{\partial L}{\partial \mathbf{a}} = (\hat{y} - y) \cdot \mathbf{w}_2

> \frac{\partial L}{\partial \mathbf{z}} = \frac{\partial L}{\partial \mathbf{a}} \odot \sigma'(\mathbf{z})

> \frac{\partial L}{\partial \mathbf{W}_1} = \frac{\partial L}{\partial \mathbf{z}} \mathbf{x}^\top

Notice the last few steps: the "input-side gradient" at each layer equals the "output-side gradient times the layer's Jacobian." The algorithm is therefore: *run a forward pass and cache all intermediate values; then starting from the loss at the final layer, propagate gradients one layer at a time back toward the input, performing a single matrix multiplication per layer*.

```
import numpy as np
```

```
def sigmoid(z): return 1 / (1 + np.exp(-z))
def sigmoid_prime(z):
    s = sigmoid(z); return s * (1 - s)

# Forward + backward pass for a two-layer network
np.random.seed(0)
x = np.array([[0.5], [0.2]])        # input
y = np.array([[1.0]])               # target
W1 = np.random.randn(3, 2) * 0.1    # hidden layer
w2 = np.random.randn(1, 3) * 0.1    # output layer

# Forward
z  = W1 @ x
a  = sigmoid(z)
y_hat = w2 @ a
L  = 0.5 * (y_hat - y) ** 2

# Backward
dL_dyhat = y_hat - y                    # scalar gradient
dL_dw2   = dL_dyhat @ a.T               # [1,3]
dL_da    = w2.T @ dL_dyhat              # [3,1]
dL_dz    = dL_da * sigmoid_prime(z)     # elementwise
dL_dW1   = dL_dz @ x.T                  # [3,2]

print("loss =", L.item())
print("dL/dW1 =\n", dL_dW1)
```

These 20 lines of code are the conceptual core of the automatic differentiation engines in modern deep learning frameworks (PyTorch, TensorFlow). What the frameworks do is generate these hand-written chain-rule steps automatically: each operator registers a forward function and a corresponding backward function (a Jacobian-vector product), and at runtime a computation graph is constructed and traversed in reverse.

Why Deep Networks Are Hard to Train

Look at the term \sigma'(z). The maximum value of the sigmoid derivative is 0.25; multiplied L times it becomes 0.25^L — as depth increases, the gradient decays exponentially, and the early layers learn nothing. This is the classic **vanishing gradient** problem. ReLU, whose derivative is 1 on the positive half-axis, solved

this problem. The way to understand vanishing gradients is to examine each factor in this chain product and ask what it actually is.

1.1.5 Real Analysis Extension: The Invertibility Problem in LLMs

> *Orientation: an open theoretical question — given the output distribution of an LLM, can the input be recovered? Introduced briefly.*

One of the central objects in real analysis is "measure" — an abstraction of length, area, and volume. A concrete application in AI is the **invertibility of generative models**: normalizing flows require each transformation layer to be a bijection with a computable Jacobian determinant, so that likelihoods can be computed exactly. Diffusion models, by contrast, abandon strict invertibility in favor of learning the score function \nabla_\mathbf{x} \log p(\mathbf{x}).

For LLMs as autoregressive generative models, "invertibility" is raised at a different level: *given a piece of output, can the prompt be uniquely recovered?* The answer is no — the sampling process injects randomness, and many prompts can produce the same output. Nonetheless, studying "approximate invertibility" has practical significance: for example, determining whether a piece of text was generated by a specific model (watermark detection), or inferring training data leakage. This is an active but nascent research direction with no established engineering tools yet. It is enough simply to be aware that this problem exists.

1.1.6 Riemannian Extension: Optimization on Manifolds

> *Orientation: how to optimize when the parameter space is not a flat Euclidean space but a curved surface (manifold). A brief introduction.*

Standard gradient descent assumes parameters live in the flat space $\mathbb{R}^n$. But in some problems parameters are naturally subject to geometric constraints: orthogonal matrices ($\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$), positive definite matrices, unit-norm vectors, low-rank matrices. The sets formed by these objects are not flat $\mathbb{R}^n$ but **manifolds** — structures that look locally Euclidean but are globally curved.

Performing gradient descent on a manifold requires two modifications. First, the gradient must be projected onto the **tangent space** at the current point (taking a straight step would leave the manifold); this is called the Riemannian gradient. Second, after taking a step along a tangent vector, the result must be pulled back onto the manifold via a **retraction** (for example, re-orthogonalizing a matrix). The full framework is implemented in libraries such as `geoopt` for PyTorch.

Concrete applications in AI: orthogonal weight initialization in Transformers, certain LoRA variants that optimize on the unit sphere, and constraints on covariance matrices when training diffusion models. This is not something most working engineers encounter daily. It is enough to take away the impression that "when parameters carry hard constraints, manifold optimization may be needed."

PART TWO

1.2 Linear Algebra

Deep learning is countless nested layers of matrix multiplication — misunderstand this and much of the field becomes opaque.

1.2.1 Linear Spaces and Matrices

> *Orientation: definitions and axioms of vectors, matrices, and tensors; clarifying the meaning of "dimension".*

Vectors, Matrices, and Tensors

A vector $\mathbf{v} \in \mathbb{R}^n$ is an ordered arrangement of n real numbers. What matters is that it supports two operations: addition $\mathbf{u} + \mathbf{v}$ and scalar multiplication $\alpha \mathbf{v}$, and that these operations satisfy 8 axioms (associativity, distributivity, etc.). Any set satisfying these axioms is called a **linear space** (or vector space). $\mathbb{R}^n$ is the most fundamental linear space, but abstractly it can be a space of continuous functions, a space of probability distributions, and so on — this is the power of "linear" thinking.

A matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ arranges $m \times n$ real numbers in a two-dimensional grid. A tensor is a higher-dimensional generalization: a three-dimensional tensor can be thought of as a stack of matrices, and a four-dimensional tensor as a stack of three-dimensional tensors. In deep learning, "tensor" is used more loosely than its strict mathematical definition — in PyTorch, any multi-dimensional array is called a tensor.

Dimension and Rank

A set of vectors $\{\mathbf{v}_1, \ldots, \mathbf{v}_k\}$ is called **linearly independent** if $\sum \alpha_i \mathbf{v}_i = \mathbf{0}$ has only the solution $\alpha_i = 0$. The **dimension** of a linear space is the size of the largest linearly independent set that can be found.

The **rank** $\text{rank}(\mathbf{A})$ of a matrix $\mathbf{A}$ is the maximum number of linearly independent columns (or rows) in the matrix. $\text{rank}(\mathbf{A}) \le \min(m, n)$. Rank directly reflects "information redundancy": a low-rank matrix can be compressed into a lower-dimensional representation. This is the starting point for SVD and LoRA later on.

In AI

Each layer's weight in a neural network is a matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, and an input vector

$\mathbf{x}$ is transformed into an output vector via $\mathbf{W}\mathbf{x}$. The semantics of the entire network can be understood as "performing transformations between a series of linear spaces, with nonlinear activations in between." An LLM's embedding maps discrete tokens into a continuous linear space, after which all operations are geometric transformations within that linear space — the intuition that "semantically similar words are close together in embedding space" has its mathematical foundation right here.

1.2.2 Linear Transformations and Matrix Operations

> *Orientation: a matrix is not just a grid of numbers — it is a "linear transformation." Connecting geometry with algebra.*

Matrices as Transformations

$\mathbf{A} \in \mathbb{R}^{m \times n}$ can be viewed as a function: it sends a vector $\mathbf{x}$ in $\mathbb{R}^n$ to $\mathbf{y} = \mathbf{A}\mathbf{x}$ in $\mathbb{R}^m$. This function has two properties: $\mathbf{A}(\mathbf{x}+\mathbf{y}) = \mathbf{A}\mathbf{x} + \mathbf{A}\mathbf{y}$ (additivity) and $\mathbf{A}(\alpha \mathbf{x}) = \alpha \mathbf{A}\mathbf{x}$ (homogeneity), together called a **linear transformation**.

A few common 2D examples help build geometric intuition:

Rotation (by angle θ): $\begin{pmatrix}\cos\theta & -\sin\theta \\ \sin\theta & \cos\theta\end{pmatrix}$
Scaling (by s_x, s_y): $\begin{pmatrix}s_x & 0 \\ 0 & s_y\end{pmatrix}$
Shear (horizontal shear): $\begin{pmatrix}1 & k \\ 0 & 1\end{pmatrix}$
Projection onto the x-axis: $\begin{pmatrix}1 & 0 \\ 0 & 0\end{pmatrix}$

Key insight: a complex transformation = a composition of simple transformations = matrix multiplication. Rotation followed by scaling = the product of two matrices.

This is why matrix multiplication matters — it is the "composition operation" for transformations.

Three Ways to Understand Matrix Multiplication

$\mathbf{C} = \mathbf{A}\mathbf{B}$ has three complementary perspectives, all commonly used in practice.

Perspective 1: Inner product. $C_{ij} = \sum_k A_{ik} B_{kj}$ — row i dotted with column j. The textbook definition.

Perspective 2: Column combination. Treat the columns of $\mathbf{B}$ as vectors; each column of $\mathbf{A}\mathbf{B}$ is the result of applying $\mathbf{A}$ to the corresponding column of $\mathbf{B}$. This helps you understand scenarios like "$\mathbf{A}$ transforming a batch of data $\mathbf{B}$."

Perspective 3: Sum of outer products. $\mathbf{A}\mathbf{B} = \sum_k \mathbf{a}_k \mathbf{b}_k^\top$ — the outer product of the k-th column of $\mathbf{A}$ and the k-th row of $\mathbf{B}$, summed over k. This perspective is particularly important for understanding SVD and LoRA decomposition.

In AI

Matrix multiplication is the most computationally expensive operation in deep learning. The Q/K/V projections in Transformers, feed-forward layers, $QK^\top$ in attention, $(QK^\top) V$ — all of these are matrix multiplications. This is also why GPU Tensor Cores and NPU Cube units are specifically optimized for matrix multiplication — when matrix multiplication is fast, all of deep learning is fast. Low-precision formats like BF16/FP8 are first applied to matrix multiplication precisely because it accounts for 90%+ of total compute.

1.2.3 Solution Spaces, Inverses, and Pseudoinverses

> *Orientation: all cases for solving \mathbf{A}\mathbf{x} = \mathbf{b} — unique solution, no solution, infinitely many solutions. The mathematical basis of least squares lives here.*

Three Cases

For the linear system \mathbf{A}\mathbf{x} = \mathbf{b} with \mathbf{A} \in \mathbb{R}^{m \times n}:

\lt td>\det(\mathbf{A}) \ne 0 \lt td>Unique solution \mathbf{x} = \mathbf{A}^{-1}\mathbf{b}

Case	Condition	Result
Square, full rank (m = n, \text{rank} = n)		
Overdetermined (m > n)	Usually no exact solution	Least squares solution
Underdetermined (m \lt n)	Usually infinitely many solutions	Minimum-norm solution

Gaussian Elimination

The classic algorithm for solving linear systems: apply row operations to [\mathbf{A} | \mathbf{b}] to reduce it to row echelon form, then back-substitute. Time complexity O(n^3). In practice, explicitly computing matrix inverses is almost never done — solving via LU decomposition is more numerically stable and faster.

Generalized Inverse (Moore-Penrose Pseudoinverse)

When \mathbf{A} is not square, or is square but not invertible, the traditional \mathbf{A}^{-1} does not exist. The **pseudoinverse** \mathbf{A}^+ is the generalization:

- If \mathbf{A} has full column rank (m \ge n, \text{rank} = n): \mathbf{A}^+ = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top

- If $\mathbf{A}$ has full row rank ($m \le n$, $\text{rank} = m$): $\mathbf{A}^+ = \mathbf{A}^\top (\mathbf{A} \mathbf{A}^\top)^{-1}$
- General case: construct via SVD

$\mathbf{x} = \mathbf{A}^+ \mathbf{b}$ gives the least squares solution (minimizing the residual norm) in the overdetermined case, and the minimum-norm solution in the underdetermined case. These two results are unified into a single formula — elegantly clean.

In AI

Linear regression: given n samples $(\mathbf{x}_i, y_i)$, fitting $y = \mathbf{w}^\top \mathbf{x}$ in matrix form as $\mathbf{X}\mathbf{w} = \mathbf{y}$ (where $\mathbf{X}$ is $n \times d$ with $n > d$, overdetermined), the least squares solution is $\mathbf{w} = \mathbf{X}^+ \mathbf{y} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ — this is the mathematical form of the earliest machine learning algorithm.

Never compute the inverse directly in practice. $(\mathbf{X}^\top \mathbf{X})^{-1}$ is numerically unstable — when columns are highly correlated, $\mathbf{X}^\top \mathbf{X}$ approaches singularity. The engineering approach is to apply QR decomposition or SVD directly to $\mathbf{X}$, which is far more stable. This is also how `numpy.linalg.lstsq` is implemented.

1.2.4 Eigendecomposition and Matrix Diagonalization

> *Orientation: finding the "intrinsic" transformation directions of a matrix — the mathematical foundation of PCA, spectral methods, and graph neural networks.*

Eigenvalues and Eigenvectors

For a square matrix $\mathbf{A}$, if there exists a nonzero vector $\mathbf{v}$ and a scalar λ such that:

> $$\mathbf{A}\mathbf{v} = \lambda \mathbf{v}$$

then $\mathbf{v}$ is called an **eigenvector** and λ the corresponding **eigenvalue** . Geometric meaning: $\mathbf{A}$ acting on $\mathbf{v}$ *only changes its length, not its direction* (when $\lambda < 0$, the direction is reversed). These are the "intrinsic" directions of the matrix — the simplest directions under its action.

Diagonalization

If $\mathbf{A}$ has n linearly independent eigenvectors $\mathbf{v}_1, \ldots, \mathbf{v}_n$, arrange them as columns to form matrix $\mathbf{V}$ and let $\boldsymbol{\Lambda}$ be the diagonal matrix of corresponding eigenvalues; then:

> $$\mathbf{A} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^{-1}$$

This is the **eigendecomposition**. Intuition: applying $\mathbf{A}$ to any vector is equivalent to "first expressing the vector as a linear combination of eigenvectors ($\mathbf{V}^{-1}$), stretching each component by its corresponding eigenvalue ($\boldsymbol{\Lambda}$), then converting back to the original coordinate system ($\mathbf{V}$)."

Symmetric matrices are a special case: *a symmetric matrix is always diagonalizable, and its eigenvectors are mutually orthogonal* — that is, $\mathbf{A} = \mathbf{Q}\boldsymbol{\Lambda}\mathbf{Q}^\top$ where $\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}$. This is called the **spectral theorem**, the ideal case for numerical stability and geometric clarity. Covariance matrices, Laplacian matrices, and Hessians are all symmetric, so the spectral theorem appears repeatedly in AI.

PCA: A Classic Application of Eigendecomposition

The goal of principal component analysis: reduce high-dimensional data $\mathbf{X} \in \mathbb{R}^{n \times d}$ to k dimensions ($k \ll d$) while preserving as much variance as possible.

Steps: center the data → compute the covariance matrix $\mathbf{C} = \frac{1}{n}\mathbf{X}^\top \mathbf{X}$ ($\mathbf{C}$ is symmetric) → eigendecompose

\mathbf{C} → take the eigenvectors corresponding to the top k eigenvalues as the new basis.

Why does this preserve the maximum variance? Because *the variance of the data along a given direction equals the quadratic form of the covariance matrix in that direction, which equals the corresponding eigenvalue* (when the direction is an eigenvector). So choosing the k largest eigenvalues is equivalent to choosing the k directions of greatest variance.

```
import numpy as np

np.random.seed(0)
X = np.random.randn(1000, 5)
# Artificially make dimensions 1 and 2 have much larger variance
X[:, 0] *= 5; X[:, 1] *= 3

X_c = X - X.mean(axis=0)                    # Center
C = X_c.T @ X_c / len(X)                    # Covariance
eigvals, eigvecs = np.linalg.eigh(C)        # eigh handles
symmetric matrices
order = np.argsort(eigvals)[::-1]
eigvals, eigvecs = eigvals[order], eigvecs[:, order]
print("Variance explained by each principal component:", eigvals
/ eigvals.sum())

# Take top 2 principal components, reduce to 2D
X_pca = X_c @ eigvecs[:, :2]
print("Variance after dimensionality reduction:",
X_pca.var(axis=0))
```

In AI

Although PCA has largely given way to learnable dimensionality reduction in the deep learning era (autoencoders, embeddings), the idea of spectral decomposition never left. **Spectral methods in graph neural networks** (GCN) diagonalize the Laplacian matrix to define convolution in the spectral domain; **Hessian analysis** uses eigenvalues to describe the curvature distribution of the loss surface; **low-rank matrix approximation** retaining only the components corresponding to large eigenvalues is the foundation of compressing model weights.

1.2.5 Singular Value Decomposition (SVD)

> *Orientation: the crown jewel of linear algebra — the most universal matrix decomposition, and the direct theoretical basis for LoRA.*

The Form of SVD

Any matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ can be decomposed as:

> $$\mathbf{A} = \mathbf{U} \boldsymbol{\Sigma} \mathbf{V}^\top$$

where $\mathbf{U} \in \mathbb{R}^{m \times m}$ and $\mathbf{V} \in \mathbb{R}^{n \times n}$ are both orthogonal matrices, and $\boldsymbol{\Sigma} \in \mathbb{R}^{m \times n}$ is a diagonal matrix (all off-diagonal entries are zero when it is not square), with diagonal entries $\sigma_1 \ge \sigma_2 \ge \ldots \ge 0$ called the **singular values**.

Relationship to eigendecomposition: if $\mathbf{A}$ is a symmetric positive definite square matrix, SVD and eigendecomposition coincide and the singular values are the eigenvalues. But SVD exists for **any matrix** (non-square, non-invertible, non-symmetric) — this is what makes it more general than eigendecomposition.

Geometric Meaning

Break down $\mathbf{A}\mathbf{x}$ into three steps: **$\mathbf{V}^\top$ rotates the coordinate system → $\boldsymbol{\Sigma}$ scales along each axis → $\mathbf{U}$ rotates again into the output space**. Therefore any linear transformation is a combination of "rotation + scaling + rotation." This conclusion is remarkably powerful.

Low-Rank Approximation

SVD can be written as a sum of outer products:

> $$\mathbf{A} = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$$

where $r = \text{rank}(\mathbf{A})$. The **Eckart-Young theorem** tells us: retaining only the largest k terms ($k \lt r$), the result $\mathbf{A}_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$ is the *optimal rank-k approximation* to $\mathbf{A}$ under either the Frobenius norm or the spectral norm. No other rank-k matrix is closer to $\mathbf{A}$.

This is the unified mathematical foundation for compression, dimensionality reduction, and denoising. Image compression, matrix completion in recommendation systems, model weight compression — all rely on this theorem.

In AI: LoRA

LoRA's hypothesis is that the weight change $\Delta \mathbf{W}$ induced by fine-tuning an LLM is *low-rank* . Mathematically: $\Delta \mathbf{W} \approx \mathbf{B}\mathbf{A}$, where $\mathbf{A} \in \mathbb{R}^{r \times d}$ and $\mathbf{B} \in \mathbb{R}^{d \times r}$ with $r \ll d$. The parameter count drops from d^2 to $2rd$ — for a 7B model with $d \approx 4096$ and $r = 16$, that is a 128× compression.

Why does this work? Empirical observation: the "effective information dimensionality" of each layer in a neural network is far lower than its parameter dimensionality, and truncating SVD to retain the top r components loses almost no performance. This is not a mathematically proven result but an empirical regularity — yet SVD provides the theoretical language for it.

```
import numpy as np

np.random.seed(0)
# Simulate a 512×512 weight matrix that is essentially rank-10
plus noise
rank_true = 10
U_true = np.random.randn(512, rank_true)
V_true = np.random.randn(rank_true, 512)
W = U_true @ V_true + 0.01 * np.random.randn(512, 512)

U, S, Vt = np.linalg.svd(W, full_matrices=False)
# Reconstruct with different ranks and observe the error
for r in [5, 10, 20, 50, 100]:
    W_r = U[:, :r] @ np.diag(S[:r]) @ Vt[:r, :]
```

```
    err = np.linalg.norm(W - W_r) / np.linalg.norm(W)
    print(f"rank {r:3d}: relative error {err:.4f}")
```

Running this reveals that the error drops sharply around rank=10 and then flattens — confirming that "low-rank approximation can reconstruct with high fidelity." This is the mathematical evidence that justifies trusting LoRA.

1.2.6 Inner Product Spaces and Distance Metrics

> *Orientation: defining "angle" and "distance" in a vector space — the mathematical foundation of embedding retrieval.*

Inner Products and Norms

The inner product maps two vectors to a scalar: \langle \mathbf{u}, \mathbf{v} \rangle = \mathbf{u}^\top \mathbf{v} = \sum u_i v_i. It satisfies three axioms: symmetry, linearity, and positive definiteness. The norm is derived from the inner product:

> \|\mathbf{v}\|_2 = \sqrt{\langle \mathbf{v}, \mathbf{v} \rangle}

This is the **L2 norm** (Euclidean length). The more general Lp norm is: \|\mathbf{v}\|_p = (\sum |v_i|^p)^{1/p}; p=1 gives the Manhattan norm and p=\infty gives the maximum norm.

Angle: Cosine Similarity

From \langle \mathbf{u}, \mathbf{v} \rangle = \|\mathbf{u}\| \|\mathbf{v}\| \cos\theta, we obtain:

> \cos\theta = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}

This is **cosine similarity**. It cares only about direction, not magnitude — two vectors pointing in the same direction have cosine similarity 1, orthogonal vectors 0, and opposite directions -1. In embedding-based semantic retrieval this is the most commonly used similarity measure: *the closer the directions of the query vector and document vector, the more semantically related they are, and magnitude differences do not matter*.

Common Distance Metrics

Distance	Formula	Use case
Euclidean distance (L2)	$\sqrt{\sum (u_i - v_i)^2}$	Position, pixel values, physical quantities
Manhattan distance (L1)	$\sum \|u_i - v_i\|$	More robust to outliers
Cosine distance	$1 - \cos\theta$	Embedding semantic matching
Hamming distance	Number of differing bits	Binary vectors, hashing
KL divergence	$\sum p_i \log(p_i / q_i)$	Between probability distributions

Strictly speaking, KL divergence is not a distance (it is asymmetric), but it is used extensively in AI — knowledge distillation, variational inference, and RLHF all rely on KL constraints to keep two distributions from drifting too far apart.

In AI

KNN and KMeans: finding nearest neighbors based on Euclidean distance. **Vector databases** (e.g., FAISS, Milvus): approximate nearest neighbor (ANN) search using cosine or inner product similarity as the core operation. **Contrastive learning** (CLIP, SimCLR): maximizing cosine similarity for positive pairs and minimizing it for negative pairs. **RAG retrieval**: embedding both query and documents, ranking by cosine similarity, and returning the top-k results.

PART THREE

1.3 Probability

The mathematical language of uncertainty — running through machine learning from loss functions to sampling-based inference.

1.3.1 Basic Probability: POTC and Balls and Bins

Orientation: Use the concrete problem of "two-choice load balancing" to clearly explain the thinking behind probabilistic modeling.

Definition of Probability

The sample space Ω is the set of all possible outcomes, and an event is a subset of Ω. Probability P is a function mapping events to [0, 1], satisfying $P(\Omega) = 1$ and countable additivity $P(\cup_i A_i) = \sum_i P(A_i)$ (when A_i are pairwise disjoint). Conditional probability is $P(A|B) = P(A \cap B) / P(B)$, and independence means $P(A \cap B) = P(A) P(B)$.

Bayes' theorem is the most important corollary:

> $P(A|B) = \frac{P(B|A) P(A)}{P(B)}$

Random Variables

A random variable X is a function from Ω to $\mathbb{R}$ — it maps "outcomes" to numerical values. A discrete RV is described by a probability mass function $p(x) = P(X=x)$; a continuous RV is described by a probability density function f(x), where $P(a \le X \le b) = \int_a^b f(x)\,dx$. Expectation is $\mathbb{E}[X] = \sum x p(x)$ or $\int x f(x) dx$, and variance is $\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2]$.

Balls and Bins Model

Throw n balls randomly into n bins. The question: how many balls are in the fullest bin? This is the mathematical abstraction of **load balancing**.

Each ball independently picks a bin uniformly at random, so the number of balls each bin receives is approximately \text{Poisson}(1). By analyzing tail probabilities, one can show: *the maximum load is with high probability approximately \Theta(\log n / \log\log n)* . For n = 1000, the fullest bin holds about 5 balls — not terrible, but already 5 times the average.

POTC: Power of Two Choices

Improved strategy: each ball independently picks *two* bins at random and goes into the less loaded one. This seemingly minor change produces a dramatic effect — *the maximum load is with high probability approximately \Theta(\log\log n)* — an exponential improvement. For n = 1000, the maximum drops from 5 balls to 2.

This is one of the most striking results in probability theory: "looking at one extra option" yields an exponential improvement. The intuition: worse bins are less likely to be chosen, pushing the distribution toward uniformity.

```
import numpy as np
import random

def simulate_balls_bins(n, strategy='random'):
    bins = [0] * n
    for _ in range(n):
        if strategy == 'random':
            bins[random.randrange(n)] += 1
        elif strategy == 'potc':
            i, j = random.randrange(n), random.randrange(n)
            bins[i if bins[i] <= bins[j] else j] += 1
    return max(bins)

n = 10000
print("Random   max load:", np.mean([simulate_balls_bins(n,
'random') for _ in range(50)]))
print("POTC     max load:", np.mean([simulate_balls_bins(n,
'potc')   for _ in range(50)]))
# Typical output: random ~6.5, POTC ~3.1
```

In AI

Balls and Bins is a fundamental tool for analyzing **hash collisions**, **parameter server load in distributed training**, and **expert routing in MoE models**. A common phenomenon in MoE is "routing collapse" — the majority of tokens are routed to a small number of experts, resulting in low utilization. This is essentially the non-independent ball version of balls-and-bins, and requires an auxiliary loss to enforce uniformity.

1.3.2 Common Probability Distributions and Fundamental Theorems

> *Orientation: Three families of distributions — discrete, light-tailed, and heavy-tailed — plus the Law of Large Numbers and the Central Limit Theorem: the intuitive foundation for feature engineering.*

Discrete Distributions

\lt td>p, p(1-p) \lt td>np, np(1-p) \lt td>\lambda, \lambda \lt td>1/p, (1-p)/p^2

Distribution	Meaning	PMF	Mean / Variance
Bernoulli Ber(p)	One coin flip	P(X=1) = p	
Binomial Bin(n, p)	Sum of n Bernoullis	\binom{n}{k} p^k (1-p)^{n-k}	
Poisson Poi(λ)	Count of rare events	\frac{\lambda^k e^{-\lambda}}{k!}	
Geometric Geo(p)	Number of trials until success	(1-p)^{k-1} p	

Poisson is the limit of Binomial as n \to \infty, p \to 0, np = \lambda. It describes "a large number of independent rare events" — user traffic, system crashes, genetic

mutations.

Light-Tailed Distributions

The **normal distribution** $\mathcal{N}(\mu, \sigma^2)$ is the most fundamental continuous distribution. Its density is:

> $$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

The "68-95-99.7" rule for the standard normal $\mathcal{N}(0,1)$: $\pm 1\sigma$ contains 68% of probability, $\pm 2\sigma$ contains 95%, and $\pm 3\sigma$ contains 99.7%.

The **exponential distribution** $\text{Exp}(\lambda)$: $f(x) = \lambda e^{-\lambda x}$, describes "waiting time between events." Memoryless property: the probability of waiting an additional t units after already waiting s units is the same as the probability of waiting t units from scratch.

Log-normal: the logarithm of the variable is normally distributed. Suitable for size variables (e.g., file sizes, incomes) — always positive and right-skewed.

Heavy-Tailed Distributions

The technical definition of heavy-tailed is "the tail decays more slowly than an exponential function." The consequence is that **extreme values are no longer rare**.

Pareto distribution: $P(X > x) \propto x^{-\alpha}$. The classic "80/20 rule." Internet traffic and file access frequencies are both heavy-tailed.

Power law: another way of expressing the same idea as Pareto — characterized by a straight line on a log-log plot.

In heavy-tailed distributions, *the sample mean is a poor estimator of the population mean* — a single extreme value can pull the mean far off. Working with heavy-tailed data requires: log transformation, using the median instead of the mean, or winsorizing with quantile truncation.

Law of Large Numbers and Central Limit Theorem

The **Law of Large Numbers**: the sample mean $\bar{X}_n$ of n i.i.d. samples converges with probability 1 to the true mean μ (given finite variance). This is the foundation of all statistical methods based on "estimating the population from a sample" — and the reason SGD works (using mini-batch gradients to estimate the full-dataset gradient).

The **Central Limit Theorem**: the sample mean of n i.i.d. samples from any distribution converges in distribution to a normal as $n \to \infty$:

> $$\sqrt{n}(\bar{X}_n - \mu) \xrightarrow{d} \mathcal{N}(0, \sigma^2)$$

This is why the normal distribution is everywhere — *"summing/averaging" smooths out the shape of the original distribution*, leaving only the normal. But note: the CLT requires **finite variance**. Heavy-tailed distributions (e.g., Pareto with $\alpha \le 2$) do not satisfy this and the CLT does not apply.

In AI

Identifying the type of distribution is a basic skill in feature engineering: light-tailed data can be standardized, while heavy-tailed data must be log-transformed first. Token frequencies in training data classically follow a Zipf (power law) distribution, which directly affects tokenizer design. MoE expert loads are naturally heavy-tailed, which is why an auxiliary loss must be used to enforce balance.

PART FOUR

1.4 Statistics

Inferring the unknown from data — the philosophical core of model training.

1.4.1 Parameter Estimation

> *Orientation: Maximum likelihood estimation is the mathematical identity of cross-entropy loss.*

Problem Setup

Suppose data $x_1, \ldots, x_n$ are independently sampled from some distribution $p(x; \theta)$, where θ is unknown. The task: estimate θ from data. There are two estimation paradigms: frequentist (θ is an unknown constant, estimated from data) and Bayesian (θ has a prior distribution, updated with data). Deep learning is predominantly frequentist.

Maximum Likelihood Estimation (MLE)

The likelihood function: the probability of observing the current data given parameter θ:

> $$L(\theta) = \prod_{i=1}^n p(x_i; \theta)$$

MLE finds the θ that maximizes $L(\theta)$. In practice we take the logarithm (turning products into sums, which is numerically more stable):

> $$\hat{\theta}_{\text{MLE}} = \arg\max_\theta \sum_{i=1}^n \log p(x_i; \theta)$$

MLE = Minimizing Cross-Entropy

This equivalence is key to understanding deep learning loss functions. Consider a classification task where the model $p(y|x; \theta)$ outputs a class distribution for each sample. MLE maximizes:

> $$\sum_i \log p(y_i \mid x_i; \theta)$$

Negating this gives minimizing the negative log-likelihood. And the **cross-entropy loss** $\text{CE} = -\sum_c y_c \log \hat{y}_c$ (cross-entropy between one-hot label y

and predicted distribution $\hat{y}$), when summed over all samples, is exactly the negative log-likelihood.

Therefore: *using cross-entropy loss for classification = performing maximum likelihood estimation*. These are not two different things — they are two ways of saying the same thing.

MAP (Maximum A Posteriori)

Add a prior $p(\theta)$ on θ, and find the θ that maximizes the posterior $p(\theta \mid \text{data}) \propto L(\theta)\, p(\theta)$. Taking the logarithm:

> $$\hat{\theta}_{\text{MAP}} = \arg\max_\theta \left[\log L(\theta) + \log p(\theta)\right]$$

If the prior $p(\theta)$ is Gaussian $\mathcal{N}(0, \sigma^2)$, then $\log p(\theta)$ contributes a $-\|\theta\|^2 / (2\sigma^2)$ term — this is **L2 regularization**. A Laplace prior corresponds to L1 regularization. This is the Bayesian interpretation of regularization.

Confidence Intervals

A point estimate $\hat{\theta}$ is a single number that tells you nothing about certainty. A confidence interval $[\hat{\theta} - c, \hat{\theta} + c]$ means "if we repeated the sampling 100 times, 95% of the resulting confidence intervals would contain the true value" — note this is the probability that the interval contains the true value, not the probability that the true value falls within the interval (in the frequentist framework, θ is a constant, so there is no probability of it "falling" anywhere).

In AI

Model training is essentially MLE. When evaluating model performance, reporting a single accuracy number is not enough — you need bootstrap confidence intervals and standard deviations across multiple runs. The statement "our model beats the baseline by 0.3%" means very little without a confidence interval; it is very likely just sampling noise.

1.4.2 Hypothesis Testing

> *Orientation: Quantitatively determining whether two models differ significantly — the foundation of A/B testing and ablation studies.*

Core Concepts

Null hypothesis H_0 : the default position we wish to disprove ("the new model is no different from the old one"). **Alternative hypothesis H_1**: the claim we wish to establish ("the new model is better").

Test statistic: a number computed from the data that has a known distribution under H_0. **p-value** : the probability of observing "the current statistic or a more extreme value" under H_0. A small p-value means "if H_0 were true, the current data would be unlikely," and we therefore reject H_0.

Common p-value thresholds are 0.05 or 0.01. These thresholds are historical conventions, not mathematical laws — in recent years many fields have recommended 0.005 or even abandoning p-values altogether in favor of Bayesian approaches, due to widespread "p-value abuse."

Two Types of Errors

\lt th>Truth: H_1 is true

	Truth: H_0 is true	
Accept H_0	Correct	Type II error (false negative, β)
Reject H_0	Type I error (false positive, α)	Correct (power 1-β)

\alpha and \beta cannot both be reduced simultaneously — only increasing the sample size can reduce both at once. Before running an experiment you should perform a **power analysis** : target effect size + desired \alpha and \beta → required sample size. Many AI experiments have too few samples, insufficient power, and cannot detect a real difference even when one exists.

t-Test

The most commonly used test. It compares whether the means of two groups are significantly different. Assuming both groups are normally distributed with equal variances, the test statistic is:

> $$t = \frac{\bar{X}_1 - \bar{X}_2}{s_p \sqrt{1/n_1 + 1/n_2}}$$

s_p is the pooled standard deviation. Under H_0, t follows a t-distribution.

Chi-Squared Test

Tests categorical data. Given a frequency table with k categories, it tests whether "observed frequencies vs. expected frequencies" are consistent. The test statistic is:

> $$\chi^2 = \sum_{i=1}^k \frac{(O_i - E_i)^2}{E_i}$$

Used for data distribution tests, tokenizer language coverage testing, MoE expert activation uniformity tests, and similar applications.

In AI

Model A achieves 85.3% on a benchmark and model B achieves 85.7% — is that 0.4% difference real? Two approaches:

1. **Run each model with multiple random seeds** and perform a t-test;
2. **Run a paired test on the same test set** (McNemar's test), because the two models are being compared on the same set of samples — this is paired data and has higher statistical power than an independent-samples test.

The minimum standard for a paper's reported results: mean ± standard deviation across multiple seeds, ideally accompanied by a p-value from a significance test.

PART FIVE

1.5 Stochastic Processes

> *Randomness evolving over time — LLM inference is fundamentally a Markov decision process.*

1.5.1 Basic Stochastic Processes

> *Orientation: Discrete and continuous stochastic processes, Poisson processes, Markov processes, stationarity — covered at a high level.*

Definition

A stochastic process $\{X_t\}_{t \in T}$ is a family of random variables indexed by time (or space). If T is discrete (e.g., $\{0, 1, 2, \ldots\}$), it is a discrete-time process; if T is continuous (e.g., $[0, \infty)$), it is a continuous-time process.

Poisson Process

A stochastic process counting the "number of occurrences of rare events." Properties:

1. $N(0) = 0$;
2. Event counts in non-overlapping time intervals are independent;
3. The number of events in the interval $[s, s+t]$ follows $\text{Poisson}(\lambda t)$.

The inter-event times follow $\text{Exp}(\lambda)$. This is the foundation of queueing theory and server request modeling.

Markov Process

Core property: **given the present, the future is independent of the past** — $P(X_{t+1} \mid X_t, X_{t-1}, \ldots) = P(X_{t+1} \mid X_t)$. A discrete-time Markov chain is described by a transition matrix $\mathbf{P}$, where $P_{ij} = P(X_{t+1}=j \mid X_t=i)$.

The key question: what does the distribution look like after a long time? The answer: if the chain is ergodic and aperiodic, it converges to a **stationary distribution** $\boldsymbol{\pi}$ satisfying $\boldsymbol{\pi}^\top \mathbf{P} = \boldsymbol{\pi}^\top$. The stationary distribution is the left eigenvector of the transition matrix (with eigenvalue 1) — linear algebra once again.

Stationarity

Strict stationarity: all finite-dimensional distributions are invariant under time shifts. Too strong a requirement, rarely used in practice. **Wide-sense stationarity**: the mean and covariance do not vary with time — $\mathbb{E}[X_t] = \mu$ holds for all t, and $\text{Cov}(X_t, X_{t+h})$ depends only on h and not on t. Many time-series methods (ARIMA) are based on the wide-sense stationarity assumption.

In AI

LLM autoregressive sampling is fundamentally a Markov process: the next token depends only on the current hidden state (although through the KV Cache it indirectly depends on all history, the "state" at each step is just the KV). **Reinforcement learning**'s MDP is a Markov process augmented with actions. The denoising process in **diffusion models** is a Markov chain. **MCMC sampling** constructs a Markov chain with a specific stationary distribution for inference. The field of stochastic processes itself need not be studied in depth, but the Markov idea is everywhere.

1.5.2 Stochastic Process Modeling of Tree of Thoughts

> *Orientation: Viewing LLM reasoning as a tree-structured stochastic process — a concrete application of stochastic processes.*

From Chain of Thought to Tree of Thoughts

Chain of Thought (CoT): prompting the LLM to "think step by step" before answering, generating an intermediate chain of reasoning. Mathematically, this is a sequential generation process: each step depends on all previous steps (via KV Cache), and the strategy is single-path sampling.

Tree of Thoughts (ToT): at each reasoning node, multiple branches are expanded, forming a tree-shaped search space, and some strategy (DFS, BFS, beam search) is used to find the best leaf-node answer.

Stochastic Process Perspective

Model ToT as a stochastic decision process with evaluation:

- **State**: the reasoning prefix generated so far;
- **Action**: the next reasoning step generated by the LLM;
- **Transition**: the next state obtained by sampling (stochastic, because LLM sampling uses temperature);
- **Evaluation**: scoring each intermediate state using the LLM itself or a separate evaluator;
- **Policy**: how to select the next branch to expand (greedy / Monte Carlo tree search / beam).

This is a perfect analogy to AlphaGo's MCTS — the only difference is that the action space changes from board positions to natural language. The randomness comes from two layers: the temperature of LLM sampling itself, and the exploration randomization in the tree search strategy.

Why This Modeling Is Useful

Single-path CoT propagates errors — once a step goes wrong, there is no way to correct it downstream. ToT provides a backtracking capability: branches with low scores are pruned, and the tree is re-expanded from an earlier node. Mathematically, this trades computational cost for reliability — one inference becomes dozens, but with a higher accuracy rate.

In terms of stochastic process theory, this is a **selective Markov process**: rather than passively observing the evolution of states, we make decisions at each step

based on an evaluation function. This is also precisely the basic setup of RL.

ToT raises GPT-4's accuracy on the Game of 24 task from 4% to 74% (*Tree of Thoughts*, Yao et al. 2023). The cost is a 5–100× increase in token consumption.

PART SIX

1.6 Information Theory

The mathematics of "information" established by Shannon — from entropy to mutual information, the home of cross-entropy loss.

1.6.1 The Core Questions of Information Theory

> *Orientation: Information theory studies two fundamental questions — how small can data be compressed? How fast can a channel reliably transmit?*

Shannon's 1948 paper defined the entire field. He posed two mathematically precise questions:

Compression: How much randomness does a data source have? Can it be represented losslessly with fewer bits? Answer: *no fewer than entropy H(X) bits per symbol*. This is the information-theoretic lower bound on data compression.

Transmission: How much information can a noisy channel reliably convey per unit time? Answer: *no more than channel capacity C*. Below this limit, encoding schemes exist that make the error rate arbitrarily small; above it, errors are unavoidable.

These two results are almost unreasonably clean — both are sharp "achievable but not exceedable" limits. They do not tell you how to achieve them (existence proofs), but they tell you "no further optimization is possible beyond this point." In engineering, information theory is a mirror: any real-world compression algorithm or communication system is measured against these limits.

1.6.2 Measuring Information

> *Orientation: Entropy, conditional entropy, mutual information, KL divergence — the direct ancestors of deep learning loss functions.*

Entropy

The entropy of a random variable X:

> $H(X) = -\sum_x p(x) \log p(x)$

The base of the logarithm determines the unit: $\log_2$ gives bits, $\ln$ gives nats. Two layers of intuition:

1. **Uncertainty**: How hard it is to predict the outcome of X. A uniform distribution has maximum entropy; a degenerate distribution (one value with probability 1) has entropy 0.
2. **Minimum encoding length**: Using an optimal prefix code to encode X, on average H(X) bits are needed per symbol.

Flipping a fair coin gives $H = 1$ bit; rolling a fair die gives $H = \log_2 6 \approx 2.58$ bits — indeed, about 2.58 bits are needed to encode a single die outcome.

Cross-Entropy and KL Divergence

The **cross-entropy** between two distributions p, q:

> $H(p, q) = -\sum_x p(x) \log q(x)$

Interpretation: Using the optimal code designed for distribution q to encode data actually drawn from distribution p, the average number of bits required per symbol. Clearly $H(p,q) \ge H(p)$, and the excess is the **KL divergence**:

> $$D_{\text{KL}}(p \parallel q) = H(p, q) - H(p) = \sum_x p(x) \log \frac{p(x)}{q(x)}$$

$D_{\text{KL}} \ge 0$, with equality if and only if $p = q$. **Asymmetry** : $D_{\text{KL}}(p \| q) \ne D_{\text{KL}}(q \| p)$ — this is not a mathematical defect, but actually useful: "approximating p with q" and "approximating q with p" are genuinely different operations, and KL's asymmetry faithfully reflects this.

Mutual Information

The amount of information shared between two random variables:

> $$I(X; Y) = D_{\text{KL}}(p(x,y) \parallel p(x)p(y)) = H(X) - H(X|Y)$$

Two equivalent forms. The first: how far the joint distribution is from independence. The second: how much uncertainty about X is reduced by knowing Y. $I(X;Y) = 0$ iff X, Y are independent.

In AI

Cross-entropy loss for classification: one-hot label $\mathbf{y}$, model prediction $\hat{\mathbf{y}}$, the loss is $H(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_c y_c \log \hat{y}_c$. Why not just use L2 loss? Because cross-entropy is equivalent to MLE, and when output probabilities saturate (close to 0 or 1) gradients do not vanish — whereas with L2 they do.

KL divergence as a constraint: In VAEs, the latent variable distribution is forced to stay close to a standard normal using KL; in RLHF, KL constrains the RL-optimized policy from drifting too far from the SFT model; in distillation, KL measures the distributional distance between student and teacher.

Mutual information as an objective: Many self-supervised learning methods (InfoNCE, CPC) maximize mutual information between different views of data —

"the semantics shared across two views" is precisely what we want to learn.

1.6.3 Compressing Information: Source Coding

> *Orientation: Huffman coding, arithmetic coding — classic schemes for compression algorithms. Brief.*

The Source Coding Theorem

Shannon's first theorem: the average code length of any lossless code is \ge H(X), and there exists a code that achieves H(X) + 1 (due to the integer-bit constraint).

Huffman Coding

A greedy algorithm: repeatedly merge the two symbols with the lowest probabilities to form a binary tree, with left branch 0 and right branch 1. Result: high-frequency symbols get short codes, low-frequency symbols get long codes. Average code length is close to entropy (gap \lt 1 bit per symbol).

Drawback: each symbol must be encoded with an integer number of bits. If a symbol has probability 0.9, its information content is only -\log 0.9 \approx 0.15 bits, yet Huffman still uses 1 bit — wasteful.

Arithmetic Coding

Maps an entire message to an interval in [0,1), then encodes it as the binary representation of a real number within that interval. Theoretically achieves the entropy limit without the integer-bit constraint. Modern compression algorithms (AV1 video, Brotli compression) are largely based on arithmetic coding or its variant ANS.

In AI

LLMs as compressors: Research from DeepMind and others has shown that large language models can serve as powerful general-purpose compressors — feed text to

the model, use the per-step conditional probabilities for arithmetic coding, and achieve compression ratios that beat gzip. The intuition: LLMs learn better $p(x_{t+1} | x_{1:t})$; the better the model's predictions, the shorter the code. This is one of the most elegant connections between information theory and LLMs: *compression is understanding*.

1.6.4 Transmitting Information: Channel Coding

> *Orientation: Adding redundancy to combat noise — the core technology of communications. Less directly relevant to AI, kept brief.*

Shannon's second theorem: a noisy channel has capacity $C = \max_{p(x)} I(X; Y)$; at transmission rate $R \lt C$, there exists an encoding scheme that drives the error probability to zero.

Implementation: add redundant bits. The simplest repetition code ($0 \to 000$, $1 \to 111$) decodes by majority vote. But it is inefficient — 3 bits to transmit 1 bit.

Practical codes: Reed-Solomon codes (CDs, QR codes), convolutional codes + Viterbi decoding, LDPC (5G), and Polar codes (also 5G, proposed by Turkish researcher Erdal Arıkan). These codes transmit at rates approaching capacity.

In AI, except for certain end-to-end communications research (replacing encoder/decoder with neural networks), engineers rarely encounter this in daily work. Knowing "this exists" is sufficient.

1.6.5 Boundaries and Security: Information Theory in ML and Cryptography

> *Orientation: Differential privacy, membership inference attacks — an information-*

> *theoretic perspective on AI security.*

Differential Privacy

An algorithm $\mathcal{M}$ is ϵ-differentially private if for any two datasets D, D' differing by a single record and any output S:

> $$P(\mathcal{M}(D) \in S) \le e^{\epsilon} \cdot P(\mathcal{M}(D') \in S)$$

Intuition: observing model outputs reveals almost nothing about whether any particular record was in the training set. Smaller ϵ means stronger privacy. Engineered in practice by adding Gaussian noise to gradients (DP-SGD).

Membership Inference Attacks

An attacker attempts to determine whether a particular record was in the training set. The method typically exploits the signal that "training samples have lower loss on the model." From an information-theoretic perspective: the model memorizes the training set, and the mutual information between the training data and model parameters satisfies $I(\text{data}; \theta) > 0$ — the larger this mutual information, the worse the privacy.

Why LLM Privacy Risks Are Serious

With sufficient capacity, LLMs can precisely memorize rare sequences from the training set (such as API keys on GitHub, personal email addresses). Membership inference and verbatim extraction attacks have both been demonstrated to be feasible. Differential privacy training can mitigate this, but at the cost of model quality — at the scale of large LLMs, this remains an unresolved engineering trade-off.

1.6.6 The Future of Information Theory

> *Orientation: Quantum information, AI communications, brain-computer interfaces — a brief glimpse at frontier directions.*

Quantum information theory: using quantum states as the unit of information (qubits), which can be superposed, introducing new concepts such as von Neumann entropy and quantum entanglement. Theoretical limits are broader than classical information theory. Still far from practical engineering.

Semantic communications: traditional communications focuses on transmitting bits accurately; semantic communications focuses on preserving meaning — allowing outputs that differ literally but are semantically consistent. AI models (especially LLMs) as semantic encoders/decoders are central to this direction. One candidate technology area for 6G.

Brain-computer interfaces: the upper bound on information rate between neural signals, intent, text, and actions is an information-theoretic problem. Neuralink and similar research are all advancing along this curve.

This section requires no deep exploration — engineering roles will not encounter this within several years. The goal is simply to build the impression that "information theory is not an old discipline; it is still actively evolving at new frontiers."

PART SEVEN

1.7 Numerical Computation

Making mathematics actually run on finite-precision computers — the area AI engineers interact with most frequently in daily work.

1.7.1 Error Analysis

> *Orientation: Floating-point numbers are not real numbers — this section is the foundation for quantization, low-precision training, and NaN debugging.*

IEEE 754 Floating-Point

Floating-point representation:

> $x = (-1)^s \cdot (1.f) \cdot 2^{e}$

s is the sign bit, f is the mantissa (fractional part), and e is the exponent. FP32 has 1+8+23 bits, FP16 has 1+5+10 bits, and BF16 has 1+8+7 bits.

Core fact: **floating-point numbers are not continuous** but form a discrete lattice whose density varies with magnitude — densely packed near small values, sparsely packed near large values. This governs all numerical behavior.

Machine Epsilon

The smallest floating-point number greater than 1 is $1 + \epsilon$, where ϵ is the **machine epsilon** . FP32 $\epsilon \approx 1.19 \times 10^{-7}$, FP16 $\epsilon \approx 9.77 \times 10^{-4}$, BF16 $\epsilon \approx 3.91 \times 10^{-3}$. BF16 has four times worse precision than FP16, but it has a larger dynamic range (same exponent bits as FP32), which is why training uses BF16 rather than FP16.

Classic Numerical Pitfalls

Pitfall 1: Catastrophic cancellation. Subtracting two nearby numbers loses precision.

```
import numpy as np
x = np.float32(1e8)
y = x + np.float32(1.0)
print(y - x)   # Expected 1.0, may actually be 0.0 — 1.0 is
swallowed by the precision of 1e8
```

Pitfall 2: Numerical overflow in softmax. Computing $\exp(x_i)/\sum\exp(x_j)$ directly causes $\exp$ to overflow to inf when x_i is large. The standard fix is to **subtract the maximum**:

> $\text{softmax}(x_i) = \frac{\exp(x_i - \max_j x_j)}{\sum_k \exp(x_k - \max_j x_j)}$

Mathematically equivalent (numerator and denominator both multiplied by \exp(-\max)), but numerically all \exp inputs are \le 0, so results lie in (0, 1] without overflow.

Pitfall 3: log-sum-exp. \log\sum\exp(x_i) also overflows when computed directly; the same subtract-the-maximum trick applies:

> \log\sum_i \exp(x_i) = \max_j x_j + \log\sum_i \exp(x_i - \max_j x_j)

These are not advanced optimizations — they are the standard techniques used by default in all numerical libraries (numpy, torch). Writing your own softmax and forgetting to subtract the maximum will cause mysterious NaNs during training.

In AI

Low-precision training: Training FP32 models with FP16/BF16. Gradients can underflow in FP16 (becoming 0 when smaller than the minimum positive value), requiring loss scaling — multiply the loss by a large number (e.g., 2^{15}) to shift gradient magnitudes into the representable range of FP16, then divide back before updating parameters. Because BF16 has a larger exponent range, loss scaling is typically unnecessary.

Quantization: INT8/INT4 inference. Maps floating-point weights to integers. This mapping introduces additional quantization error, and the accumulation of errors determines the compression limit. INT4 works for LLMs (AWQ, GPTQ), but going lower to INT2/INT3 causes severe accuracy degradation — not because the algorithms fail, but because the errors exceed the model's tolerance for perturbation.

Numerical stability checklist: subtract the maximum wherever \exp appears; add a small \epsilon wherever \log appears (\log(0) = -\infty); check denominators in all divisions; replace matrix inversion with solve; use Cholesky instead of checking whether the Hessian is positive definite.

1.7.2 Classical Numerical Analysis Algorithms

> *Orientation: Linear systems, matrix decompositions, eigenvalue solvers — the "chassis" of scientific computing libraries.*

LU Decomposition

$\mathbf{A} = \mathbf{L}\mathbf{U}$, where $\mathbf{L}$ is lower triangular (with unit diagonal) and $\mathbf{U}$ is upper triangular. Steps to solve $\mathbf{A}\mathbf{x} = \mathbf{b}$: $\mathbf{L}\mathbf{y} = \mathbf{b}$ (forward substitution, $O(n^2)$) → $\mathbf{U}\mathbf{x} = \mathbf{y}$ (back substitution, $O(n^2)$). Total: $O(n^3)$ for decomposition + $O(n^2)$ for solving. In practice, partial pivoting (PLU) is used to ensure numerical stability.

QR Decomposition

$\mathbf{A} = \mathbf{Q}\mathbf{R}$, where $\mathbf{Q}$ is orthogonal and $\mathbf{R}$ is upper triangular. More stable than LU, commonly used for least squares and eigenvalue computations. Construction methods: Gram-Schmidt (unstable), Householder reflections (stable, used in practice), Givens rotations.

Cholesky Decomposition

Specialized for symmetric positive definite matrices: $\mathbf{A} = \mathbf{L}\mathbf{L}^\top$. Computational cost is half that of general LU. Used repeatedly in Gaussian processes, Bayesian inference, and Kalman filtering, because covariance matrices are always symmetric positive definite.

Eigenvalue Solvers

Small matrices: solve the characteristic polynomial $\det(\mathbf{A} - \lambda \mathbf{I}) = 0$ directly. Large matrices: the **QR algorithm** (iteratively applying QR decomposition until convergence to diagonal/triangular form) is the standard approach — it is what `numpy.linalg.eig` uses under the hood.

When only the top k eigenvalues are needed : use **power iteration** . Starting from a random vector $\mathbf{v}_0$, repeatedly apply $\mathbf{v}_{k+1} = \mathbf{A}\mathbf{v}_k / \|\mathbf{A}\mathbf{v}_k\|$, which converges to the

eigenvector corresponding to the largest eigenvalue. Combined with deflation or the Lanczos method, multiple leading eigenvectors can be obtained.

PageRank is power iteration — repeatedly multiplying the web link matrix converges to the steady-state probability of each page. Google's original core algorithm.

Linear Programming

Objective: $\min \mathbf{c}^\top \mathbf{x}$, subject to $\mathbf{A}\mathbf{x} \le \mathbf{b}$, $\mathbf{x} \ge \mathbf{0}$. The **simplex method** traverses vertices of a polyhedron — classical but exponential in the worst case. **Interior-point methods** (Karmarkar) traverse the interior in polynomial time and are standard in modern solvers. SVM primal optimization and LASSO solving can both be reduced to LP/QP.

In AI

Scientific computing libraries (numpy, scipy) encapsulate these algorithms. You will rarely write them by hand, but you need to *know which API to choose*: use `np.linalg.solve` (LU) to solve linear systems — not `inv`; use `np.linalg.lstsq` (SVD/QR) for least squares; use `scipy.linalg.cho_solve` for symmetric positive definite systems. Choosing the wrong API can result in numerical accuracy and speed differences of an order of magnitude.

1.7.3 Interpolation and Function Fitting

> *Orientation: Fitting a function to a point cloud — from classical interpolation to least squares.*

The Difference Between Interpolation and Fitting

Interpolation: the fitted curve must pass through every given point. Commonly used when exact values are known and the goal is to fill in the gaps. **Fitting**: the

curve is not required to pass through every point; instead, total error is minimized. Commonly used for noisy data. Machine learning uses fitting, not interpolation.

Common Interpolation Methods

Linear interpolation: connect two points with a straight line. Simple, but not smooth (derivative is discontinuous at nodes).

Lagrange interpolation: n+1 points uniquely determine a polynomial of degree n.

> $$L(x) = \sum_{i=0}^n y_i \prod_{j \ne i} \frac{x - x_j}{x_i - x_j}$$

Drawback: high-degree polynomials oscillate violently near the endpoints (Runge's phenomenon) — adding more points can make the fit worse.

Cubic splines: piecewise cubic polynomials with continuous second derivatives across segments. Avoids the oscillation of high-degree polynomials; the preferred choice in engineering.

Least Squares Fitting

Given n data points (x_i, y_i), fit a line $y = a + bx$ by minimizing $\sum (y_i - a - bx_i)^2$. Closed-form solution:

> $$b = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}, \quad a = \bar{y} - b\bar{x}$$

Generalized to multiple variables: $\mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ — back to the pseudoinverse.

In AI

A neural network is an "ultra-high-degree, constrained, nonlinear function fitter." After understanding least squares and Runge's phenomenon, you will better appreciate why **regularization** is needed — in regions with few data points there are no constraints, so the model can exhibit Runge-like violent oscillations (overfitting); L2 regularization, Dropout, and data augmentation all suppress this type of oscillation.

1.7.4 Optimization Algorithms for Machine Learning

> *Orientation: SGD, Momentum, Adam — the second key section of this chapter, on par with 1.1.4.*

The Problem with Batch Gradient Descent

Standard gradient descent computes the gradient over the full dataset at each step — precise but slow. **Stochastic gradient descent (SGD)**: estimates the gradient using a single sample per step — fast but highly noisy in direction. **Mini-batch SGD**: a compromise, using a small batch (32/64/256/...) per step. This is the standard approach in modern deep learning.

SGD's gradient is an unbiased estimate of the true gradient, with variance inversely proportional to batch size. Why is noise actually beneficial? Empirically, noise helps models *escape poor local minima and converge to wider valleys*. "Wider valley → better generalization" has substantial empirical support (though theory has not fully resolved it).

Momentum

SGD oscillates back and forth in narrow ravines (large slope in the transverse direction, small slope in the longitudinal direction). Momentum adds "inertia":

> \mathbf{v}_{t+1} = \beta \mathbf{v}_t + \nabla L(\boldsymbol{\theta}_t), \quad \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \mathbf{v}_{t+1}

\beta \approx 0.9. Effect: oscillations in transverse directions cancel out while progress in the correct direction accumulates. Convergence is faster and more stable.

Adam

The most popular optimizer. Core idea: *each parameter uses an independent learning rate, adaptively adjusted based on the first and second moments of its gradient history*.

> \mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1) \mathbf{g}_t (first moment, ~Momentum)

> \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2) \mathbf{g}_t^2 (second moment, exponential moving average of squared gradients)

> \hat{\mathbf{m}}_t = \mathbf{m}_t / (1 - \beta_1^t), \quad \hat{\mathbf{v}}_t = \mathbf{v}_t / (1 - \beta_2^t) (bias correction)

> \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}

\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8} are the default values. **Bias correction** is a detail but an important one: at the start, \mathbf{m}_0 = \mathbf{v}_0 = 0 causes early estimates to be biased low; dividing by (1 - \beta^t) corrects for this.

Intuition: parameters with large gradients automatically get a small learning rate (because \sqrt{\mathbf{v}} is large), while parameters with small gradients get a large learning rate — automatic per-dimension adjustment. This is what "adaptive learning rate" means.

```
import numpy as np

def adam_step(params, grads, state, lr=1e-3, b1=0.9, b2=0.999,
eps=1e-8):
    t = state['t'] + 1
    m = state.get('m', [np.zeros_like(p) for p in params])
    v = state.get('v', [np.zeros_like(p) for p in params])
    new_params = []
    for i, (p, g) in enumerate(zip(params, grads)):
        m[i] = b1 * m[i] + (1 - b1) * g
        v[i] = b2 * v[i] + (1 - b2) * g * g
        m_hat = m[i] / (1 - b1 ** t)
```

```
        v_hat = v[i] / (1 - b2 ** t)
        new_params.append(p - lr * m_hat / (np.sqrt(v_hat) +
eps))
    return new_params, {'t': t, 'm': m, 'v': v}
```

30 lines is all of Adam. PyTorch's `torch.optim.Adam` is mathematically equivalent to this code, with added engineering details such as fused kernels and sparse gradient support.

AdamW

Adam + decoupled weight decay. In the original Adam, L2 regularization and adaptive learning rates are coupled, reducing effectiveness. AdamW pulls weight decay out of the gradient computation and applies it directly to the parameters:

> \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \left(\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} + \lambda \boldsymbol{\theta}_t\right)

Modern LLM training (GPT, LLaMA, Qwen) almost universally uses AdamW, not Adam.

Convergence Speed

Theoretically, for convex functions: GD converges at O(1/t), accelerated gradient methods (Nesterov) at O(1/t^2), and Newton's method at second-order rate O(\rho^{2^t}). But for non-convex problems with SGD, most of these results do not apply; in practice, empirical patterns matter more. Learning rate scheduling (warmup + cosine decay) and batch size tuning have a greater impact on actual convergence speed.

In AI

Training LLMs essentially comes down to AdamW + warmup + cosine decay + gradient clipping. This combination works across a wide range of scales. To save memory: 8-bit Adam (quantized optimizer states); for larger batches: LAMB

(AdamW + layer-wise learning rate scaling). These are all variants of Adam — everything traces back to the same core idea.

1.7.5 Numerical Integration, Differentiation, and Root Finding

> *Orientation: Numerical integration (Simpson's rule), numerical solutions to differential equations, root finding — common engineering tools.*

Numerical Integration

Trapezoidal rule: piecewise linear approximation to the integrand. $\int_a^b f(x)\, dx \approx \frac{h}{2}[f(x_0) + 2f(x_1) + \ldots + 2f(x_{n-1}) + f(x_n)]$.

Simpson's rule: piecewise parabolic approximation, three points per segment. Much higher accuracy; commonly used in engineering.

These rules cannot be used for high-dimensional integration (the number of grid points grows exponentially with dimension); **Monte Carlo integration** is required instead: $\int f(x)\, p(x)\, dx \approx \frac{1}{N} \sum_i f(x_i)$, $x_i \sim p$. Variance decreases at rate $1/\sqrt{N}$, independent of dimension. This is the universal tool for importance sampling, expected-value estimation in RL, and Bayesian inference.

Ordinary Differential Equations

Numerical solution of $y' = f(t, y)$, $y(0) = y_0$.

Explicit Euler: $y_{n+1} = y_n + h\, f(t_n, y_n)$. Simple but unstable — large step sizes blow up.

Implicit Euler: $y_{n+1} = y_n + h\, f(t_{n+1}, y_{n+1})$. Requires solving an equation, but much better stability. *Stiff equations* (coexisting time constants of very different scales) require implicit methods.

Runge-Kutta 4 (RK4): uses a weighted average of multiple intermediate points within each step to achieve fourth-order accuracy; the general-purpose engineering

solution.

Root Finding

Bisection: find $f(x) = 0$ on $[a, b]$, requiring $f(a) f(b) < 0$. Halves the interval at each step. Slow to converge but unconditionally stable.

Newton's method: $x_{n+1} = x_n - f(x_n)/f'(x_n)$. Second-order convergence (extremely fast), but requires a computable derivative, a good initial guess, and no poles. Failure rate is not low.

In AI

Diffusion models: the sampling process solves an SDE (or an equivalent ODE), using DDPM (naive Euler), DDIM (improved step size), or DPM-Solver (high-order methods). The difference between "sampling needs 50 steps" and "sampling needs 5 steps" is the accuracy gap between ODE solvers.

Neural ODEs: replace the discrete layers of a neural network with a continuous time parameter, where the forward pass solves an ODE $\frac{d\mathbf{h}}{dt} = f(\mathbf{h}, t, \theta)$ and the backward pass uses the adjoint method. Niche but conceptually elegant.

Newton's method in LLM optimization: second-order optimizers (K-FAC, Shampoo) use Newton's method ideas to approximate the Hessian, converging faster than Adam in certain scenarios. Has not become mainstream, but offers advantages in large-batch training (batch size in the thousands).

Chapter 2: Systems Software

From CPU to DPDK

What does systems software mean for AI engineers? Three things. First, **training and inference both run on top of systems** — no matter how fast the GPU, data traveling from disk to device memory must pass through PCIe, the kernel, and the file-system cache; every layer can become a bottleneck. Second, **AI infrastructure is itself systems software** — CUDA/CANN are driver-layer stacks, the vLLM scheduler borrows from OS process scheduling, and PagedAttention maps directly onto virtual-memory paging. Third, **in the era of AI-assisted programming, judgment about low-level systems is worth more than raw implementation skill** — AI can write the code, but architectural decisions, performance analysis, and fault diagnosis still require a human.

This chapter unfolds 28 sections in stack order: chips → OS → languages/compilers → data storage → communication. As in Chapter 1, coverage is intentionally asymmetric: memory management, concurrency and synchronization, Rust ownership, compiler optimizations, and DPDK/VPP receive deep treatment; device drivers, traditional relational databases, and certain transport-protocol details are covered more briefly. Proprietary internal technology stacks (specific internal middleware, private protocols, etc.) are replaced throughout with open-source general-purpose equivalents — master DPDK, understanding any proprietary equivalent is just one more layer of abstraction.

PART ONE

2.1 CPU & Core Architecture

AI training depends on GPUs and NPUs, but the CPU is the prerequisite for understanding everything else.

2.1.1 CPU Overall Architecture

> *Scope: ISA, pipeline, cache, multi-core — the skeleton of modern processors.*

ISA: The Contract CPU Exposes to Software

The Instruction Set Architecture (ISA) is the programming model a CPU exposes to software. It defines which instructions exist, how many registers are available, and how memory is addressed. The three mainstream ISAs:

ISA	Style	Examples	Use Cases
x86 / x86-64	CISC	Intel / AMD	PCs, servers
ARM	RISC	Apple M-series, Ampere	Mobile, cloud, embedded
RISC-V	RISC (open)	SiFive, Alibaba T-Head	Emerging: IoT to HPC

The CISC (Complex Instruction Set) vs. RISC (Reduced Instruction Set) divide is no longer as sharp as it once was — modern x86 CPUs decompose complex instructions into micro-operations (μops) at the front end, making the back-end execution units essentially identical to RISC. From a software perspective, though, the ISA still determines which assembly instructions you can use, whether vector extensions are AVX-512 or SVE, and whether a TSO memory model is in play.

RISC-V occupies a special position because it is **open** — anyone can implement it freely and add their own extensions. It is reshaping the landscape of embedded and accelerator chips at a remarkable pace.

Microarchitecture: The Reality Beneath the ISA

The same ISA can yield several times the performance difference depending on the microarchitecture. Modern high-performance CPU cores contain these key components:

Pipeline: Overlaps the five stages — fetch, decode, execute, memory access, writeback — so that one instruction completes per cycle. Branches disrupt the pipeline, which is why there is a **branch predictor** that uses historical statistics to guess the next instruction. Accuracy routinely exceeds 95%. A misprediction flushes the pipeline at a cost of 15–20 cycles.

Out-of-order execution: Hardware detects when instructions have no dependencies and issues them in parallel. Code written sequentially by the programmer is reordered by hardware according to the dataflow. The Spectre/Meltdown vulnerabilities exploited side effects of this out-of-order execution.

SIMD: Single Instruction, Multiple Data. AVX-512 can compute 8 doubles with a single instruction. Before GPUs dominated, CPU inference relied on SIMD as its primary workhorse for deep learning.

Cache Hierarchy

Memory is two orders of magnitude slower than the CPU (100+ ns vs. <1 ns). The solution is a three-level cache:

Level	Size	Latency	Scope
L1 (data + instr)	32KB + 32KB	~4 cycles	per-core
L2	256KB–1MB	~12 cycles	per-core
L3 (LLC)	several MB – tens of MB	~40 cycles	shared

DRAM	GB	~200 cycles	global

Understanding the cache is the key to understanding performance. **Sequential array access** lets the prefetcher predict the next cache line accurately, at near-zero cost; **random linked-list traversal** produces a cache miss on almost every access, slowing things down by tens of times. This is why tensors in AI almost universally use contiguous memory layouts.

```
// The dramatic effect of cache on performance
#define N 16384
int a[N][N];

// Version 1: row-major access, cache-friendly
for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++) sum += a[i][j];

// Version 2: column-major access, strides N*sizeof(int) bytes
each time
for (int j = 0; j < N; j++)
    for (int i = 0; i < N; i++) sum += a[i][j];

// Version 2 is typically 5–10x slower than Version 1 — purely
because of cache misses
```

Multi-Core and Cache Coherence

When multiple cores share memory, the system must guarantee that "what I wrote, you can see." Hardware maintains cache-line state through the **MESI protocol** (Modified/Exclusive/Shared/Invalid). If one core modifies a cache line, copies held by other cores are marked Invalid; the next access on those cores must reload the line.

This introduces a subtle trap — **False Sharing**. Two unrelated variables happen to reside on the same cache line (64 bytes). Two cores each write their respective variables frequently, and each write triggers a cache invalidation on the other core. The fix is **cache-line alignment** — annotate critical variables with `alignas(64)`, or pad them to 64 bytes.

In AI

AI training's data preprocessing, tokenization, and sampling logic all run on the CPU. One frequently overlooked detail: **DataLoader CPU performance is often the training bottleneck**. The GPU computes at blazing speed while the CPU cannot feed it data fast enough. The remedies are multi-process DataLoaders, binding workers to NUMA nodes, and using pinned memory to reduce copies — all of which rest on a basic understanding of CPU architecture.

2.1.2 Network Processing Units

> *Scope: dedicated hardware for accelerating network packet processing — from NP chips to smart NICs.*

Why Dedicated Units Are Needed

Line-rate processing of 100 Gbps traffic, assuming 64-byte small packets, means **148.8 Mpps** (million packets per second). The per-packet budget is **6.7 ns** — not even enough time for a single DRAM access. General-purpose CPUs cannot keep up with pure software approaches; hardware is mandatory.

Hardware Acceleration Approaches

NP (Network Processor): A multi-core processor designed specifically for packet processing, with an instruction set that includes header parsing, table lookups, and checksum operations. Each core runs a segment of microcode, processing packets in a pipeline fashion. Examples: Broadcom XLP, Intel IXP.

ASIC (switching chip): Fully hardened for fixed protocols. The Broadcom Tomahawk and Marvell Teralynx chips inside Ethernet switches are ASICs — table lookups, forwarding, and VLAN processing are all done in hardware. Latency can reach the hundreds-of-nanoseconds range, but functionality is fixed at fabrication time.

Smart NIC / DPU: NVIDIA BlueField, Intel IPU, AWS Nitro. Essentially a NIC with an ARM SoC on board that can run a full OS, offloading networking, storage, and security functions from the host CPU to the NIC. This is the dominant trend in data-center-scale acceleration.

P4 programmable pipeline: Positioned between ASICs and NPs. The hardware is a fixed pipeline, but the behavior of each pipeline stage is defined in P4 and loaded at power-on. Example: Barefoot Tofino.

Common Hardware Acceleration Features

RSS (Receive Side Scaling): The NIC hashes packets on the 5-tuple (source IP, destination IP, protocol, source port, destination port) and distributes them to receive queues on different CPU cores, enabling parallel processing without conflicts.
Hardware LRO/GRO: Combines multiple small packets from the same flow into a larger packet before handing them to the protocol stack, reducing the number of syscalls.
TSO/GSO: The reverse — the protocol stack hands a large segment to the NIC, which splits it into MTU-sized frames on its own.
Checksum/Crypto offload: TCP checksums and TLS encryption/decryption are performed entirely by the NIC.
ECMP hardware implementation: Equal-Cost Multi-Path routing; the hardware distributes traffic by hash.

In AI

Large-model training clusters (where GPUs communicate over RDMA via InfiniBand/RoCE) depend on NIC hardware capabilities. GPUDirect RDMA lets the NIC read and write GPU memory directly, bypassing CPU memory. The latency of a single AllReduce operation is dominated by the NIC's RDMA performance, not GPU compute throughput.

2.1.3 GPU and NPU Acceleration

> *Scope: the carrier of AI compute — hardware for parallel computation and tensor acceleration. Brief here; details in Chapter 4.*

This section overlaps with Chapter 4 / Section 5.2. Here we only establish the basic reference frame.

The GPU Parallel Model

The GPU's DNA is graphics rendering — shading a massive number of independent pixels is inherently parallel. Modern GPUs repurpose this capability for general-purpose computation (GPGPU). An NVIDIA H100 has approximately **16,896 CUDA cores + 528 Tensor Cores**, with FP16 Tensor Core throughput of roughly 989 TFLOPS (dense) / 1,979 TFLOPS (2:4 sparse).

The GPU programming model is called SIMT (Single Instruction, Multiple Threads) — 32 threads form a warp and a single instruction acts on all 32 data elements simultaneously. This is conceptually the same as SIMD; the difference is that if threads within a warp diverge at a branch (if statement), the GPU serializes execution of both paths — known as warp divergence — causing a performance collapse.

NPU: Tensor-First Design

An NPU (Neural Processing Unit) is designed from the ground up for tensor computation. The core architecture is the **systolic array**: a two-dimensional grid of MAC (multiply-accumulate) units through which data flows like a pulse — each unit performs a multiply-accumulate on a fixed schedule. Google's TPU is the most well-known systolic array design.

Huawei's Ascend uses the **DaVinci architecture**: a Cube unit for matrix multiplication, a Vector unit for element-wise operations, and a Scalar unit for control flow. Operator developers must explicitly dispatch computation to the appropriate unit.

Memory Structure

GPU/NPU memory hierarchies are more extreme:

Level	Capacity	Bandwidth
Registers	64KB per thread	extremely fast
Shared memory / SRAM	~100KB per block	~several TB/s
HBM	80GB (H100)	~3 TB/s

HBM bandwidth is large, but still 10× slower than on-chip SRAM. The essence of operator optimizations like **FlashAttention** is to keep computation in SRAM as much as possible and minimize round-trips to HBM.

2.1.4 SoC Heterogeneous Computing

> *Scope: a single chip integrating CPU/GPU/NPU/accelerators — the paradigm for modern embedded and edge AI. Brief.*

SoC (System on Chip) is not a new concept. Mobile processors have been SoCs since the iPhone era — a single chip integrating CPU big and little cores, a GPU, an ISP (image signal processor), a neural network accelerator, and a modem. The data-center SoC trend is even more pronounced: AWS Graviton and Ampere Altra both integrate CPUs, memory controllers, PCIe, and network subsystems on a single die.

Challenges of Heterogeneous Computing

When multiple compute units share memory and buses on a single chip, the challenges are:

1. **Data movement**: Copying data between CPU memory and GPU/NPU memory is the dominant overhead. Unified memory (CUDA Unified Memory, Apple Silicon's Unified Memory Architecture) attempts to solve this, but truly zero-copy operation requires hardware coherence support.
2. **Scheduling**: Which code runs on the CPU and which runs on the accelerator? Frameworks like PyTorch and TensorFlow decide automatically, but edge

cases are numerous.

3. **Power wall**: All units share the chip's power budget. Running the NPU at full tilt forces the CPU to throttle.

Common Offload Engines

Network device SoCs commonly include, beyond the CPU and NP:

- **Compression engine** (hardware-accelerated DEFLATE/LZ4)
- **Encryption/decryption engine** (AES, SHA, Chinese national standard SM4)
- **Regex matching engine** (DFA execution, used for IPS intrusion detection)
- **Table-lookup engine** (TCAM, used for route lookups)

These are not directly relevant to AI scenarios, but understanding the concept of "offloadable computation" is highly instructive for understanding DPUs and smart NICs — they are all, at their core, extensions of the SoC philosophy into the networking domain.

PART TWO

2.2 Operating Systems

The OS is the invisible infrastructure beneath all higher-level software — AI framework scheduling, containers, and networking all pass through it.

2.2.1 OS Overview

> *Scope: what an OS is, what it does, and the differences between the major OS families.*

An operating system does two things: **manage hardware resources** (CPU, memory, disk, network) and **provide abstractions to applications** (processes, files,

sockets). Dijkstra's old saying: "Abstraction is not about hiding details — it is about creating a new, precise level of description." The OS is exactly such a level.

Major OS Families

Family	Kernel	Examples	Use Cases
Unix-like	Monolithic	Linux, FreeBSD	Servers, containers, supercomputers
macOS / iOS	XNU (hybrid)	Darwin	Desktop, mobile
Windows	NT (hybrid)	Windows 10/11	Desktop, enterprise servers
Android	Custom Linux	AOSP	Mobile
RTOS	Real-time microkernel	FreeRTOS, VxWorks, Zephyr	Embedded, industrial control

AI infrastructure runs on Linux virtually 100% of the time. The Linux kernel spans 30 million lines of source code, but the core subsystems you interact with day-to-day number only five or six: process scheduling, memory management, VFS, the network stack, block devices, and character devices.

Monolithic Kernel vs. Microkernel

Monolithic kernel (Linux): the file system, network stack, and drivers all live in kernel space and communicate via direct function calls. Fast, but a bug in any subsystem can crash the entire machine.

Microkernel (seL4, Mach): the kernel retains only the most essential IPC, scheduling, and memory-protection primitives; all other services run in user space and communicate through message passing. More resilient, but IPC overhead is significant.

In practice, most modern OSes are hybrids — Linux itself has grown loadable kernel modules (LKMs) and eBPF as dynamic mechanisms, nudging it incrementally toward the microkernel direction.

Real-Time OS

The primary metric for an RTOS is not throughput but **worst-case response latency**. Scheduling must be predictable — a high-priority task that becomes ready must receive CPU time within a bounded interval. Linux is not an RTOS by default, but the PREEMPT_RT patch brings it close to real-time capability. Industrial control, automotive electronics, and robotics all require an RTOS.

2.2.2 Processes and Scheduling

> *Scope: processes are the basic unit of CPU resource allocation; the scheduler decides who runs and who waits. Focus on CFS and modern scheduling challenges.*

Processes and Threads

In Linux, processes and threads look the same to the kernel — both are represented by a `task_struct`. The creation call determines whether it is a process (`fork`) or a thread (`clone` with flags like CLONE_VM). The essential distinction: processes have independent address spaces; threads share one.

A `task_struct` contains:

- PID, parent process, user/group information
- Run state (RUNNING, INTERRUPTIBLE, UNINTERRUPTIBLE, ZOMBIE, ...)
- Scheduling policy and priority
- Open file table, signal handler table
- Memory map (mm_struct)
- Context (registers, stack)

fork and Copy-on-Write

`fork()` creates a child process; the traditional semantics copy the entire address space. A naive implementation would be very slow — because the vast majority of child processes immediately `exec` a new program, making the copied memory useless from the start.

Copy-on-write (COW) is the elegant solution: at fork time nothing is copied; all pages are simply marked read-only with their reference count incremented. When either the parent or child writes to a page, a page fault fires and only then does the kernel copy that one page. Result: fork is nearly as fast as spawning a new thread.

Orphans and Zombies

If the parent dies before the child, the child becomes an **orphan process** and is adopted by init (PID 1).

If the child dies before the parent calls wait, the child becomes a **zombie process** — the kernel retains its PID and exit status until the parent reads them. Zombies consume no memory but do occupy a PID table entry. Orphan vs. zombie is a classic Unix interview question.

CFS: Linux's Default Scheduler

The core idea of CFS (Completely Fair Scheduler) is the "ideal multi-tasking CPU" — if the CPU were infinitely fast, N tasks should each receive exactly 1/N of the compute simultaneously. The real CPU is time-shared; CFS approximates this ideal.

Implementation: each task maintains a **virtual runtime** (vruntime); each time it is scheduled, vruntime += actual run time × weight. The scheduler always picks the task with the lowest vruntime. Tasks are organized in a red-black tree, so selecting the next task costs $O(\log n)$.

The nice value ([-20, 19]) influences the rate at which vruntime advances via weight: a lower nice value means higher weight, slower vruntime growth, and more frequent selection. This is a smooth implementation of "priority."

Note: Since Linux 6.6 (late 2023), the default fair scheduler has switched from CFS to **EEVDF** (Earliest Eligible Virtual Deadline First). EEVDF preserves virtual-time fairness while introducing virtual deadlines, improving tail latency and reducing scheduling jitter for interactive tasks. The underlying data structure is still a red-black tree, so $O(\log n)$ complexity is unchanged.

Real-Time Scheduling

Beyond CFS, Linux also has real-time scheduling classes (SCHED_FIFO, SCHED_RR). FIFO has no time slice — a high-priority task runs until it voluntarily yields or blocks, making it suitable for hard real-time scenarios. RR is FIFO with time-slice round-robin.

There is also **DEADLINE scheduling** (EDF algorithm): each task declares (runtime, deadline, period), and the kernel guarantees that runtime is completed within each period without exceeding the deadline. This is currently the closest Linux comes to hard real-time scheduling.

Priority Inversion

A classic failure mode: low-priority task L holds a lock; medium-priority task M preempts L (without releasing M's hold on L); high-priority task H waits for the lock — H is indirectly blocked by M, a priority inversion. The 1997 Mars Pathfinder mission suffered exactly this bug.

Fix: **priority inheritance** — while L holds the lock, it is temporarily elevated to H's priority so M cannot preempt it. **Priority ceiling** — statically assigns a higher ceiling. Both mechanisms are implemented in RT-mutex.

Cost of Context Switching

A single context switch costs roughly 1–5 μs and involves: saving registers, switching the page table (reloading CR3, partially invalidating the TLB), switching the kernel stack, and the scheduling decision. That sounds fast, but accumulated overhead is significant under a 1,000 Hz timer interrupt with frequent preemption.

In AI training, **CPU affinity** (via taskset or sched_setaffinity) is commonly used to pin DataLoader workers to fixed cores, reducing both context switches and cache misses.

2.2.3 Memory Management

> *Scope: virtual memory, page tables, allocators, HugePages, OOM — the section most worth digging into in the entire chapter.*

The Virtual Memory Contract

Every process sees an independent, contiguous address space that appears to belong entirely to it (128 TB of user space on x86-64). This is the illusion that virtual memory provides. Hardware (the MMU) and the OS (kernel page-table management) cooperate to maintain it.

4-level page tables (x86-64): a 48-bit virtual address is split into 9+9+9+9+12 bits. The first four 9-bit groups index four levels of page tables; the final 12 bits are the offset within the page. Why this design? A single-level page table covering the entire address space would require 2^{36} \times 8 B = 512 GB per process — clearly impractical. Multi-level tables allocate on demand — only virtual memory ranges that are actually used have page table entries.

TLB

Walking all four levels of page tables on every memory access means four extra memory accesses — far too slow. Hardware uses a **TLB** (Translation Lookaside Buffer) to cache recently used virtual-to-physical mappings. A TLB hit incurs ~0 overhead; a miss triggers a full page walk costing ~100 cycles.

A TLB holds only tens to hundreds of entries — random access across a large array causes the TLB miss rate to spike.

HugePages

Normal pages are 4 KB; 512 TLB entries cover 4 KB × 512 = 2 MB of data. **HugePages** are 2 MB (THP) or 1 GB (`hugetlbfs`), so a single TLB entry covers 2 MB or 1 GB. The benefit is pronounced for large-memory workloads such as databases and AI training.

DPDK, databases, and the JVM all strongly recommend HugePages — not to save memory, but to reduce TLB misses.

Physical Memory Allocation

The kernel's physical page allocator is the **Buddy System**: physical memory is divided into power-of-two-sized blocks (4 KB, 8 KB, 16 KB, … 4 MB), with a free list for each size. On allocation, the smallest block greater than or equal to the request is found; if it is too large, it is *split in half* (into buddies) — one half goes to the requester and the other enters the next-smaller free list. On release, the kernel checks whether the buddy is also free; if so, they are merged.

This algorithm balances avoiding external fragmentation with fast allocation speed. But 4 KB granularity is too coarse for small objects (tens of bytes), so a second layer is added:

Slab/Slub: a dedicated allocator for kernel objects. A set of pages is pre-allocated as a "slab" and divided into fixed-size objects (e.g., a `task_struct` at 2 KB), with a free list maintained at object granularity. Allocation and deallocation are O(1) and cache-friendly.

User-Space malloc

User-program malloc does not call the kernel directly. glibc's ptmalloc, jemalloc, and tcmalloc all obtain large blocks from the kernel via `brk()` or `mmap()` and then manage small-object allocation themselves. The core idea behind jemalloc (Redis, Rust default) and tcmalloc (Google) is **per-thread caches** — each thread maintains a local small-object cache so allocation is lock-free; the global pool is consulted only when the cache is exhausted. This is far faster than ptmalloc under multi-threaded workloads.

Page Reclamation

When physical memory runs low, the kernel must reclaim pages. Linux maintains two LRU lists: **active** and **inactive**. Accessing a page promotes it to the head of the active list; pages that go unaccessed for a long time migrate to the tail of the inactive list. Reclamation begins from the inactive tail.

The **kswapd** kernel thread reclaims pages in the background, maintaining a certain amount of free memory. The reclamation paths are:

1. Clean page-cache pages: discarded immediately (re-read from file on next access)
2. Dirty page-cache pages: written back to disk first, then freed
3. Anonymous memory (heap, stack): swapped out to the swap partition (if enabled)

Swap is commonly disabled on server machines — an OOM kill is preferable to letting a program silently degrade in performance. AI training machines must have swap disabled.

OOM Killer

When even swap cannot help, the kernel invokes the OOM Killer to select and terminate a process. The scoring algorithm weighs memory consumption, running time, and `oom_score_adj` (tunable). In practice: critical services should be given a negative `oom_score_adj` to reduce the chance of being killed.

In AI

Most AI training memory problems arise in GPU VRAM; CPU memory is relatively abundant. However, these scenarios trip teams up frequently:

- **fork with multiple DataLoader workers**: Due to COW, large objects in the parent process (such as dataset indices) are copied page by page as child processes "touch" them, eventually duplicating all of that memory. Python multi-process training OOMs are often caused by this.
- **page cache overwhelming low-memory machines**: Reading large datasets fills memory with page cache; on some kernel versions it is not released quickly enough.
- **zero-copy requirements**: Ideally, training data travels from disk to GPU without the four-copy path of "disk → kernel → user space → GPU." Combining `mmap` + `O_DIRECT` + GPU pinned memory can reduce the number of copies.

```
# PyTorch pinned memory example: enables DMA for host→device
copies
train_loader = DataLoader(
    dataset,
```

```
    batch_size=32,
    num_workers=8,
    pin_memory=True,   # allocate page-locked memory; GPU can
DMA directly
)
# Pair with non_blocking=True in the training loop for
asynchronous copies
for x, y in train_loader:
    x = x.cuda(non_blocking=True)
    y = y.cuda(non_blocking=True)
```

2.2.4 Storage and File Systems

> *Scope: VFS abstraction, inodes, page cache, I/O scheduling — the foundation for loading large datasets.*

VFS: Everything Is a File

One of Unix's core philosophies. Files, directories, pipes, sockets, devices, and /proc information — all are accessed via `open/read/write/close`. VFS (Virtual File System) is the unified interface layer provided by the kernel, beneath which different concrete file systems are mounted: ext4, xfs, btrfs, overlayfs (containers), nfs (network).

inode

An inode is the metadata object for a file, containing: size, owner, permissions, timestamps, and pointers to data blocks. **The filename is not in the inode** — filenames are stored in the directory (which is itself a file whose content is a name-to-inode mapping).

Hard link: two directory entries point to the same inode; the inode's reference count is incremented. Deleting a file actually removes a directory entry and decrements the reference count; the file is truly freed only when the count reaches zero.

Symbolic link (symlink): a special file type whose content is the target path. Resolved recursively.

Page Cache

When a file is read, the kernel caches the data in the page cache (in physical memory). A second read of the same data is served directly from cache with no disk I/O. Writes also go to the page cache first by default (write-back), with the kernel flushing to disk in the background.

The "buff/cache" shown by the `free` command is the page cache. It is not "wasted" memory — it is memory put to efficient use and can be reclaimed at any time.

I/O Scheduling

In the era of spinning disks, I/O schedulers (CFQ, deadline) focused on merging adjacent requests and minimizing seek time. With SSDs, seek cost has disappeared; Linux now defaults to `mq-deadline` or `none` (NVMe multi-queue with direct dispatch).

Synchronous vs. Asynchronous I/O

Standard `read/write` is synchronous and blocking. High-performance scenarios use:

- **O_DIRECT**: bypasses the page cache and DMA's directly into the user buffer. Commonly used by databases.
- **aio** (Linux AIO): traditional asynchronous I/O, only truly asynchronous under O_DIRECT; limited capabilities.
- **io_uring** (2019+): modern asynchronous I/O using shared ring-buffer submission/completion queues, capable of syscall-less I/O in polling mode. AI training data loaders, databases, and high-performance web servers are all migrating to it.

```
// io_uring core ring-buffer structure (simplified)
struct io_uring_sqe {  // Submission Queue Entry
    __u8  opcode;      // IORING_OP_READ, _WRITE, ...
    __s32 fd;
    __u64 off;
    __u64 addr;
    __u32 len;
```

```
    // ...
};
struct io_uring_cqe {  // Completion Queue Entry
    __u64 user_data;
    __s32 res;
    __u32 flags;
};
// User space fills SQEs and reads CQEs directly — no syscall
needed (in polling mode)
```

In AI

Training datasets can easily reach terabytes, making load performance critical. A few practical points:

- **Packed formats like WebDataset and TFRecord**: pack tens of millions of small files into a small number of large files, avoiding inode/directory-traversal overhead.
- **Prefetching + multiple workers**: DataLoader's `prefetch_factor` controls how many batches each worker prefetches ahead.
- **Beyond POSIX semantics**: in distributed training, backends such as NFS, Lustre, and S3 have performance profiles that differ drastically from local disks — you must benchmark to know where the bottleneck is.

2.2.5 Exceptions and Interrupts

> *Scope: the mechanism by which the CPU communicates with the outside world — hardware interrupts, soft interrupts, tasklets, and workqueues.*

Interrupt Classification

Hardware interrupts (IRQ): triggered by hardware; the CPU immediately suspends its current work and jumps to the interrupt handler. An interrupt vector table maps each interrupt number to a handler function.

Software interrupts / exceptions: triggered by the CPU itself — divide by zero, page fault, system call (a syscall is essentially a trap).

Two-Phase Interrupt Handling

There is an iron rule for interrupt handling: it **must be fast** — interrupt context cannot sleep or acquire potentially blocking locks. But events like a network packet arriving sometimes require complex processing (parsing protocols, routing, copying data).

Linux's solution is "top half + bottom half":

Top half: the interrupt handler itself. Does only the bare minimum — acknowledges the hardware, moves data into a memory buffer, schedules the bottom half. Runs with interrupts disabled; must be as short as possible.
Bottom half: interrupts re-enabled, schedulable. Three mechanisms:

Mechanism	Characteristics	Use Cases
softirq	Static types, per-CPU, cannot sleep	Network RX/TX, timers, block I/O
tasklet	Built on softirq, dynamically created; a given tasklet runs on only one core at a time	Medium-overhead drivers
workqueue	Kernel thread context, can sleep	Deferred work that needs to sleep

The network packet receive path is a classic softirq user — after a hardware interrupt enqueues the skb, NET_RX_SOFTIRQ hands it to the protocol stack in the bottom half. This is why `ksoftirqd` frequently pins an entire core under high network load.

Interrupt Affinity and Load Balancing

By default, all interrupts may be handled by a single core (especially with cheap NICs). `/proc/irq/<n>/smp_affinity` can bind an interrupt to specific cores. Multi-queue NICs, combined with RSS, distribute different queues' interrupts across different cores for true parallelism.

In AI

AI relevance here is limited. However, GPU drivers are interrupt-intensive — when a CUDA kernel completes, the GPU fires an interrupt, and the driver notifies user space in the bottom half. Interrupt handling overhead is non-negligible in high-QPS inference services (thousands of requests per second).

2.2.6 Concurrency, Mutual Exclusion, and Synchronization

> *Scope: the basic tools for multi-core concurrency — atomics, locks, RCU, memory model. Worth digging into deeply.*

The Nature of Races

When multiple threads read and write shared data without synchronization, things go wrong. The simplest counterexample:

```
int counter = 0;
// Two threads, each running:
for (int i = 0; i < 1000000; i++) counter++;
// Final counter is much less than 2000000
```

`counter++` is not atomic — it is actually three steps: load, add, store. When two threads interleave these steps, updates are lost.

Atomic Operations

Hardware provides atomic instructions: the `LOCK` prefix on x86, LDREX/STREX on ARM. C11's `_Atomic`, C++11's `std::atomic`, and Rust's `AtomicUsize` are all language-level wrappers around these instructions.

The most important atomic primitive is **CAS** (Compare-And-Swap):

```
// Atomically: if *ptr == expected, write new and return true;
// otherwise, update expected with *ptr and return false
bool cas(int *ptr, int *expected, int new);
```

CAS is the foundation of all lock-free data structures. The typical pattern is a "loop until CAS succeeds":

```
int old, new;
do {
    old = *counter;
    new = old + 1;
} while (!cas(counter, &old, new));
```

Types of Locks

Spinlock: busy-waits on failure. Suitable for extremely short critical sections on multi-core systems. Must be used in kernel interrupt handlers (cannot sleep).

Mutex: sleeps on failure and is woken when the lock becomes available. Suitable for longer critical sections. Higher overhead than a spinlock (context switch), but does not waste CPU cycles.

Read-write lock: multiple readers share access; writers get exclusive access. Advantageous in read-heavy, write-rare scenarios. Downside: writer starvation (a continuous stream of readers can keep a writer waiting indefinitely).

Semaphore: a lock with a count, allowing N concurrent holders. Commonly used for resource pools.

Condition variable: not a lock, but a primitive for "waiting until a condition becomes true." Typically used together with a mutex.

RCU

Read-Copy-Update is the most elegant concurrency primitive in the Linux kernel. It is suited for scenarios with overwhelmingly more reads than writes. The core idea:

1. Readers **take no lock at all** — they read the old-version pointer directly
2. The writer copies the data, modifies the copy, then **atomically replaces the pointer**
3. The old version is freed with a delay — only after all readers that had already started finish (the grace period)

The RCU read path has near-zero overhead, almost identical to single-threaded access. The cost is write complexity, a temporary doubling of memory, and delayed reclamation. Linux's routing table, namespace structures, and module list all use RCU.

```
// RCU reader: completely lock-free
rcu_read_lock();
struct foo *p = rcu_dereference(global_ptr);
// use p...
rcu_read_unlock();

// RCU writer
struct foo *new = kmalloc(...);
*new = (struct foo){ .x = 42, ... };
struct foo *old = rcu_dereference(global_ptr);
rcu_assign_pointer(global_ptr, new);   // atomic pointer
replacement
synchronize_rcu();                      // wait for all readers
to finish
kfree(old);                             // safe to free
```

Memory Model

Programmers assume their code executes in order, but both CPUs and compilers reorder instructions. In a single thread this is imperceptible (data dependencies are preserved), but in a multi-threaded context reordering can make another core observe an "impossible" state.

Example:

```
// Thread A:                Thread B:
x = 1;                       if (flag) {
flag = 1;                        assert(x == 1);  // may fail!
                             }
```

The CPU or compiler may move `flag = 1` ahead of `x = 1` — Thread B sees flag=1 but x is still 0.

Fix: **memory barriers**. x86 uses the TSO (Total Store Order) memory model, which is relatively strong — stores are ordered, but loads can be reordered. ARM is

weakly ordered — almost anything can be reordered, and explicit barriers are mandatory.

Higher-level language abstractions: C++11's `std::memory_order` and Rust's `Ordering` offer five levels: Relaxed, Acquire, Release, AcqRel, and SeqCst. SeqCst is the strongest (sequentially consistent) and most expensive; Relaxed is the weakest (only atomicity is guaranteed) and fastest. Using these correctly is the hallmark of a high-level systems engineer.

MESI and Cache Coherence

Multi-core cache coherence is guaranteed by the hardware MESI protocol. Cache lines migrate between cores through Modified/Exclusive/Shared/Invalid state transitions.

A frequently overlooked observation: **the cost of a lock is not "the lock primitive itself" — it is cache-line migration**. After core A holds a lock and core B then contends for it, the lock variable's cache line migrates from A to B, costing ~100 cycles per migration. This explains why performance collapses under heavy lock contention.

Lock-Free Data Structures

Lock-free queues, stacks, and hash tables exist but are hard to write correctly — subtle mistakes lead to ABA problems and memory reclamation hazards. Industrial-grade implementations (Folly, Crossbeam) have been tuned over many years, but writing your own lock-free structures is not worth the risk.

In AI

PyTorch's internals make extensive use of lock-free atomics (Tensor reference counting, parameter server updates). The `c10::intrusive_ptr` in the C10 library is faster than `std::shared_ptr` because it embeds the reference count, avoiding an extra allocation.

Parameter servers and asynchronous SGD in training clusters are concurrency problems through and through. NCCL synchronization for single-machine multi-

GPU training also implicitly involves memory barriers — ensuring that AllReduce results are visible to all ranks.

2.2.7 Device Management and System Calls

> *Scope: syscalls are the only interface between user space and the kernel — io_uring and eBPF are reshaping it.*

Device Driver Model

Linux divides devices into three classes:

Character devices: streaming reads and writes (serial ports, keyboard, `/dev/random`)
Block devices: random block access (HDDs, SSDs)
Network devices: do not use `read/write`; use the socket API

A driver registers an operations table (`file_operations`) with the kernel — function pointers for open, read, write, ioctl, mmap, and others. Applications reach the device through the file system (`/dev/xxx`); calls are dispatched to the driver.

DMA and Zero-Copy

DMA (Direct Memory Access) lets a device read and write memory without involving the CPU. The traditional network receive path: NIC DMA's into a kernel buffer → kernel copies into the socket buffer → user-space `read` copies again into the user buffer. Three copies total.

Zero-copy techniques: `sendfile` (file to socket entirely within the kernel), `splice` (zero-copy through a pipe), `mmap` + `write` (maps the file into user space). Nginx and Kafka use sendfile extensively.

System Calls

A syscall is the only way user space enters the kernel. x86-64 uses the `syscall` instruction (the old `int 0x80` is obsolete), with arguments passed in specific

registers. One syscall costs ~100 ns — not slow, but the cumulative cost becomes significant across tens of millions of iterations in a loop.

vDSO (virtual Dynamic Shared Object): implements a few high-frequency, lightweight syscalls (such as `gettimeofday`) as user-space functions, avoiding a kernel entry. Provided transparently through dynamic linking.

eBPF

eBPF (extended Berkeley Packet Filter) is the most important new mechanism added to Linux in recent years. It allows users to inject verified small programs into kernel hot paths (networking, tracing, security) without modifying the kernel or loading a module.

Applications: **tcpdump** uses BPF for packet filtering; **bpftrace** for dynamic tracing; **Cilium** for container networking (replacing iptables); **Katran** for Layer-4 load balancing (Facebook). AI observability platforms (fine-grained monitoring of training jobs) are increasingly adopting eBPF.

eBPF programs pass through a verifier that guarantees: no loops, no out-of-bounds accesses, bounded execution time. This makes them safe to run in kernel space.

Containers

Containers are not virtual machines — they are ordinary Linux processes that happen to use namespaces (for view isolation) and cgroups (for resource limits).

namespaces: eight kinds — PID, net, mount, IPC, UTS, user, cgroup (4.6+), and time (5.6+) — each isolating one dimension of resource visibility. A container seeing itself as PID 1 is the effect of the PID namespace.
cgroups: enforce quotas on CPU, memory, block I/O, and network bandwidth. Kubernetes resource limits are translated directly into cgroup settings.

Docker, containerd, and Podman are all higher-level wrappers around these two primitives. AI training almost always runs inside containers (for reproducibility and scheduling); understanding namespaces and cgroups is indispensable when troubleshooting performance or permission issues.

PART THREE

2.3 Programming Languages & Compilers

Language determines what you can think; the compiler determines how fast you can run.

2.3.1 The Rust Language

> *Position: ownership model + no GC + zero-cost abstractions — the new baseline for systems programming.*

Ownership: The Core of Rust

Rust's most distinctive feature is its **ownership** system. Three rules:

1. Every value has exactly one owner.
2. When the owner goes out of scope, the value is automatically dropped (resources freed).
3. Assignment or passing a value as an argument moves ownership by default (unless the type implements `Copy`).

These rules eliminate three classic C/C++ bugs at the root: use-after-free, double-free, and memory leaks. The compiler performs static checks at compile time with no runtime overhead.

```
fn main() {
    let s1 = String::from("hello");
    let s2 = s1;           // ownership moves; s1 is no longer
valid
    // println!("{}", s1); // compile error!
```

```
    println!("{}", s2);    // OK
} // s2 goes out of scope; the string is automatically freed
```

Borrowing: Temporary Access Without Moving

Moving ownership everywhere would be cumbersome — borrowing lets you "use it temporarily without owning it":

```
fn len(s: &String) -> usize { s.len() }

fn main() {
    let s = String::from("hello");
    let n = len(&s);       // pass an immutable borrow
    println!("{}, len {}", s, n);  // s is still valid
}
```

There are two kinds of borrows: immutable `&T` (multiple can coexist) and mutable `&mut T` (exclusive). The core rule: **a mutable borrow is mutually exclusive with any other borrow**. This is enforced at compile time, directly eliminating data races — if concurrent code compiles, it is (at the ownership + borrow level) race-free.

Lifetimes

A borrow must not outlive the value it borrows from. Most of the time the compiler infers this automatically; complex cases require explicit annotations:

```
// 'a is a lifetime parameter: the returned borrow cannot
outlive
// the shorter of the two inputs
fn longer<'a>(s1: &'a str, s2: &'a str) -> &'a str {
    if s1.len() > s2.len() { s1 } else { s2 }
}
```

Lifetimes are the first mental hurdle for Rust beginners. Once you clear it, you realize how much "is this pointer still valid?" in C/C++ relies on intuition and convention — Rust turns it into the compiler's job.

Zero-Cost Abstractions

Rust's traits, iterators, and generics are **zero-cost abstractions**: after compilation they run as fast as hand-written C. An iterator chain like `v.iter().filter(...).map(...).sum()`, after LLVM optimization, produces assembly essentially identical to a hand-written for loop — no virtual dispatch overhead, no boxing. This is fundamentally different from the abstractions in Java or Python.

No GC, But No Manual malloc/free Either

Rust has no garbage collector — the ownership model makes memory management a compile-time decision. The compiler knows every drop point; resource cleanup is implemented in the `Drop` trait (similar to C++ RAII). `Box<T>`, `Rc<T>`, and `Arc<T>` handle exclusive heap allocation, single-threaded reference counting, and multi-threaded reference counting, respectively.

Async

Rust's `async/await` produces **zero-cost Futures** — each async function compiles into a state machine. The runtime (tokio, async-std) is responsible for scheduling and advancing state machines. This lets Rust async squeeze out performance close to C, though the ecosystem is still maturing.

In AI

PyTorch is Python + C++, not Rust. But Rust is heavily used in AI infrastructure surrounding it: **tokenizers** (HuggingFace), **burn** (a pure-Rust deep learning framework), **candle** (a Rust inference framework), **certain components of vLLM**, and various vector databases (Qdrant, LanceDB).

Rust-Python interop is bridged via **PyO3**, now the preferred way to write Python extensions — far safer than writing C extensions.

2.3.2 The Modern Evolution of C

> *Position: C is still the general-purpose low-level language — let's look at how it's evolving, and at C++ as an alternative path for systems programming. Brief.*

This section covers the broader "C-family evolution." C is not dead — kernels, drivers, DPDK, and Redis are still written in C. But the C toolchain and its ecosystem keep improving:

Sanitizer suite: AddressSanitizer (ASan) detects out-of-bounds accesses and use-after-free; ThreadSanitizer (TSan) detects data races; UBSan detects undefined behavior. Enable with `-fsanitize=address` at compile time; runtime overhead is 2–5×, but it catches the vast majority of memory bugs.
Static analysis: Clang Static Analyzer, Coverity, PVS-Studio. Catches code that compiles cleanly but fails at runtime.
Fuzz testing: libFuzzer, AFL++. Automatically generates inputs to probe for crashes. OpenSSL and SQLite have both found large numbers of bugs through fuzzing.

C++ in Systems Programming

C++ is far more expressive than C, but its complexity has exploded. Modern C++ (C++17/20/23) makes writing contemporary code much more pleasant with `std::unique_ptr`, `std::optional`, ranges, and more. The main problem is that **dangerous features cannot be disabled** — the safe subset of C++ exists, but must be enforced by discipline.

Performance-wise, C++ and Rust are close (both have zero-cost abstractions, neither has a GC). The fundamental difference is that Rust performs ownership checks at compile time, while C++ relies on the programmer. Databases, browsers, and game engines still make heavy use of C++ (LLVM, Chromium, Unreal).

Language Selection Considerations

If you were starting a network device firmware, operating system, or database from scratch today, the general guidance on language choice is:

- **Kernel/drivers**: C (mainstream on Linux) or Rust (Rust for Linux has been merged into the mainline; multiple subsystem drivers have been rewritten in Rust;

the ecosystem should mature after 2025).
- **Middleware/services**: Rust first, Go second (GC is acceptable), C++ when the team has experience.
- **Upper-layer applications**: Go, Python, Java.

2.3.3 Compiler Fundamentals

> *Position: lexical/syntactic/semantic analysis → IR → code generation. LLVM as the main thread.*

The Classic Three-Stage Compiler

Front end: source code → AST → intermediate representation (IR)

- **Lexical analysis** (lexing): character stream → token stream. Tools: flex, or hand-written.
- **Syntactic analysis** (parsing): token stream → AST. Tools: bison, ANTLR, or hand-written recursive descent.
- **Semantic analysis**: type checking, scope resolution, generating a type-annotated AST.
- **IR generation**: AST → IR (e.g., LLVM IR).

Middle end: optimizations on the IR (next section).

Back end: IR → target assembly. Includes instruction selection, register allocation, and instruction scheduling.

LLVM and SSA

LLVM's dominance rests on two things: its modular design (front end and back end decoupled) and its **SSA** (Static Single Assignment) IR.

SSA requires that every variable be assigned exactly once. Traditional code:

```
x = 1;
x = x + 2;
```

```
y = x * 3;
```

In SSA form:

```
x1 = 1
x2 = x1 + 2
y1 = x2 * 3
```

Each variable has exactly one definition, making def-use analysis trivial — "which definition does this use refer to?" is immediately obvious. At control-flow merge points, **φ functions** (phi) select the value from the appropriate path:

```
if (cond) x1 = 1;
else      x2 = 2;
x3 = φ(x1, x2);
y  = x3 + 1;
```

SSA lets all dataflow optimizations (constant propagation, dead-code elimination, common subexpression elimination) be implemented with simple algorithms — that is LLVM's strength.

LLVM IR Example

```
// C source: int add(int a, int b) { return a + b; }

// LLVM IR (SSA form)
define i32 @add(i32 %a, i32 %b) {
entry:
  %sum = add i32 %a, %b
  ret i32 %sum
}
```

LLVM IR is platform-independent — this IR can be compiled by different back ends (x86, ARM, RISC-V, GPU). This is the architectural foundation of cross-platform compilers.

Back End

Three main tasks for the back end:

1. **Instruction selection**: IR instructions → target architecture instructions. One IR instruction may map to multiple assembly instructions (or vice versa). Common algorithms: tree pattern matching, BURS.
2. **Register allocation**: IR is written assuming an unlimited number of registers; the back end must map them to a finite set of physical registers. The classic algorithm is **graph coloring** — color the register interference graph so variables with different colors can share a physical register. LLVM uses a more sophisticated linear-scan variant.
3. **Instruction scheduling**: On out-of-order CPUs, reordering instructions at compile time reduces pipeline stalls.

In AI

AI compilers (TVM, MLIR, XLA) apply the ideas of general-purpose compilers to tensor computation. **MLIR** (Multi-Level IR) is an extension of the LLVM project that provides multiple levels of intermediate representation — from PyTorch's ATen operator level, to the HLO tensor level, to the affine loop level, to the LLVM level — all unified in a single framework. PyTorch 2.0's TorchDynamo + Inductor and Triton's MLIR path are both products of this trend.

2.3.4 Compiler Optimizations

> *Position: dozens of optimizations working together can make code several times faster — with direct implications for AI kernel optimization.*

Local Optimizations

Constant propagation: `x = 5; y = x + 3;` → `y = 8;`

Constant folding: Evaluate at compile time what can be evaluated at compile time. `86400 * 30` is not left to be multiplied at runtime.

Common subexpression elimination (CSE):

```
a = x * y + 1;
b = x * y + 2;
// →
tmp = x * y;
a = tmp + 1;
b = tmp + 2;
```

Dead code elimination: Remove computations whose results are never used.

Strength reduction: Replace expensive operations with cheaper ones. `x * 8` → `x << 3`; replace `i*c` in loops with an accumulating variable.

Intraprocedural Optimizations

Inlining: Replace a function call with the function body. This eliminates call/return overhead, and more importantly, exposes the inlined code and its call-site context to other optimizations — "inlining opens the optimization window."

Cost: code bloat. The compiler uses heuristics to decide whether to inline: always inline small functions, inline hot functions, inline functions called only once. The `inline` keyword is merely a hint; the final decision belongs to the compiler.

Loop optimizations (important):

- **Loop unrolling**: `for (i=0;i<4;i++) a[i]=0;` → 4 store instructions. Reduces branch overhead and exposes parallelism.
- **Loop fusion**: Merge two adjacent independent loops into one. Improves cache locality.
- **Loop fission**: Split one large loop into several smaller ones. Enables vectorization and exposes parallelism.
- **Loop interchange**: Swap the order of nested loops so the inner loop accesses memory contiguously.
- **Loop-invariant code motion** (LICM): Hoist computations that don't change inside the loop to outside it.
- **Vectorization**: Transform scalar loops into SIMD instructions. `for (i=0;i under AVX2 becomes a vector instruction that processes 8 floats at a time.`

Interprocedural Optimizations

Optimizations that cross function boundaries: inlining, interprocedural constant propagation, dead function elimination. Require whole-program visibility — **LTO** (Link Time Optimization) lets the linker see all IR before optimizing, which is more powerful than the traditional approach of compiling each `.o` independently.

PGO

Profile-Guided Optimization: first compile a version with profiling enabled, run it under a real workload to collect a profile, then recompile — letting the compiler use the profile to drive optimization decisions (which branches are more likely taken, which functions are hot). Chromium and MySQL both use PGO; it typically yields a 5–15% performance improvement.

Limits of Auto-Vectorization

The compiler's auto-vectorization is powerful but not unlimited. It gives up in the following situations:

- Data dependencies (loop body contains `a[i] = a[i-1] + 1`)
- Uncertain pointer aliasing (the `restrict` keyword can help the compiler)
- Function calls blocking it (the `pure` attribute can help)
- Complex branches

So for performance-critical paths, inspecting the compiler-generated assembly (`gcc -S` or Compiler Explorer) is an essential skill for experts.

In AI

Hand-tuning AI kernels is applying these compiler optimization ideas to tensor computation:

- **Operator fusion** = loop fusion + inlining, reducing memory round-trips through intermediate tensors.
- **Tiling** = loop blocking, keeping data resident in cache/SRAM.
- **Vectorization to SIMD/Tensor Cores** = the tensor version of auto-vectorization.
- **Auto-tuning** (TVM, Triton AutoTune) = the PGO idea — empirically search different parameters for the best configuration.

Once you understand general compiler optimizations, AI compilers are just the same ideas applied to a specific domain.

PART FOUR

2.4 Databases

An increasingly important piece of AI infrastructure — from relational databases to vector databases.

2.4.1 Database Fundamentals

> *Position: storage engines, transactions, query optimization, logging — the four pillars of relational databases.*

Storage Engines: B+Tree vs LSM-Tree

Two mainstream index architectures.

B+Tree (MySQL InnoDB, PostgreSQL): a balanced multi-way tree whose leaf nodes hold data pages linked in a list (convenient for range scans). Reads are fast (\log_B n per lookup); updates are in-place. Downside: frequent random writes are unfriendly to SSDs.

LSM-Tree (RocksDB, Cassandra, LevelDB): Log-Structured Merge Tree. Writes first land in memory (memtable); when full, the memtable is flushed as an immutable SSTable to disk. Background compaction merges old SSTables. Reads may need to check multiple levels and consult bloom filters. Writes are extremely fast (purely sequential I/O); reads are somewhat slower; write amplification is the main concern.

Choosing: read-heavy OLTP → B+Tree; write-intensive (logs, time-series) → LSM.

Row Storage vs Column Storage

Row storage (OLTP): a row's data is physically contiguous. Fast for reading or writing whole rows. MySQL, PostgreSQL.

Column storage (OLAP): a column's data is physically contiguous. OLAP queries typically read only a few columns (`SELECT avg(salary) FROM emp`) and have high filter selectivity; columnar storage dramatically reduces I/O. Achieves high compression ratios (values in the same column are similar). ClickHouse, Parquet, DuckDB.

ACID Transactions

A (Atomicity): a transaction either fully completes or has no effect. **C** (Consistency): transactions preserve data constraints. **I** (Isolation): concurrent transactions do not interfere with each other. **D** (Durability): once committed, data is not lost.

Implementation relies on two mechanisms: **locking** (enforcing I) and **logging** (enforcing A and D).

Locking and Isolation Levels

The SQL standard defines four isolation levels, each preventing a stricter set of anomalies:

Level	Dirty Read	Non-Repeatable Read	Phantom Read
Read Uncommitted	Possible	Possible	Possible

Read Committed	No	Possible	Possible
Repeatable Read	No	No	Possible
Serializable	No	No	No

In practice, implementations use **2PL** (Two-Phase Locking) or **MVCC** (Multi-Version Concurrency Control). MVCC is the modern mainstream: each write creates a new version; readers see the snapshot from when their transaction began, so reads and writes do not block each other. PostgreSQL, MySQL InnoDB, and Oracle all use MVCC.

WAL and Recovery

WAL (Write-Ahead Log): write to the log before modifying data pages (sequential log I/O is faster than random page I/O). After a crash, redo committed transactions and undo uncommitted ones using the log. The **ARIES** algorithm is the industry-standard recovery protocol: Analysis → Redo → Undo.

Query Optimizer

SQL is declarative — it says "what," not "how." The optimizer translates SQL into an execution plan.

The process: SQL parsing → logical plan (relational algebra) → logical optimization (predicate pushdown, subquery rewriting) → physical plan (choosing concrete operators: hash join vs nested loop join, index scan vs full table scan) → cost estimation (based on statistics: table size, column distributions) → selecting the lowest-cost plan.

The optimizer is the "brain" of a database; a good one can outperform a bad one by an order of magnitude.

Vectorized Execution

The traditional execution engine uses the **row-at-a-time Volcano model**: each operator's `next()` returns one row. Simple, but per-row overhead is high (virtual dispatch, cache misses).

Vectorized execution: each `next()` returns a batch of rows (e.g., 1024). Overhead is amortized across the batch; a single SIMD instruction processes multiple rows; cache utilization improves. ClickHouse, DuckDB, and Apache Arrow are all vectorized — roughly 10× faster than the Volcano model.

In AI

In AI infrastructure, relational databases remain the primary store for metadata — users, model versions, training job records, and experiment tracking. PostgreSQL + MLflow is a common combination. But the databases most relevant to AI are the three covered in the following sections: graph, time-series, and vector.

2.4.2 Graph Databases

> *Position: native storage of nodes and edges — the foundation for knowledge graphs, social networks, and reasoning chains.*

Graph Storage

A relational database can store a graph (two tables: a node table and an edge table), but querying multi-hop relationships requires repeated joins and performs poorly. Graph databases use **native storage** for adjacency information — each node directly holds pointers to its outgoing edge list, so traversing k hops is k pointer dereferences.

The Property Graph Model

The mainstream graph model is the **property graph**: nodes have labels (types) and properties (key-value pairs); edges have direction, type, and properties. For example, a social graph: `(Alice :Person {age: 30})-[:FRIEND {since: 2020}]->(Bob :Person)`.

Query languages: **Cypher** (Neo4j, most popular), **Gremlin** (TinkerPop, cross-database), **SPARQL** (RDF graphs, semantic web).

Cypher Example

```
// Find friends-of-friends of Alice who are not already Alice's direct friends
MATCH (alice:Person {name: 'Alice'})-[:FRIEND]->(friend)-[:FRIEND]->(fof)
WHERE NOT (alice)-[:FRIEND]->(fof) AND fof <> alice
RETURN DISTINCT fof.name
```

The equivalent SQL query requires self-joins and NOT EXISTS subqueries — worse in both readability and performance.

Common Graph Algorithms

BFS/DFS: the most fundamental traversals
Shortest path: Dijkstra (no negative weights), Bellman-Ford (handles negative weights), A* (with heuristic)
PageRank: importance of pages/nodes
Community detection: Louvain, Label Propagation
Connected components: Union-Find
Centrality: degree, betweenness, closeness centrality

Representative Products

Product	Type	Characteristics
Neo4j	Native graph	Birthplace of Cypher; broad community
JanusGraph	Distributed graph	Built on HBase/Cassandra
TigerGraph	Distributed native graph	Large-scale analytics
NebulaGraph	Distributed graph	developed in China; Chinese community
DGraph	Distributed graph	Written in Go; GraphQL interface

In AI

Knowledge graph RAG: Traditional vector RAG performs poorly on "multi-hop reasoning" (questions requiring multiple associative steps). GraphRAG (Microsoft) first extracts entity-relationship graphs from documents, then combines vector retrieval and graph traversal at query time to improve complex question answering.

Graph neural networks (GNNs): node classification, link prediction, recommendation. Training typically loads the graph into memory, but large graphs require the graph database to perform sampling (sampling subgraphs by neighbor for mini-batch training).

Reasoning chain tracing: Every step an agent executes (LLM call → tool call → result) can be stored as a graph for easy retrospective analysis.

2.4.3 Time-Series Databases

> *Position: data streams ordered by timestamp — purpose-built for monitoring, IoT, and financial market data.*

Characteristics of Time-Series Data

Time-series data differs from ordinary relational data in several ways:

- **Ordered by time**: the time dimension is a natural index
- **Append-heavy**: old data is rarely modified
- **High write rate**: monitoring may generate millions of points per second
- **Fixed query patterns**: range aggregation, downsampling, time windows
- **Data has a lifecycle**: old data is downsampled or deleted

Key Technology: Compression

Adjacent values in time-series data vary little, enabling very high compression ratios. Two classic techniques:

Delta-of-Delta (timestamp compression): store `Δ(Δt)`. For regularly sampled data, Δ is nearly constant, so ΔΔ is mostly zero; variable-length encoding reduces it to a few bits.

XOR encoding (value compression): XOR adjacent floating-point values; high-order bits are mostly zero. The compression algorithm from the Gorilla (Facebook) paper achieves 1.37 bytes per point in practice — a 10×+ compression over the raw 16 bytes (8 bytes each for timestamp and value).

Storage Structure

Most time-series databases are based on LSM-tree variants. Write path:

1. Form a series by (metric, tags)
2. Each series is appended to independently in time order
3. Partition by time window (e.g., one chunk per 2 hours)
4. Compress chunks to disk as they age out

Downsampling

Raw 1-second-granularity data is retained for 7 days, then downsampled to 1-minute granularity for 30 days, then to 1-hour granularity for 1 year. Automatic continuous aggregates materialize the rollups, avoiding the cost of recomputing them at query time.

Representative Products

Product	Key Characteristics
InfluxDB	Most popular; InfluxDB 2.x (2024) migrated to Apache Arrow / DataFusion engine; query language shifted to SQL/InfluxQL; Flux is deprecated
Prometheus	Standard monitoring stack; PromQL; pull model
TimescaleDB	PostgreSQL extension; SQL compatible

VictoriaMetrics	Prometheus-compatible; stronger performance
IoTDB	developed in China; targets industrial IoT
TDengine	developed in China; SQL interface

PromQL Example

```
-- Total HTTP request QPS over the past 5 minutes, aggregated by
service
sum by (service) (rate(http_requests_total[5m]))

-- 99th-percentile response latency
histogram_quantile(0.99, sum by (le)
(rate(http_duration_seconds_bucket[5m])))
```

In AI

Observability metrics for AI training and inference are stored heavily in time-series databases: GPU utilization, VRAM usage, training loss curves, inference QPS/TTFT/P99 latency. Grafana + Prometheus + metrics exposed by model-serving frameworks is the standard setup.

Training data for **time-series forecasting models** (ARIMA, Prophet, Transformer variants such as Informer) is pulled directly from time-series databases.

2.4.4 Vector Databases

> *Position: nearest-neighbor search over high-dimensional vectors — the foundation of RAG, semantic retrieval, and recommendation.*

Problem Definition

Given N vectors of dimension d and a query vector \mathbf{q}, find the k vectors closest to \mathbf{q}. Distance is typically

measured by L2 distance or cosine similarity.

Brute-force search is O(Nd). With N = 10^8 and d = 768, each query takes several seconds — unacceptable. So the core of a vector database is **approximate nearest neighbor** (ANN) search: accept a small probability of returning a non-exact nearest neighbor in exchange for sub-millisecond query latency.

The ANN Algorithm Family

LSH (Locality-Sensitive Hashing): use random hyperplanes to partition the space into buckets; similar vectors fall into the same bucket with high probability. Simple, but recall is low.

IVF (Inverted File): cluster all vectors with k-means first; at query time, search only the nearest few clusters. Typical usage: IVF_PQ (vectors within clusters are stored using PQ quantization).

PQ (Product Quantization): split a d-dimensional vector into m segments of d/m dimensions each; independently quantize each segment with k-means into an 8-bit codebook index. For d=128, 8 segments of 256 classes each compress the original 512 bytes down to 8 bytes — a 64× reduction. Distance computation is approximated via lookup tables.

HNSW (Hierarchical Navigable Small World): a multi-layer graph index; the top layer is sparse, the bottom layer is dense. Queries start at the top layer and greedily "hop" toward the target. Recall and speed are best-in-class in most scenarios; it is currently the mainstream choice.

Intuition Behind HNSW

Think of all vectors as nodes in a graph, with each node connected to several nearby neighbors. A query starts at an entry point and at each step jumps to the neighbor closest to the query vector, until no neighbor is closer than the current node — a local optimum.

HNSW adds **hierarchy**: multiple graph layers, where each layer is a sparse subset of the layer below; edges at higher layers span

greater distances, while edges at lower layers are short. A query first takes large steps at the top layer to approach the target region, then descends layer by layer for refinement. Think of it as a two-dimensional generalization of a skip list.

Representative Products

Product	Characteristics
FAISS	Facebook library; most mature; non-distributed
Milvus	developed in China; distributed; supports multiple index types
Qdrant	Written in Rust; strong performance; filter-friendly
Weaviate	Written in Go; built-in multi-modal support
pgvector	PostgreSQL extension; good fit when already using PG
Chroma	Lightweight embedded; good for prototyping
LanceDB	Written in Rust; zero-copy; multi-modal

Hybrid Retrieval

Pure vector retrieval underperforms keyword search on technical terms and exact matches. **Hybrid retrieval** fuses vector (semantic similarity) and BM25 (keyword) results using RRF (Reciprocal Rank Fusion) or other reranking methods. Industrial RAG systems almost universally use hybrid retrieval.

Filtering and Metadata

Real-world use cases require not just "similar," but also "only search within this user's documents" or "only results after 2024." This requires the vector database to support ANN with filters. Two approaches:

Pre-filter: filter by metadata first, then run ANN. If very few results survive the filter, brute-force search is used.
Post-filter: run ANN first, then filter. But this may return fewer than k results.

Qdrant and Milvus implement **filterable HNSW** — checking filter conditions while traversing the HNSW graph, balancing speed and correctness.

In AI

RAG (Retrieval-Augmented Generation): documents → chunks → embeddings → stored in a vector database; query → embedding → ANN to fetch top-k → concatenated into the prompt → LLM answers. This is currently the most prevalent LLM application pattern.

Semantic search, recommendation systems, image-text retrieval, and deduplication are all direct applications of vector databases. **Long-term memory**: an agent's conversation history is stored as vectors and retrieved on demand — an increasingly important pattern in LLM applications.

PART FIVE

2.5 Message Middleware

The abstraction layer for inter-process/inter-node communication — a unified view of message queues, RPC, and publish-subscribe.

2.5.1 High-Performance Message Bus

> *Position: The foundation of all inter-process/service communication patterns — RPC, Pub/Sub, message queues.*

Classification of Communication Patterns

The underlying abstractions of middleware all revolve around a few fundamental communication semantics:

Pattern	Semantics	Examples
Unicast/Multicast	Point-to-point / point-to-many	UDP, shared memory
RPC	Request-response	gRPC, Thrift
Pub/Sub	Publish-subscribe, loose coupling	Kafka, NATS, ROS
Message Queue	Persistent, ordered, consumed	RabbitMQ, Kafka
Stream Processing	Continuous computation over event streams	Flink, Kafka Streams

Core Technical Points of RPC

gRPC is the de facto standard for modern RPC. Key elements:

IDL (Interface Definition Language): Protobuf describes messages and services, automatically generating stubs for each language
Serialization: Protobuf is binary, compact, and cross-language. 3–10x smaller than JSON with faster parsing
Transport: HTTP/2 foundation — multiplexing, streaming, header compression
Streaming modes: Unidirectional/bidirectional streams
Interceptors: Authentication, rate limiting, tracing

The Loose Coupling of Pub/Sub

The design philosophy of Pub/Sub: producers and consumers are unaware of each other. Publishers deliver messages to a topic; subscribers register interest in topics to receive messages. Scaling the system means adding consumers or partitions.

Kafka is the canonical example of "persistent-log Pub/Sub." Each topic is a partitioned append-only log; consumers read by offset and can replay history. A log retention period controls storage.

Zero-Copy and High Performance

All high-performance message middleware employs zero-copy optimizations:

- User-space ring buffer with lock-free multi-producer multi-consumer support
- `sendfile`/`splice` (kernel-level zero-copy)
- mmap shared memory (single-machine IPC)
- RDMA (cross-machine)

In single-machine scenarios, a shared-memory ring queue with atomic pointers can achieve 20M+ msg/s. In cross-machine scenarios, RDMA can reach line rate.

Service Discovery

The mandatory question in distributed service communication: "What IP and port is the service I need to call?" — that's service discovery. Several approaches:

DNS: The simplest approach; TTL governs update speed
Client-side registries: Consul, Zookeeper, etcd — services register on startup, clients query
Service Mesh: Istio, Linkerd — a proxy intercepts traffic, transparently injecting discovery and governance
Kubernetes Service: Built-in DNS + kube-proxy

Reliability Guarantees

Three tiers of message delivery semantics:

at-most-once: Delivered at most once; may be lost — simple and high-performance
at-least-once: Delivered at least once; may be duplicated — requires idempotent consumers
exactly-once: Delivered exactly once — expensive, requires transactions or deduplication

Kafka's exactly-once holds *within Kafka itself* (via transactional messages); end-to-end exactly-once involving external systems remains difficult.

In AI

Message middleware sees considerable use in AI systems:

- **Training job scheduling**: Producers (users) submit jobs; consumers (training workers) process them
- **Pre-inference queuing**: A queue in front of high-concurrency LLM services smooths traffic spikes
- **Data ingestion → feature engineering pipeline**: Kafka as the intermediary
- **Distributed RL**: Actors generate experience; learners consume from the queue
- **Agent tool-call bus**: LLM → tool router → individual tool processes

PART SIX

2.6 Network Stack: DPDK & VPP

Two main paths to high-performance packet forwarding — DPDK for kernel bypass, VPP for vectorized processing.

2.6.1 Bottlenecks of the Kernel Network Stack

> *Position: Why bypass the kernel — understanding the reason DPDK/VPP exists.*

The Linux Network Stack Path

A TCP packet traveling from the NIC to the application must pass through:

```
flowchart TB NIC[Network Card] --> IRQ[Hardware IRQ] IRQ --> SI[Softirq NET_RX] SI --> NP[netif_receive_skb] NP --> IP[IP layer] IP --> TCP[TCP layer] TCP --> SKB[socket buffer] SKB --> SYS[syscall read] SYS --> APP[Application]
```

Every layer has a cost:

- Hardware interrupt overhead
- sk_buff struct allocation/deallocation (once per packet, heavy pressure on the memory allocator)
- Multi-layer protocol stack (Ethernet, IP, TCP) parsed layer by layer
- User/kernel space data copies
- System call overhead
- Scheduler invocation per packet

This architecture is superb for generality, but its performance ceiling is roughly **1–2 Mpps per core**. To handle 10G small-packet line rate (14.88 Mpps) or 100G line rate (148.8 Mpps), the kernel stack simply cannot keep up.

Root Causes of the Bottleneck

Interrupt storms: At high packet rates, millions of hardware interrupts per second force the CPU to spend the majority of its time on interrupt handling. The NAPI mechanism partially mitigates this with an "interrupt fires → disable interrupts → poll" model, but is still limited.

Memory allocation: Receiving each packet requires a kmalloc for an sk_buff and a kfree when done. The allocator itself becomes the bottleneck.

Locks: The protocol stack contains global locks in many places (routing table, connection table). Multi-core scalability is poor.

Cache pollution: sk_buff migration across cores and the large protocol stack codebase cause frequent I-cache misses.

Two Paths to a Breakthrough

Bypass the kernel: User space directly manipulates the NIC. DPDK, Netmap, Snabb.

Optimize the kernel: XDP (eBPF filtering/forwarding at the driver layer), io_uring, AF_XDP.

The latter is maturing rapidly (XDP can reach 15 Mpps+ per core), but DPDK remains the preferred choice for maximum performance and is covered in detail in the next section.

2.6.2 DPDK Architecture

> *Position: The industry-standard framework for user-space networking that bypasses the kernel.*

DPDK's Design Philosophy

DPDK (Data Plane Development Kit) is a user-space networking framework open-sourced by Intel in 2013 (internal development began around 2010). Four core decisions:

1. **User-space driver**: UIO/VFIO is used to unbind the NIC from the kernel; a user-space process directly mmaps the device registers.
2. **Polling instead of interrupts**: The application main loop polls the receive ring continuously — no interrupt overhead.
3. **Pre-allocated huge-page memory**: All required memory is allocated once at startup; no malloc at runtime.
4. **Per-core isolation**: One thread per core, pinned to a fixed NIC queue, with minimal shared data.

Result: 10 Mpps+ per core with ease; single-server 100G line-rate forwarding is achievable.

Architecture Diagram

```
    flowchart LR subgraph UserSpace["User Space DPDK App"]
 APP[Application Logic] MBUF[mbuf pool] RING[rte_ring] PMD[Poll
 Mode Driver] end subgraph Kernel["Kernel (minimal)"] UIO[UIO /
VFIO] end subgraph HW["Hardware"] NIC[NIC] end APP --> RING RING
    --> PMD PMD -->|mmap regs| UIO UIO --> NIC PMD --> MBUF
```

Key Components

EAL (Environment Abstraction Layer): Abstracts the OS and hardware, providing a unified interface (memory, CPU, devices). `rte_eal_init()` is the entry point of every DPDK program.

PMD (Poll Mode Driver): User-space NIC driver. One PMD per NIC type (Intel ixgbe, Mellanox mlx5, virtio, etc.).

mbuf: DPDK's equivalent of sk_buff, but much simpler — a fixed-size header plus data, no linked list. Pre-allocated in a mempool.

rte_ring: Lock-free MPMC (multi-producer multi-consumer) ring queue — the primary primitive for user-space IPC.

mempool: Object pool. Allocation/deallocation in O(1); per-core cache reduces contention.

Minimal DPDK Application

```
#include <rte_eal.h>
#include <rte_ethdev.h>
#include <rte_mbuf.h>

#define RX_RING_SIZE 1024
#define NUM_MBUFS    8191
#define BURST_SIZE   32

int main(int argc, char *argv[]) {
    rte_eal_init(argc, argv);

    // Create mempool
    struct rte_mempool *mbuf_pool = rte_pktmbuf_pool_create(
        "MBUF_POOL", NUM_MBUFS, 250, 0,
        RTE_MBUF_DEFAULT_BUF_SIZE, rte_socket_id());
```

```
    // Configure port 0
    uint16_t port = 0;
    struct rte_eth_conf cfg = {0};
    rte_eth_dev_configure(port, 1, 1, &cfg);
    rte_eth_rx_queue_setup(port, 0, RX_RING_SIZE,
rte_socket_id(), NULL, mbuf_pool);
    rte_eth_tx_queue_setup(port, 0, RX_RING_SIZE,
rte_socket_id(), NULL);
    rte_eth_dev_start(port);

    // Main loop: poll for received packets, echo them back
    struct rte_mbuf *bufs[BURST_SIZE];
    while (1) {
        uint16_t nb_rx = rte_eth_rx_burst(port, 0, bufs,
BURST_SIZE);
        if (unlikely(nb_rx == 0)) continue;

        // Process packets... (omitted here)

        uint16_t nb_tx = rte_eth_tx_burst(port, 0, bufs, nb_rx);
        // Free any packets that failed to transmit
        for (uint16_t i = nb_tx; i < nb_rx; i++)
            rte_pktmbuf_free(bufs[i]);
    }
}
```

This code runs on a dedicated CPU core to which the NIC is bound and can easily reach several Mpps of throughput — 10–50x faster than a kernel-stack socket program.

The Cost

DPDK is not a free lunch:

- **Dedicated CPU**: The polling core runs at 100% utilization even when there are no packets
- **No kernel protocol stack**: TCP/UDP/ARP must be implemented yourself or via user-space stacks like mTCP or F-Stack
- **Development complexity**: Standard socket libraries cannot be used
- **Hardware dependency**: The NIC must have DPDK support

DPDK is therefore suited to **dedicated forwarding appliances** (firewalls, load balancers, NFV) and not to general-purpose

applications.

2.6.3 Key DPDK Mechanisms

> *Position: The secrets behind DPDK's performance — HugePages, lock-free rings, NUMA, and RSS.*

HugePages

DPDK mandates the use of HugePages (2 MB or 1 GB). The reason was covered in Section 2.2.3: TLB coverage expands dramatically, eliminating TLB misses. A typical DPDK startup command specifies how many 2 MB huge pages to reserve on each NUMA node.

Lock-Free Ring Queue

rte_ring is one of DPDK's most elegant components. It is an MPMC ring queue based on two pairs of producer/consumer head/tail pointers:

```
// Simplified rte_ring core variables
struct rte_ring {
    uint32_t size;           // Ring capacity (power of 2 for
efficient & masking)
    struct {
        uint32_t head;       // Next position to be written
        uint32_t tail;       // Last committed position
    } prod, cons;
    void *ring[size];
};
```

Enqueue flow: atomic CAS increments prod.head by n (reserving slots) → copy data → wait for prod.tail to catch up to the original head → advance prod.tail. This two-phase approach allows multiple producers to *reserve concurrently and commit in order*, guaranteeing that data in the ring is complete when seen by a consumer.

NUMA Awareness

Modern servers have multiple sockets; cross-socket memory access doubles latency (remote DRAM vs. local DRAM). DPDK's `rte_socket_id()` lets you allocate mempools and queues on a specific NUMA node. Production code must NUMA-align all resources — NIC, CPU core, and memory pool must reside on the same socket.

RSS and Multi-Queue

A single queue on a single core has a throughput ceiling. DPDK works with the NIC's RSS to distribute packets from different flows across multiple receive queues, each bound to a core, enabling true parallelism.

RSS hashes on the 5-tuple — *packets belonging to the same flow always go to the same queue* (preserving intra-flow ordering), while different flows are distributed evenly across cores. This design underlies all modern high-speed forwarding.

CPU Affinity

DPDK threads must be pinned to cores (the `--lcores` argument), preventing scheduler migrations that cause cache misses. Combined with the `isolcpus` kernel parameter to remove these cores from the Linux scheduler, they become fully dedicated.

Batch Processing

All DPDK APIs operate in batches — `rte_eth_rx_burst` receives 32 packets at a time, `rte_eth_tx_burst` transmits 32 at a time — amortizing per-packet overhead to near zero. Application logic should also batch: parse 32 Ethernet headers, look up 32 flow table entries, and so on, which is friendly to both the cache and the CPU pipeline.

Zero-Copy

The mbuf data region is directly the DMA region — the NIC DMAs packets into mbufs; the application reads them directly; on

transmit, the NIC DMAs directly from the mbuf. The entire data path is zero-copy.

2.6.4 VPP Vectorized Forwarding

> *Position: A high-performance, programmable forwarding framework built on top of DPDK, based on the idea of "applying the same operation to a batch of packets consecutively."*

VPP's Core Innovation: Graph Nodes + Vectorization

VPP (Vector Packet Processing) is the FD.io project (based on Cisco open-source code). Its two core concepts:

Forwarding graph: The processing pipeline is a directed graph; nodes are "processing stages" (e.g., ethernet-input, ip4-lookup, ip4-rewrite). Packets enter from source nodes, pass through a series of nodes, and reach a sink (transmit or drop). Each node does exactly one thing and is pluggable/extensible.

Vectorization: *Instead of processing one packet at a time, process a batch at a time.* The traditional "each packet traverses the entire forwarding pipeline" is replaced by "each node processes the entire batch before passing it to the next node."

Why Vectorization Is So Fast

Suppose a batch of 256 packets with 10 processing stages. Traditional approach:

```
for packet in 256:
    for stage in 10:
        do_stage(packet)
```

Each stage's code is fetched 256 times; each fetch may cause an I-cache miss.

Vectorized approach:

```
for stage in 10:
    for packet in 256:
        do_stage(packet)
```

The stage code is fetched only once, staying warm in the I-cache. The 256 loop iterations allow data to remain in the D-cache (prefetcher-friendly). The improvement in **I-cache hit rate** is the primary reason VPP is fast — not SIMD vectorization.

This is the network-forwarding equivalent of the compiler optimization known as *loop interchange* — swapping loop order to improve cache locality.

VPP Node Example

```
// Simplified sketch of the ip4-lookup node
static uword
ip4_lookup_node_fn(vlib_main_t *vm, vlib_node_runtime_t *node,
                   vlib_frame_t *frame) {
    u32 *from = vlib_frame_vector_args(frame);
    u32 n_left = frame->n_vectors;   // Number of packets in
this batch

    // Process 4 at a time (use prefetch to hide cache miss
latency)
    while (n_left >= 4) {
        vlib_buffer_t *b0, *b1, *b2, *b3;
        b0 = vlib_get_buffer(vm, from[0]);
        b1 = vlib_get_buffer(vm, from[1]);
        b2 = vlib_get_buffer(vm, from[2]);
        b3 = vlib_get_buffer(vm, from[3]);

        // Prefetch the cache line for the next batch
        CLIB_PREFETCH(vlib_get_buffer(vm, from[4]),
CLIB_CACHE_LINE_BYTES, LOAD);
        // ...

        // Parallel route lookup for 4 packets
        ip4_fib_mtrie_lookup(...);   // for b0
        ip4_fib_mtrie_lookup(...);   // for b1
        ip4_fib_mtrie_lookup(...);   // for b2
```

```
        ip4_fib_mtrie_lookup(...);   // for b3

        from += 4; n_left -= 4;
    }

    // Handle remaining 0-3 packets individually
    while (n_left > 0) { /* ... */ }

    return frame->n_vectors;
}
```

This "4-at-a-time + prefetch" pattern is the hallmark of VPP code. It uses instruction-level parallelism and explicit prefetching to hide memory latency behind computation.

What VPP Can Do

- IPv4/IPv6 route forwarding (FIB)
- MPLS
- Tunneling: VXLAN, GRE, etc.
- NAT
- Basic firewall ACL
- Load balancing
- IPSec

Performance: 10+ Mpps per core; dual-socket servers can achieve 100 Gbps+ line rate.

Relationship with DPDK

VPP uses DPDK as its default I/O driver layer but also supports AF_XDP, Netmap, and others. VPP handles the "packet processing framework"; DPDK handles "talking to hardware." The two are complementary and are commonly used together.

2.6.5 TCP / UDP / Raw IP

> *Position: Transport-layer protocols — you use sockets every day, but you should know*

> *what's happening underneath.*

The IP Layer

IP is an unreliable packet delivery service. Each IP packet is routed independently — *there is no concept of a "connection"*; intermediate routers forward each packet independently based on a lookup. The IPv4 header is 20 bytes (with optional fields); the IPv6 header is 40 bytes (but with simplified fields).

Key IP mechanisms: **fragmentation** (when the next-hop MTU is smaller than the packet), **TTL** (decremented by one at each hop; discarded at zero to prevent loops), and **checksum** (header only).

UDP

The thinnest transport layer — an 8-byte header (source port, destination port, length, checksum). Connectionless, no retransmission, no congestion control, no ordering guarantee.

Suitable for: DNS queries, real-time audio/video, game state synchronization, custom reliability layers (QUIC is built on UDP).

TCP: The Three Core Mechanisms

Connection: Established with a three-way handshake; closed with a four-way handshake. Maintains state such as sequence numbers, acknowledgment numbers, and window size.

Reliable delivery: Retransmission based on sequence numbers and ACKs. Timeout retransmit + fast retransmit (retransmit on 3 duplicate ACKs without waiting for timeout).

Congestion control: AIMD (Additive Increase Multiplicative Decrease). The congestion window grows exponentially during slow start, then linearly after reaching the threshold, and is halved on packet loss. Algorithms include Reno, Cubic (Linux default), and BBR (Google — models bandwidth/delay rather than relying on loss feedback).

TCP Performance Tuning

Parameters to pay attention to in production deployments:

- **Buffer sizes**: `net.core.rmem_max/wmem_max`. Long fat networks (high bandwidth, high latency) require large buffers.
- **TCP_NODELAY**: Disables the Nagle algorithm, sending small packets immediately — must be enabled for low-latency interactive workloads.
- **TCP_QUICKACK**: Disables delayed ACKs.
- **Congestion algorithm**: `net.ipv4.tcp_congestion_control`. BBR generally outperforms Cubic for data center or intercontinental links.
- **TCP Fast Open**: Carries data in the SYN packet, saving one RTT.

Raw IP / Raw Socket

The application constructs IP packets directly (any protocol can be customized). Requires root privileges. Used for: ping (ICMP), routing protocols (OSPF, BGP), custom protocols. `socket(AF_INET, SOCK_RAW, IPPROTO_XXX)`.

In AI

AI training/inference communication primarily uses:

- **TCP**: gRPC/REST APIs, control plane for distributed training
- **RDMA** (covered in Chapter 4, Section 5.7): Training data plane, completely bypassing TCP/IP
- **UDP**: Occasionally used in custom AllReduce implementations

2.6.6 TLS and QUIC

> *Position: Two paradigms for encrypted transport — TLS over TCP vs. QUIC (UDP + encryption).*

TLS Handshake

TLS 1.3 (2018) simplified the handshake to 1 RTT (TLS 1.2 required 2 RTT):

1. Client Hello: carries supported algorithms and ECDHE public key share
2. Server Hello + Certificate + Finished: server returns selected algorithm, certificate, its own ECDHE share, and signature
3. Client Finished: client verifies the certificate; both sides derive the session key

TLS 1.3 **0-RTT** mode: A client with an existing session can send request data in the very first packet (using a cached PSK). Zero latency overhead, but vulnerable to replay attacks.

QUIC's Design Motivation

TCP + TLS has several inherent limitations:

1. TCP handshake (1 RTT) + TLS handshake (1 RTT) = at least 2 RTT before data can flow
2. TCP is implemented in the OS; upgrades are slow (congestion algorithm deployment cycles are measured in years)
3. TCP's head-of-line blocking: a single lost packet stalls the entire stream
4. TCP connections are identified by a 4-tuple; when a phone switches between Wi-Fi and 4G, the IP changes and the connection breaks

QUIC (Quick UDP Internet Connections) reinvents the transport layer on top of UDP, solving all four problems:

- **1 RTT handshake** (first connection), **0 RTT** (reconnection) — encryption and reliable transport are merged
- **User-space implementation**: A library-level component; upgrading only requires a library update
- **Multiple streams**: Many independent streams within a single connection; packet loss in one stream does not block others
- **Connection ID**: Not tied to an IP address; connections survive network changes

HTTP/3

HTTP/3 = HTTP/2 semantics + QUIC transport. Compared to HTTP/2 over TCP+TLS: faster handshake, multiplexing without head-of-line blocking, and smooth network transitions. Standardized by the IETF in 2022; already supported by major browsers and CDNs.

Implementations

QUIC libraries: Google's QUICHE, Meta's mvfst, Cloudflare's quiche, Microsoft's msquic. All are user-space implementations.

In AI

LLM inference APIs (OpenAI, Anthropic, etc.) largely support HTTP/3, reducing time-to-first-token latency. Mobile AI applications (which switch networks frequently) benefit even more noticeably from QUIC.

2.6.7 Forwarding Management and the Control Plane

> *Position: The data plane uses DPDK/VPP to forward packets at speed; the control plane is responsible for distributing policies and synchronizing routing tables. Brief section.*

Control Plane vs. Data Plane

Modern network devices separate CP and DP:

Control plane: Routing protocols (BGP, OSPF), policy computation, configuration management. Runs on general-purpose CPUs; Python/Go implementations are fine. Being a bit slower is acceptable.
Data plane: Per-packet processing. DPDK/VPP/NP/ASIC. Must be fast.

The CP computes routes and pushes them to the DP's forwarding table (FIB). The DP only performs table-lookup forwarding; it

does not participate in routing computation.

SDN

Software-Defined Networking takes this separation to its logical extreme: **extract the control plane from every individual device and centralize it in a single controller**. The controller pushes forwarding rules via protocols such as OpenFlow, gNMI, and P4Runtime.

Advantages: global-view optimization, unified policy, simplified devices (only the data plane needs to be implemented). Disadvantage: the controller becomes a single point of failure.

Control Plane Configuration Protocols

CLI: Traditional — SSH into a device and type commands
SNMP: Legacy monitoring; read-heavy, write-light
NETCONF + YANG: Structured configuration with transaction support
gNMI: gRPC-based, streaming telemetry
OpenConfig: Vendor-neutral YANG models

Routing Protocols in Brief

BGP: The Internet backbone protocol — path-vector based, inter-AS routing. TCP port 179; rich policy support.
OSPF: IGP (Interior Gateway Protocol) — link-state algorithm (Dijkstra shortest path). Fast convergence; used within an AS.
IS-IS: Similar to OSPF; commonly used by ISPs.

In AI

The network topology of AI training clusters (fat-tree, dragonfly) has an enormous impact on performance. The control plane itself, however, has little direct bearing on AI; this section is included only for general awareness.

2.6.8 Logging and Observability

> *Position: The trinity of Metrics + Logs + Traces — without observability there is no production operations.*

The Three Pillars of Observability

Metrics: Numerical values stored as time series. QPS, CPU, latency. Low-frequency sampling, small volume, easily aggregated. Prometheus is the standard.

Logs: Textual records of discrete events. High cardinality, large volume. ELK (Elasticsearch + Logstash + Kibana) and Loki are common choices.

Traces: Distributed tracing of a single request across multiple services. Each span records start time, duration, and tags. Jaeger, Zipkin, Tempo.

Structured Logging

Traditional `printf`-style logs are difficult for machines to process. The modern approach is structured logging — JSON or logfmt format:

```
{"time":"2024-05-01T10:30:15Z","level":"INFO","service":"inference",
 "request_id":"abc123","model":"gpt-4","ttft_ms":234,"tokens":156}
```

Structure enables efficient search and aggregation. Go's slog, Python's structlog, and Rust's tracing all push in this direction.

OpenTelemetry

OpenTelemetry (OTel) is a CNCF graduated project that aims to **unify observability data collection standards** — language SDKs, data models, and transport protocol (OTLP). A single SDK simultaneously produces metrics, logs, and traces that can be

routed to different backends (Prometheus, Jaeger, commercial APM).

For any new service being written today, choosing OTel is almost always the right call.

Sampling

Full-volume traces are too expensive — storing 100K requests per second would overwhelm any disk. Sampling strategies:

Head-based sampling: Decided at the entry point — based on probability or rules
Tail-based sampling: Decided after the trace completes (can target errors and slow requests specifically)
Adaptive sampling: Dynamically adjusts the sampling rate

eBPF Observability

Traditional observability requires intrusive instrumentation of the application. eBPF lets you capture any function call, any syscall, and any socket event in the kernel without modifying the application. **Pixie**, **Cilium Hubble**, and **Parca** are all eBPF-driven observability tools.

In AI

Key observability metrics specific to LLM inference:

- **TTFT** (Time to First Token): Critical for the frontend experience
- **TPOT** (Time Per Output Token): Steady-state generation speed
- **tokens/sec throughput**: System-level throughput
- **KV cache hit rate**: Effectiveness of prefix caching
- **Batch size distribution**: Scheduler health
- **GPU utilization**: Hardware efficiency

Training observability: loss curves, gradient norms, learning rate, per-rank synchronization latency, data loading delay. TensorBoard, Weights & Biases, and MLflow are purpose-built platforms.

Chapter 3: AI Model Technology

From Traditional ML to Performance Engineering

AI engineers in the LLM era still need traditional machine learning — not out of nostalgia, but because in scenarios with **limited data, interpretable features, and tight latency constraints** (traffic classification on network devices, KPI anomaly detection), a single XGBoost tree is often a better fit than an LLM. The structure of this chapter reflects that view: traditional ML first, then fundamental operators, then classic deep learning models, then training optimization, and finally performance engineering and domain applications. Each layer is the foundation for the next, yet each also holds an independent place in modern AI systems.

All 36 sections go deep. The key anchors are: 3.1.2 Tree Models (full derivation from decision trees to XGBoost), 3.2.3 Convolution (including im2col and Winograd), 3.3.2 LSTM gate derivations, 3.5.3 Fine-Tuning & Alignment (the full SFT/DPO/RLHF pipeline), 3.6.3 Operator Optimization (Roofline + tiling + FlashAttention), and 3.7.2 Traffic Classification feature engineering in practice. The other sections are equally substantive — every section is written at the level of a knowledgeable colleague explaining it for 30 minutes, not as a textbook survey.

PART ONE

3.1 Traditional Machine Learning

Decades of accumulated knowledge before deep learning — still the best answer in many scenarios.

3.1.1 K-means and KNN

> *Overview: Two of the most fundamental distance-based algorithms — an introduction to clustering and classification.*

KNN: Nearest Neighbor Classification

K-Nearest Neighbors requires no training — it simply memorizes all training samples. At prediction time, it finds the k training samples closest to the query point and uses their labels to vote (classification) or average (regression). The only parameters are k and the distance metric.

It may seem simple, but in low-dimensional, data-dense settings it performs well. Its theoretical properties are also elegant — as $n \to \infty$, KNN's error rate is at most twice the Bayes optimal error rate. This is one of the rare "distribution-free" guarantees in machine learning.

KNN's problems are all engineering ones:

- **Prediction time**: O(nd) per query. For large datasets, use KD-Tree, Ball Tree, or approximate nearest neighbors (see Chapter 2, vector databases) to speed things up.
- **Curse of dimensionality**: In high-dimensional space, all points are roughly equidistant, and KNN breaks down.
- **Feature scale sensitivity**: A feature with range [0, 1] and a feature with range [0, 10000] computed directly using Euclidean

distance will be completely dominated by the latter. Normalization is mandatory.

K-means: Nearest Centroid Clustering

Given data $\{\mathbf{x}_i\}$, partition them into k clusters so that the **sum of squared distances from each point to its cluster center is minimized**:

> $\min_{\{\mathbf{c}_j\}, \{S_j\}} \sum_{j=1}^k \sum_{\mathbf{x} \in S_j} \|\mathbf{x} - \mathbf{c}_j\|^2$

This is a combinatorial optimization problem (NP-hard); K-means finds an approximate solution via alternating optimization:

1. Initialize k cluster centers (random selection or K-means++)
2. **Assignment**: assign each point to the nearest cluster center
3. **Update**: recompute each cluster center as the mean of its assigned points
4. Repeat steps 2–3 until centers no longer change

```
import numpy as np

def kmeans(X, k, max_iter=100, tol=1e-4):
    # K-means++ initialization: spread initial centers far apart
    n = len(X)
    centers = [X[np.random.randint(n)]]
    for _ in range(k - 1):
        # For each point, compute the squared distance to the
nearest existing center
        d2 = np.min([np.sum((X - c)**2, axis=1) for c in
centers], axis=0)
        # Sample the next center proportional to squared
distance
        idx = np.random.choice(n, p=d2/d2.sum())
        centers.append(X[idx])
    centers = np.array(centers)

    for it in range(max_iter):
        # Assign
        dists = ((X[:, None] - centers[None, :])**2).sum(axis=2)
```

```
        labels = dists.argmin(axis=1)
        # Update
        new_centers = np.array([X[labels == j].mean(axis=0) for
j in range(k)])
        if np.linalg.norm(new_centers - centers) < tol:
            break
        centers = new_centers
    return centers, labels
```

A few pitfalls with K-means. First, **sensitivity to initialization** — different seeds can produce completely different clusters. K-means++ initialization (as done in the code above) significantly improves this. Second, **you must specify k** — typically the elbow method (plot k vs. SSE, find the kink) or the silhouette score helps with selection. Third, **it assumes spherical clusters** — elongated or ring-shaped distributions are beyond its reach; DBSCAN or spectral clustering are needed for those cases.

In the Networking Domain

KNN was widely used in early traffic classification work — extract features from a connection (packet size distribution, inter-arrival times) and do KNN matching against a labeled traffic database. The downside is that as the traffic database grows, prediction latency becomes unacceptable. Tree models or small NNs are now the common replacements.

K-means is used for anomaly detection: normal traffic clusters into a few groups, and points that don't belong to any cluster (far from the nearest center) are potentially anomalous. But real anomalies are often too sparse and scattered for K-means to handle well; Isolation Forest is more commonly used instead.

3.1.2 Tree Models: Decision Trees, Random Forests, XGBoost

> *Overview: The king of tabular data. Deep dive: from decision trees to GBDT to XGBoost, with full derivations.*

Decision Trees

Each internal node of the tree is a feature test (e.g., `age < 30`), and leaf nodes produce predictions. The key to building a tree is choosing split points. For classification, the goal is to select splits that maximally improve the **purity** of child nodes. Common purity measures are the **Gini impurity** and **entropy**:

> $$\text{Gini} = 1 - \sum_c p_c^2, \quad H = -\sum_c p_c \log p_c$$

Here p_c is the fraction of class c at a node. After a split, the weighted average impurity of child nodes is compared to the parent; the difference is called the information gain. The split with the highest information gain is chosen greedily.

Decision trees are naturally interpretable ("age > 30 → high income → ..."), require no feature normalization, handle missing values, and support both numerical and categorical features. But a single tree overfits easily — grow it deep enough and each leaf holds one sample, giving perfect training-set fit and catastrophic test-set performance. Pruning (pre-pruning: limit depth/leaf count; post-pruning: grow fully then prune) is necessary.

From Single Trees to Ensembles

The intuition behind ensemble learning: many weak learners voting beats a single strong learner. Two main lines:

Bagging (Bootstrap AGGregating): each tree is trained on a bootstrap sample (sampling with replacement) of the original data, and at each split only a random subset of features is considered. Predictions from multiple trees are averaged or voted. **Random Forest (RF)** is bagging applied to decision trees. The primary benefit is variance reduction — a single tree has high variance and low bias; averaging many trees dramatically reduces variance.

Boosting: trees are trained *sequentially*, with each tree learning the residuals that the previous tree failed to capture. **GBDT** (Gradient Boosting Decision Tree), **XGBoost**, and **LightGBM** all follow this approach.

The Mathematics of GBDT

Take regression as an example. The model is $F(x) = \sum_{t=1}^T f_t(x)$, where f_t is the t-th tree. At each round we fit the *negative gradient* — the direction of steepest descent for the current loss.

For squared loss $L(y, F(x)) = \frac{1}{2}(y - F(x))^2$, the negative gradient is exactly the residual $r_i = y_i - F_{t-1}(x_i)$ — so GBDT under squared loss is just "fitting the residuals of the previous step." For other losses (log loss, Huber), the pseudo-residuals are the corresponding gradients.

The key insight of GBDT is that "training the t-th tree" is reduced to an ordinary regression problem (fit the pseudo-residuals), which is then added to the cumulative prediction.

XGBoost's Innovations

XGBoost is an engineered refinement of GBDT, proposed by Tianqi Chen in 2014. It introduces several important ideas.

Second-order Taylor expansion. Traditional GBDT uses only first-order gradients; XGBoost uses both first and second order. Expanding the loss L around F_{t-1} to second order:

> $L(y, F_{t-1} + f_t) \approx L(y, F_{t-1}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2$

$g_i = \partial L / \partial F$, $h_i = \partial^2 L / \partial F^2$. This yields a closed-form optimal solution for the split gain.

Closed-form leaf weights. Let the t-th tree map x to leaf j with value w_j. Adding L2 regularization $\Omega(f) = \gamma T +$

$\frac{1}{2}\lambda \sum_j w_j^2$ (where T is the number of leaves):

> $$\text{Obj}^{(t)} = \sum_j \left[(\sum_{i \in I_j} g_i) w_j + \frac{1}{2} (\sum_{i \in I_j} h_i + \lambda) w_j^2 \right] + \gamma T$$

Setting the derivative with respect to w_j to zero gives the optimal leaf weight:

> $$w_j^* = -\frac{G_j}{H_j + \lambda}, \quad G_j = \sum_{i \in I_j} g_i, \quad H_j = \sum_{i \in I_j} h_i$$

Substituting back gives the minimum loss for a fixed tree structure:

> $$\text{Obj}^* = -\frac{1}{2} \sum_j \frac{G_j^2}{H_j + \lambda} + \gamma T$$

This yields a closed-form split gain: splitting a node into left and right children L, R, the gain equals the original loss minus the post-split loss:

> $$\text{Gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\right] - \gamma$$

All split candidates are ranked by this gain and the maximum is selected. γ controls whether to split at all (no split if gain is below γ); λ prevents overfitting.

Engineering optimizations: histogram approximation (LightGBM discretizes features into 256 bins, reducing $O(n \times d)$ to $O(n \times \text{bins})$); sparsity awareness (missing values automatically pick a default direction); column-block parallelism; GPU acceleration.

```
import xgboost as xgb
import numpy as np
```

```
# A typical pipeline for binary classification of network
traffic
X_train, y_train = load_data()   # feature matrix and 0/1 labels

model = xgb.XGBClassifier(
    n_estimators=500,       # number of trees
    max_depth=6,            # tree depth
    learning_rate=0.05,     # per-tree contribution scaling
    subsample=0.8,          # use 80% of samples per tree
(randomization, prevents overfitting)
    colsample_bytree=0.8,   # use 80% of features per tree
    reg_lambda=1.0,         # L2 regularization
    reg_alpha=0.1,          # L1 regularization
    min_child_weight=1,     # minimum Hessian sum per leaf,
controls overfitting
    tree_method='hist',     # histogram algorithm, faster
training
    eval_metric='logloss',
    early_stopping_rounds=50,
)
model.fit(X_train, y_train,
          eval_set=[(X_val, y_val)],
          verbose=50)

# Feature importance
importance = model.feature_importances_
for feat, imp in sorted(zip(feature_names, importance),
key=lambda x: -x[1])[:10]:
    print(f"{feat}: {imp:.4f}")
```

When to Choose Tree Models

XGBoost/LightGBM remain top-tier baselines on **tabular data** (rows are samples, columns are features). Over 90% of top Kaggle solutions on tabular competitions include GBDT. Advantages:

- No feature normalization required
- Automatically handles feature interactions (deep trees can express complex nonlinearities)
- Fast training (millions of samples in minutes)
- Fast inference (bounded tree depth, hundreds of microseconds per sample)
- Interpretable feature importance

When does deep learning take over? For *unstructured data* like images, text, and audio — deep learning learns features end-to-end and nearly always outperforms hand-crafted features combined with trees. But as long as your data is in row-column tabular form, XGBoost is almost always the first choice.

3.1.3 SVM

> *Overview: Maximum-margin classification — once dominant, now a reference baseline.*

Maximum-Margin Hyperplane

The objective of binary linear SVM: find a hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$ that separates two classes while maximizing the margin (distance to the nearest point). This is the **maximum-margin classifier**.

Geometric intuition: many hyperplanes can separate two classes, but the "wide" one is more tolerant of noise. Maximizing the margin is equivalent to minimizing $\|\mathbf{w}\|$. Adding classification constraints $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \ge 1$ yields a convex quadratic program:

> $$\min_{\mathbf{w}, b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{s.t. } y_i(\mathbf{w}^\top \mathbf{x}_i + b) \ge 1, \forall i$$

Dual Form and Support Vectors

Applying Lagrangian duality, the dual problem is:

> $$\max_\alpha \sum_i \alpha_i - \frac{1}{2}\sum_{i,j} \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j, \quad \alpha_i \ge 0, \sum_i \alpha_i y_i = 0$$

After solving for α^*, $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$. A key observation: **the vast majority of $\alpha_i = 0$** ; only the points on the margin boundary have $\alpha_i > 0$ —

these are the **support vectors**, which is where the name SVM comes from. Prediction only requires the support vectors:

> $$f(\mathbf{x}) = \text{sign}\left(\sum_{i \in \text{SV}} \alpha_i y_i \mathbf{x}_i^\top \mathbf{x} + b\right)$$

The Kernel Trick

What about linearly inseparable data? Map the data to a higher-dimensional space $\phi(\mathbf{x})$ where linear separability holds. Notice that the dual only involves $\mathbf{x}_i^\top \mathbf{x}_j$ — replacing this with $\phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$ suffices. And this inner product can be defined as a **kernel function** $K(\mathbf{x}_i, \mathbf{x}_j)$ computed directly, without explicitly computing ϕ — this is the kernel trick.

Common kernels:

- **Linear kernel**: $K = \mathbf{x}_i^\top \mathbf{x}_j$ (equivalent to no kernel)
- **Polynomial kernel**: $K = (\mathbf{x}_i^\top \mathbf{x}_j + c)^d$
- **RBF kernel** (Gaussian kernel): $K = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$ — corresponds to an infinite-dimensional space, most commonly used

The hyperparameter γ of the RBF kernel controls the "curvature" of the decision boundary — large γ makes the boundary cling tightly to training points (overfitting); small γ produces a smoother boundary. Grid search over γ and regularization parameter C is the standard approach.

Soft Margin

Real data often has noise and some inseparable samples. A hard margin (perfectly separate everything) is too strict. The soft margin introduces slack variables $\xi_i \ge 0$:

> $$\min \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_i \xi_i, \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \ge 1 - \xi_i$$

C controls how much misclassification to tolerate — large C is strict, small C is lenient.

Current Status

In the 2000s, SVMs were the state of the art for text and image classification. Their prominence declined in the deep learning era, but they retain value in **small-sample, high-dimensional, interpretable** settings — bioinformatics and certain text classification tasks still use them. The downside is training complexity of O(n^2) to O(n^3), making them impractical on large datasets.

3.1.4 N-gram and Label Propagation

> *Overview: Two old but useful tools — N-gram statistical language models and LPA for graph semi-supervised learning.*

N-gram Language Models

N-gram is a probabilistic model that predicts the next word from the previous N-1 words, based on counts:

> $$P(w_n \mid w_1, ..., w_{n-1}) \approx P(w_n \mid w_{n-k+1}, ..., w_{n-1}) = \frac{\text{count}(w_{n-k+1}...w_n)}{\text{count}(w_{n-k+1}...w_{n-1})}$$

Typical choices are k = 2 (bigram) or k = 3 (trigram). Before neural language models, this was the mainstream approach in NLP.

The core problem is **sparsity** — unseen combinations in the training set have zero probability. Smoothing techniques address this:

Add-one smoothing (Laplace): add 1 to all counts — the simplest approach
Good-Turing: uses the frequency of items appearing r+1 times to estimate the probability of items appearing r times

Kneser-Ney: the most sophisticated, considers "how many distinct contexts a word appears as a continuation of"

Today LLMs dominate NLP and N-grams are no longer the lead approach. But **character-level N-grams** remain in use: unknown language detection, spell correction, and fast approximate matching (MinHash shingles).

LPA: Label Propagation Algorithm

Semi-supervised learning on graphs: a small number of nodes have labels, most do not. LPA assumes "**tightly connected nodes should have the same label**" and iteratively propagates labels along edges:

1. Set initial labels for labeled nodes; unlabeled nodes start empty or with a uniform distribution
2. Each node sets its label distribution to the (weighted) average of all its neighbors' label distributions
3. Fix (clamp) the labels of labeled nodes
4. Repeat until convergence

In matrix form: let $F \in \mathbb{R}^{n \times C}$ be the class distribution for each node, and W the adjacency matrix (or a normalized version). Update $F \leftarrow D^{-1} W F$, where D is the degree diagonal matrix. After clamping labeled nodes, repeat. This iteration has a closed-form solution (convergence can be proved).

LPA's strengths: computationally cheap, requires no training, and especially well-suited for large-graph community detection and label diffusion. Its weakness: heavily dependent on graph structure — poor graph construction leads to poor results.

In the Networking Domain

N-gram remains active in DGA domain detection — malware-generated random domain names can be distinguished from legitimate ones by computing character-level N-gram perplexity (malicious domains have higher perplexity).

LPA has direct applications in community detection over network topologies (finding tightly connected device subgroups) and malicious IP propagation detection (known malicious IPs influence suspicion scores of connected nodes via the graph).

3.1.5 Regression and Prediction

> *Overview: From linear regression to generalized regression — the oldest method in machine learning, still important today.*

Linear Regression

$y = \mathbf{w}^\top \mathbf{x} + b + \epsilon$; solve for $\mathbf{w}$, b via least squares. The closed-form solution was covered in Chapter 1. Three commonly overlooked points:

Assumptions: the error ϵ is i.i.d. normal (zero mean). When this assumption is violated, the estimator remains unbiased but is no longer minimum-variance; in more serious cases, conclusions may be misleading. Always inspect residual plots in regression analyses.

Multicollinearity: when features are highly correlated, $\mathbf{X}^\top \mathbf{X}$ becomes near-singular and the estimate of $\mathbf{w}$ has high variance and is unstable. Ridge (L2) or PCA dimensionality reduction are the remedies.

Outliers: squared loss is extremely sensitive to outliers (a single outlier contributes loss proportional to the square of its distance). Huber loss (squared for small residuals, linear for large ones) is more robust.

Logistic Regression

Despite the name "regression," it performs classification. The model is:

> $$P(y=1|\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b), \quad \sigma(z) = \frac{1}{1+e^{-z}}$$

Training uses maximum likelihood (equivalent to minimizing cross-entropy). There is no closed-form solution; gradient descent or IRLS (Iteratively Reweighted Least Squares) is used.

The **log-odds** of logistic regression are linear: $\log\frac{P(y=1)}{P(y=0)} = \mathbf{w}^\top \mathbf{x} + b$. This gives the coefficients w_i a direct interpretation — a unit increase in feature x_i increases the log-odds by w_i, i.e., multiplies the odds by e^{w_i}.

Ridge and Lasso

The two canonical regularization methods.

Ridge (L2): minimize $\|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 + \lambda \|\mathbf{w}\|_2^2$. Analytical solution: $\mathbf{w} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$. Larger λ shrinks coefficients more.

Lasso (L1): minimize $\|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 + \lambda \|\mathbf{w}\|_1$. No analytical solution (L1 is non-differentiable); solved via coordinate descent or ISTA. Key property: **Lasso drives irrelevant feature coefficients exactly to zero** — automatic feature selection.

Why does L1 produce sparse solutions? Geometric intuition: the L2 level set is a sphere, and its tangency with the loss function's level set is almost never on a coordinate axis; the L1 level set is a diamond (with corners), whose corners lie on the coordinate axes, making it more likely that the tangency point corresponds to some coefficients being exactly zero.

ElasticNet = L1 + L2 combined, balancing Lasso's sparsity with Ridge's stability under multicollinearity.

Generalized Linear Models (GLM)

An extension of linear regression to settings where "the prediction target is not a real number." The core idea: use a "link function" g to map the linear predictor $\eta = \mathbf{w}^\top \mathbf{x}$ to distribution parameters.

Distribution	Link Function	Task
Normal	Identity $\mu = \eta$	Real-valued regression
Bernoulli	Logit $\log\frac{p}{1-p} = \eta$	Binary classification (logistic)
Poisson	Log $\log\mu = \eta$	Count prediction
Gamma	Log $\log\mu = \eta$	Positive reals (duration, cost)

Poisson regression is useful for predicting counts like network connections per day or attacks per hour — ordinary linear regression can produce negative predictions, while Poisson regression guarantees positive outputs.

In the Networking Domain

Linear/logistic regression in the networking domain primarily serves as a **baseline** — any complex model should be compared against logistic regression; failing to outperform LR signals a problem with features, data, or the task itself. Ridge/Lasso are used for interpretable traffic-prediction models (coefficients indicate which features matter). Poisson regression is used for arrival-rate modeling.

3.1.6 Time Series: DTW and DBA

> *Overview: Classic algorithms for time series similarity and clustering — still occupying a unique niche in KPI monitoring and pattern mining.*

Why Euclidean Distance Fails for Time Series

Two KPI curves may be similar in shape but shifted in time — Euclidean distance, which aligns them point-by-point, will

conclude they are very different. DTW (Dynamic Time Warping) measures shape similarity through *nonlinear temporal alignment*, allowing one segment to correspond to multiple segments of the other.

The DTW Algorithm

Given sequences $X = (x_1, ..., x_n)$ and $Y = (y_1, ..., y_m)$, DTW finds an alignment path $\{(i_k, j_k)\}$ such that:

1. It starts at $(1, 1)$ and ends at (n, m)
2. Each step goes $(i, j) \to (i+1, j)$, $(i, j+1)$, or $(i+1, j+1)$
3. The total point-to-point distance along the path is minimized

Solved via dynamic programming. Define $D(i, j)$ as the optimal alignment cost from $X_{1:i}$ to $Y_{1:j}$:

> $$D(i, j) = |x_i - y_j| + \min\{D(i-1, j), D(i, j-1), D(i-1, j-1)\}$$

Filling the table takes $O(nm)$ time and space. The DTW distance is $D(n, m)$.

```
import numpy as np

def dtw(x, y):
    n, m = len(x), len(y)
    D = np.full((n+1, m+1), np.inf)
    D[0, 0] = 0
    for i in range(1, n+1):
        for j in range(1, m+1):
            cost = abs(x[i-1] - y[j-1])
            D[i, j] = cost + min(D[i-1, j], D[i, j-1], D[i-1, j-
1])
    return D[n, m]
```

Optimizations: the **Sakoe-Chiba band** restricts $|i - j| \le w$, reducing complexity from $O(nm)$ to $O(nw)$; **FastDTW** uses multiscale recursion to approximate in $O(n)$.

DBA: Dynamic Time Warping Barycenter Averaging

With a DTW distance defined, it is natural to want to cluster time series — for example, replacing Euclidean distance with DTW inside K-means. But K-means requires computing cluster centers (averages) at each step, and the DTW average of time series is not the pointwise average (which distorts shapes). DBA is the "time-series average under DTW semantics":

1. Initialize a candidate average sequence (e.g., randomly select one member)
2. Compute DTW alignment between each member and the candidate
3. Take the pointwise average of the aligned positions to form a new candidate
4. Repeat until convergence

DBA + K-means is the classical time series clustering approach. It is widely used in KPI pattern discovery (what curve patterns recur frequently) and building libraries of anomalous patterns.

Modern Alternatives

Deep learning for time series now offers Transformer variants (Informer, Autoformer, PatchTST) and MLP-based models (N-BEATS), which generally outperform DTW on large datasets. But DTW's strengths are **no training required, works with few samples, and is fully interpretable** — the alignment path between two curves can be visualized intuitively. It remains worth using for initial analysis and small-sample scenarios.

PART TWO

3.2 Basic Neural Network Operators

> *The atomic operations of deep learning — each with its own independent optimization space.*

3.2.1 Matrix Multiply and Fundamental Operations

> *Overview: GEMM is everything — 70%+ of deep learning computation is matrix multiplication. Also covered: reductions, casts, and elementwise operations.*

GEMM

General Matrix Multiply: $\mathbf{C} = \alpha \mathbf{A} \mathbf{B} + \beta \mathbf{C}$, $\mathbf{A} \in \mathbb{R}^{M \times K}$, $\mathbf{B} \in \mathbb{R}^{K \times N}$, $\mathbf{C} \in \mathbb{R}^{M \times N}$. This is the core API of all linear algebra libraries (BLAS level 3).

The naive triple-loop has complexity O(MNK):

```
for (i = 0; i < M; i++)
    for (j = 0; j < N; j++)
        for (k = 0; k < K; k++)
            C[i][j] += A[i][k] * B[k][j];
```

The naive implementation performs extremely poorly — each access to `B[k][j]` jumps across rows causing cache misses. High-performance GEMM in practice (OpenBLAS, MKL, cuBLAS) uses multi-level blocking: the inner loops operate within L1/L2 cache, and small-tile results are accumulated into larger tiles.

In AI, matrix multiplication also has **batched GEMM**: perform multiple independent GEMMs simultaneously (e.g., one per sample in a batch) — specifically optimized to reduce kernel launch overhead.

Reduction Operations (Reduce)

Sum, maximum, or average across a given dimension. Typical examples:

`sum(x, dim=-1)`: sum along the last dimension, common in the softmax denominator
`mean(x, dim=(2,3))`: spatial average pooling
`max(x, dim=-1)`: subtract the maximum for numerical stability in softmax

GPU reductions have elegant implementations — tree-structured reduction (halving the number of active threads at each step), or warp-level primitives (`__shfl_down_sync`). A naive loop is O(n) serial; an optimized version can achieve O(\log n) parallelism.

Cast and Precision Conversion

Converting between FP32, FP16, BF16, INT8, and INT4. Seemingly simple, but full of subtleties:

FP32 → FP16: can underflow (small values become 0) or overflow (large values become inf). Loss scaling must be used during training.
BF16 ↔ FP32: BF16 is just FP32 with truncated mantissa — conversion is cheap and dynamic range is preserved.
FP ↔ INT: requires scale and zero-point; this is quantization (see section 3.6.2).

Elementwise Operations

Unary: `abs`, `neg`, `exp`, `log`, `sqrt`. Binary: `+`, `-`, `*`, `/`, `pow`, `max`.

The **performance bottleneck for these is memory bandwidth**, not compute. Performing `x * 2` on a tensor of N elements reads N values and writes N values, but only performs N multiplications — arithmetic intensity is just 0.5 FLOP/byte. A GPU Tensor Core can deliver 1000 TFLOPS, but if the operation is fully memory-bound, the actual throughput is capped by memory bandwidth.

This is why **operator fusion** has the greatest benefit for elementwise operators — fusing `relu(bn(conv(x)))` into a single kernel means the data is read once and written once, eliminating the intermediate tensor round-trips through HBM. This is the core idea behind FlashAttention and PyTorch inductor optimizations.

Comparison and Selection Operations

`greater`, `equal`, `where(cond, a, b)`, `topk`. `where` is conditional selection, commonly used in mask implementations (the causal mask in attention is just `where(mask, score, -inf)`). `topk` is critical in inference sampling (top-k sampling) and MoE routing (selecting the top-k experts).

Broadcasting

When performing elementwise operations on two tensors with different shapes, numpy/PyTorch broadcasting rules align each dimension from the right: dimensions must either be equal or one of them must be 1 (expandable).

Typical use case: adding bias to every row. If `x` has shape `[B, N, D]` and `bias` has shape `[D]`, `x + bias` automatically broadcasts bias to every batch position.

Engineering note: **broadcasting can produce unexpectedly large intermediate tensors**. `[N, 1] + [1, N]` yields `[N, N]` — memory explosions often originate here.

3.2.2 Data Movement Operators

> *Overview: Transpose, Gather, Scatter, Concat — no computation, just data shuffling, but the cost can be surprisingly high.*

Transpose

Reordering dimensions. Mathematically an identity transformation; in practice it means rearranging memory. Transposing `[B, N, H, D]` to `[B, H, N, D]` changes the physical memory address of every element.

The key optimization for transpose on GPU is **shared memory tiling** — read a tile of the matrix into on-chip shared memory, then write it back to global memory in the new order, avoiding strided memory accesses.

But the best transpose is **no transpose at all** — in many cases a "virtual transpose" suffices (changing only the shape/stride metadata without touching the data). PyTorch's `.transpose()` returns a view; a real copy is made only when contiguous memory is subsequently required (e.g., when passing to a CUDA kernel).

Reshape / View

Changes the shape without changing the total number of elements. If the original data is contiguous in memory, reshape is nearly free (only metadata changes). When it is not contiguous, a `contiguous()` call must first copy the data.

Gather / Scatter

Gather: read values from a source tensor by index. `y[i] = x[idx[i]]`. Embedding lookup is a gather — given an input sequence of token IDs, fetch the corresponding rows from the embedding table.

Scatter: the reverse — write into a target tensor by index. `y[idx[i]] = x[i]`.

Both present **irregular memory access** challenges on GPU: adjacent threads access non-adjacent memory locations, causing severe cache misses. For large embedding tables (vocabulary sizes of 100k+), gather is frequently a bottleneck during inference. Optimization approaches include bucket-sorting by index for batch access and using TLB-friendly layouts.

Concat / Split

Concatenate or split tensors. Concat requires allocating new memory and copying; Split can often be zero-copy (if the split dimension keeps each fragment contiguous).

Typical use: in MHA, `concat(head_1, ..., head_H, dim=-1)` merges multi-head outputs. But modern implementations often merge Q/K/V into a single large Linear and reshape to expose the head dimension, avoiding an explicit concat and saving one operator.

Pad

Padding the edges of a tensor with zeros (or other values). Before convolution, padding preserves output shape = input shape; variable-length sequences are padded to the same length for batching.

The cost of padding: **waste**. In a batch where the longest sequence is 2048 tokens and the shortest is 16, both are padded to 2048 — all computation on the short sequence is wasted. This is why continuous batching (see Chapter 4, section 5.13) is so valuable for LLM inference — it avoids padding waste.

The Hidden Cost of Data Movement

Rule of thumb: **in a deep learning model, data movement accounting for 20–40% of total time is common**. Unlike matrix multiplications, these costs are not conspicuous — they are scattered throughout the model but accumulate significantly. When profiling reveals that transpose, reshape, and cat operations account for a high fraction, it usually means the model architecture can be simplified.

3.2.3 Convolution and Pooling

> *Overview: The lifeblood operators of CNNs. Deep dive: im2col unfolding and Winograd acceleration.*

Definition of 2D Convolution

Input $\mathbf{X}$ with shape `[C_in, H, W]`, convolution kernel $\mathbf{W}$ with shape `[C_out, C_in, kH, kW]`, output $\mathbf{Y}$ with shape `[C_out, H', W']`:

> $$Y_{c, h, w} = \sum_{c'} \sum_{i, j} W_{c, c', i, j} \cdot X_{c', h+i, w+j}$$

(This is technically cross-correlation; in engineering it is universally called convolution.) Each output position is the inner product between the kernel and the corresponding input region.

Parameters: stride (sliding step), padding (how much to add to edges), dilation (sampling interval within the kernel), and groups (grouped convolution, where each group only sees a subset of channels).

Naive Implementation and im2col

Direct implementation requires 7 nested loops: three output dimensions (C, H, W) × three kernel dimensions (C, H, W) × summing over input C. This performs poorly on GPU.

im2col is the classic technique: *convert convolution into a matrix multiplication*. The idea:

1. Unfold each $kH \times kW \times C_{in}$ local patch from the input into a row; stack patches for all output positions into a large matrix $\mathbf{X}_{col}$ with shape `[C_in×kH×kW, H'×W']`
2. Reshape the convolution kernel to `[C_out, C_in×kH×kW]`
3. Perform the matrix multiply: $\mathbf{Y} = \mathbf{W} \cdot \mathbf{X}_{col}$, with shape `[C_out, H'×W']`
4. Reshape the result to `[C_out, H', W']`

This allows calling a GEMM kernel directly, achieving near-peak throughput. The cost is a memory expansion of $\mathbf{X}_{col}$ by a factor of `kH × kW` — each input pixel is duplicated to multiple locations.

```
import numpy as np

def im2col(X, kh, kw, stride=1):
    # X: [C, H, W]
    C, H, W = X.shape
    oh, ow = (H - kh) // stride + 1, (W - kw) // stride + 1
    cols = np.zeros((C * kh * kw, oh * ow))
    for y in range(oh):
        for x in range(ow):
            patch = X[:, y*stride:y*stride+kh,
x*stride:x*stride+kw]
            cols[:, y*ow + x] = patch.flatten()
    return cols, (oh, ow)

def conv2d_im2col(X, W):
    # X: [C, H, W], W: [C_out, C, kh, kw]
    C_out, _, kh, kw = W.shape
    cols, (oh, ow) = im2col(X, kh, kw)       # [C*kh*kw, oh*ow]
    W_mat = W.reshape(C_out, -1)              # [C_out, C*kh*kw]
    Y = W_mat @ cols                           # [C_out, oh*ow]
    return Y.reshape(C_out, oh, ow)
```

Winograd Algorithm

Offers a remarkable speedup for small kernels (3×3). The core idea: through a transformation, reduce the 9 multiplications needed by direct convolution to just 4 (per 2×2 output tile).

For the F(2, 3) transform: computing 2 outputs y_1, y_2 directly requires 6 multiplications. Winograd achieves this via:

> \mathbf{y} = \mathbf{A}^\top[(\mathbf{G}\mathbf{w}) \odot (\mathbf{B}^\top \mathbf{x})]

where \mathbf{G}, \mathbf{B}, \mathbf{A} are fixed transformation matrices (\odot is elementwise multiply). When expanded, only 4 multiplications are needed (a 33% reduction). Extending to F(2×2, 3×3): a 2×2 output tile requires 36 multiplications directly, but only 16 with Winograd (a 56% reduction).

The cost: the transforms introduce extra additions and subtractions, and the benefit only applies to small kernels (with F(6×6, 3×3), the transform overhead begins to outweigh the multiplication savings). Libraries like cuDNN and ACL automatically choose between Winograd, im2col+GEMM, and FFT based on input and kernel size.

Depthwise Separable Convolution

Standard convolution: `[C_out, C_in, kH, kW]` has C_{out} C_{in} kH kW parameters.

MobileNet introduced **depthwise separable convolution**: factorize into a depthwise step (independent kH×kW convolution per channel) plus a pointwise step (1×1 convolution for cross-channel mixing). Parameters: C_{in} kH kW + C_{in} C_{out}. For kH = kW = 3 and C = 64, this reduces parameters by roughly 9×.

The tradeoff is a slight reduction in representational capacity, but it is worthwhile in mobile/embedded settings — the foundation of MobileNet and EfficientNet.

Pooling

A downsampling operation. **Max pooling**: take the maximum over each 2×2 window, halving spatial dimensions. **Average pooling**: take the mean. **Global pooling**: compress an entire feature map into a single value (used for feature aggregation before classification).

The roles of pooling:

1. Reduce computation (H, W halved)
2. Expand the receptive field (the next convolutional layer sees a wider range)
3. Approximate translation invariance

Modern architectures sometimes replace pooling with stride-2 convolutions, making the downsampling learnable.

3.2.4 Activation Functions

> *Overview: The source of "nonlinearity" in neural networks — without them, no matter how deep the network, it is equivalent to a single linear layer.*

Why Nonlinearity Is Needed

Composing linear layers still yields a linear transformation: $W_2(W_1 x + b_1) + b_2 = (W_2 W_1) x + (W_2 b_1 + b_2)$. The entire network collapses to a single linear transform — insufficient expressive power. Activation functions introduce nonlinearity, allowing the network to approximate any continuous function (the universal approximation theorem).

Sigmoid

$\sigma(x) = 1/(1 + e^{-x})$. Output is in (0, 1). The oldest activation, still used in logistic regression and LSTM gates.

Problem: **vanishing gradients**. $\sigma'(x) = \sigma(x)(1 - \sigma(x))$, with a maximum of 0.25. During backpropagation through a deep network, gradients are multiplied at each layer and quickly vanish. Sigmoid is not used in hidden layers of deep CNNs or Transformers.

Tanh

$\tanh(x) = (e^x - e^{-x})/(e^x + e^{-x})$. Output is in (-1, 1); a centered version of sigmoid with maximum gradient value of 1, slightly better. Historically common in RNNs.

ReLU

$\text{ReLU}(x) = \max(0, x)$. AlexNet in 2012 made ReLU mainstream. Advantages: extremely simple to compute; derivative is constant 1 on the positive half (solves vanishing gradients).

Disadvantage: **Dead ReLU** — once a neuron's output stays at 0, its gradient is also 0 and it never updates again. Particularly prone to occurring with large learning rates.

ReLU Variants

LeakyReLU: $\max(\alpha x, x)$, $\alpha = 0.01$. Small slope on the negative side prevents dead neurons.
PReLU: α becomes a learnable parameter.
ELU: uses $e^x - 1$ on the negative side, smooth.

GELU and SiLU/Swish

Two activations popular in the Transformer era:

GELU: $x \cdot \Phi(x)$, where Φ is the standard normal CDF. Equivalent to "dropout with probability proportional to input magnitude." Used in BERT, GPT, and related models. In practice, an approximation is used: $0.5 x (1 + \tanh(\sqrt{2/\pi}(x + 0.044715 x^3)))$.

SiLU / Swish: $x \cdot \sigma(x)$. Simpler. Used in the LLaMA family (specifically SwiGLU, discussed below).

GELU and SiLU look similar in shape — both are smooth versions of ReLU with a slight dip on the negative side. Empirically they perform slightly better than ReLU on large models (around 1%).

SwiGLU / GeGLU

Used in the FFN of LLaMA and Qwen. Not a pure activation function but a "gated linear unit" combined with activation:

> $$\text{SwiGLU}(x) = \text{SiLU}(W_1 x) \odot (W_2 x)$$

Two parallel linear transformations: one passed through an activation and one not, then elementwise multiplied. Equivalent to learning a multiplicative gate at each position. The cost is that parameters change from `[d, 4d] + [4d, d]` to `[d, 8d/3] + [d, 8d/3] + [8d/3, d]` (to keep total parameter count unchanged, the intermediate dimension is typically reduced from 4d to 8d/3).

Softmax

Strictly not an "activation function," but plays a similar role — converting a vector into a probability distribution:

> $$\text{softmax}(\mathbf{x})_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Properties: outputs sum to 1, differentiable everywhere, monotone. Used at classification output layers and for attention weights.

A numerically stable implementation must subtract the maximum value (see Chapter 1, section 2.8.1).

3.2.5 Normalization

> *Overview: BatchNorm, LayerNorm, RMSNorm — pillars of training stability.*

Why Normalization Is Needed

Training deep networks suffers from **internal covariate shift**: small changes to earlier layer parameters cause the input distribution of later layers to shift dramatically, destabilizing training. Normalization forces each layer's input back toward a standard distribution, making training more stable and allowing larger learning rates.

BatchNorm

For each feature (along the channel dimension) across the batch, subtract the batch mean and divide by the batch standard deviation, then rescale and shift with learnable parameters γ, β:

> $$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$$

During training, μ_B and σ_B^2 are computed from the current batch; during inference, running mean/var accumulated

during training (or from the final batch) is used.

BatchNorm was revolutionary for CNN training — introduced in 2015, it instantly became the standard, making it possible to train very deep networks like ResNet with 100+ layers.

Drawbacks:

- Sensitive to batch size: statistics are unstable with small batches (recommended ≥ 16)
- Different behavior during training vs. inference: a common source of subtle bugs
- Not suitable for RNNs/Transformers (variable sequence lengths make batch-dimension statistics meaningless)

LayerNorm

Normalizes over the *feature dimension of each individual sample* (not the batch dimension):

> $$\mu = \frac{1}{D}\sum_i x_i, \quad \sigma^2 = \frac{1}{D}\sum_i (x_i - \mu)^2$$

Each sample is normalized independently with no dependence on batch. Transformers use LayerNorm throughout.

Behavior is identical during training and inference, and it is insensitive to batch size. Has become standard in NLP.

RMSNorm

A simplified version of LayerNorm: removes mean centering (deemed unimportant) and only normalizes magnitude:

> $$\text{RMS}(x) = \sqrt{\frac{1}{D}\sum_i x_i^2}, \quad y_i = \frac{x_i}{\text{RMS}(x)} \cdot \gamma_i$$

Eliminates the mean computation and the β bias parameter. LLaMA, Qwen, and DeepSeek all use RMSNorm — *slightly less computation, no difference in accuracy*: exactly the kind of trade-off worth making in LLMs.

GroupNorm / InstanceNorm

Variants of BatchNorm. GroupNorm partitions channels into groups and normalizes within each group (no batch dependence). InstanceNorm normalizes each sample and channel independently (commonly used in image style transfer). BatchNorm, GroupNorm, and InstanceNorm form a continuum along the axis of channel-dimension granularity.

Pre-Norm vs. Post-Norm

The placement of normalization in Transformers has been debated.

Post-Norm (original Transformer): `LN(x + Sublayer(x))`. Less stable; training large models requires warmup.

Pre-Norm (GPT series): `x + Sublayer(LN(x))`. The residual connection links input and output directly, improving gradient flow and making large model training more stable.

All modern large models use Pre-Norm.

PART THREE

3.3 Classic Deep Learning Models

The dominant architectures before Transformers — each still holds independent value in its own domain.

3.3.1 CNN

> *Role: The cornerstone of computer vision. Understanding CNN means understanding the core concept of "inductive bias."*

The Inductive Bias of CNNs

The secret behind CNN's success lies in two assumptions:

1. **Locality**: Relevant features are typically spatially close (the meaning of a pixel is related to its neighbors)
2. **Translation equivariance**: Features are recognized the same way regardless of where they appear in the image (a cat is a cat whether it's in the top-left or bottom-right)

Convolution neatly encodes both points — the kernel is local and shared across the entire image. Parameter efficiency is orders of magnitude higher than a fully connected MLP, and the training data requirement is much lower.

Evolution of Classic Architectures

LeNet (1998): The ancestor of CNNs, designed for handwriting recognition. A few convolutional layers + pooling + fully connected layers.

AlexNet (2012): The landmark that revived deep learning. 8 layers, using ReLU, Dropout, data augmentation, and GPU training. Reduced ImageNet top-5 error from 26% to 15%, shaking the entire CV community.

VGG (2014): Deepened the network by repeatedly stacking small 3×3 convolutional kernels. Proved that "depth matters." 16–19 layers.

Inception / GoogLeNet (2014): Parallel multi-scale convolutions (1×1, 3×3, 5×5). Introduced 1×1 convolutions for dimensionality reduction (fewer parameters).

ResNet (2015): Residual connections. This is the pivotal innovation in CNN history.

Residual Connections: The Secret to Training Deep Networks

After VGG, researchers found that continuing to add depth hurt rather than helped — not due to overfitting (training accuracy also dropped), but due to optimization difficulty. ResNet proposed: instead of learning F(x), learn F(x) - x, and output x + F(x).

```
flowchart LR X[x] --> F[Conv-BN-ReLU-Conv-BN] X --> ADD((+)) F -
-> ADD ADD --> RELU[ReLU] RELU --> Y[y]
```

Why does it work? Two perspectives:

Gradient perspective: Gradients along the residual path are passed directly to the input, bypassing the nonlinearities and multiplications of intermediate layers — alleviating vanishing gradients.

Identity mapping perspective: If the optimal behavior for a layer is the identity mapping (no transformation), the residual version only needs F = 0, which is far easier than learning a full identity F(x) = x.

ResNet enabled training 152-layer and even 1000-layer networks, completely unlocking depth.

More Recent Architectures

DenseNet (2016): Each layer is densely connected to all previous layers.

EfficientNet (2019): Finds the optimal scaling ratio across depth, width, and resolution.

ConvNeXt (2022): Redesigns CNNs using Transformer design philosophy (large kernels, GELU, LayerNorm), demonstrating that CNNs and Transformers each have their own strengths.

Where CNNs Stand Today

In CV, Transformers (ViT) already dominate large models, but CNNs still lead for small models and edge deployment — their inductive bias wins on small data, and their regular computation graphs are more amenable to hardware optimization. In embedded systems, industrial control, and medical imaging, CNNs remain the first choice.

3.3.2 RNN / LSTM / GRU

> *Role: The dominant sequence modeling architectures before Transformers. Deep dive into LSTM gating mechanisms.*

Vanilla RNN

At each time step, the current input $\mathbf{x}_t$ and the previous hidden state $\mathbf{h}_{t-1}$ are combined into a new hidden state:

> $$\mathbf{h}_t = \tanh(W_{xh} \mathbf{x}_t + W_{hh} \mathbf{h}_{t-1} + \mathbf{b})$$

Unrolled over a time sequence:

```
flowchart LR X1[x_1] --> H1[h_1] X2[x_2] --> H2[h_2] X3[x_3] --> H3[h_3] H1 --> H2 H2 --> H3 H1 --> Y1[y_1] H2 --> Y2[y_2] H3 --> Y3[y_3]
```

The same weight W_{hh} is reused at every time step. This allows RNNs to handle sequences of arbitrary length, with parameter count that does not grow with sequence length.

The problem: **vanishing/exploding gradients**. During backpropagation, gradients pass through T matrix multiplications by W_{hh} (where T is the sequence length). If the eigenvalues of W_{hh} are less than 1, gradients decay exponentially; greater than 1, they grow exponentially. This causes vanilla RNNs to remember only very short histories.

LSTM Gating

Long Short-Term Memory (1997) solves this problem through **explicit memory cells and gating mechanisms**. The core innovation is introducing a **cell state** $\mathbf{c}_t$, which provides a "highway" for gradients to travel long distances.

Three gates govern how information flows:

> $\mathbf{f}_t = \sigma(W_f [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad (\text{forget gate})$

> $\mathbf{i}_t = \sigma(W_i [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad (\text{input gate})$

> $\mathbf{o}_t = \sigma(W_o [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad (\text{output gate})$

> $\tilde{\mathbf{c}}_t = \tanh(W_c [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) \quad (\text{candidate cell})$

> $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$

> $\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$

Intuition:

Forget gate decides how much information to retain from the old cell state (near 1: retain; near 0: forget)
Input gate decides how much new information to accept
Output gate decides how much to expose from the cell state to the outside

The key gradient channel is `c_t = f_t * c_{t-1} + ...` — when the forget gate stays close to 1 for a long time, `c_t` retains long history through addition, and gradients during

backpropagation decay almost not at all. This is the meaning of "long short-term memory."

GRU

Gated Recurrent Unit (2014) simplifies LSTM: it merges the cell state and hidden state, and combines the forget and input gates into an "update gate." 25% fewer parameters, with empirical performance comparable to LSTM.

> $$\mathbf{z}_t = \sigma(W_z [\mathbf{h}_{t-1}, \mathbf{x}_t]) \quad (\text{update gate})$$

> $$\mathbf{r}_t = \sigma(W_r [\mathbf{h}_{t-1}, \mathbf{x}_t]) \quad (\text{reset gate})$$

> $$\tilde{\mathbf{h}}_t = \tanh(W [\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t])$$

> $$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$$

Bi-RNN and Deep RNN

Bidirectional RNN: Runs one pass forward and one pass backward; the representation at each position is a concatenation from both directions. Well-suited for understanding tasks (each word's representation takes context from both sides into account).

Multi-layer RNN: The output of one layer serves as the input to the next, stacked several layers deep. Greater expressive power.

Why RNNs Gave Way to Transformers

The fundamental limitation of RNNs is that they **cannot be parallelized** — time steps must be computed sequentially. GPU throughput cannot be fully utilized. In 2017, Transformers replaced the recurrence with self-attention entirely, enabling

full parallelism during training with even better results. Today LSTM/GRU in industry mainly survives in: small-model scenarios (embedded time-series forecasting), classic baselines, and research.

The Revival of Mamba

In 2023, Mamba (selective SSM) brought "RNN-like architectures" back into the spotlight — training can use parallel scanning, while inference is O(1) per step. It challenges Transformers in long-context scenarios. The spirit of RNNs is not dead; it just needs cleverer design (see Chapter 4, section 5.6).

3.3.3 GNN and GAT

> *Role: Graph Neural Networks — the mainstream paradigm for processing non-Euclidean data.*

Why Graph Data Needs Specialized Models

Characteristics of graphs: variable number of nodes, variable number of neighbors, no fixed ordering. The "locality + translation equivariance" assumption of CNNs does not hold (there is no regular grid on a graph). RNNs don't work either (graphs have no natural sequential order). GNNs are designed specifically for graph data.

The Message-Passing Paradigm

Almost all mainstream GNNs operate within the "message passing" framework. Each layer performs two steps:

1. **Aggregation**: Each node collects messages from its neighbors
2. **Update**: Updates its representation based on the aggregated result and its current representation

> $$\mathbf{h}_v^{(l+1)} = \text{UPDATE}\left(\mathbf{h}_v^{(l)}, \text{AGG}\left(\{\mathbf{h}_u^{(l)}: u \in \mathcal{N}(v)\}\right)\right)$$

$\mathcal{N}(v)$ is the set of neighbors of v. Different GNNs differ in how AGG and UPDATE are implemented.

GCN

Graph Convolutional Network (2017). The aggregation is a normalized weighted average:

> $$\mathbf{h}_v^{(l+1)} = \sigma\left(\sum_{u \in \mathcal{N}(v) \cup \{v\}} \frac{1}{\sqrt{\deg(u) \deg(v)}} W^{(l)} \mathbf{h}_u^{(l)}\right)$$

The normalization factor $1/\sqrt{d_u d_v}$ prevents high-degree nodes from dominating the aggregation. The derivation comes from a spectral-domain approximation of the graph Laplacian — the details are complex, but the idea is simple: take a weighted average of neighbors, then apply a linear transformation.

GraphSAGE

GCN requires computing the Laplacian over the entire graph, which does not scale to large graphs. GraphSAGE instead uses **neighbor sampling**: each node randomly samples k neighbors to aggregate from. This enables mini-batch training and supports very large graphs (billions of nodes).

GAT: Graph Attention

In GCN, neighbor weights are determined by graph structure (degree normalization), and cannot express "this neighbor is more important." GAT introduces attention:

> $$\alpha_{vu} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^\top [W \mathbf{h}_v \| W \mathbf{h}_u]))}{\sum_{k \in \mathcal{N}(v)} \exp(\text{LeakyReLU}(\mathbf{a}^\top [W \mathbf{h}_v \| W \mathbf{h}_k]))}$$

> $$\mathbf{h}_v^{(l+1)} = \sigma\left(\sum_{u \in \mathcal{N}(v)} \alpha_{vu} W \mathbf{h}_u\right)$$

Each node computes attention weights over its neighbors and aggregates with those weights. $\|$ denotes concatenation and $\mathbf{a}$ is a learnable vector. Multi-head attention is supported (just like Transformers).

GAT's advantages lie in heterogeneous graphs (where neighbor importance varies) and interpretability (weight visualization).

Applications

Node classification (predicting a category for each node), link prediction (predicting whether an edge should exist between two nodes), graph classification (properties of an entire graph), graph generation. Used in: social network recommendations, molecular property prediction, knowledge graph completion, network topology analysis.

Over-Smoothing

GNNs cannot be stacked as deep as CNNs — after a few layers, all node representations converge (because each layer averages over neighbors). In practice, 2–3 layers is common; going deeper requires specialized designs (residual connections, PairNorm, etc.).

3.3.4 GAN and VAE

> *Role: The two major generative model paradigms — adversarial vs. variational. Now superseded by diffusion models, but their ideas live on.*

What Generative Models Do

Given data samples $\{x_i\} \sim p_{\text{data}}$, learn a model p_θ such that samples drawn from p_θ "look like" real

data. Two major challenges:

1. The distribution of high-dimensional data (images, audio) is extremely complex
2. Directly maximizing likelihood $\max \prod p_\theta(x_i)$ requires a normalizing constant that is intractable in high dimensions

GAN and VAE each sidestep these problems from completely different directions.

VAE: Variational Inference

Assume the data is generated by a latent variable z: $z \sim p(z) = \mathcal{N}(0, I)$, $x \sim p_\theta(x|z)$. We want to maximize the likelihood $\log p_\theta(x) = \log \int p_\theta(x|z) p(z) dz$, but the integral is intractable.

VAE approximates it using a **variational lower bound** (ELBO):

> $\log p_\theta(x) \ge \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - D_{\text{KL}}(q_\phi(z|x) \| p(z))$

Here $q_\phi(z|x)$ is the encoder (maps x to a distribution over z), and $p_\theta(x|z)$ is the decoder (reconstructs x from z). Both are neural networks.

Training jointly optimizes two terms: the reconstruction term (the decoder can recover x from z) and the KL term (q stays close to the prior $p(z)$). Training uses the **reparameterization trick** : $z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon$, $\epsilon \sim \mathcal{N}(0, I)$ — making sampling differentiable.

VAE advantages: explicit latent space (supports interpolation and controllable generation), stable training. Disadvantage: generated samples tend to be blurry (reconstruction loss is pixel-level MSE, which averages across outcomes).

GAN: Adversarial Game

An entirely different approach. Two networks compete:

Generator G : takes noise z as input, outputs fake sample G(z)
Discriminator D: takes a sample as input, judges whether it is real or fake (outputs a 0–1 probability)

Objective function:

> $\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))]$

D wants to classify real samples as real and fake samples as fake; G wants to fool D. The two compete until G generates samples that D cannot distinguish from real ones — at that point G has learned p_{data}.

GAN advantages: sharp samples with high visual quality. Disadvantages: **extremely unstable training** (achieving game equilibrium is hard), mode collapse (the generator produces only a few types of samples), no explicit likelihood.

Notable improvements: **WGAN** replaces JS divergence with Wasserstein distance, resulting in much more stable training; **StyleGAN** (NVIDIA) achieves fine-grained style control and was the primary technology behind early deepfakes.

The Rise of Diffusion Models

From 2020 onward, **diffusion models** rapidly replaced GAN/VAE as the mainstream approach to image generation.

Core idea: progressively add noise to data until it becomes pure noise (forward process, fixed), then train a neural network to denoise (reverse process, learned). During generation, start from pure noise and iteratively denoise over T steps to obtain a sample.

Advantages: extremely stable training (a pure denoising objective, no adversarial component), high sample quality, controllable (guidance, conditioning). DALL-E 2, Stable Diffusion, Imagen, and Sora all use diffusion.

Trade-off: slow sampling (requires 20–50 steps, each a full forward pass). Acceleration directions: DDIM, DPM-Solver,

Consistency Models (1-step generation).

PART FOUR

3.4 Data & Feature Engineering

"Garbage in, garbage out" — data quality sets the ceiling on model performance.

3.4.1 Sample Scoring, Generation, and Augmentation

> *Role: What to do when data is scarce — synthetic samples, augmentation, and active learning.*

Data Augmentation

Multiple "new" samples are generated from a single real sample through transformations, expanding the training set. Domain-specific approaches:

Domain	Common Augmentations
Image	Flip, rotation, crop, color jitter, Cutout, Mixup, CutMix, AutoAugment
Text	Synonym replacement, back-translation, random insertion/deletion, EDA, Back-translation
Audio	Time stretching, pitch shifting, background noise injection, SpecAugment (spectral masking)

Time Series	Time warping, noise injection, window slicing, DBA synthesis

Mixup is a particularly interesting augmentation: both samples and labels are interpolated as convex combinations, $x' = \lambda x_1 + (1-\lambda) x_2$, $y' = \lambda y_1 + (1-\lambda) y_2$. The model learns that "a linearly interpolated sample should have a linearly interpolated label" — the regularization effect is significant and broadly benefits classification tasks.

Sample Scoring

Not all samples are equally important. Scoring methods:

Difficulty: How large is the current model's loss on this sample (loss-based)
Uncertainty: The model's prediction confidence (entropy, variance)
Representativeness: How different this sample is from other samples
Influence: The effect on test-set performance when this sample is removed from training (influence function)

Curriculum learning: Train on easy samples first, then gradually introduce harder ones — similar to how humans learn. For certain tasks this can significantly accelerate convergence.

Active learning: When labeling budget is limited, use the model to select the most worthwhile samples to label (typically those with the highest uncertainty) for human annotation. Reduces labeling cost.

Synthetic Data

Two main approaches:

Rule-based generation: Generate samples using domain rules (e.g., synthesize adversarial network traffic from known malicious attack rules, or synthesize autonomous driving scenarios using a physics engine). Controllable, but may be too "clean" to match the real distribution.

Model-based generation: Use generative models (GAN, diffusion, LLM) to produce synthetic samples. Particularly popular in the LLM era:

Self-Instruct: Have a strong model generate instructions + responses, which serve as training data for a weaker model
Distillation: A small model learns from a large model; the large model's outputs serve as "synthetic labels"
Rejection Sampling: The large model generates multiple answers; a scoring function selects the good ones for the training set

Risk of synthetic data: **model collapse** — if successive generations of models are trained on data synthesized by the previous generation, the distribution drifts over generations and ultimately loses diversity. Mixing in real data is essential for safety.

Handling Class Imbalance

In extreme cases (1% positive, 99% negative), a model that simply predicts all negatives achieves 99% accuracy — but is completely useless. Remedies:

Resampling: Oversample the minority class, undersample the majority class
SMOTE: Generate new samples by interpolating between minority-class examples
Class weights: Assign higher loss weight to the minority class
Focal Loss: $-\alpha(1-p)^\gamma \log(p)$ — down-weights easy samples (high p) so the model focuses on hard ones

3.4.2 Preprocessing and Feature Construction

> *Role: Cleaning, normalization, encoding — in traditional ML, feature engineering often matters more than model selection.*

Data Cleaning

Real-world data is messy: missing values, outliers, type inconsistencies, duplicates. Cleaning is unglamorous work but must be done:

Missing values: Drop rows, fill with mean/median, predict with a model, or add a "is missing" flag feature
Outliers: Detect using IQR ($Q_1 - 1.5 \cdot \text{IQR}, Q_3 + 1.5 \cdot \text{IQR}$) or Z-score, then drop, clip, or flag
Duplicates: Exact or approximate deduplication

Numerical Feature Processing

Normalization: Scale to [0, 1] — $x' = (x - \min)/(\max - \min)$. Sensitive to outliers.

Standardization: Subtract mean and divide by standard deviation — $x' = (x - \mu)/\sigma$. More robust; preferred for normally distributed data.

Log transformation: For heavy-tailed or skewed data, apply $\log(1+x)$ before further processing. Network traffic packet sizes, file sizes, and costs are typical heavy-tailed distributions.

Binning: Discretize a continuous variable into intervals. Can be equal-width, equal-frequency, or decision-tree-based. Benefits include handling nonlinear relationships, robustness to outliers, and strong interpretability.

Categorical Feature Encoding

One-Hot: One binary column per category. Simple, but causes dimensionality explosion when there are many categories.

Label Encoding: One integer per category. Suitable for tree models (trees can handle categorical features), but not for linear models (which would incorrectly assume a numeric ordering).

Target Encoding: Map each category to the mean of the target variable for samples in that category. Information-rich but

carries leakage risk — must be computed on the training set before being applied to validation/test sets.

Embedding: Map categories to low-dimensional continuous vectors (learned through a network). The standard approach for handling categorical features in deep learning.

Temporal Features

Using raw timestamps directly tends to perform poorly. Common expansions: hour of day, day of week, day of month, month, quarter, is_weekend, holiday flag, days since a specific event, etc.

Cyclic encoding: Hour is not a linear quantity (11 PM and midnight are close but numerically far apart). Use $\sin(2\pi h/24)$, $\cos(2\pi h/24)$ to encode and preserve periodicity.

Text Features

Traditional: TF-IDF, N-grams, bag of words. Modern: use pretrained models (BERT, Sentence-BERT) to extract embeddings directly as features.

Feature Crosses

Multiply or combine two features. Tree models handle this automatically (deep trees naturally express feature interactions), but linear models require explicit feature engineering. Factorization Machines (FM) and FFM in recommender systems are automated feature crossing methods.

3.4.3 Dimensionality Reduction Algorithms

> *Role: The curse of dimensionality, and the remedies — PCA, t-SNE, UMAP.*

PCA

The math was covered in Section 2.3.4 of Chapter 1. Engineering notes:

- **Must standardize first** — PCA is variance-sensitive; without standardization, features with larger ranges dominate
- Choosing how many components: look at cumulative explained variance ratio (typically retain 95% of variance)
- PCA is **linear** dimensionality reduction — it cannot handle nonlinear structure (e.g., a Swiss roll)

Variants: Kernel PCA uses the kernel trick for nonlinear structure; Incremental PCA supports streaming data; Sparse PCA produces sparse principal components for better interpretability.

t-SNE

t-distributed Stochastic Neighbor Embedding (2008). Dimensionality reduction designed specifically for **visualization** to 2D/3D.

Idea: compute pairwise "similarity" in high-dimensional space (Gaussian kernel), compute similarities in low-dimensional space using a t-distribution (heavy-tailed), and minimize the KL divergence between the two. High-dimensional neighbors remain neighbors in low dimensions.

t-SNE produces stunning results — the 10 digit classes of MNIST separate cleanly in 2D. But there are pitfalls:

- The `perplexity` hyperparameter has a large effect (typical range 5–50); different values yield different results
- Global structure is not preserved (distances between clusters are meaningless)
- Results differ across runs (random initialization)
- Only for visualization; cannot be used as features for downstream models

UMAP

Uniform Manifold Approximation and Projection (2018). Faster than t-SNE with comparable or better quality, and better global structure preservation. Based on Riemannian geometry and algebraic topology (sounds intimidating, but in practice it's just a library call).

The modern first choice for visualization. Also supports **transform** (projecting new data into an already-trained space), which t-SNE strictly does not.

Autoencoder Dimensionality Reduction

The "encoder" part of a neural network: input x → low-dimensional latent vector z → decode back to $\hat{x}$. Minimize reconstruction loss $\|x - \hat{x}\|^2$. z is the reduced representation.

Stronger than PCA for nonlinear structure. VAE is the probabilistic version of an autoencoder. But training is time-consuming; for small to medium scale, PCA is often sufficient.

When to Use Which

PCA: As features for downstream models, large datasets, speed required, interpretability needed
t-SNE / UMAP: Data exploration, visualizing clusters
Autoencoder: Nonlinear structure, end-to-end scenarios

3.4.4 Feature Selection

> *Role: Picking the truly useful features from many — reduce overfitting, improve interpretability, lower inference cost.*

Three Categories of Methods

Filter methods: Evaluate each feature independently, without involving the model. Fast.

- Variance threshold: Drop features with very low variance (nearly constant)
- Correlation coefficient: Drop features with low correlation to the target (linear relationship)
- Mutual information: Captures "correlation" for nonlinear relationships
- Chi-squared: Categorical features vs. categorical targets

Wrapper methods: Use a model to evaluate the performance of different feature subsets, searching greedily.

- **Forward selection**: Start from empty; each round add the feature that gives the largest performance gain
- **Backward elimination**: Start from all features; each round remove the feature with the smallest impact on performance
- **RFE** (Recursive Feature Elimination): Train a model, inspect coefficients/importance, remove the least important feature, retrain, and repeat

Embedded methods: Models with built-in feature selection capability.

- Lasso regression: L1 regularization drives irrelevant feature coefficients to zero
- Tree model feature importance

Pitfalls of Feature Importance

Tree model feature importance has multiple definitions:

Gain: Total information gain from all splits using this feature
Split count: Number of times this feature is used to split
Permutation importance: How much model performance drops after shuffling this feature

Different definitions give different rankings. More importantly: **feature importance ≠ causality**. Highly correlated features may share importance (they are predictive jointly); removing one causes its importance to be redistributed to others. A feature may also be purely a leakage signal (e.g., "hospital ID" predicting disease — not because it has a causal relationship,

but because different hospitals see different disease distributions).

SHAP

SHapley Additive exPlanations (2017). Uses Shapley values from game theory to give a fair attribution of each feature's contribution to a *single prediction*. More fine-grained than global feature importance — you can see "for this sample, how much did each feature contribute."

```
import shap, xgboost

model = xgboost.XGBClassifier().fit(X_train, y_train)
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)

# Single-sample explanation
shap.force_plot(explainer.expected_value, shap_values[0],
X_test.iloc[0])

# Global feature importance (by SHAP values)
shap.summary_plot(shap_values, X_test)
```

Explainable AI in industry is largely built on SHAP. It is standard in scenarios with legal explainability requirements, such as financial risk control and medical diagnosis.

PART FIVE

3.5 Training Optimization Algorithms

> *From supervised to self-supervised to alignment — the core techniques driving the evolution of model capabilities.*

3.5.1 Supervised Learning

> *Overview: The most classical paradigm — given labeled data, train a model to fit it. The details hold a wealth of knowledge.*

The Basic Loop

Supervised learning is the foundational paradigm of deep learning. Given data (x_i, y_i), a model f_θ, a loss ℓ, and an optimizer update:

> $$\theta_{t+1} = \theta_t - \eta \nabla_\theta \sum_i \ell(f_\theta(x_i), y_i)$$

Choosing a Loss Function

Task	Loss
Regression	MSE, MAE, Huber
Binary classification	Binary cross-entropy
Multi-class classification	Categorical cross-entropy
Multi-label classification	Binary CE per label
Sequence generation	Token-level cross-entropy
Object detection	Classification loss + bbox regression + IoU
Segmentation	Pixel-wise CE + Dice loss

Overfitting and Regularization

The primary enemy of supervised learning is overfitting — perfect on the training set, poor on the test set. Countermeasures stack up in layers:

L2 regularization (weight decay): $\ell + \lambda \|\theta\|^2$. Keeps parameters small and decision boundaries smooth.
Dropout: Randomly drops a fraction of neurons (e.g., 50%) during training, forcing the model not to rely on any single neuron. At inference time all neurons are kept but scaled by the dropout probability.
Data augmentation: Increases data diversity.
Early stopping: Stop training when validation performance stops improving.
Label smoothing: Replace hard labels [0, 1, 0] with [0.05, 0.9, 0.05] to prevent the model from becoming overconfident.

Train / Validation / Test Split

The classic 70/15/15 or 80/10/10 split. Key points:

- **Look at the test set only once at the very end** — repeatedly tuning on the test set constitutes data leakage
- **Stratified sampling**: preserve class proportions
- **Time-series splitting**: split by time, not randomly (using future data for training is cheating)
- **Group-based splitting**: data from the same user or device must not appear in both train and test sets (information leakage)

Cross-Validation

When data is scarce, use K-fold cross-validation: divide the data into K folds, train on K-1 folds and validate on the remaining one, rotate K times and average the results. This eliminates the randomness of a single split.

For time series, use **time series split**: progressively expand the training window.

Training Diagnostics

The loss curve reveals a great deal:

- Training loss not decreasing: learning rate too small, vanishing gradients, insufficient model capacity, or data issues
- Training loss oscillating or exploding: learning rate too large or numerical instability
- Training loss decreasing but validation loss not: overfitting
- Both decreasing but with a large gap: overfitting, but manageable
- Neither decreasing: fundamental problem with the model or data

3.5.2 Self-Supervised Pre-Training

> *Overview: The foundational paradigm for LLMs and large foundation models — "creating" labels from unlabeled data.*

Core Idea

Labeled data is expensive — but unlabeled data (web pages, code, video) is virtually unlimited. Self-supervised learning designs an "auxiliary task" whose labels can be *automatically generated from the data itself*. The representations learned on this auxiliary task are often highly useful for downstream tasks.

Pre-Training Objectives for NLP

MLM (Masked Language Modeling, BERT): Randomly mask 15% of tokens and have the model predict the masked ones. The label is the original word. Encoder-only architecture.

CLM (Causal Language Modeling, GPT): Given the first t tokens, predict the (t+1)-th. The label is the next word. Decoder-only architecture.

Span Corruption (T5): Mask a contiguous span of tokens and have the model generate the masked span.

The success of the GPT family demonstrates that CLM scales best — more data and larger models yield stronger capabilities, with zero-shot/few-shot transfer to a wide range of tasks.

Pre-Training for CV

Contrastive learning (SimCLR, MoCo): Apply two different augmentations (random crop, color jitter, etc.) to the same image, and train the representations of these two views to be similar while being distant from representations of other images.

MAE (Masked Autoencoders): Inspired by MLM, mask 75% of image patches and have the model reconstruct them. Simple yet remarkably effective.

DINO (self-distillation): A student network mimics the output of a teacher network without requiring explicit negative samples.

Why It Works

A central finding of self-supervised pre-training: **representation learning can emerge from "prediction" tasks**. In order to predict masked words or reconstruct images, the model must learn grammar, semantics, and object concepts — this knowledge is a byproduct of "solving the task," but it also happens to be exactly what downstream tasks need.

Scaling Laws

OpenAI and DeepMind observed that model capability improves as a power law of **data volume × model size × compute**, without saturating for a very long time. This is the scaling law:

> $$L(N, D) \approx \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{D_c}{D}\right)^{\alpha_D}$$

Loss as a function of parameter count N and data volume D follows power-law decay. This turns large model training into an "engineering problem" (just add compute to improve) rather than a "research problem" (requiring algorithmic breakthroughs).

Chinchilla (2022) refined this: for a given compute budget, the optimal allocation is to scale data and parameter counts in

roughly equal proportion (rather than simply stacking more parameters as was previously done).

3.5.3 Fine-Tuning and Alignment

> *Overview: From pre-trained model to usable product — the complete pipeline of SFT, DPO, and RLHF.*

This section goes deep; these are the most critical training techniques of the LLM era.

Three-Stage Training Pipeline

Modern LLMs go through three steps from pre-training to release:

```
flowchart LR PT[Pretrain
Massive text CLM] --> SFT[SFT
Instruction-response pairs] SFT --> RM[Reward Model
Preference data] RM --> RLHF[RLHF / DPO
Align to human preferences] RLHF --> REL[Released Model]
```

Pre-training falls under Section 3.5.2 and produces the **base model** — it holds vast world knowledge and can complete text, but does not know how to "converse." The subsequent three steps transform it into a conversational assistant.

SFT: Supervised Fine-Tuning

Train on human-annotated "instruction → ideal response" pairs. The loss is CLM — the model learns to "generate the response following the instruction" from this data.

```
# SFT data format (simplified)
{
    "instruction": "Explain what gradient descent is",
    "response": "Gradient descent is an optimization algorithm
that computes the gradient of the loss function..."
}
```

```
# Construct training sequence (with system prompt and role
markers)
prompt = f"<|system|>You are a helpful assistant<|user|>
{instruction}<|assistant|>"
full   = prompt + response + "<|end|>"

# Loss is computed only on the response portion (prompt portion
is loss-masked)
labels = [-100] * len(tokenize(prompt)) + tokenize(response + "
<|end|>")
```

The amount of SFT data is crucial but does not need to be enormous — tens of thousands of high-quality instruction samples are enough to turn a base model into a usable conversational model. OpenAI's InstructGPT paper revealed the approximate scale.

The limitation of SFT: it only teaches the model "this kind of answer is correct," not "this kind of answer is wrong." What the model learns is imitation, not preference ranking. This is where RLHF or DPO comes in.

RLHF: The Classic Approach

Reinforcement Learning from Human Feedback. Three steps:

Step 1: Collect preference data. For the same prompt, have the SFT model generate multiple responses and have humans annotate which is better — producing (x, y_{\text{win}}, y_{\text{lose}}) triples.

Step 2: Train a Reward Model. A neural network r_\phi (typically initialized from the SFT model) learns to "score good responses high and bad responses low." The loss uses the Bradley-Terry preference model:

> \mathcal{L}_{\text{RM}} = -\log \sigma(r_\phi(x, y_{\text{win}}) - r_\phi(x, y_{\text{lose}}))

Step 3: Optimize the policy with PPO. Use the reward model's scores as reinforcement signals. The policy \pi_\theta generates

responses, receives rewards, and updates θ to increase the probability of high-reward responses.

To prevent the policy from drifting too far from the original SFT model (maintaining fluency and avoiding reward-hacking tricks), a KL penalty is added:

> $$\mathcal{L}_{\text{PPO}} = -\mathbb{E}[r_\phi(x, y) - \beta \cdot D_{\text{KL}}(\pi_\theta \| \pi_{\text{SFT}})]$$

In practice, the clipping variant of the PPO algorithm is used.

Engineering Complexity of RLHF

A single RLHF training run requires four models to be loaded simultaneously:

- Policy (the model being trained)
- Reference (frozen SFT model, for computing KL)
- Reward Model (frozen, for scoring)
- Critic (value function estimate for PPO)

GPU memory is tight, training is unstable, hyperparameter tuning is difficult, and reward hacking risk is high. This motivated the development of DPO.

DPO: Bypassing RL

Direct Preference Optimization (2023). The key insight: the combination of PPO + KL penalty + RM is mathematically equivalent to a closed-form classification loss. Specifically, under the Bradley-Terry preference model, the optimal policy and the reference policy have an explicit relationship:

> $$\pi^*(y|x) \propto \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)$$

Solving for r:

> $$r(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)}$$

Substituting back into the RM preference loss:

> $$\mathcal{L}_{\text{DPO}} = -\log \sigma\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)$$

No reward model, no RL, no critic — only the policy and reference models are needed, with a standard supervised loss. Training is stable, simple, and achieves results comparable to or better than PPO.

This has been the most important simplification in alignment training over the past two years. Today a large number of open-source models (Zephyr, Tulu, Nous-Hermes) use DPO rather than PPO.

Newer Variants

IPO: Fixes a problem with DPO under noisy preference data
KTO: Replaces pairwise comparisons with single-point binary feedback (good/not good), making data easier to collect
ORPO: Combines SFT and preference alignment into a single training pass
RLAIF: Uses AI instead of humans to label preference data, enabling scale

The Goal of Alignment

At the technical level: make model outputs conform to "the distribution of human preferences." But preferences themselves are pluralistic — the three goals of helpful, harmless, and honest sometimes conflict (being too helpful might assist bad actors; being too harmless might refuse legitimate requests). In practice, this is balanced through diverse RLHF data.

Constitutional AI (Anthropic) and rule-based preferences represent directions for improving on "pure human annotation" — using explicit rules to generate preference comparisons, reducing annotation inconsistency.

Parameter-Efficient Fine-Tuning (PEFT)

Full fine-tuning requires updating all model parameters, which is extremely costly in the LLM era. **Parameter-Efficient Fine-Tuning** trains only a tiny fraction of parameters while achieving performance close to full fine-tuning:

- **LoRA** (Low-Rank Adaptation): Freeze the original weights W_0 and inject a low-rank bypass matrix BA ($r \ll d$). The number of trainable parameters can be reduced 100–1000×. At inference time, BA can be merged back into W_0 with zero additional latency.
- **QLoRA**: Builds on LoRA by quantizing the frozen weights to NF4 (4-bit), combined with double quantization and a paged optimizer, enabling fine-tuning of a 65B model on a single consumer-grade GPU.
- **Adapter**: Insert small bottleneck modules into each Transformer layer and train only the Adapter parameters.
- **Prefix Tuning / P-Tuning**: Prepend trainable virtual tokens to the input.

Detailed mathematical derivations and code implementations are available in Chapter 5, Paper Deep-Dive P11 (LoRA / QLoRA).

3.5.4 Reinforcement Learning

> *Overview: From games to alignment — core RL concepts and several key algorithms.*

The MDP Framework

The mathematical backbone of RL is the Markov Decision Process (MDP): state s, action a, transition $P(s'|s,a)$, reward $R(s,a)$, discount factor γ. The objective is to learn a **policy** $\pi(a|s)$ that maximizes the expected cumulative return:

> $$J(\pi) = \mathbb{E}_\pi\left[\sum_{t=0}^\infty \gamma^t R(s_t, a_t)\right]$$

Two Main Lines

Value-based: Learn the state value function $V(s)$ or the state-action value $Q(s, a)$. Representatives: Q-learning, DQN.

Policy-based: Directly learn the policy π_θ. Representatives: REINFORCE, PPO.

Actor-Critic: A combination of both — the policy (actor) generates actions while the value function (critic) evaluates their quality. All modern mainstream methods (A3C, PPO, SAC) use the AC architecture.

Q-learning and DQN

The Bellman optimality equation for Q-learning:

> $$Q^*(s, a) = R(s, a) + \gamma \mathbb{E}_{s'}[\max_{a'} Q^*(s', a')]$$

Q^* can be approximated via iterative updates. When the state space is large, the Q-table becomes intractable, so DQN (2015) uses a neural network to approximate Q. Key techniques:

Experience replay: Store (s, a, r, s') tuples in a buffer and sample randomly during training — breaks temporal correlations
Target network: Use lagged weights to compute the target $y = r + \gamma \max_{a'} Q_{\text{target}}(s', a')$, stabilizing training

DQN was the first to achieve superhuman performance on Atari games, marking RL's emergence from the ivory tower.

Policy Gradient and PPO

The policy gradient theorem:

> $$\nabla_\theta J(\pi_\theta) = \mathbb{E}_\pi[\nabla_\theta \log \pi_\theta(a|s) \cdot A(s, a)]$$

$A(s, a) = Q(s, a) - V(s)$ is the advantage function (how much better this action is compared to average).

Naive policy gradient has enormous variance and unstable updates. TRPO (2015) and PPO (2017) stabilize training by limiting the magnitude of each update. PPO uses **clipping** instead

of TRPO's hard constraint — simple, efficient, and has become the default RL algorithm:

> $\mathcal{L}^{\text{CLIP}} = \mathbb{E}\left[\min(r_t A_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) A_t)\right]$

$r_t = \pi_\theta / \pi_{\theta_{\text{old}}}$ is the probability ratio of new to old policy. This prevents r from deviating too far from 1 (typical $\epsilon = 0.2$).

Applications

RL has achieved success in games (AlphaGo, AlphaStar, OpenAI Five), robotics, autonomous driving, and LLM alignment (RLHF). However, RL training is notoriously difficult — sample efficiency is low, reward engineering is tricky, and hyperparameter tuning is an art.

In the networking domain, RL is used for congestion control (variants of PCC and BBR), routing optimization, resource scheduling, and ad bidding, but industrial deployment remains relatively rare — supervised learning and rule-based systems are the default in industry, with RL as a supplement.

3.5.5 Distillation, Transfer, and Multi-Task Learning

> *Overview: Using one model to accelerate another, using one task to help another — three parameter-efficient and data-efficient techniques.*

Knowledge Distillation

A large model (teacher) "teaches" its knowledge to a small model (student). Proposed by Hinton in 2015: the small model learns not only from hard labels but also from the teacher's soft labels (the full probability distribution).

> $\mathcal{L} = \alpha \cdot \text{CE}(y_{\text{true}}, p_s) + (1-\alpha) \cdot T^2 \cdot \text{KL}(p_t^T \| p_s^T)$

T is the temperature, used to "soften" the distribution (T > 1 flattens it). At high temperature, the teacher's "runner-up class" information is preserved — for example, "this image looks most like a cat but also somewhat like a dog."

DistilBERT, TinyBERT, and TinyLLaMA are all distillation products. Compression ratios are typically 2–5×, with only a few percentage points of performance loss.

Transfer Learning

Transfer a model trained on a source domain to a target domain. Typical workflow:

1. Load a pre-trained backbone
2. Replace the final layer with an output head for the target task
3. Freeze the backbone and train only the head (linear probing); or train all parameters together but with a smaller learning rate (fine-tuning)

The key decision is **what to freeze and what to train**. With little data: freeze most layers. With more data: train everything, but use a smaller learning rate for the early layers (which capture general features).

Domain Adaptation goes a step further — the target domain may be unlabeled, and feature distribution alignment methods are used for transfer.

Multi-Task Learning

A single model learns multiple tasks simultaneously. Shares a backbone with one head per task.

Advantages:

- Shared parameters, less memory
- Implicit regularization; one task may help another generalize

- Low-resource tasks benefit from high-resource tasks

Challenge: **task conflict** — gradients from different tasks may point in opposite directions, pulling against each other. Mitigations:

Loss weighting: Assign higher weight to more important tasks
Gradient projection (PCGrad): Project conflicting gradients to remove the conflicting component
MoE: Different tasks activate different experts

T5 and FLAN are canonical examples of multi-task training — hundreds of NLP tasks trained together, producing general-purpose models.

Continual Learning

A model has already learned task A and now needs to learn task B. Naive fine-tuning causes **catastrophic forgetting** — learning B overwrites knowledge of A. Mitigations include: EWC (add larger regularization on parameters important for A), Replay (mix in a portion of task A data), and Adapter (add small modules for each task, freezing the main trunk).

3.5.6 Federated Learning

> *Overview: Distributed training where data never leaves the local device — the solution for privacy-compliant scenarios.*

Motivation

Hospitals have patient data but cannot share it, phones have user inputs but cannot upload them, banks have transaction data subject to compliance restrictions. Centralized training is not an option — federated learning keeps data on-device and uploads only model updates.

FedAvg

The most basic federated algorithm (Google, 2017):

1. The server distributes the current global model θ_t to participating clients
2. Each client trains locally for a few rounds, obtaining a local update θ_t^k
3. Clients upload $\theta_t^k - \theta_t$ (or directly θ_t^k) to the server
4. The server computes a weighted average: $\theta_{t+1} = \sum_k \frac{n_k}{n} \theta_t^k$

Challenges

Non-IID data: Different clients have different data distributions (one client's photos are all cats, another's are all dogs), causing FedAvg to converge slowly or not at all. Improvements: FedProx adds a proximal term; SCAFFOLD uses control variates to correct bias.

Communication cost: Uploading and downloading large models is expensive. Compression (gradient quantization, Top-K sparsification) is the main optimization direction.

Heterogeneity: Wide variation in client device performance. Asynchronous FedAsync and hierarchical FL address this.

Privacy: Even uploading only updates can leak information (model inversion attacks can recover inputs from gradients). **Differential privacy** (adding noise) and **secure aggregation** (aggregating over encrypted values so the server sees only the sum, not individual contributions) are the mainstream defenses.

Applications

Google's Gboard (mobile keyboard) uses federated learning to train language models on-device. In the medical domain, multi-institution collaborative training (FeTS challenge) trains tumor segmentation models across multiple hospitals. In networking: collaborative training of multi-site attack detection models.

Industrial deployment remains limited — engineering complexity is high and performance lags behind centralized training. This

may become more important as privacy regulations tighten (GDPR, CCPA).

3.5.7 AutoML

> *Overview: Automating the machine learning pipeline — hyperparameter search, architecture search, and feature engineering.*

Hyperparameter Search

Traditional methods:

Grid search: Enumerate all combinations. Explodes in high dimensions.
Random search: Sample each hyperparameter independently at random. Bergstra (2012) demonstrated this outperforms grid search in most scenarios — most hyperparameters are actually insensitive and don't need a fine grid.
Bayesian optimization: Use a Gaussian process or TPE to model the "hyperparameter → validation loss" function, and select the next point to try using an acquisition function (EI, UCB). Optuna, Hyperopt, and Ax are the mainstream libraries.
Hyperband / BOHB: Early-stop poor configurations and allocate more budget to promising ones.

NAS: Neural Architecture Search

Automatically find the best network structure. Google's NASNet (2017) used RL to search and achieved state-of-the-art on ImageNet, but at enormous compute cost (thousands of GPU-days).

Subsequent improvements:

DARTS: Relax discrete architecture choices into continuous weights and optimize with gradient descent — runs in a few GPU-days
EfficientNet: A representative architecture family found by NAS

One-shot NAS: Train a single supernet; all candidate architectures are subgraphs of it

NAS was prominent in CV's early days. In the era of large models, Transformers dominate and NAS is less relevant for mainstream architecture selection. It remains useful for small models and hardware-targeted scenarios.

Automated Feature Engineering

Libraries like Featuretools and TSFresh automatically generate large numbers of features from tabular or time-series data (aggregations, cross-features, time windows), followed by feature selection to keep the useful ones. This can significantly reduce manual effort in traditional ML pipelines.

End-to-End AutoML Systems

H2O AutoML, AutoGluon, Auto-sklearn, Google Vertex AI AutoML — upload your data and the system automatically handles preprocessing, feature engineering, model selection, hyperparameter tuning, and ensembling. Suitable for teams without ML experts who need to get started quickly.

That said, experienced ML engineers typically outperform AutoML — domain knowledge provides priors that AutoML cannot know (e.g., knowing that certain feature combinations are meaningless). AutoML is an accelerator, not a replacement.

PART SIX

3.6 AI Performance Engineering

> *Taking models from demo to production — comprehensive optimization of inference latency, throughput, and power consumption.*

3.6.1 NPU Performance Modeling

> *Focus: Estimating theoretical peak performance and identifying real bottlenecks — the Roofline model.*

The Roofline Model

Given a hardware platform (compute P FLOPS, bandwidth B bytes/s) and a kernel (arithmetic intensity I = FLOPs / bytes accessed), the performance upper bound is:

> $$\text{Perf} \le \min(P, I \cdot B)$$

Plotted on a log-log graph: the x-axis shows arithmetic intensity, the y-axis shows performance. $I \cdot B$ forms the sloped "bandwidth roof" and P forms the horizontal "compute roof." The ridge point $I^* = P / B$ is the **hardware balance point**.

- $I < I^*$: memory-bound. Spending effort to increase compute is pointless; the goal is to reduce memory accesses.
- $I > I^*$: compute-bound. The hardware's compute is saturated; the only options are upgrading hardware or reducing FLOPs through algorithmic improvements.

Computing Arithmetic Intensity for Typical Operators

GEMM: $M \times K \times N$ multiplication reads $MK + KN + MN$ elements. Arithmetic intensity is approximately $O(\min(M, N, K))$. High for large matrices — compute-bound.

Elementwise: 1 FLOP per element, reading and writing 2 elements. Arithmetic intensity of 0.5 FLOP/byte. Severely memory-bound.

Attention (standard implementation): Arithmetic intensity varies with sequence length n. Short sequences are memory-bound; long sequences gradually become compute-bound. FlashAttention completely rewrites attention into a compute-bound form.

The Balance Point on H100

NVIDIA H100 SXM5: FP16 Tensor Core compute is approximately 989 TFLOPS (dense) / 1979 TFLOPS (2:4 sparse), with HBM3 bandwidth of 3.35 TB/s. Using dense compute, the balance point is $I^* = 989000 / 3350 \approx 295$ FLOP/byte; using 2:4 structured sparsity gives $I^* \approx 591$ FLOP/byte.

This means 295 floating-point operations must be performed for every byte read from HBM to fully saturate compute. Most kernels fall short — which is exactly why tiling, operator fusion, and on-chip SRAM utilization are central to optimization.

The Unique Characteristics of NPUs

NPUs typically feature:

- Tensor core compute >> general-purpose compute (most compute capacity is inaccessible without GEMM workloads)
- On-chip SRAM significantly larger than GPU shared memory (on the order of several MB)
- Slightly lower HBM bandwidth but better energy efficiency

This means the Roofline ridge point on NPUs may sit at a higher arithmetic intensity — making efficient use of on-chip SRAM even more critical.

3.6.2 Model Architecture Optimization, Quantization,

Pruning, and Lookup Tables

> *Focus: Reducing computation at the algorithm level — the three pillars of model compression.*

Architecture Optimization

Designing for efficiency from the start:

Depthwise separable convolutions (MobileNet): 1/9 the parameters and computation
Grouped convolutions (ShuffleNet, ResNeXt): reduced computation
GQA (Grouped Query Attention): multiple Q heads share a single set of K and V. The mainstream approach since LLaMA2, significantly reducing KV Cache size
MLA (Multi-head Latent Attention, DeepSeek): compresses KV into a latent space, saving even more
MoE: sparse activation — large total parameter count but only a small fraction is activated per forward pass

Quantization

This topic has come up several times before; here it is treated systematically.

PTQ (Post-Training Quantization): quantize after training, without retraining. Calibrate on a few hundred samples to estimate activation ranges, then compute scale/zero-point.

- Weight-only INT8: the most conservative option, nearly lossless
- Weight-only INT4: GPTQ / AWQ / AutoGPTQ, the mainstream approach for LLM inference
- W8A8 (both weights and activations in INT8): accelerates inference but with accuracy loss

QAT (Quantization-Aware Training): simulates quantization during training (using fake-quantization nodes), allowing the model to learn to function under quantized conditions. Higher accuracy than PTQ, at the cost of requiring retraining.

The key idea behind AWQ: observations show that "important weights" correspond to "channels with large activation magnitudes." Direct quantization causes the quantization error on these important weights to dominate the output. AWQ first scales up the weights of important channels (reducing their relative quantization error), then inversely scales the activations — mathematically equivalent, but with quantization error redistributed.

The key idea behind GPTQ: treats quantization as column-wise optimization, using Hessian information to absorb the error from already-quantized columns into the remaining columns. Algorithmically complex but achieves the best results at INT4.

Pruning

Unstructured pruning: sets individual weights with small absolute values to zero. High compression ratio, but sparse matrix acceleration requires special hardware support.

Structured pruning: removes entire channels, entire heads, or entire layers. Directly accelerates inference without special hardware support.

2:4 sparsity: forces exactly 2 out of every 4 consecutive weights to be zero. Natively supported by NVIDIA Ampere and later Tensor Core hardware, delivering a direct 2× speedup. Slightly inferior to unstructured pruning in terms of accuracy, but an engineering win overall.

Pruning typically requires **fine-tuning to recover accuracy** — accuracy always drops after pruning, but fine-tuning for a few epochs recovers most of it. Iterative pruning (alternating rounds of pruning and fine-tuning) yields the best results.

Lookup Tables Instead of Computation

In some cases, results can be precomputed offline and stored in a lookup table, replacing runtime computation with a table lookup for significant speedups. Typical examples:

Softmax LUT: precomputed \exp(x) table for discrete value domains
Embedding Cache: prefetched embeddings for high-frequency tokens
Activation LUT: nonlinear activations approximated with piecewise linear functions plus table lookups
PQ in retrieval: precomputed codebook distance tables for product-quantized vectors replacing real-time distance computation

Well-suited for embedded scenarios — table lookups are faster than computation and consume less power.

Distillation

Covered in section 3.5.5 above. A small model learns from a large one, achieving the highest compression ratios (10× or more), but requiring training from scratch. Typically combined with quantization and pruning — distill to a small model, then quantize to INT4, for a final speedup of 20–50×.

3.6.3 Operator Optimization

> *Focus: Squeezing hardware performance at the operator level — tiling, fusion, and FlashAttention in practice.*

This section is a technically dense deep dive.

Tiling: Moving the Problem into On-Chip SRAM

The standard optimization for GEMM. In a naive implementation, \mathbf{A}, \mathbf{B}, \mathbf{C} all reside in HBM and every element is read repeatedly. Tiling divides the matrices into small blocks, loading each block entirely into SRAM to be computed there.

```
// Three-level tiling: block (HBM→SRAM) / warp / thread
#define BM 128
#define BN 128
```

```
#define BK 16

for (int bi = 0; bi < M; bi += BM) {
    for (int bj = 0; bj < N; bj += BN) {
        // Result tile: BM × BN, held in SRAM
        float C_tile[BM][BN] = {0};

        for (int bk = 0; bk < K; bk += BK) {
            // Load A[bi:bi+BM, bk:bk+BK] and B[bk:bk+BK,
bj:bj+BN] from HBM to SRAM
            load_to_sram(A_tile, A + bi*K + bk, BM, BK);
            load_to_sram(B_tile, B + bk*N + bj, BK, BN);

            // Perform small matrix multiply within SRAM
            for (int i = 0; i < BM; i++)
                for (int j = 0; j < BN; j++)
                    for (int k = 0; k < BK; k++)
                        C_tile[i][j] += A_tile[i][k] * B_tile[k]
[j];
        }
        // Write C_tile back to HBM
        store_from_sram(C + bi*N + bj, C_tile, BM, BN);
    }
}
```

The number of times each element is read from HBM drops from O(N) in the naive implementation to O(N/BN) — reducing memory bandwidth pressure by a factor of BN. In practice, cuBLAS and CUTLASS add further levels (block / warp / thread fragment) but the principle is the same.

The Engineering Elegance of FlashAttention

Standard attention: $Q, K, V \in \mathbb{R}^{n \times d}$, computing $S = QK^\top$ (shape $n \times n$), $P = \text{softmax}(S)$, $O = PV$. The memory bottleneck lies in S and P — with $n = 4096$, $d = 64$, S alone occupies 64MB, and a single round trip through HBM costs 128MB.

The core idea of FlashAttention: **fuse attention into a single kernel, never materializing S or P**. The procedure:

1. Partition Q into blocks $Q_1, Q_2, \ldots$, each fitting in one SRAM tile
2. Partition K, V into blocks K_j, V_j
3. Outer loop over Q_i, inner loop over K_j, V_j
4. For each pair (Q_i, K_j), compute partial S_{ij}, partial softmax, and partial output $O_i \mathrel{+}= P_{ij} V_j$
5. Use an **online softmax** algorithm to merge partial results across different j

The online softmax uses a running max and running sum:

> $$m_i^{\text{new}} = \max(m_i, \max(S_{ij}))$$

> $$l_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \cdot l_i + \sum_k e^{S_{ij,k} - m_i^{\text{new}}}$$

> $$O_i^{\text{new}} = \frac{l_i}{l_i^{\text{new}}} e^{m_i - m_i^{\text{new}}} O_i + \frac{1}{l_i^{\text{new}}} \sum_k e^{S_{ij,k} - m_i^{\text{new}}} V_{j,k}$$

Each new block K_j, V_j updates the running max, running sum, and running output. The final result is equivalent to a full softmax, but the $n \times n$ matrices S and P are never materialized.

Benefits:

- HBM accesses reduced from $O(n^2)$ to $O(n^2 / \text{tile_size})$
- Memory footprint reduced from $O(n^2)$ to $O(n)$ (only intermediate m, l, O are stored)
- 2–4× speedup, with greater gains for longer sequences

FlashAttention-2 further optimizes warp scheduling; FlashAttention-3 is designed specifically for the Hopper architecture, using TMA (Tensor Memory Accelerator) and WGMMA to overlap asynchronous computation with data movement, achieving 840 TFLOPS/s (85% utilization) in BF16 and 1.3 PFLOPS/s in FP8; FlashAttention-4 is written with CuTeDSL and optimizes both Hopper and Blackwell. Today it is a mandatory component of every major inference engine.

Operator Fusion

What `torch.compile` in PyTorch 2.0 does under the hood is automatic operator fusion. Classic fusion patterns:

`Conv + BN + ReLU`: BN parameters can be folded into Conv — three-in-one
`GEMM + Bias + Activation`: handled in a single CUTLASS epilogue
`LayerNorm`: inherently a three-step fusion of "subtract mean, divide by std, then scale"
`RMSNorm + SwiGLU`: a common fusion pattern in LLMs

Evaluate performance gains for manual fusion. The general rule: *adjacent elementwise, broadcast, and reduction operators should always be fused.*

Operator Autotuning

For the same operator, the optimal tile size and thread layout differ across shapes and hardware. Triton AutoTuner, TVM Ansor, and cuDNN's algorithm selection all follow this approach — for each target shape, they try multiple configurations and record the best one. This is done once at compile time or on first run, with results cached for subsequent use.

3.6.4 Software Stack Optimization

> *Focus: Frameworks, runtimes, and scheduling — system-level optimization above the operator layer.*

Graph-Level Optimization

The computation graph passes through a series of optimization passes:

Constant folding, dead code elimination
Operator fusion
Memory planning (reusing tensor buffers)
Layout optimization (NCHW ↔ NHWC)

Operator selection (choosing the fastest among multiple operator implementations)

ONNX Runtime, TensorRT, OpenVINO, and CoreML are all graph compilers of this kind. They typically deliver 2–5× speedups for inference deployment.

Scheduling Optimization

Concurrent kernel execution, CPU-GPU pipelining, and asynchronous data copies are all important for both training and inference. CUDA Streams allow multiple kernels to run simultaneously on the GPU, as long as they are independent.

A classic optimization: **overlapping communication and computation**. In distributed training, gradients produced by each layer's backward pass immediately begin AllReduce while the previous layer continues its backward pass. With proper configuration, communication is entirely hidden behind computation, and scaling efficiency approaches linear.

Memory Optimization

Gradient Checkpointing: the forward pass stores only a subset of intermediate activations; the rest are recomputed during the backward pass. Trades roughly 30% extra computation for 2–5× memory savings. Essential for training large models.

ZeRO (see Chapter 4, section 5.10): shards optimizer states, gradients, and parameters across devices so each device holds only a fraction.

CPU Offload: moves infrequently accessed state (e.g., optimizer states) to CPU memory, transferring back when needed. Saves GPU memory but introduces latency.

PagedAttention (Chapter 4, section 5.13): virtual memory paging for KV Cache during inference.

I/O Optimization

Training bottlenecks often lie in data loading:

- Multi-process DataLoader + pinned memory
- Prefetching
- Compressed formats (WebDataset, TFRecord)
- Distributed file systems (Lustre, GlusterFS) or object storage (S3)

Inference bottlenecks:

- Model loading: use `mmap` + safetensors instead of pickle
- First-request cold start: warm up the model, enable keep-alive

Kernel Runtimes

Triton, CUDA Graphs, and CUTLASS each have their place:

Triton: the preferred tool for writing new kernels, with Python-like syntax
CUDA Graphs: records a series of kernels as a graph and launches the entire graph in a single call. Reduces launch overhead (critical for small-batch inference)
CUTLASS: a high-performance GEMM and related template library

PART SEVEN

3.7 AI in Networking

AI is more than ChatGPT — it has mature applications in network operations, security, and performance optimization.

3.7.1 Traffic Classification: Requirements and Challenges

> *Focus: Traffic identification in the age of encryption — traditional methods have failed, making AI the only viable option.*

Why Traffic Classification Is Needed

Network devices need to know "what kind of traffic this is" in order to apply QoS policies, security rules, billing, and compliance auditing. Traditional methods:

Port-based: 80 for HTTP, 443 for HTTPS, 53 for DNS...
Deep Packet Inspection (DPI): matching characteristic strings in the payload (User-Agent, SNI, protocol signatures)

Both approaches worked in the early 2010s but are no longer viable today —

1. **Port obfuscation**: applications no longer use fixed ports; P2P and VPN traffic uses random port assignments
2. **TLS ubiquity**: HTTPS accounts for the vast majority of traffic; payloads are encrypted, making DPI blind
3. **QUIC**: even the handshake is encrypted (once TLS 1.3 Encrypted ClientHello becomes widespread, SNI is no longer visible)
4. **CDN/cloud**: a single IP address can carry hundreds or thousands of applications

As a result, industry today relies almost exclusively on AI methods — identifying traffic from **statistical features and behavioral patterns**, without depending on payload content.

Classification Granularity

Traffic classification is a multi-level problem:

Traffic type: Web / Video / VoIP / Gaming / File Transfer
Application identification: Netflix / YouTube / TikTok / Zoom
Behavioral identification: casual browsing / video conferencing / file download / malicious scanning
User/device type: IoT device / mobile / desktop

Finer granularity demands more complex features and models.

Challenges in Data Acquisition

Where does training data come from? Several approaches:

Controlled packet capture: capturing pcap traces from known applications; labels are precise but volume is limited
Public datasets: ISCX, USTC, CICIDS, MAWI — labels are not always accurate
Weak supervision: using traditional methods to produce coarse-grained labels, with AI refining them
Self-supervised: exploiting the natural structure of traffic (forward/backward flows, connection establishment sequences)

Deployment Location and Constraints

Where the classifier is deployed determines the available resources:

Edge devices (routers, CPE): compute < 10 GFLOPS; must be extremely lightweight
Core devices (datacenter firewalls): GPU/NPU available; can run larger models
Cloud-side analysis: offline deep analysis; model size is unconstrained

Edge deployment is the most demanding — XGBoost with a few dozen hand-crafted features is the typical solution; in recent years, compact CNN/Transformer models have also become viable in this context.

3.7.2 Traffic Classification: Features and Models

> *Focus: Feature engineering in practice — which features actually matter and which models to use.*

Three Categories of Features

Statistical features (per-flow): statistics computed over a single flow (a sequence of packets sharing the same 5-tuple).

- Total packet count, total byte count, duration
- Packet size statistics: min / max / mean / std / median / percentiles
- Inter-packet interval statistics: same as above
- Directionality: forward/backward packet count ratio, byte ratio
- TCP flag occurrence counts (SYN, ACK, PSH, RST...)
- Window size mean and variance

Temporal features: the temporal patterns of packets, without aggregation.

- Size sequence of the first N packets (N = 20–100)
- Arrival time sequence of the first N packets
- Direction sequence of the first N packets (binary: forward = 0, backward = 1)

Header features (when available): TLS ClientHello cipher suite list, SNI (when unencrypted), TCP options, QUIC version, TLS ALPN, etc.

Hand-Crafted Features + Tree Models

The most common industrial approach. A few dozen features combined with XGBoost / LightGBM. Fast to train, fast to infer, interpretable, and accurate enough (90%+) for common tasks.

```
import numpy as np
import xgboost as xgb

def extract_flow_features(packets):
    """Extract features from a sequence of packets in a single
flow"""
    sizes    = np.array([p.size for p in packets])
    times    = np.array([p.timestamp for p in packets])
    fwd      = np.array([p.direction == 'fwd' for p in packets])

    feat = {}
    # Basic statistics
    feat['n_packets']      = len(packets)
```

```
    feat['total_bytes']    = sizes.sum()
    feat['duration']       = times[-1] - times[0] if
len(packets) > 1 else 0
    feat['bytes_per_sec']  = feat['total_bytes'] /
max(feat['duration'], 1e-6)

    # Packet size distribution
    for stat, fn in [('mean', np.mean), ('std', np.std),
                     ('min', np.min), ('max', np.max),
                     ('p25', lambda x: np.percentile(x, 25)),
                     ('p50', lambda x: np.percentile(x, 50)),
                     ('p75', lambda x: np.percentile(x, 75))]:
        feat[f'size_{stat}'] = fn(sizes)

    # Directional distribution
    feat['fwd_ratio']      = fwd.mean()
    feat['fwd_bytes']      = sizes[fwd].sum()
    feat['bwd_bytes']      = sizes[~fwd].sum()
    feat['fwd_bwd_ratio']  = feat['fwd_bytes'] /
max(feat['bwd_bytes'], 1)

    # Inter-arrival time (iat)
    if len(times) > 1:
        iat = np.diff(times)
        feat['iat_mean'] = iat.mean()
        feat['iat_std']  = iat.std()
        feat['iat_max']  = iat.max()

    # Size sequence of the first 20 packets (as individual
features)
    for i in range(20):
        feat[f'pkt_size_{i}'] = sizes[i] if i < len(sizes) else
0

    return feat

# Training
X = np.array([extract_flow_features(f).values() for f in flows])
y = np.array(labels)

model = xgb.XGBClassifier(
    n_estimators=300, max_depth=8, learning_rate=0.1,
    tree_method='hist', eval_metric='mlogloss'
)
model.fit(X_train, y_train, eval_set=[(X_val, y_val)])
```

```
# Inspect important features
importance = sorted(zip(feature_names,
model.feature_importances_),
                    key=lambda x: -x[1])
print("Top 10 features:", importance[:10])
```

Deep Learning Approaches

Treating a flow as a "sequence" and processing it directly with 1D-CNN or LSTM:

```
import torch.nn as nn

class FlowCNN(nn.Module):
    """Input: sequence of (size, iat, direction) tuples for the
first N packets"""
    def __init__(self, num_classes, seq_len=50):
        super().__init__()
        # 3 channels: size (normalized), iat (normalized),
direction (0/1)
        self.conv = nn.Sequential(
            nn.Conv1d(3, 32, kernel_size=5, padding=2),
nn.ReLU(),
            nn.Conv1d(32, 64, kernel_size=5, padding=2),
nn.ReLU(),
            nn.MaxPool1d(2),
            nn.Conv1d(64, 128, kernel_size=3, padding=1),
nn.ReLU(),
            nn.AdaptiveAvgPool1d(1),
        )
        self.fc = nn.Linear(128, num_classes)

    def forward(self, x):
        # x: [B, 3, seq_len]
        h = self.conv(x).squeeze(-1)
        return self.fc(h)
```

Advantages: end-to-end learning without hand-crafted features. Disadvantages: large data requirements, high computational cost, black-box nature. Difficult to deploy at the edge.

A recent trend is **FlowTransformer**-style work — treating packet sequences as token sequences and processing them with small

Transformers (3–6 layers). Strong performance but heavier.

Practical Deployment Considerations

Early classification: waiting until a flow ends before classifying is not an option — by then it has already passed through. Classification must be completed within the first N packets (N = 5–20). Feature extraction must be done incrementally, updating as each packet arrives.

Concept drift: applications update frequently (Netflix changes video encoding, Zoom changes transport protocols), causing traffic patterns to shift and model accuracy to degrade. Periodic retraining or online learning is required.

Class imbalance: dominant applications (YouTube, Netflix) account for 90% of traffic while niche applications are extremely rare. Use focal loss or oversampling.

Out-of-distribution samples in production: how to handle new applications not seen during training? Introduce an "Other" class combined with a confidence threshold.

3.7.3 Log Anomaly Detection: Requirements and Challenges

> *Focus: Finding "something's wrong" in massive log streams — one of the foundational problems in automated operations.*

The Scenario

Network devices, servers, and applications generate logs: system logs, protocol logs (BGP state changes), security logs (failed logins), application logs (error / warn). Hundreds of devices produce hundreds of millions of entries per day — manual inspection is infeasible.

The goal is to detect "anomalous patterns" in real time and alert operators. Anomaly definitions are often fuzzy — there are known patterns that directly indicate failures (supervised learning), but more often the challenge is "something never seen before" (anomaly detection, unsupervised).

Two Approaches

Rule-based: regex matching against specific patterns triggers alerts. Simple, interpretable, effective with zero examples. But rule maintenance becomes a nightmare — novel failure types always slip through. Mainstream SIEM (Security Information and Event Management) systems still rely heavily on rules.

Machine learning-based: learns "what is normal" from historical logs and flags deviations as anomalies. Not a replacement for rules but a complement — rules catch known issues, ML catches unknown ones.

Challenges with Log Data

Log data presents several unique challenges:

Templated structure: the same event type appears with different parameters. `"Connection from 1.2.3.4 failed"` and `"Connection from 5.6.7.8 failed"` represent the same event type with different parameters. Analysis requires **log parsing** — extracting logs into "template + parameters" (algorithms such as Drain and Spell).

Mixture of unstructured and semi-structured data: syslog is unstructured; JSON logs are structured. Modern systems are gradually moving toward structured logging, but legacy systems remain chaotic.

Temporal context dependency: individual log entries may appear normal, but "1000 consecutive login failures followed by a success" is anomalous — context must be considered.

Extreme class imbalance: anomalies account for less than 0.1% of logs.

Scarce ground truth: who is going to label millions of log entries?

3.7.4 Log Anomaly Detection: Methods

> *Focus: From template extraction to sequence modeling, to LLM-based anomaly explanation.*

The Pipeline

```
flowchart LR LOG[Raw Logs] --> PARSE[Log Parsing
Drain / Spell] PARSE --> TEMP[Template ID Sequence] TEMP -->
FEAT[Feature Extraction] FEAT --> DETECT[Detection Model] DETECT
--> ALERT[Anomaly Alert] DETECT --> EXPLAIN[LLM Explanation]
```

Log Parsing: The Drain Algorithm

Drain (2017) is the most widely used log parsing algorithm. The core idea: log templates have fixed prefixes while variable parts are primarily parameters. It uses a fixed-depth parse tree — classifying logs by length, branching level by level on the first few tokens, and performing fine-grained matching at the leaves.

The result is a "template library": each log entry is mapped to a template ID along with its parameter values. Subsequent analysis operates on template ID sequences rather than raw text.

Classical Methods: PCA / Isolation Forest

Represent each time window as a vector of template occurrence counts, then apply anomaly detection algorithms:

PCA: normal samples cluster tightly in the low-dimensional principal component space; anomalies lie far away. High reconstruction error indicates an anomaly.
Isolation Forest: randomly partitions the data; anomalous points are isolated with fewer splits on average. No distributional

assumptions required.

One-Class SVM: finds the tightest boundary enclosing normal samples.

Simple, fast, and unsupervised. At baseline levels, capable of detecting coarse-grained anomalies such as traffic spikes and unusual error template patterns.

Deep Learning: DeepLog / LogAnomaly

DeepLog (2017) was the first deep learning log anomaly detection system with significant industrial impact. The idea: treat log template IDs as "words" and a complete log segment as a "sequence." Use an LSTM to learn "given the previous k templates, predict the next one" — a wrong prediction signals an anomaly.

```
import torch.nn as nn

class DeepLog(nn.Module):
    def __init__(self, num_templates, hidden_size=128,
num_layers=2):
        super().__init__()
        self.emb  = nn.Embedding(num_templates, 64)
        self.lstm = nn.LSTM(64, hidden_size, num_layers,
batch_first=True)
        self.fc   = nn.Linear(hidden_size, num_templates)

    def forward(self, x):
        # x: [B, seq_len] template IDs
        e = self.emb(x)
        h, _ = self.lstm(e)
        return self.fc(h[:, -1])   # predict distribution over
next template

# At inference time: if the actual next template is not in the
top-k predictions, flag as anomaly
def detect(model, seq, k=9):
    logits = model(seq)
    topk = logits.topk(k).indices.squeeze()
    return seq[-1] not in topk
```

LogAnomaly further accounts for anomalies in parameter values. More recent work (LogBERT, NeuralLog) replaces the LSTM with a Transformer.

Log Analysis in the LLM Era

LLMs have changed the game — feeding log snippets directly to an LLM and asking "what anomalies are here, and what might be causing them?" Advantages:

- No log parsing required; natural language is processed directly
- Joint reasoning across log patterns
- Output is a human-readable explanation, not merely an "anomalous/normal" verdict

Disadvantages:

- Context window limitations restrict how many logs can be examined at once
- High inference cost makes real-time processing difficult
- Hallucinations

Industrial practice: a two-stage approach. Classical models perform coarse-grained filtering, then an LLM provides explanation and root-cause analysis for suspicious segments — combining the strengths of both.

The Evaluation Challenge for Anomaly Detection

How do you evaluate without ground truth? Several compromise approaches:

- Manual review over a period of time, annotating recall
- Injecting known failures (chaos engineering) and checking whether they are detected
- Comparing against the alerting system — checking whether the model also flags events at alert timestamps

An important post-deployment metric: **alert suppression rate** — how much the total alert count is reduced without missing

critical failures. The greatest pain point for operations teams is not missed alerts but "alert fatigue."

3.7.5 KPI Time-Series Forecasting: Requirements and Challenges

> *Focus: Predicting QPS, CPU, and bandwidth — the foundation of capacity planning and proactive operations.*

The Scenario

Core KPI time series in network operations: bandwidth utilization, CPU/memory, connection count, packet loss rate, latency, and error rate. Typical forecasting needs:

Capacity planning: predicting traffic trends over the next three months to inform expansion decisions
Auto-scaling: predicting load for the next hour to scale up proactively and avoid instability
Anomaly alerting: predicting the "normal" trajectory and alerting when actual values deviate
SLA management: predicting potential SLA violations in advance to enable early intervention

Properties of KPI Time Series

KPI time series data exhibits characteristic patterns:

Trend: long-term upward or downward movement
Seasonality: daily cycles (daytime/nighttime), weekly cycles (weekends), annual cycles
Residual: noise and anomalies

A typical QPS curve is a superposition of all three components.

Change points are particularly tricky — deployments, promotions, and failure recoveries can cause abrupt shifts in statistical

properties. Forecasting models must either be able to detect change points or adapt quickly after one occurs.

Challenges

Multivariate: CPU, memory, and QPS are correlated with one another; joint forecasting outperforms independent forecasting.

Multi-scale: forecasts are needed at the second, minute, hour, and day level; different scales require different models.

Cold start: insufficient historical data when a new service launches.

Anomaly contamination: training data that includes past failure periods can cause the model to treat those anomalies as "normal." Anomalous periods must be removed in advance.

Uncertainty quantification: point forecasts are insufficient; prediction intervals are needed — "QPS over the next hour will likely be 5000–7000" is more useful for decision-making than "approximately 6000."

3.7.6 KPI Time-Series Forecasting: Methods

> *Focus: From ARIMA to Prophet to Transformer — the evolution and when to use each.*

Classical Statistical Methods

ARIMA (AutoRegressive Integrated Moving Average):

> \phi(L)(1-L)^d y_t = \theta(L) \epsilon_t

Three parameters (p, d, q): p is the autoregressive order, d is the number of differencing steps (to achieve stationarity), and q is the moving average order. Parameters are selected using ACF/PACF plots or information criteria (AIC, BIC).

ARIMA performs well on stationary linear series, is computationally inexpensive, and is interpretable. For strong seasonality, SARIMA (seasonal ARIMA) is used. It remains the preferred baseline in many settings today.

Exponential smoothing: Holt-Winters triple exponential smoothing separates trend and seasonality. The kind built into Excel — simple, adequate, and old-fashioned.

Prophet

Open-sourced by Facebook in 2017. Decomposes the time series into **trend + seasonality + holidays + noise**, modeling each component independently:

> $y(t) = g(t) + s(t) + h(t) + \epsilon$

- Trend g: piecewise linear or logistic
- Seasonality s: Fourier series
- Holidays h: user-specified list of dates

Prophet's strength is being **analyst-friendly** — business knowledge can be incorporated manually (next week's major sale event, a new feature launch, annual subscription renewals). Parameters are interpretable, and the model can be tuned when things go wrong.

```
from prophet import Prophet
import pandas as pd

# Data must have two columns: (ds, y)
df = pd.DataFrame({'ds': timestamps, 'y': qps_values})

m = Prophet(
    yearly_seasonality=False,
    weekly_seasonality=True,
    daily_seasonality=True,
    changepoint_prior_scale=0.05,  # higher values allow more
trend changes
)
m.add_country_holidays(country_name='CN')
m.fit(df)
```

```
# Forecast the next 24 hours with a 5-minute granularity
future = m.make_future_dataframe(periods=24*12, freq='5min')
forecast = m.predict(future)
# forecast contains yhat (point forecast), yhat_lower,
yhat_upper (80% confidence interval)
```

Deep Learning Methods

LSTM / GRU: the most straightforward deep learning approach for time series. Takes a historical window as input and outputs the next step (or multiple steps). Multi-step forecasting commonly uses seq2seq or iterative prediction.

N-BEATS (2019): fully MLP-based, using a "backcast/forecast" architecture. Outperformed all RNN and Transformer methods in the M4 time series competition. Demonstrates that MLPs remain competitive for time series.

Temporal Fusion Transformer (2020): a Transformer variant designed specifically for time series. Supports multivariate inputs, static covariates, and future-known covariates (such as pre-scheduled promotion dates).

PatchTST (2023): divides the time series into patches (analogous to ViT's treatment of images), then encodes them with a Transformer. Simple yet powerful.

Informer / Autoformer: Transformer variants optimized for long-sequence forecasting.

Foundation Models

Since 2024, **time series foundation models** have emerged: Chronos (Amazon), TimesFM (Google), Moirai. Pretrained on large and heterogeneous time series corpora, they enable zero-shot forecasting — new series can be used directly without task-specific training.

The practical picture is mixed: Prophet/ARIMA is sufficient for simple series; deep models outperform on complex multivariate

and long-horizon tasks; foundation models are the only option when no historical data is available.

Choosing a Method

Scenario	Recommended Method
Small data (< 1000 points), need a quick solution	Prophet / ARIMA
Moderate data, univariate	Prophet / N-BEATS
Strongly correlated multivariate	Temporal Fusion Transformer / PatchTST
Long sequences (1000+ points)	Informer / PatchTST
Cold start, no history	Time series foundation model zero-shot
Interpretability required	Prophet + SHAP
Edge device inference	ARIMA / compact LSTM

Evaluation

Common metrics:

MAE: Mean Absolute Error. Robust to outliers.
RMSE: Root Mean Squared Error. Penalizes large deviations heavily.
MAPE: Mean Absolute Percentage Error. Relative error, scale-independent, but unstable when true values are near zero.
sMAPE: Symmetric MAPE, addressing the issue above.
Quantile Loss: evaluates interval forecasts.

In practice, **directional accuracy** often matters more — whether the model correctly predicted "up or down" is more actionable than the precise value. Directional accuracy is a useful supplementary metric.

Chapter 4: LLM Technology

From Transformer to Agent

An An LLM is no longer just a "model"—it is an entire technology stack. From low-level Transformer operators, to distributed training strategies, to inference-side scheduling optimizations, to RAG and Agent on the application side, every layer involves substantial engineering decisions and algorithm choices. This chapter ties these layers together so you can see the whole picture from top to bottom.

37 sections, all deep dives. Key anchors include: complete derivation of Self-Attention (4.2.2), RoPE rotation matrices (4.3.2), DeepSeek's MLA low-rank decomposition (4.4.1), Mamba's selective state space (4.4.3), ZeRO three-stage memory sharding (4.7.2), PagedAttention's paging mechanism (4.8.3), acceptance rate analysis for speculative decoding (4.8.5), mathematics of AWQ/GPTQ (4.9.1), reranking and query rewriting in advanced RAG (4.10.2), and ReAct loop implementation (4.11.1). Other sections are written at the same level of depth: conceptual intuition, minimal derivation, and engineering details.

When discussing concrete products, we focus on **open-source** (Transformers, vLLM, TGI, llama.cpp, DeepSpeed, FSDP, LangChain, LlamaIndex, etc.), with OpenAI/Anthropic APIs mentioned for comparison. This way you see the full ecosystem landscape without being tied to any single vendor.

PART ONE

4.1 LLM Overview

Establish a coordinate system for the entire LLM field—where capabilities come from and how the landscape divides.

4.1.1 From LM to LLM

> *Position: How Scaling transforms "language models" into "general reasoning systems"—history, key milestones, Scaling Law.*

The Old Problem of Language Models

The definition of a language model has never changed: given the first t-1 tokens, predict the probability distribution of the t-th token.

> $P(w_t \mid w_1, w_2, \ldots, w_{t-1})$

From Shannon's information theory in 1948, to N-grams in the 1980s, to Bengio's neural language models in 2003, to Word2Vec and GloVe—for decades, NLP researchers have been finding better ways to model this conditional probability. But no one could have predicted that "stacking" this simple objective large enough would lead to emergent reasoning, programming, and mathematical abilities.

Key Breakthroughs

2017 Transformer: Google's "Attention is All You Need." Completely replaces RNNs, training is parallelizable, and results are superior. Foundation for everything that followed.

2018 BERT / GPT-1: Establishes the pre-training + fine-tuning paradigm. BERT uses masked language modeling for pre-training, dominating various downstream tasks; GPT uses causal language modeling, taking the generation route.

2020 GPT-3: 175B parameters, can perform tasks via few-shot learning without fine-tuning. "In-context learning" becomes the new paradigm—as models grow larger, they can learn new tasks from prompts. This is the turning point where LLM transforms from "feature extractor" to "general reasoner."

2022 ChatGPT: Brings RLHF to product grade; models can now "chat." Overnight, LLM transforms from a research topic to a consumer product.

2023 GPT-4 / Claude 2 / LLaMA-2: Multimodal, 100K+ context windows, and strong open-source models emerge. The ecosystem begins to bifurcate—closed-source commercial vs. open-weight.

2024 DeepSeek-V3 / o1 / Claude 3.5 / Qwen2.5: Open-source catches up with the first tier of closed-source models; "thinking at test time" (test-time compute) becomes a new dimension, allowing models to "think" for extended periods before output.

2025 DeepSeek-R1 / Qwen3 / LLaMA 4 / Claude 4 / GPT-5 / Gemini 3: Explosion of reasoning models. DeepSeek-R1 launches in early 2025, achieving GPT-4o-level reasoning via pure RL but at a tenth the cost, triggering a global open-source race; Qwen3 introduces a "thinking mode" toggle, covering both reasoning and non-reasoning scenarios in a single model; DeepSeek's subsequent V3.1 unifies thinking/non-thinking modes, while V3.2 introduces Sparse Attention (DSA) to further reduce costs; LLaMA 4 family (Scout/Maverick) uses MoE for the first time; Claude 4/4.5 emphasizes agent stability, while OpenAI GPT-5 and Google Gemini 3 compete for top reasoning benchmarks.

Early 2026: Open-source MoE densifies further—Qwen3-Next 80B-A3B, Qwen3.5-397B-A17B, Kimi K2, GLM-4.5/4.6, and MiniMax-M2 all emerge from late 2025 through early 2026. On the closed-source side, OpenAI releases GPT-4.2 (2025.12), followed by GPT-4.4 on 2026.3.5; the Claude series evolves to Opus 4.6 / Sonnet 4.6 /

Haiku 4.5. Anthropic's **Claude Mythos Preview** in 2026.4 is a critical milestone—this unreleased frontier model exhibits "emergent-level" abilities in cybersecurity, automatically discovering thousands of zero-day vulnerabilities in mainstream operating systems and browsers. Anthropic makes an exception by not releasing it publicly, instead providing it exclusively to 50+ major enterprise partners through Project Glasswing for defensive purposes. This is the first time a major AI company has proactively constrained a frontier model's release for safety reasons since OpenAI's restricted GPT-2 release in 2019.

In March 2026, DeepSeek releases **V4** (next-generation flagship) and **R2** (R1's successor reasoning model with significantly improved multilingual reasoning); Alibaba releases **Qwen3.6-Plus Preview**, supporting 1M context and always-on CoT reasoning. Major disruption in video generation: OpenAI announces on 2026.3.24 the **shutdown of the entire Sora product line** (including standalone app and API), redirecting resources to robot world simulation research; meanwhile, Kling 3.0 becomes the first natively 4K 60fps video generation model.

The gap between open-source and closed-source narrows further to 3-6 months. Anthropic's MCP (Model Context Protocol) becomes the standard for Agent tool calling throughout 2025, widely adopted across Claude, Cursor, and the VSCode ecosystem. In 2025.12, MCP is donated to the Linux Foundation's Agentic AI Foundation (jointly founded by Anthropic, Block, and OpenAI), becoming the industry-standard protocol.

Scaling Law: The Mathematical Explanation for All This

Why does scaling up enable emergent abilities? Two papers provide the answer: Kaplan 2020 and Chinchilla 2022. **Loss is a power-law function of parameter count N, data quantity D, and compute budget C.**

> $$L(N, D) \approx E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}$$

E is the irreducible entropy of the data (theoretical lower bound), and A/N^\alpha, B/D^\beta are compressible portions. The power law means that as long as compute continues to scale, loss continues to decrease—no saturation has been observed.

More intriguingly, Chinchilla's finding: given a compute budget C = 6ND, the optimal point is when N and D grow in roughly equal proportion (with a specific ratio of about 1:20 tokens/param). GPT-3's 175B parameters actually didn't have enough training data (300B tokens); Chinchilla proved that 70B parameters with 1.4T tokens performs better. This guided the design of the LLaMA family.

Emergent Abilities

Scaling is not linear—certain abilities suddenly appear when model size crosses a threshold (in-context learning, CoT reasoning, multi-step math). This is called "**emergent abilities**." Small models can't learn from limited prompts; large models master a task from just two or three examples.

Some 2023 work argued that emergence is an illusion of metric choice (BIG-Bench uses strict accuracy, appearing to "emerge" as it jumps from 0 to non-zero; switching to log-likelihood smooths it out). But even so, the phenomenon remains: **there are real qualitative differences in capability between large and small models.**

From Scaling to Test-Time Compute

In late 2024, the rules of the game changed again. OpenAI's o1 demonstrated that models don't just scale during training—**test-time scaling also offers enormous gains.** Letting models "think" for extended periods (generating many reasoning tokens before outputting an answer) can double performance. OpenAI subsequently released **o3 and o3-mini**, further boosting reasoning—o3 achieves breakthrough results (87.5%) on the ARC-AGI benchmark, exceeding the human average for the first time. DeepSeek-R1, QwQ, and Claude 3.7 Sonnet's thinking mode all follow this path.

Throughout 2025, this direction dominated LLM evolution. Key developments:

DeepSeek-R1 (2025.1): An open-source reasoning model trained via pure RL (GRPO algorithm) without requiring human annotations of reasoning traces—overturning the assumption that "alignment data is essential." The R1-0528 version exceeds GPT-4o on AIME math problems.

Hybrid thinking/non-thinking modes: Qwen3 pioneers a "mode toggle"—users flip a switch to enable or disable extended reasoning. DeepSeek V3.1 further merges both modes into one model, with chat templates controlling behavior. This design became standard for open-source models in the second half of 2025.

Thinking budget: The product side gradually exposes a "thinking budget" parameter, letting users control reasoning length—the same question can be answered with 1K tokens of thinking (fast) or 16K tokens (deep).

Test-time compute transforms "one forward pass determines capability" into "pay more compute for better answers"—a significant shift for the product side. This is one of the major axes of LLM evolution in the coming years.

4.1.2 Open-Source vs Closed-Source Ecosystems

> *Position: Two different development paths—open weights vs API services—and their technical/commercial tradeoffs.*

Closed-Source Commercial Model

Typical examples: OpenAI's GPT series, Anthropic's Claude series, Google's Gemini. Model weights are not public; access is via API. Advantages: frontier capabilities (new releases often lead open-source by 6-12 months), managed service (no ops burden), comprehensive multimodal abilities.

Tradeoffs: data must leave your infrastructure (privacy compliance), costs scale with usage, vendor lock-in, limited or no fine-tuning. Best for application development companies and non-sensitive workloads.

Open-Source Weight Models

Main families (as of early 2026):

Family	Latest	License	Characteristics
LLaMA	LLaMA 4 Scout / Maverick	Community License	Meta-led; LLaMA 4 first to use MoE architecture
Qwen	Qwen3 (0.6B-235B MoE), Qwen3-Next 80B-A3B	Apache 2.0	Alibaba; thinking mode toggle; strong in Chinese/English; full MoE and dense lineup
DeepSeek	DeepSeek-V3.1/V3.2, R1	MIT	MLA, DSA architectural innovations; strong reasoning; extremely low cost
Mistral	Mistral Large 3, Ministral	Partial Apache 2.0	European open-source pioneer
Kimi (Moonshot)	Kimi K2, K2-Instruct	Permissive License	Emerged in 2025; strong long context and reasoning
GLM (Zhipu)	GLM-4.5, GLM-4.6	Permissive License	Chinese MoE; shared experts + attention bias
MiniMax	MiniMax-M2	Open Weight	Chinese; Flash Attention variant
Phi	Phi-4	MIT	Microsoft; strong small models
Gemma	Gemma 2, Gemma 3	Gemma License	Google's open-source version based on Gemini
GPT-OSS	gpt-oss-20b, gpt-oss-120b	Apache 2.0	OpenAI's first open-source in 2025; first since GPT-2

Granite	Granite 4 (Mamba-Attention hybrid)	Apache 2.0	IBM; enterprise use cases; architectural innovation
Nemotron	Nemotron 2	Open	NVIDIA; Mamba2-Transformer hybrid

Note that "open-source" is a spectrum—LLaMA's community license has commercial restrictions (customers with 700M+ monthly active users need separate licensing), so it's strictly "open-weight." Mistral, Qwen, DeepSeek, Kimi, GLM, and GPT-OSS are true Apache/MIT, enabling unlimited commercial use.

A notable 2025 shift: **Chinese open-source models become the absolute driving force**—Qwen, DeepSeek, Kimi, GLM, MiniMax, Xiaomi MiMo and others surge collectively. The Chinchilla principle (data quantity matching parameter count) is widely adopted, with training data scaling from 15T tokens in the LLaMA-3 era to 30-40T.

Capability Gaps Are Narrowing

A striking 2024 trend: the gap between open-source flagship models and closed-source frontiers narrows to 6 months. By 2025, it shrinks to 3-6 months—DeepSeek-V3.2, Qwen3-235B-A22B, and Kimi K2 match or exceed contemporary closed-source models in math reasoning, coding, and general benchmarks. DeepSeek-R1 even surpasses GPT-4o-level models on some math benchmarks while costing 20x less.

The "DeepSeek Moment" in February 2025: After R1's release, the global AI community is shaken; 94% of GitHub Trending's weekly rankings are AI-related. Closed-source vendors accelerate: OpenAI launches GPT-5, GPT-4.1, and GPT-4.2 (2025.12, with consecutive Codex-specific versions) in 2025; releases GPT-4.4 in 2026.3 unifying reasoning, coding, and computer use (1M context, 75% OSWorld-Verified), and rarely, open-sources GPT-OSS-20B/120B. Anthropic releases Claude Sonnet 4.5, Opus 4.6, Haiku 4.5, and previews Claude Mythos Preview in 2026.4—the gap

reopens, but often closes again within months as the next open-source generation catches up.

Closed-source maintains advantages in a few areas: multimodal completeness (Gemini 3's video understanding, GPT-4o/GPT-5's real-time voice chat); ultra-long context (Gemini's practical 1M+, GPT-4.4's usable 1M tokens); Agent tool-calling stability (Claude's Computer Use maturity); safety guardrails and compliance. Plus domain-specific "emergent abilities"—cybersecurity breakthroughs like Mythos Preview remain beyond current open-source models.

Deployment Modes

Deployment options for open-source models:

Local inference (laptop/workstation): llama.cpp, Ollama, LM Studio. Quantized to INT4, 7B models run on 5GB RAM.
Self-hosted service (data center): vLLM, TGI (HuggingFace), SGLang, LMDeploy. High concurrency, high throughput.
Edge deployment (embedded devices): llama.cpp with ARM NEON, MNN, MediaPipe. Phones, routers, IoT.
Cloud-hosted: AWS Bedrock, Azure AI, Alibaba Cloud PAI—third-party vendors host open-source models and provide APIs.

Inference Engine Landscape

Section 4.8 dives deep; here's the landscape (as of early 2026):

Engine	Language	Primary Use	Characteristics
Transformers	Python	Research/Prototype	HuggingFace official; broadest compatibility
vLLM	Python+CUDA	Production Service	PagedAttention, continuous batching; growth accelerates after v1 architecture rewrite in 2025
SGLang	Python+CUDA	Complex Workflows	RadixAttention, structured generation;

			one of fastest-growing engines in 2025
TGI	Rust	Production Service	HuggingFace product; comprehensive ecosystem
TensorRT-LLM	C++	NVIDIA Production	Highest performance; hardware-bound
NVIDIA Dynamo	C++/Python	Massive-Scale Inference	NVIDIA 2025 release; unifies vLLM/SGLang/TRT-LLM backends; distributed scheduling
LMDeploy	Python	Production Service	Shanghai AI Lab; recommended by DeepSeek and others
llama.cpp	C++	Local/Edge	Pure CPU capable; comprehensive quantization
MLC-LLM	Python+TVM	Cross-Hardware	WebGPU, ARM, Chinese chips
Ollama	Go	Personal Use	User-level layer on llama.cpp

By 2025, the inference engine landscape forms a duopoly: vLLM + SGLang, borrowing from and competing with each other. DeepSeek's V3 release explicitly recommends both, further solidifying their positions.

Application-Layer Frameworks

LangChain: Most comprehensive features, largest ecosystem; criticized for "over-abstraction." Good for rapid prototyping.

LlamaIndex: RAG-focused; strongest data connection and retrieval capabilities.

DSPy (Stanford): Treats LLM calls as "programmable components" with automatic prompt optimization.

Haystack: Enterprise-grade RAG framework; high production readiness.

Roll Your Own: Many teams find that calling the OpenAI SDK directly with a bit of glue code is cleaner than using a framework—LLM calls are fundamentally HTTP requests, and thicker abstraction layers only make debugging harder.

PART TWO

4.2 Transformer Architecture

The foundation of all LLMs — master this section and everything downstream falls into place.

4.2.1 Overall Structure and Information Flow

> *Position: The complete path of a token from model entry to output — establishing the macro picture.*

Overall Flow

From user input "Hello world" to the model predicting the next token, the steps are:

```
flowchart TB INP[User Input Text] --> TOK[Tokenization] TOK --> EMB[Token Embedding] EMB --> POS[+ Position Encoding] POS --> L1[Transformer Block x N] L1 --> NORM[Final LayerNorm/RMSNorm] NORM --> LMH[LM Head Linear] LMH --> SOFT[Softmax] SOFT --> SAM[Sample next token]
```

Each **Transformer Block** is a repeated unit of the same structure. GPT-3 has 96, LLaMA-3-70B has 80. Each block does two things:

1. **Self-Attention**: lets each position "see" other positions, enabling lateral information exchange
2. **FFN (Feed-Forward Network)**: independently transforms each position, enhancing expressiveness

Each operation is wrapped in "normalization + residual connection." Modern LLMs use Pre-Norm — normalization before the residual. The original Transformer used Post-Norm (normalize after residual), which is less stable.

Shape Evolution

Tracking a token ID tensor of shape `[B, L]` (B=batch, L=seq len) through the model:

Stage	Shape
Input token IDs	[B, L]
After Embedding	[B, L, d_model]
Inside each attention layer	[B, H, L, d_head] (H heads, d_head = d_model/H)
Attention layer output	[B, L, d_model]
Inside FFN	[B, L, d_ff] (d_ff is typically 4×d_model)
Final output	[B, L, vocab_size]

Parameter Distribution

The parameters of a Transformer Block are mainly in:

Attention's four linear layers Q/K/V/O: 4 \times d_{model}^2

FFN's two linear layers: $2 \times d_{model} \times d_{ff} = 8 \times d_{model}^2$ (with $d_{ff} = 4 d_{model}$)

So **FFN accounts for 2/3 of the parameters per layer**, attention accounts for 1/3. This motivates MoE — sparsifying the large FFN significantly reduces computation.

Total model parameters $\approx 12 \times N_{layers} \times d_{model}^2$ + embedding table ($V \times d_{model}$, V = vocabulary size). LLaMA-3-70B has 80 layers, $d_{model}=8192$, giving ≈ 64B, plus embedding and miscellaneous ≈ 70B.

Forward and Backward Pass

During training, given the entire sequence, one forward pass produces the predicted distribution for every position. The loss is **cross-entropy computed at every position** (predicting the next token), summed and then backpropagated. One forward + backward pass costs approximately $6 \times N_{params} \times N_{tokens}$ FLOPs.

During inference, starting from a prompt, only the next token is predicted each time. See Section 4.8 for details.

4.2.2 Self-Attention: Complete Derivation

> *Position: The soul operator of LLMs — deriving every detail, including why we divide by √d.*

Problem Setup

Input: L tokens, each a d-dimensional vector $\mathbf{x}_i \in \mathbb{R}^d$. Goal: make the output $\mathbf{y}_i$ at each position i a **dynamic weighted combination** of all input positions, with weights determined by the "similarity" between $\mathbf{x}_i$ and other positions.

The Roles of Query / Key / Value

Core idea: each position plays three roles:

Query: $\mathbf{q}_i = W_Q \mathbf{x}_i$ — "what kind of information am I looking for"
Key: $\mathbf{k}_i = W_K \mathbf{x}_i$ — "what kind of information am I"
Value: $\mathbf{v}_i = W_V \mathbf{x}_i$ — "what am I contributing"

When computing the output for position i: compare $\mathbf{q}_i$ against all $\mathbf{k}_j$ to get weights α_{ij}, then use α_{ij} to compute a weighted average over all $\mathbf{v}_j$. This three-way separation gives the model expressiveness — the relevance of "i to j" and "j to i" can differ.

Matrix Derivation

Stack all tokens: $\mathbf{X} \in \mathbb{R}^{L \times d}$. Compute:

> $$\mathbf{Q} = \mathbf{X} W_Q, \quad \mathbf{K} = \mathbf{X} W_K, \quad \mathbf{V} = \mathbf{X} W_V$$

These are all $L \times d$ matrices (W_* are $d \times d$ projections). Then compute similarity:

> $$\mathbf{S} = \mathbf{Q} \mathbf{K}^\top \in \mathbb{R}^{L \times L}$$

S_{ij} is the dot product of query i and key j — larger values indicate greater "similarity."

Why Divide by √d

There is a critical detail here. $\mathbf{q}_i \cdot \mathbf{k}_j = \sum_{k=1}^d q_k \cdot k_k$. If each dimension of q and k is independent with mean 0 and variance 1, the variance of the dot product is d (standard deviation $\sqrt{d}$).

When d is large, the range of dot products is wide — some very large, some very small. When fed into softmax, softmax is dominated by the maximum value (the "sharpening" effect of softmax). If s_1 = 30, s_2 = 25, a difference of 5 causes softmax to almost exclusively attend to s_1. Even worse: gradients vanish — softmax has near-zero gradients in the "winner-takes-all" regime.

The original Transformer paper's fix: divide by \sqrt{d_k} to bring the dot product variance back to 1:

> \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q} \mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}

This is "Scaled Dot-Product Attention."

Causal Mask

LLMs are causal (each position can only see itself and prior positions), so the upper triangle of \mathbf{S} is set to -\infty, which becomes 0 after softmax:

```
import torch

def causal_attention(Q, K, V):
    # Q, K, V: [B, H, L, d_head]
    d_k = Q.size(-1)
    S = Q @ K.transpose(-2, -1) / d_k ** 0.5   # [B, H, L, L]
    L = S.size(-1)
    mask = torch.triu(torch.ones(L, L, dtype=torch.bool),
diagonal=1)
    S = S.masked_fill(mask, float('-inf'))
    A = torch.softmax(S, dim=-1)
    return A @ V   # [B, H, L, d_head]
```

Computational Complexity

Computing \mathbf{Q}\mathbf{K}^\top: O(L^2 d). Softmax: O(L^2). Multiply by \mathbf{V}: O(L^2 d). Total: O(L^2 d).

Memory: the \mathbf{S} matrix is O(L^2); at L=4096, that's 16M elements × FP16 = 32MB per head per sample. This is the "quadratic complexity bottleneck" of attention — the fundamental challenge for long contexts. FlashAttention, sparse attention, and Mamba all address this.

4.2.3 Multi-Head Attention

> *Position: Splitting attention into multiple heads — why it works and how to implement it.*

Motivation

Single-head attention can only learn one "attention pattern" — it might learn coreference, or syntactic proximity, but a single head can only represent one. Multiple heads let the model learn multiple patterns simultaneously: each head has independent W_Q^h, W_K^h, W_V^h projections that learn different patterns.

Implementation

A naive implementation runs H separate attention passes and concatenates. In practice, for efficiency, one large projection is used and split into heads:

1. Project \mathbf{X} to d_{model}-dimensional Q/K/V
2. Reshape into H subspaces of d_{head} dimensions each: d_{model} = H \times d_{head}
3. Each head independently performs attention
4. Concatenate back to d_{model} dimensions, then apply output projection W_O

```
import torch
import torch.nn as nn

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_heads):
        super().__init__()
        assert d_model % n_heads == 0
```

```
        self.H = n_heads
        self.d_head = d_model // n_heads
        self.Wq = nn.Linear(d_model, d_model, bias=False)
        self.Wk = nn.Linear(d_model, d_model, bias=False)
        self.Wv = nn.Linear(d_model, d_model, bias=False)
        self.Wo = nn.Linear(d_model, d_model, bias=False)

    def forward(self, x, mask=None):
        B, L, D = x.shape
        Q = self.Wq(x).view(B, L, self.H,
self.d_head).transpose(1, 2)
        K = self.Wk(x).view(B, L, self.H,
self.d_head).transpose(1, 2)
        V = self.Wv(x).view(B, L, self.H,
self.d_head).transpose(1, 2)
        S = Q @ K.transpose(-2, -1) / self.d_head ** 0.5
        if mask is not None: S = S.masked_fill(mask, float('-
inf'))
        A = torch.softmax(S, dim=-1)
        out = A @ V
        out = out.transpose(1, 2).reshape(B, L, D)
        return self.Wo(out)
```

Common Parameters

Model	d_model	Heads H	d_head
GPT-2 small	768	12	64
GPT-3 175B	12288	96	128
LLaMA-3 70B	8192	64	128
LLaMA-3 405B	16384	128	128

d_{head} is generally fixed at 64 or 128 — too small and each head has insufficient capacity; too large and there are too few heads to justify multi-head attention.

Visualizing Attention Heads

Tools like BertViz and LLM Attention Viewer can visualize the attention of each head. Empirically, different heads do learn

different patterns — some heads attend to end-of-sentence punctuation, some to nearest neighbors, some perform coreference resolution. But there are also many "redundant" heads (removing them doesn't affect performance) — this inspired subsequent head pruning research.

4.2.4 FFN, Residual Connections, and Normalization

> *Position: The other half of the Transformer Block — FFN enhances expressiveness, residual connections and normalization ensure trainability.*

The Role of FFN

Attention enables communication between positions, but **attention output remains in the linear subspace of token representations** — attention alone limits model expressiveness. FFN independently applies a nonlinear transformation at each position, projecting the representation into a high-dimensional nonlinear space and back.

> $$\text{FFN}(\mathbf{x}) = W_2 \cdot \text{activation}(W_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$$

W_1 maps $d_{model} \to d_{ff}$ (expanding), W_2 maps $d_{ff} \to d_{model}$ (contracting). d_{ff} is typically 4× d_{model}.

Activation Function Evolution

Original Transformer: ReLU. **BERT/GPT era**: GELU. **LLaMA era**: SwiGLU.

The Mechanism of SwiGLU

SwiGLU is not an ordinary activation function but a "gated linear unit." It modifies the FFN to:

> \text{SwiGLU}(\mathbf{x}) = W_3 \cdot \left(\text{SiLU}(W_1 \mathbf{x}) \odot (W_2 \mathbf{x}) \right)

Three linear projections W_1, W_2, W_3 are used. W_1 goes through SiLU, W_2 does not, and the two are multiplied element-wise. W_3 projects the result back to d_{model}.

The element-wise product (W_2 \mathbf{x}) acts as a "gate" — it determines which dimensions pass through and which are suppressed. More expressive than a plain MLP.

Parameter control: to keep the total parameter count equivalent to a 4× FFN, the hidden dimension is changed from 4d to \frac{8}{3}d. LLaMA, Qwen, Mistral, and DeepSeek all do this.

```
class SwiGLU(nn.Module):
    def __init__(self, d_model, d_ff=None):
        super().__init__()
        d_ff = d_ff or int(8 * d_model / 3)
        self.W1 = nn.Linear(d_model, d_ff, bias=False)
        self.W2 = nn.Linear(d_model, d_ff, bias=False)
        self.W3 = nn.Linear(d_ff, d_model, bias=False)

    def forward(self, x):
        return self.W3(nn.functional.silu(self.W1(x)) *
self.W2(x))
```

Residual Connections

Chapter 3 Section 4.4.1 covered ResNet's residual idea. Transformer uses it too: each sublayer's (attention and FFN) output is added to its input. \mathbf{y} = \mathbf{x} + \text{Sublayer}(\mathbf{x}).

Benefits: gradients flow through the residual path directly to deeper layers, preventing vanishing gradients; if a layer doesn't learn a useful transformation, the residual keeps it close to identity mapping without degrading performance; enables training 80+ layer models.

Pre-Norm vs Post-Norm

Where normalization is placed matters.

Post-Norm (original Transformer): `y = LayerNorm(x + Sublayer(x))`. Normalize after residual addition.

Pre-Norm (since GPT-2): `y = x + Sublayer(LayerNorm(x))`. Normalize before the sublayer; residual is added directly to the input.

Pre-Norm wins on **training stability**: the residual path is "clean," allowing gradients to flow unimpeded. Post-Norm's residual path passes through LayerNorm, scaling the gradients and making deep networks harder to train.

Trade-off: Pre-Norm's final output variance accumulates across layers (one residual addition per layer), so Pre-Norm models need a final LayerNorm at the end. All modern LLMs use Pre-Norm.

RMSNorm as a Replacement for LayerNorm

Mentioned in Chapter 3 Section 4.3.5. LayerNorm: \frac{x - \mu}{\sigma} \gamma + \beta, requiring mean, variance, and bias computations.

RMSNorm simplifies: divide only by the root mean square, no mean subtraction, no bias.

> \text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\sqrt{\frac{1}{d}\sum_i x_i^2 + \epsilon}} \odot \boldsymbol{\gamma}

Empirically performs similarly to LayerNorm, but faster (saves one mean computation and one subtraction) and has fewer parameters (no \beta). LLaMA uses RMSNorm, and most subsequent models have followed suit.

4.2.5 Encoder/Decoder Divide

> *Position: Three Transformer variants — Encoder-only, Decoder-only, Encoder-Decoder — and where each is used.*

Three Architectures

The original Transformer (2017) is Encoder-Decoder — designed for translation tasks: the encoder reads the source language, and the decoder attends to the encoder output and generates the target language. Three lines evolved from this.

Encoder-Only (BERT Family)

Only an encoder stack, with bidirectional attention (each position sees all positions). Training objective: MLM (Masked Language Modeling) — mask 15% of tokens and have the model predict them.

Strength: top-tier for understanding tasks (classification, NER, sentence similarity), because every position's representation has full context.

Representatives: BERT (2018), RoBERTa, DeBERTa, Electra. Still widely used today as "text encoders" — most embedding models for sentence embeddings and RAG are in this family.

Decoder-Only (GPT Family)

Only a decoder stack, with unidirectional attention (causal mask — each position only sees itself and prior positions). Training objective: CLM (Causal LM) — predict the next token.

Strength: naturally fits generation. Also, emergent abilities like in-context learning and few-shot learning appear mainly in decoder-only large models.

Representatives: GPT series, LLaMA, Qwen, DeepSeek, Mistral — essentially all modern LLMs.

Encoder-Decoder (T5/BART)

Full encoder-decoder architecture. Diverse training objectives: T5 uses span corruption (mask a span and generate it); BART uses

denoising (recover from various corruptions).

Strength: tasks with clear input-output separation (translation, summarization, Q&A) — encoder focuses on understanding, decoder focuses on generation.

Representatives: T5 (Google), BART, mT5, FLAN-T5. Today's mainstream LLMs have largely abandoned this route — decoder-only is simpler and scales better.

Why Decoder-Only Won

1. **Simpler training**: one objective (next token), all text is training data. Encoder-Decoder requires designing input-output pairs.
2. **In-context learning**: put task instructions and examples in the prompt, the model self-adapts — no specialized architecture design needed.
3. **Generation capability**: even for understanding tasks, having decoder-only write out an answer works (and is often stronger, because it can do CoT).
4. **Scaling behavior**: empirically, decoder-only responds best to parameter increases.

Encoder-only remains active in scenarios that require embeddings — because decoder-only doesn't naturally produce "sentence-level representations" (one must approximate using the last token or average pooling). But for generation tasks, decoder-only dominates.

4.2.6 Transformer Evolution

> *Position: From the 2017 original to today — what are the key changes.*

Evolution Checklist

Over 7 years, the original Transformer has been modified in many places. Each of these changes can be understood using the tools

covered in the preceding sections. The table below shows the major changes from 2017 to 2024:

Component	Original Transformer (2017)	LLaMA-3 (2024)
Normalization	Post-Norm + LayerNorm	Pre-Norm + RMSNorm
Positional Encoding	Sinusoidal absolute	RoPE rotary positional encoding
Activation	ReLU	SwiGLU
Attention	MHA	GQA
Bias	All linear layers have bias	All removed
Dropout	Used	Mostly omitted (sufficient data prevents overfitting)
Tokenizer	WordPiece	SentencePiece/BPE, large vocabulary

Themes of Change

Roughly three themes:

Training stability: Pre-Norm, RMSNorm, removing bias, improved weight initialization — enabling large models to converge.

Compute/memory efficiency: GQA, SwiGLU, FlashAttention, MoE — faster and less memory for the same parameter count.

Long context: RoPE, YaRN, sliding window attention, MLA — enabling models to process longer sequences.

What Hasn't Changed

Interestingly, **the Transformer skeleton hasn't changed in 7 years**: stacked "Attention + FFN + residual" blocks. All

improvements are refinements to individual components, without any architectural revolution.

Candidate disruptors Mamba, RWKV, and RetNet gained research traction, but by the end of 2024 mainstream commercial deployments remain pure Transformer. This shows that Transformer's generality is extremely strong, and the improvement space is far from exhausted.

PART THREE

4.3 Positional Encoding and Long Context

Attention itself has no notion of order — positional encoding provides it. Long contexts require positional encoding to "extrapolate."

4.3.1 History of Positional Encoding

> *Position: From sinusoidal absolute encoding, to learned, to relative positional encoding, to RoPE — the evolutionary logic.*

Why Positional Encoding Is Needed

Attention is a "set operation" — if you permute the input token order, the attention output for each position permutes accordingly, but *semantically* each position's representation is unchanged. "The cat sat on the mat" and "mat the on sat cat The" would produce identical intermediate representations. Clearly

wrong. So positional information must be injected at each position.

Sinusoidal Absolute Encoding (Original Transformer)

Assign each position a d-dimensional vector using sinusoids at different frequencies:

> $\text{PE}(\text{pos}, 2i) = \sin(\text{pos} / 10000^{2i/d})$

> $\text{PE}(\text{pos}, 2i+1) = \cos(\text{pos} / 10000^{2i/d})$

Added to the token embedding. Advantage: no parameters, and has some "guessing" ability when extrapolating beyond training length (because it's a continuous function). Problem: extrapolation is actually terrible in practice — a model trained to position 2048 breaks at position 2049.

Learned Positional Encoding

GPT-2 simplified: give each position a learnable embedding. Supports whatever length it was trained on — exceeding that limit breaks immediately. More expressive within the training window but completely unable to extrapolate.

Relative Positional Encoding

A shift in approach: what truly matters is not the absolute position "word #5" but the relative distance "how many words apart."

T5's approach: add a bias b_{i-j} to the attention score, where b is learnable. The same bias is used for the same distance. ALiBi (2022) is more aggressive: use a fixed linear decay $-m \cdot |i - j|$ as a bias with no learnable parameters. ALiBi trained to 1024 length can extrapolate to 4096+.

RoPE

Proposed by RoFormer (2021). Today almost all mainstream models (LLaMA, Qwen, DeepSeek, Mistral) use it. Deep dive in the next section.

Comparison

Method	Extrapolation	Parameters	Representative Models
Sinusoidal absolute	Poor	None	Original Transformer
Learned absolute	None	L×d	GPT-2
T5 bias	Limited	Small	T5
ALiBi	Good	H (one m per head)	BLOOM, MPT
RoPE	Good (+ YaRN etc.)	None	LLaMA/Qwen/DeepSeek

4.3.2 RoPE Deep Dive

> *Position: Rotary positional encoding — mathematically elegant, simple to implement, the de facto standard.*

Core Idea

RoPE does not add a position vector to the input; instead, it *rotates Q and K* — so that $\mathbf{q}_m \cdot \mathbf{k}_n$ naturally encodes the relative position m - n.

Two-Dimensional Case

Start with the 2D case. Treat each pair of adjacent dimensions of Q and K as a complex number (or 2D plane vector). For position m, rotate $\mathbf{q}_m$ by angle $m\theta$:

> $$\begin{pmatrix} q_m^{(1)} \\ q_m^{(2)} \end{pmatrix} \to \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} q_m^{(1)} \\ q_m^{(2)} \end{pmatrix}$$

Similarly rotate $\mathbf{k}_n$ by angle $n\theta$. Then compute the dot product:

> $$\mathbf{q}_m^{\text{RoPE}} \cdot \mathbf{k}_n^{\text{RoPE}} = f(m - n)$$

Depends only on $m - n$! This is exactly the property that relative positional encoding seeks.

Intuition: treat the vectors as complex numbers q_m, k_n. Rotating by $m\theta$ is equivalent to multiplying by $e^{im\theta}$. The dot product becomes $q_m e^{im\theta} \cdot (k_n e^{in\theta})^* = q_m k_n^* e^{i(m-n)\theta}$ — depends only on $m-n$.

High Dimensions: Pairwise Rotation

For d-dimensional Q and K vectors, pair up dimensions two-by-two, each pair independently performs a 2D rotation, but with different frequencies:

> $$\theta_i = 10000^{-2(i-1)/d}, \quad i = 1, \ldots, d/2$$

Low-index dimensions use high frequency ($\theta_1 = 1$, rotating 1 radian per step); high-index dimensions use low frequency ($\theta_{d/2} \approx 10000^{-1}$, rotating very little per step). This lets different dimensions capture positional differences at different scales.

Implementation

```
import torch

def build_rope_freqs(d_head, max_seq_len, base=10000):
    freqs = 1.0 / (base ** (torch.arange(0, d_head, 2).float() /
d_head))
```

```
    t = torch.arange(max_seq_len).float()
    freqs_cis = torch.outer(t, freqs)
    return torch.polar(torch.ones_like(freqs_cis), freqs_cis)

def apply_rope(x, freqs_cis):
    x_complex =
torch.view_as_complex(x.float().reshape(*x.shape[:-1], -1, 2))
    x_out = x_complex * freqs_cis[:x.size(-2)]
    return torch.view_as_real(x_out).flatten(-2).type_as(x)

Q = apply_rope(Q, freqs_cis)
K = apply_rope(K, freqs_cis)
```

Some details: RoPE only applies to Q and K, not V — V carries content information and shouldn't carry position; `freqs_cis` can be precomputed and cached, with zero overhead during inference.

Three Key Advantages of RoPE

1. **Relative position**: naturally encodes distance rather than absolute position
2. **Decay property**: inner products naturally decay with distance — matching the intuition that "distant words have weaker associations"
3. **Extrapolation potential**: unseen positions can be extrapolated — continuous trigonometric functions don't suddenly break (though significant extrapolation still needs YaRN and similar techniques)

4.3.3 Long Context Extrapolation

> *Position: How to use a model trained on 4K context for 128K — PI, NTK-aware, YaRN, LongRoPE.*

The Problem

Training seen positions 0 to 4K; at inference a 64K sequence arrives — high-frequency rotation angles have cycled through millions of radians at position 64K, a rotation state the model has never seen, causing performance collapse.

PI: Position Interpolation

Proposed by Meta in 2023. The idea is straightforward: *compress* inference-time positions back into the training range. If training length is L_{train} and target length is L_{new}, map position m to $m \cdot L_{\text{train}} / L_{\text{new}}$.

Mathematically equivalent to reducing RoPE rotation frequencies by a factor of $L_{\text{new}}/L_{\text{train}}$. Downside: high-frequency information becomes blurry; the model needs slight fine-tuning (a few hundred steps) to adapt.

NTK-aware Scaling

PI compresses all dimensions equally. NTK-aware (community-proposed in 2023) observed: **low-frequency dimensions shouldn't be compressed (they haven't completed a single cycle yet)**; only compress high frequencies.

Concretely: increase RoPE's base from 10000 to a larger value (e.g., 40000). This makes all frequencies smaller, but affects originally-small low frequencies less — precisely preserving large-scale information.

Benefit: 2-4× extrapolation without fine-tuning, with minimal quality degradation.

YaRN

Proposed by Nous Research in 2023 (Yet another RoPE extensioN). Builds on NTK-aware with finer adjustments: divides dimensions into three tiers by frequency (high-frequency unchanged, mid-frequency NTK scaling, low-frequency PI interpolation); adds temperature scaling to compensate for variance changes from length variation.

YaRN with minimal fine-tuning (~1B token scale) can extend LLaMA-2 from 4K to 128K with minimal quality loss. The mainstream approach in the open-source community today.

LongRoPE

Proposed by Microsoft in 2024, an upgrade to YaRN. Core innovation: **rather than using the same scaling factor for all RoPE dimensions, search for the optimal per-dimension scaling** — some dimensions compressed more, others less. Significant search cost, but the payoff: feasible on LLaMA-3 at 2M tokens.

Native Long-Context Training

Extrapolation is the easy approach, but training from scratch is better. Major labs increasingly train directly on long documents:

LLaMA-3: 8K pretraining, then extended to 128K using 800M tokens
Qwen2.5-1M: extended from 32K to 256K/1M using staged long-document training
Gemini 1.5: claims 1M or even 10M context; technical details not public

The Practical Meaning of Context Length

A model "supporting" 128K does not mean it "uses" 128K well. The **Lost in the Middle** phenomenon: information in the middle of long contexts tends to be ignored by the model. Evaluation benchmarks: Needle in a Haystack (hide a key sentence in a long document and ask the model to find it); RULER (comprehensive multi-task long-context evaluation); LongBench. Many models claiming 100K context see accuracy plummet beyond 50K. Real-world engineering requires testing before trusting claims.

PART FOUR

4.4 Attention Optimization Variants

The quadratic complexity of attention is LLM's greatest bottleneck — the evolution from MHA to MLA to Mamba.

4.4.1 MQA / GQA / MLA

> *Position: Successive generations of solutions for reducing KV Cache — from MQA to DeepSeek's MLA.*

Why KV Cache Is the Bottleneck

During inference, generating each new token requires computing the new token's Q, K, V; appending the new K, V to the existing cache; and having the new Q perform attention over all K, V. Recomputing all prior K, V each time costs O(L^2) computation. **KV Cache** caches previously computed K, V so a new token only computes its own share, bringing total computation to O(L).

But KV Cache consumes GPU memory. Estimate: `2 × layers × H × d_head × seq_len × 2 bytes (FP16)`. LLaMA-3-70B at L=8K requires KV Cache ≈ 2.5 GB per sample. At batch=10 that's 25GB — more than the model weights. At long contexts, KV Cache is the primary memory killer.

MHA / MQA / GQA

Standard MHA: H heads with independent Q, K, V per head. KV Cache stores H sets of K and H sets of V. This is the baseline.

MQA (Noam Shazeer 2019): the extreme — all heads share a single K, V; only Q is independent per head. KV Cache shrinks by H×!

But quality degrades noticeably — a single K, V lacks expressiveness. Used by PaLM and Falcon.

GQA: a compromise between MHA and MQA. H heads divided into G groups, each group sharing one K, V. When G = H it degrades to MHA; when G = 1 it degrades to MQA. LLaMA-3-70B uses G = 8 (64 heads sharing 8 K, V groups), shrinking KV Cache by 8× with almost no quality loss.

Today LLaMA, Qwen, and Mistral all use GQA.

```
class GroupedQueryAttention(nn.Module):
    def __init__(self, d_model, n_heads=64, n_kv_heads=8):
        super().__init__()
        self.H = n_heads
        self.G = n_kv_heads
        self.d_head = d_model // n_heads
        self.Wq = nn.Linear(d_model, n_heads * self.d_head,
bias=False)
        self.Wk = nn.Linear(d_model, n_kv_heads * self.d_head,
bias=False)
        self.Wv = nn.Linear(d_model, n_kv_heads * self.d_head,
bias=False)
        self.Wo = nn.Linear(d_model, d_model, bias=False)

    def forward(self, x):
        B, L, _ = x.shape
        Q = self.Wq(x).view(B, L, self.H,
self.d_head).transpose(1, 2)
        K = self.Wk(x).view(B, L, self.G,
self.d_head).transpose(1, 2)
        V = self.Wv(x).view(B, L, self.G,
self.d_head).transpose(1, 2)
        K = K.repeat_interleave(self.H // self.G, dim=1)
        V = V.repeat_interleave(self.H // self.G, dim=1)
        S = Q @ K.transpose(-2, -1) / self.d_head ** 0.5
        A = torch.softmax(S, dim=-1)
        out = (A @ V).transpose(1, 2).reshape(B, L, -1)
        return self.Wo(out)
```

MLA: DeepSeek's Innovation

DeepSeek-V2 introduced MLA (Multi-head Latent Attention), which further compresses KV Cache.

Core idea: **instead of storing K and V, only store their low-rank latent representations**. Specifically:

> $$\mathbf{c}_{\text{kv}} = W_{DKV} \mathbf{x} \in \mathbb{R}^{d_c}$$

$d_c \ll d_{model}$. This compressed vector $\mathbf{c}_{\text{kv}}$ is stored in the cache. When K, V are needed, recover them via up-projection:

> $$\mathbf{k} = W_{UK} \mathbf{c}_{\text{kv}}, \quad \mathbf{v} = W_{UV} \mathbf{c}_{\text{kv}}$$

Analogous to LoRA — using low-rank decomposition to compress matrices. But MLA's elegance is: **the up-projection matrix W_{UK} can be merged with Q's projection, so no actual up-projection is needed during inference**.

Specifically, the attention score $\mathbf{q}^\top \mathbf{k} = \mathbf{q}^\top W_{UK} \mathbf{c}_{\text{kv}} = (\mathbf{q}^\top W_{UK}) \mathbf{c}_{\text{kv}}$. Absorb W_{UK} into Q's projection upfront; at inference time compute $\tilde{\mathbf{q}}^\top \mathbf{c}_{\text{kv}}$ directly — KV Cache only contains the compressed vectors.

DeepSeek-V2 uses $d_c = 4 d_{head}$, shrinking KV Cache by 90%+ compared to equivalent MHA, with even better quality (low-rank decomposition acts as a form of regularization).

MLA's Handling of RoPE

MLA has one awkward point: applying RoPE to K breaks the math of "merging matrices." DeepSeek's solution is to split K into two halves — **a portion without positional information** (goes through MLA compression) + **a small portion with RoPE** (not compressed, stored normally). This half/half split is an engineering compromise.

DSA: DeepSeek Sparse Attention (Introduced in V3.2)

DeepSeek's V3.2, released in October 2025, introduced a further innovation — Sparse Attention (DSA). Motivation: in long-context settings, MLA has already compressed KV Cache very small, but attention itself remains O(L^2) computation. DSA adds "fine-grained sparsity" on top of MLA — each Q only attends to an important subset of K, rather than all of them.

Key innovations:

1. **Dynamic top-k selection**: for each query, use a lightweight scoring function to find the most relevant small set of K; only perform attention on these K. Different queries select different K.
2. **Seamless integration with MLA**: sparse + low-rank compression together reduce long-context compute cost by ~50% with almost no quality loss.
3. **Unified thinking/non-thinking**: V3.2 inherits V3.1's hybrid mode — the same model can answer quickly or reason deeply.

DSA is the first time, after years of sparse attention research, that "quality matches dense attention with significantly lower cost" has been achieved in a large model. This also shows that the sparse attention direction covered in 4.4.2 hasn't failed — previous approaches just weren't fine-grained enough.

V3.2 also has a "Speciale" variant that pushes reasoning capability to reportedly near the level of top closed-source models — the open-source benchmark at the end of 2025.

4.4.2 Sparse Attention

> *Position: Abandoning "every position attends to all positions," attending only to a key subset — replacing O(L^2) with O(L \log L) or even O(L).*

The Idea

Most weights in full attention are actually small — most position pairs have weak associations. Sparse attention computes selectively, saving time and memory.

Sliding Window

Simplest approach: each position only attends to neighbors within a window of size w. Complexity O(L w); linear when w is fixed. Used by Longformer, BigBird, Mistral (window 4096). Problem: distant information can't reach. Solution: after stacking multiple layers, the receptive field grows (layer k has receptive field k w).

Dilated Attention

Strided attention — attend to every d-th position. Similar to dilated convolution. After a few layers, covers a large range.

Global Token

Designate a few "global tokens" (e.g., [CLS]) that can attend to all positions and all positions can attend to them. Other positions only attend locally. Global tokens serve as "information aggregation points." BigBird combines Sliding Window + Dilated + Global, providing both local detail and long-range bridging.

Routing Attention

Dynamically select which positions to attend to. Reformer uses LSH (Locality-Sensitive Hashing) to find similar Q, K pairs; Routing Transformer uses clustering. These methods are effective on some tasks but complex to implement.

Sliding Window + Attention Sink

StreamingLLM (2023) observed an interesting phenomenon: the initial few tokens (attention sinks) are crucial for maintaining stable attention distributions — even if semantically

unimportant, the model assigns them large attention weights (acting as normalization).

For streaming scenarios: retain initial few tokens + recent window + discard the middle — the model can handle arbitrarily long inputs without degradation. Practically useful.

Why Sparse Attention Hasn't Dominated

Despite theoretical elegance, sparse attention in practice has *not replaced* full attention in large models. Reasons:

1. FlashAttention optimizes full attention's constants to the extreme; the practical impact of L^2 complexity is less severe than theory suggests
2. Capability gains from model scaling far exceed efficiency from sparse attention
3. Hard-coded sparse patterns impose inductive biases that may not suit all tasks

The typical hybrid approach in today's open-source models is Mistral's "Sliding Window + full attention" — sliding in shallow layers, full attention in the last few layers — saving memory without sacrificing expressiveness.

4.4.3 Linear Attention and Mamba

> *Position: The resurgence of RNNs — replacing KV Cache with "state," O(L) inference, challenging Transformer.*

The Linear Attention Idea

Standard attention: \text{softmax}(QK^\top) V. If softmax can be removed, this can be rewritten as:

> \text{LinAttn}(Q, K, V) = Q (K^\top V)

Complexity: $K^\top V$ is $(d \times L)(L \times d) = d \times d$, $O(L d^2)$. Then $Q (K^\top V)$ is $O(L d^2)$. Total $O(L d^2)$ — linear!

But softmax can't simply be dropped — it ensures weight normalization and stability. Removing it consistently degrades quality. Various linear attention works (Linformer, Performer, Linear Transformer) propose different "softmax approximation" schemes, but none match full attention quality.

SSM: State Space Models

An idea borrowed from control theory. A linear state space system:

> $$\mathbf{h}_t = A \mathbf{h}_{t-1} + B \mathbf{x}_t, \quad \mathbf{y}_t = C \mathbf{h}_t$$

$\mathbf{h}_t$ is the "state" — a fixed-size vector that stores historical information. This is an RNN structure.

Early SSM work (S4, HiPPO) used carefully designed A matrices (HiPPO matrices) to enable long-term memory in the state. But A, B, C are fixed for every input — no ability to "selectively attend" to different positions. Performs worse than Transformer.

Mamba's Selectivity

Mamba's (2023, Albert Gu) key innovation: **making B, C, Δ (step size) depend on the input $\mathbf{x}_t$**. The "memory dynamics" at each position differ — the model can selectively "remember" or "forget."

> $$B_t = \text{Linear}_B(\mathbf{x}_t), \quad C_t = \text{Linear}_C(\mathbf{x}_t), \quad \Delta_t = \text{softplus}(\text{Linear}_\Delta(\mathbf{x}_t))$$

This gives the model "dynamic content addressing" capability — similar selectivity to attention, but using $O(L)$ state updates rather than $O(L^2)$ full comparisons.

Hardware-Aware Scanning

Selective SSM has an engineering challenge: input-dependent B_t, C_t means traditional "FFT-accelerated convolution" doesn't work. Mamba uses a "hardware-aware parallel scan" algorithm — based on prefix-sum ideas, maintaining parallelism on GPUs.

Training: forward pass can use parallel scan, O(L) FLOPs but O(\log L) wall-clock time.

Inference: pure recurrence, each step O(1) — fixed state size. No KV Cache! Memory usage doesn't grow with sequence length.

Mamba vs Transformer Comparison

\lt td>O(1) state update

Dimension	Transformer	Mamba
Training	Fully parallel	Parallel scan (O(L))
Inference	O(L) attention + O(L) KV Cache	Fixed
Inference memory	Grows with L (KV Cache)	
Long sequences	Limited by KV Cache	Naturally supported
Interpretability	Attention visualization	State not directly observable
Precise recall	Strong (attention precisely points)	Weak (state compression is lossy)

Mamba's weakness is in "precise recall tasks" — "given the first 1000 tokens, what is position 473?" Mamba tends to forget; Transformer can point precisely.

Hybrid Architectures

Early work like Jamba (AI21) and Zamba explored "Transformer + Mamba hybrid" — most layers use Mamba (fast), a few layers use

attention (precise recall). By 2025, this direction became one of the mainstream approaches:

NVIDIA Nemotron 2: Mamba2-Transformer hybrid; long-context speed superior to pure Transformer
IBM Granite 4: Mamba-Attention hybrid + MoE; enterprise production use
Qwen3-Next 80B-A3B (2025.9): introduces **Gated Delta Networks** (Mamba-like) + sparse MoE; achieves 235B non-hybrid model performance with only 3B active parameters
Qwen3.5 full lineup: all adopt a Gated Delta Networks + MoE hybrid architecture

Mamba-3 (ICLR 2026) further improves: introduces complex-valued state updates and MIMO (multi-input multi-output) architecture; at 1.5B scale, improves average downstream accuracy by 0.6 percentage points over Gated DeltaNet, while achieving comparable perplexity to Mamba-2 with half the state size.

This trend shows: Mamba is not "replacing Transformer" but serving as a complementary layer for long-context and low-memory scenarios, combining with attention to form a "better architecture."

There's also a path of continuing innovation within pure Transformer — DeepSeek V3.2's DSA optimizes within attention itself, achieving similar effects without taking the Mamba route. Both technical paths run in parallel through 2025-2026.

PART FIVE

4.5 Tokenizer and Embedding

The first gate for text entering the model — tokenizer details affect every aspect of LLM

> *capability.*

4.5.1 The BPE Family

> *Position: From BPE to WordPiece to SentencePiece — the three main lines of modern tokenizers.*

Why Not Character-Level, Why Not Word-Level

Two extremes:

Character-level: small vocabulary (128 ASCII characters in English), one token per character. Problem: long sequences — a 5-letter word becomes 5 tokens, causing O(L^2) explosion in attention.

Word-level: natural vocabulary (one token per word). Problem: vocabulary explosion (millions of words in English), OOV (out-of-vocabulary words not seen during training), unable to handle spelling variations. Chinese "word" boundaries are fuzzy, making it more problematic.

Subword is the compromise: common words are a single token; rare words are split into meaningful subwords. "unbelievable" → ["un", "believ", "able"].

BPE (Byte-Pair Encoding)

Originated as a data compression algorithm in 1994; borrowed for NMT tokenization in 2015.

Training:

1. Initialize vocabulary as all characters
2. Count the frequency of adjacent token pairs in the corpus
3. Merge the most frequent pair and add it to the vocabulary
4. Repeat steps 2-3 until the vocabulary reaches the target size (e.g., 50K)

```
from collections import Counter

def train_bpe(corpus, vocab_size):
    words = Counter()
    for line in corpus:
        for w in line.split():
            words[' '.join(list(w)) + ' </w>'] += 1
    vocab = set()
    for w in words: vocab.update(w.split())
    while len(vocab) < vocab_size:
        pairs = Counter()
        for w, freq in words.items():
            toks = w.split()
            for i in range(len(toks) - 1):
                pairs[(toks[i], toks[i+1])] += freq
        if not pairs: break
        best = pairs.most_common(1)[0][0]
        merged = ''.join(best)
        new_words = {}
        for w, freq in words.items():
            new_w = w.replace(' '.join(best), merged)
            new_words[new_w] = freq
        words = new_words
        vocab.add(merged)
    return vocab
```

Inference: apply the trained merge rules recursively to new text to get the token sequence.

WordPiece

Google's variant used in BERT. Difference: instead of merging "the most frequent pair," merge "the pair that maximizes language model likelihood." Similar results but theoretically cleaner.

SentencePiece

Open-sourced by Google in 2018. Addresses several pain points:

1. **No pre-tokenization**: trains directly on raw text, doesn't rely on the concept of "words" — friendly to languages without spaces like Chinese, Japanese, Korean

2. **Reversible**: tokenization can perfectly restore the original string (including spaces) — via a special ▁ symbol marking word beginnings
3. **Unigram algorithm**: a different merging strategy from BPE; starts with a large vocabulary and prunes

LLaMA, T5, mT5, and ChatGLM all use SentencePiece.

BBPE: Byte-level BPE

The variant used by GPT-2. First convert text to UTF-8 byte sequences (256 possible values), then apply BPE on the byte sequence. Benefit: **zero OOV** — any Unicode character can be represented (at least as a byte sequence), no [UNK] token needed. GPT-3/4, LLaMA, Qwen, DeepSeek all use BBPE. It is the de facto standard.

4.5.2 Modern Tokenizer Engineering

> *Position: Vocabulary size selection, Chinese/multilingual handling, tiktoken vs SentencePiece — engineering details.*

Vocabulary Size

Historical typical choices:

Model	Vocabulary Size
BERT	30K
GPT-2	50K
GPT-3	50K
LLaMA-2	32K
LLaMA-3	128K
Qwen 2.5	152K

DeepSeek-V2	100K
GPT-4o	200K

The trend toward larger vocabularies is clear. Benefits: shorter sequences, more multilingual-friendly (small vocabularies fragment Chinese characters excessively). Trade-offs: embedding table and LM Head grow larger (both are `vocab × d_model`); rare tokens are undertrained, resulting in poor embedding quality.

Chinese Token Rate

A common benchmark: token count for the same Chinese text across different tokenizers.

Tokenizer	Chinese chars per token
GPT-2 (50K)	≈ 0.5 (two tokens per Chinese character)
GPT-3.5/4 (cl100k)	≈ 1.5
LLaMA-2 (32K)	≈ 0.6
LLaMA-3 (128K)	≈ 1.3
Qwen (152K)	≈ 1.6
DeepSeek (100K)	≈ 1.5

Chinese domestic LLMs deliberately optimize Chinese token rates — packing more Chinese content into the same context window, outputting more Chinese characters for the same generation budget.

tiktoken

OpenAI's open-source BPE tokenizer, implemented in Rust, extremely fast. GPT-3.5 and GPT-4 use cl100k_base; GPT-4o uses o200k_base. Differences from SentencePiece: tiktoken is pure BPE; no training support, only encode/decode; peak performance.

HuggingFace Tokenizers

Rust implementation with Python bindings. Supports BPE, WordPiece, Unigram, BBPE, and more. Trainable, serializable, fast. The de facto standard in the open-source ecosystem — nearly all open-source model tokenizers are produced with the Tokenizers library.

Special Token Design

Modern LLMs have a pile of special tokens beyond regular ones: BOS/EOS; conversation roles (system/user/assistant); tool calls (tool_call/tool_response); reasoning delimiters (reasoning_start/end); large reserved pools for future features. Special token design is critical for conversation templates, prompt injection defense, and tool-calling protocols. LLaMA-3 reserves 250 special tokens; Qwen2.5 has 22 — wide variation across vendors.

Tokenizer Pitfalls

Trailing spaces: `"hello"` and `"hello "` tokenize differently; often overlooked, causing prompt template errors.

Chat template: different models have different conversation formats. HuggingFace Transformers' `apply_chat_template` is the standard — don't manually concatenate strings.

BOS/EOS handling: some models add BOS before fine-tuning, others don't. Mixing them causes training/inference inconsistency.

Unicode normalization: differences between NFC/NFD can cause identically-appearing characters to tokenize differently. Unify during the data cleaning phase.

4.5.3 Embedding Models

> *Position: From Word2Vec to BGE — the model family specialized for text similarity, a key component of RAG.*

From Word Embeddings to Sentence Embeddings

Early Word2Vec and GloVe were "word embeddings" — one fixed vector per word. But "bank" means different things in "river bank" and "bank account"; fixed vectors can't handle this.

The BERT era brought "contextual embeddings" — the same word has different vectors in different contexts. But BERT output is token-level; how do you get a representation for a whole sentence?

CLS token: during BERT pretraining, [CLS] represents the whole sentence, but in practice this representation performs poorly for similarity tasks
Mean pooling: average representations over all tokens; simple but loses precision
Sentence-BERT: fine-tune specifically with a dual-encoder structure to make "similar sentences have similar representations" — qualitative improvement for similarity tasks

Modern Embedding Models

Model	Dimensions	Context	Characteristics
OpenAI text-embedding-3	256-3072	8K	Commercial; Matryoshka adjustable
Cohere Embed v3	1024	512	Commercial; strong multilingual
BGE (BAAI)	512/768/1024	512-8K	Open-source; strong in Chinese and English
Qwen3-Embedding	1024/2048/4096	32K	Open-source; long context
NV-Embed	4096	32K	Open-source; top MTEB rankings
E5	1024	512	Microsoft open-source; multilingual

Training Methods

Modern embedding training is multi-stage: weakly supervised pretraining (contrastive learning from web text pairs); supervised fine-tuning (refining with labeled similar pairs); hard negative mining (finding pairs that look similar but aren't).

Contrastive Learning Loss

The core of embedding model training is contrastive loss; the typical InfoNCE loss:

> $$\mathcal{L} = -\log \frac{\exp(\text{sim}(q, p^+) / \tau)}{\exp(\text{sim}(q, p^+) / \tau) + \sum_{p^-} \exp(\text{sim}(q, p^-) / \tau)}$$

q, p^+ are positive pairs; p^- are negative samples; \tau is the temperature. Goal: make q more similar to p^+ and farther from all p^-.

Matryoshka Embedding

A novel idea: train a 1024-dimensional embedding, but also make the **first 64, first 256, and first 512 dimensions** individually valid. Users can truncate as needed — take the full vector for quality, take the first N dimensions for speed.

OpenAI text-embedding-3 and some open-source models (like Matryoshka-BGE) support this feature. Flexible trade-off between storage and performance.

Embedding Quality Evaluation

MTEB (Massive Text Embedding Benchmark) is the industry standard — 56 tasks evaluating embedding models. Covers classification, clustering, retrieval, STS, and reranking. The leaderboard changes every few weeks, but NV-Embed, BGE-M3, and Qwen3-Embedding are in the first tier. Real-world selection requires testing on your specific use case.

PART SIX

4.6 Mixture of Experts

Sparse activation makes "trillion-parameter models" computationally feasible — nearly all the largest open-source models use MoE.

4.6.1 MoE Fundamentals

> *Position: Why MoE, how MoE — core mathematics and basic architecture.*

Motivation

Transformer parameters are mainly in FFN (computed in 4.2.1 as 2/3 of the total). FFN applies the same processing to every token — but different tokens may need different processing (nouns need different features than verbs). MoE's idea: replace a single FFN with **multiple "expert" FFNs**, each token activating only 1-2 experts. Effect: total parameters multiply, but compute barely increases.

Structure

A MoE layer:

1. **Router**: compute a score for each expert given input $\mathbf{x}$: $g_i = \text{softmax}(W_{router} \mathbf{x})_i$
2. **Top-K selection**: select the K highest-scoring experts (typically K=1 or 2)
3. **Routing**: send $\mathbf{x}$ only to selected experts; other experts don't compute
4. **Weighted combination**: $\mathbf{y} = \sum_{i \in \text{top-k}} g_i \cdot \text{Expert}_i(\mathbf{x})$

Compute-Parameter Trade-off

Given N total experts, each with the same parameters as the original FFN, activating K experts:

Total parameters: approximately N \times original FFN parameters + attention layer parameters
Active parameters (actually participating in forward pass): K \times original FFN + attention
Compute: proportional to active parameters

Mixtral 8x7B: 8 experts each ≈ 7B, activating 2. Total parameters 47B (not 56B, because attention is shared); active ≈ 13B — *inference like a 13B model, capability close to a 47B dense model*. This is MoE's core value proposition.

Routing Function

```
class Router(nn.Module):
    def __init__(self, d_model, n_experts, top_k=2):
        super().__init__()
        self.W = nn.Linear(d_model, n_experts, bias=False)
        self.top_k = top_k

    def forward(self, x):
        logits = self.W(x)
        top_k_logits, top_k_idx = logits.topk(self.top_k,
dim=-1)
        top_k_gates = top_k_logits.softmax(dim=-1)
        return top_k_idx, top_k_gates
```

Load Balancing

The biggest problem: if training is improper, **routing degenerates — always sending traffic to a few experts**, starving the others.

Solution: **load balancing loss**. Define f_i = fraction of tokens routed to expert i; P_i = average gate value for expert i over all tokens. Add an auxiliary loss:

> $\mathcal{L}_{\text{aux}} = N \sum_i f_i P_i$

Ideally $f_i = P_i = 1/N$, $\mathcal{L}_{\text{aux}} = 1$. If an expert is overloaded, both f_i and P_i are large, increasing the loss and pushing gradients to rebalance routing.

Sparse vs Dense

Almost all new MoE models choose the **fine-grained expert** architecture — many small experts (DeepSeek-V3 has 256, each small). Better than "8 large experts": more specialized experts, more diverse combinations, more natural load balancing.

Shared Experts

DeepSeek-V2/V3 uses "shared experts + routed experts": 1-2 shared experts that all tokens pass through, with the other N-2 selected by routing. Intuition: some knowledge (grammar, basic logic) is needed by all tokens and doesn't need to be routed each time.

4.6.2 MoE Engineering Practice

> *Position: MoE looks good on paper, is hard in engineering — training, deployment, communication, and load balancing all need to be solved.*

Training Challenges

Instability: routing is discrete (top-k selection), making gradients hard to propagate. Early training sees routing thrash around with no expert specialization — requires very long warmup to stabilize.

Expert parallelism: 8 experts can be placed on 8 GPUs, but this requires cross-device communication at every layer (tokens distributed by routing, then collected after computation). All-to-all communication becomes a bottleneck.

Dynamic shape: different experts receive different numbers of tokens — a small expert might receive only a few, a large one might receive tens of thousands. GPUs prefer regular batch shapes; dynamic shapes make kernel optimization difficult.

Expert Parallelism

MoE's distributed strategy has its own name: Expert Parallelism (EP). Distribute N experts evenly across P GPUs, N/P experts per GPU.

Forward flow: each GPU's tokens go through attention (normal TP/DP); Router determines which expert each token needs; **All-to-all**: each GPU sends tokens to the GPU holding the corresponding expert; each GPU's active experts process the received tokens; **All-to-all**: send results back to the GPU where the tokens originated.

Two all-to-all communications with large data volume, but acceptable under high-speed interconnects (NVLink + IB). DeepSpeed-MoE and Megatron-LM both implement this.

Token Dropping

Under load imbalance, an expert may receive more tokens than its capacity (capacity factor). Handling options: Drop (tokens exceeding capacity skip this layer — simple but loses information); Capacity Bias (add capacity penalty during training to encourage uniform routing).

Real large models set capacity factor to 1.25 (allowing 25% overflow), with minimal dropping.

MoE Inference Deployment

Inference is trickier than training:

Memory usage: all experts must be loaded (since routing destination is unknown in advance). Mixtral 8x7B inference requires 80GB+ — not the "feels like 13B" experience.

Batching efficiency: under small batch sizes, most experts idle (no tokens routed to them), GPU utilization is extremely low. MoE inference performance only shines at large batch sizes.

Expert placement: for single-machine multi-GPU inference, how to assign experts? Heuristic: place high-frequency experts on GPUs with closer communication; cold experts go farther or offload to CPU. vLLM, SGLang, and TGI all have specialized optimizations for MoE inference.

Representative Models

Model	Total Params	Active Params	Expert Config
Switch Transformer (2021)	1.6T	-	Top-1 routing
Mixtral 8x7B (2023)	47B	13B	8 expert, top-2
Mixtral 8x22B (2024)	141B	39B	8 expert, top-2
DeepSeek-V2 (2024)	236B	21B	160 routed + 2 shared, top-6
DeepSeek-V3 (2024)	671B	37B	256 routed + 1 shared, top-8
Qwen2 MoE	57B	14B	64 expert, top-8
Grok-1 (xAI)	314B	79B	8 expert, top-2
Qwen3-235B-A22B (2025.4)	235B	22B	8 routed, no shared experts
Qwen3-Next 80B-A3B (2025.9)	80B	3B	High expert count + shared; extreme sparsity
LLaMA 4 Maverick (2025)	-	-	2 active experts, hidden 8192 (few but large)

DeepSeek-V3.1 / V3.2 (2025)	671B	37B	Inherits V3 + DSA sparse attention + hybrid thinking
Kimi K2 (2025)	Large-scale MoE	-	Moonshot; strong reasoning and long context
GLM-4.5 / 4.6 (Zhipu)	MoE	-	Shared experts (similar to DeepSeek); retains attention bias
gpt-oss-120b (2025)	120B	-	OpenAI's first open-source MoE; retains attention bias

DeepSeek-V3 is the peak of open-source MoE in 2024; V3.1 unifies reasoning and non-reasoning; V3.2 introduces DSA to further reduce costs, becoming the open-source reasoning benchmark by end of 2025 — 671B total parameters but only 37B activated during inference.

2025 MoE Architecture Trends:

Fine-grained experts mainstream: shifting from 8 large experts to dozens or hundreds of small ones, explosive combination diversity
Shared experts diverge: DeepSeek/GLM retain shared experts (all tokens pass through); Qwen3 goes the opposite direction by removing shared experts
Extreme sparsity: Qwen3-Next 80B activates only 3B (3.75%); Qwen3.5-35B-A3B activates only 3B (8.6%)
MoE + Mamba hybrid: mixed architectures like Qwen3-Next, Granite 4, Nemotron 2
Engineering optimizations: DeepSeek-V3's multi-token prediction, FP8 training, and All-to-All communication optimization dramatically improve MoE training economics

PART SEVEN

4.7 Distributed Training

Models above the tens-of-billions scale don't fit on a single GPU — must span across GPUs and machines.

4.7.1 Data Parallelism (DDP)

> *Position: The simplest distributed strategy — complete model on each GPU, different batch shards.*

Basic Idea

N GPUs, each holding a complete model replica. A large batch B split into N parts, B/N per GPU. Each GPU computes its own forward loss and backward gradients. Gradients are **AllReduced** to aggregate into the full-batch average gradient; each GPU updates parameters with this gradient — parameters stay synchronized.

AllReduce

The core communication primitive of DDP. Given one tensor (gradient) per GPU, compute the sum (or average) of all GPUs' tensors, with each GPU ending up with the same result.

Multiple implementations exist; the most popular is **Ring AllReduce**: split each GPU's tensor into N chunks; first phase sends rings, each round each GPU adds one chunk to the next, after N-1 rounds each GPU has the complete sum of one chunk; second phase broadcasts each GPU's "completed chunk" around the ring, after N-1 more rounds all GPUs have all complete chunks.

Communication volume: each GPU sends and receives 2(N-1)/N \cdot S (S = tensor size), nearly independent of N — excellent scalability. NCCL is the de facto standard implementation on GPUs.

PyTorch DDP

```
import torch
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP

dist.init_process_group(backend='nccl')
local_rank = int(os.environ['LOCAL_RANK'])
torch.cuda.set_device(local_rank)

model = MyModel().cuda()
model = DDP(model, device_ids=[local_rank])

from torch.utils.data.distributed import DistributedSampler
sampler = DistributedSampler(dataset)
loader = DataLoader(dataset, sampler=sampler,
batch_size=per_rank_bs)

for batch in loader:
    loss = model(batch).mean()
    loss.backward()     # DDP automatically all-reduces
gradients
    optimizer.step()
```

Gradient Accumulation

Sometimes a large batch is desired but a single GPU can't hold the full batch. Gradient accumulation: run multiple forward+backward passes without calling step, accumulating until enough, then step. Combined with DDP to reduce gradient sync frequency (only one all-reduce just before step).

DDP Limitations

DDP's primary limitation: **each GPU must fit the complete model + gradients + optimizer state**.

Computing single-GPU requirements for a 7B FP16 model: parameters 14GB; gradients 14GB; Adam state (m, v, FP32) 56GB; activations ~10GB. Total ~100GB; A100 80G isn't enough, nor is H100 80G. So training above 7B requires sharding — enter ZeRO.

4.7.2 ZeRO Three Stages

> *Position: DeepSpeed's flagship technology — sharding optimizer state, gradients, and parameters to achieve linear memory reduction.*

Core Insight

In DDP, each GPU stores complete parameters, gradients, and optimizer state — but each GPU actually only needs to process its own portion of parameter updates; the state for other parameters is redundant. ZeRO (Zero Redundancy Optimizer, Microsoft 2020) says: **shard these redundancies across different GPUs**, fetching them via communication when needed.

Three Stages

ZeRO's three stages progressively shard more:

Stage	What's Sharded	Communication Overhead	Memory Savings (N GPUs)
ZeRO-1	Optimizer state	Same as DDP	Optimizer /N
ZeRO-2	+ Gradients	Slightly more	Optimizer and gradients /N
ZeRO-3	+ Parameters	Noticeably more	Everything /N

ZeRO-1

Simplest. Each GPU still stores complete parameters and gradients, but only 1/N of optimizer state (only responsible for updating 1/N of parameters).

Update flow: all GPUs forward; all GPUs backward, getting complete gradients (AllReduce as in DDP); each GPU only updates its responsible 1/N parameters (using its stored optimizer state); AllGather updated parameter shards to synchronize globally.

Adam state is the biggest component (8 bytes/parameter); ZeRO-1 already saves most of it.

ZeRO-2

Gradients are also not stored in full. During backward, ReduceScatter (reduce and scatter simultaneously), so each GPU ends up with only the gradients for its responsible 1/N parameters. Communication volume similar to ZeRO-1 (just different operation names). Further memory savings: no more full gradients.

ZeRO-3

Parameters are also sharded. Each GPU permanently holds only 1/N of parameters. Computing a layer during forward: AllGather that layer's complete parameters (all GPUs contribute their shard); immediately release after the layer completes (keep only your own shard); AllGather again for the next layer. Backward is the same — temporarily gather when needed, discard after use.

Memory reduced to the extreme: 7B model on 8 GPUs requires only ~12-15GB per GPU. Trade-off: forward and backward both require an extra AllGather round, noticeably increasing communication overhead.

FSDP

PyTorch's official ZeRO-3 implementation is called FSDP (Fully Sharded Data Parallel). Simpler interface than DeepSpeed, better integration with the PyTorch ecosystem.

```
from torch.distributed.fsdp import FullyShardedDataParallel as
FSDP
from torch.distributed.fsdp import MixedPrecision

mp = MixedPrecision(
    param_dtype=torch.bfloat16,
    reduce_dtype=torch.bfloat16,
    buffer_dtype=torch.bfloat16,
)
model = FSDP(
```

```
    model,
    mixed_precision=mp,
    sharding_strategy=ShardingStrategy.FULL_SHARD,  # ZeRO-3
    auto_wrap_policy=size_based_auto_wrap_policy,
    device_id=local_rank,
)
```

ZeRO-Offload and Infinity

More extreme ZeRO variants:

ZeRO-Offload: offload optimizer state and gradients to CPU memory, pulling back when updates are needed. Can train 10B+ models on a single GPU, but 2-3× slower.
ZeRO-Infinity: further offload to NVMe SSD. Theoretically enables trillion-parameter training on a single machine; in practice extremely slow. Suitable for research, not production.

When to Use ZeRO

Based on parameter count: < 1B: DDP suffices; 1B-10B: ZeRO-2 for balance; 10B-100B: ZeRO-3 or FSDP; 100B+: ZeRO-3 + tensor parallelism + pipeline parallelism.

4.7.3 Tensor Parallelism and Pipeline Parallelism

> *Position: Splitting individual layer computations (TP), splitting the layer sequence (PP) — distributing "large models" across multiple GPUs.*

Tensor Parallelism (TP)

Split the matrix multiplication within a single layer. Most typical: Linear layer $\mathbf{Y} = \mathbf{X} W$.

Column-parallel: split W by columns into $[W_1, W_2, \ldots]$, each GPU holds a portion. After computation, each GPU has

different columns of $\mathbf{Y}$; concatenate or AllGather to merge.

Row-parallel: split W by rows; input $\mathbf{X}$ is also split by columns correspondingly. Each GPU computes a partial result; AllReduce to sum.

Megatron-LM's attention and FFN TP combination: in Attention, W_Q, W_K, W_V are column-parallel (each GPU computes partial heads), W_O is row-parallel — no communication inside attention, one AllReduce at the exit; in FFN, W_1 is column-parallel, W_2 is row-parallel — no communication inside FFN, one AllReduce at the exit.

This requires only 2 AllReduces per layer. Communication is a bottleneck but manageable.

TP constraint: requires high-speed interconnect. TP works within a GPU group connected by NVLink (typically 8 GPUs); InfiniBand bandwidth across nodes isn't sufficient to support TP.

Pipeline Parallelism (PP)

Split an L-layer model into P segments, one segment per GPU. A batch of data flows sequentially across GPUs.

The naive implementation has a "bubble" problem — most GPUs idle most of the time. Solution: **split the batch into micro-batches and pipeline execution** (1F1B schedule).

GPipe (2018): split batch into M micro-batches, perform M sequential forwards then M backwards. Bubble fraction $(P-1)/(P+M-1)$; small when M is large.

PipeDream / 1F1B: interleave forward and backward passes; fewer bubbles.

Interleaved 1F1B (Megatron): each GPU holds multiple non-contiguous segments; bubbles further reduced.

TP vs PP

	TP	PP
What's split	Inside single layer	Layer sequence
Communication	Multiple times inside each layer	Small amount between segments (activation transfer)
Bandwidth requirement	High (NVLink)	Medium (IB works)
Scope	Typically within 8 GPUs	Typically cross-node
Constraints	Layer must be divisible	Bubbles; micro-batch scheduling complex

4.7.4 3D Parallelism and Training Stack

> *Position: Combining DP + TP + PP — the standard approach for 100B+ models. Also covering Megatron/DeepSpeed/FSDP.*

3D Parallelism

Real large-model training uses all three dimensions together:

TP: within node (NVLink 8-GPU group), splits single layers
PP: cross-node (IB connected), splits layer sequence
DP: the "remaining" dimension beyond the above two, splits data

Example with 512 GPUs (64 nodes × 8 GPUs): TP=8 (within node), PP=8 (8 groups of nodes), DP=8 (data parallelism dimension). 8 \times 8 \times 8 = 512.

Model state per GPU: holds \frac{1}{8} of single-layer parameters (TP); holds \frac{1}{8} of layers (PP); sees \frac{1}{8} of the batch (DP). Actual memory pressure per GPU ≈ model / 64, manageable.

Adding ZeRO

Apply ZeRO on the DP dimension to further save memory: ZeRO-1 (optimizer state sharded along DP dimension); FSDP (parameters, gradients, optimizer all sharded along DP).

GPT-4-scale training is rumored to use TP=8 + PP=16 + DP=64 + ZeRO-1, across 8192 H100s total.

Activation Recomputation

Backward pass needs intermediate activations from the forward pass. Storing them all consumes significant memory. **Gradient Checkpointing**: only keep activations at a few "checkpoint" positions; recompute from the nearest checkpoint during backward. Typically only keep each Transformer block's input; recompute one forward per block during backward — 30% extra compute in exchange for 3-5× reduction in activation memory. Essential for training 100B+ models.

Training Stack Comparison

Framework	Developer	Key Features
DeepSpeed	Microsoft	ZeRO origin; complete 3D parallelism
Megatron-LM	NVIDIA	TP/PP originator; extreme GPU optimization
FSDP	PyTorch official	Good integration; active community
Megatron-DeepSpeed	Microsoft + NVIDIA	Fusion of both
Colossal-AI	HPC-AI Tech	Chinese-developed; comprehensive strategies
torchtitan	PyTorch official	Emerged 2024; modern and clean

DeepSpeed and Megatron are industry de facto standards. Meta trains the LLaMA series using their own + FSDP approach. HuggingFace's accelerate library is a higher-level wrapper.

Training Recipe

Typical configuration for one 100B-scale model training run (analogous to LLaMA-3-70B):

- Hardware: 1024 H100s (128 nodes)
- Parallelism: TP=8, PP=4, DP=32, ZeRO-1
- Data: ~15T tokens
- Duration: ~8 weeks
- Optimizer: AdamW, bf16 mixed precision
- LR schedule: linear warmup + cosine decay
- Batch: global 8192 sequences × 8192 tokens ≈ 67M tokens/step

Electricity + hardware depreciation: tens of millions of dollars. So large-scale model training is a game for giants — but once trained and open-sourced, everyone benefits.

PART EIGHT

4.8 Inference Engines and Scheduling

Training is just the beginning — inference optimization is the key to whether LLMs can be practically deployed.

4.8.1 Inference Workload Characteristics

> *Position: Prefill vs Decode — LLM inference's two phases have fundamentally different bottlenecks.*

Two-Phase Model

LLM inference has two phases:

Prefill: process the entire input prompt (e.g., 1000 tokens); one forward pass computes the KV Cache for all tokens and predicts the next token.

Decode: based on the existing KV Cache, generate one new token per step — each forward pass processes only 1 new token but must read all historical KV Cache to perform attention.

A typical conversation: 100-token prompt + 500-token response = 1 prefill (100 tokens) + 500 decode steps.

Compute Characteristic Differences

	Prefill	Decode
Per-forward processing	Entire prompt (hundreds to thousands of tokens)	1 token
Compute	Large; compute-bound	Small; memory-bound
Primary cost	GEMM	Reading KV Cache
Batching benefit	Small (prompt is already large)	Large (combining multiple requests)

Decode's bottleneck is **memory bandwidth**. Generating each token requires reading all layers' KV Cache from HBM for attention. Model weights also need a full pass. Compute is minimal but data movement is extensive — GPU compute utilization drops to 5-10%. This characteristic drives the entire inference optimization field.

Key Metrics

TTFT (Time To First Token): time from request to first token returned. Determined by prefill. User-perceived latency.
TPOT (Time Per Output Token): average time between tokens. Determined by decode. Determines generation speed.

Throughput: system-wide tokens/s — most important for service providers.
Concurrency: how many requests can be served simultaneously.

Latency and throughput trade off. Large batches increase throughput but lengthen TPOT — how to schedule is the core problem of inference engines.

Hardware Bottleneck Analysis

H100 with LLaMA-3-70B FP16 decode: model weights 140GB; HBM bandwidth 3 TB/s; theoretical limit 3000 / 140 ≈ 21 tokens/s (single request).

Measured decode 25-40 tokens/s (vLLM data) — essentially approaching the bandwidth ceiling. Improvements only possible via: compressing weights (quantization), compressing KV Cache, or batching multiple requests (sharing bandwidth across requests).

4.8.2 KV Cache

> *Position: The soul of LLM inference — cutting O(L^2) complexity to O(L), but introducing memory pressure.*

Why It's Needed

During decode, generating the t-th token requires only the new token's $\mathbf{q}_t$ for attention, but needs **all prior positions' K, V** . If $\mathbf{k}_1, \ldots, \mathbf{k}_{t-1}$ were recomputed each time, that's O(t) extra computation per step — total $O(t^2)$. KV Cache caches all prior K, V — each new token only computes its own $\mathbf{k}_t$, $\mathbf{v}_t$ and appends to the cache. Total computation O(t).

Memory Footprint

LLaMA-3-70B (80 layers, 64 heads, d_head=128, GQA=8):

KV per token per layer: 2 \times 8 \times 128 \times 2 = 4 KB (FP16)

KV per token total: 80 \times 4 KB = 320 KB

A 10K token conversation: 3.2 GB per sample

A single H100 80G can serve only ~20 such conversations simultaneously — memory is the concurrency ceiling.

Problems with Naive KV Cache

Early inference engines (original Transformers, early TGI) allocated each sample's KV Cache as **contiguous, maximum-length** pre-reserved space.

Problems: wasted memory (reserving max_length=4096 but only using 500, 87.5% wasted); internal fragmentation (different request lengths, poor space utilization); external fragmentation (new requests need large contiguous blocks; small blocks freed by short requests don't suffice); low concurrency ceiling. This is what PagedAttention was designed to solve.

4.8.3 PagedAttention and vLLM

> *Orientation: OS virtual memory applied to KV Cache — vLLM's core innovation.*

Core Idea

PagedAttention's (UC Berkeley 2023) inspiration comes directly from OS virtual memory: **physical memory is divided into fixed-size pages; logical addresses are mapped to physical pages via a page table**.

Applied to KV Cache: partition GPU memory into fixed-size blocks (e.g., each block holds KV for 16 tokens); each request maintains a block table mapping its logical token positions → physical blocks; blocks are allocated on demand, not pre-

reserved; the Attention kernel looks up actual positions via the block table.

Results

- **Memory utilization**: from ~40% to ~96% (official data)
- **Concurrency**: 4-10× more concurrent requests on the same 80G memory
- **Throughput**: 2-4× improvement over prior approaches

Attention Kernel Modification

Standard attention kernels assume K, V are in contiguous memory. PagedAttention's kernel must look up K, V positions via the block table:

```
// simplified pseudocode
for each query q_i:
    output = 0; max_score = -inf; sum_exp = 0
    for each logical token j in query's block_table:
        block_id  = block_table[j / BLOCK_SIZE]
        inner_idx = j % BLOCK_SIZE
        k_j = K_pool[block_id * BLOCK_SIZE + inner_idx]
        v_j = V_pool[block_id * BLOCK_SIZE + inner_idx]
        score = q_i @ k_j / sqrt(d)
        // online softmax update
        new_max = max(max_score, score)
        sum_exp = sum_exp * exp(max_score - new_max) + exp(score
- new_max)
        output  = output  * exp(max_score - new_max) + exp(score
- new_max) * v_j
        max_score = new_max
```

The extra indirection has minor overhead, far outweighed by the memory pressure saved.

Copy-on-Write and Sharing

PagedAttention also supports KV Cache sharing — similar to OS fork+COW. Scenario: beam search or parallel sampling where multiple candidates share the same prefix. Naive approach: one

KV Cache per candidate; PagedAttention lets multiple sequences' block tables point to the same physical block, copying only on write.

Also applies to **Prefix Caching**: multiple requests share the same system prompt; the prompt portion's KV Cache can be shared — computed only once. QPS for high-concurrency LLM services increases dramatically.

vLLM and Others

The reference implementation of PagedAttention. vLLM has grown from a research project to the de facto standard open-source inference engine: PagedAttention + continuous batching + speculative decoding; broad HuggingFace model compatibility; TP and multi-GPU support; OpenAI-compatible API; active community with fast updates.

Not the only option. SGLang has its own **RadixAttention** (prefix sharing organized as a radix tree), faster for complex workflows (multi-turn, multi-candidate). TGI (HuggingFace's Rust implementation) is stable. TensorRT-LLM is extremely optimized on NVIDIA platforms but has a closed ecosystem.

4.8.4 Continuous Batching

> *Position: The evolution of dynamic batching — efficiently parallelizing requests of different lengths rather than waiting for a full batch.*

Traditional Static Batching

Early approach: a batch of requests starts and ends together, padded to the longest length.

Problems: must wait for a full batch before starting — high latency; when request lengths vary greatly, short requests idle waiting for long ones — low throughput; finished requests wait for others in the same batch, GPU idles.

Continuous Batching

The approach popularized by Orca (2022) and vLLM (2023). Core: **schedule at the iteration (each decode step) granularity, not the whole request**.

Each decode step: check current active requests, form a batch for one forward; update each request's output; finished requests return immediately and free resources; new requests join the batch in the next iteration.

GPU utilization rises from 40% to 80%+.

Prefill/Decode Scheduling

A detail: prefill and decode have very different compute characteristics, so mixing them in scheduling isn't always most efficient. Two styles:

Iteration-level mixing (early vLLM): each iteration may have both prefill and decode requests. Prefill is compute-bound, decode is memory-bound; mixing complements each other — but iteration time is prolonged by prefill, affecting decode TPOT.

Chunked Prefill (supported by vLLM, SGLang): split long-prompt prefill into chunks; each iteration mixes a small amount of prefill with decode — balancing the two.

Prefill-Decode separation (SplitWise, DistServe): prefill and decode run on different GPUs/machines — prefill machines are compute-intensive, decode machines are bandwidth-intensive. KV Cache transferred cross-node. Suitable for ultra-large scale.

Preemption and Swap-Out

What to do when GPU memory is full? Preempt some requests — swap their KV Cache to CPU (or simply discard and recompute).

Policies: First-Come-First-Served (FCFS), priority (premium users first), or swap fairness. vLLM implements these mechanisms; robust in production environments.

4.8.5 Speculative Decoding

> *Position: A small model guesses, a large model verifies — 2-3× inference speedup with no quality loss.*

Basic Idea

The primary bottleneck of decode is that each forward produces only 1 token. Can multiple tokens be produced at once? Standard methods can't — each token depends on the previous one.

Speculative Decoding (independently proposed by Google and DeepMind): use a **small model (draft)** to quickly generate K candidate tokens; use a **large model (target)** in one forward pass to verify whether these K tokens conform to its distribution; accept conforming tokens (sequentially from the front), reject from the first non-conforming token onward.

Key insight: the large model's single forward computed logits for K positions, completing "verification + next token generation" in one step.

Why the Distribution Is Unchanged

Speculative decoding's mathematical guarantee: **the output distribution is identical to using the large model directly** (statistically). Derivation:

Let the small model distribution be q(x), the large model distribution be p(x). The acceptance probability for candidate token x:

> $$\min\left(1, \frac{p(x)}{q(x)}\right)$$

On rejection, sample from the **adjusted distribution**:

> $$p'(x) = \frac{\max(0, p(x) - q(x))}{\sum_{x'} \max(0, p(x') - q(x'))}$$

This rejection sampling mechanism guarantees the final sampling distribution is exactly p(x).

Acceptance Rate

Efficiency depends on the "alignment" between the small and large models. Higher alignment → higher acceptance rate → greater speedup.

Typical pairing: LLaMA-3-70B + LLaMA-3-8B: acceptance rate ~70%, speedup 2-3×; same-family small/large model pairings work best — same family means similar training distributions.

Medusa and EAGLE

Variants: instead of an independent small model, add a few "heads" to the target model to directly predict multiple future tokens.

Medusa (2024): add K extra heads on top of the original model, each predicting one position of the next K tokens. Adding a few hundred M parameters; K candidates generated in parallel during inference.

EAGLE / EAGLE-2: more advanced — not "multi-head prediction" but a lightweight autoregressive sub-network that generates candidates from the main model's hidden states. Acceptance rate can reach 80%+, speedup 3×+.

Engineering Trade-offs

Speculative decoding trade-offs: extra memory (small model or extra heads); complex implementation with interaction pitfalls with batching/KV Cache; large benefit under small batch; diminishing benefit under large batch (memory-bound pressure reduced).

vLLM, TGI, and TensorRT-LLM all support speculative decoding. Significant benefit for high-QPS online serving; small benefit for internal tooling services — may not be worth the complexity.

PART NINE

4.9 Inference Compression

Quantization is not an "add-on" to inference speedup; it is the prerequisite for running open-source large models at all.

4.9.1 AWQ / GPTQ Deep Dive

Position: The two mainstream LLM quantization methods — mathematical principles and engineering differences.

LLM Quantization Specifics

Chapter 3 Section 4.7.2 covered general quantization principles. LLMs are special: enormous weight count (70B × 2B = 140GB); INT4 compression brings this to 35GB — fits on a single A100/H100; activations have extreme outliers — some dimensions have very large values that suffer severe loss with standard quantization; high precision requirements — a single wrong token can derail an entire response.

GPTQ

GPTQ (2022) uses second-order information for column-by-column quantization. Core observation: quantization error introduced by one column of weights can be compensated by adjusting the remaining columns.

Mathematically, solving:

> \arg\min_{\hat{W}} \| \mathbf{X}(W - \hat{W}) \|^2

Use a small amount of calibration data to approximate the Hessian; quantize column by column: for each column j, find the nearest quantized value $\hat{w}_j$; compute error $\delta = w_j - \hat{w}_j$; "absorb" the error into remaining unquantized columns to minimize total error; continue to the next column.

Algorithm complexity $O(d^3)$ (d = layer width); quantizing one layer takes a few minutes. INT4 quantization achieves 95%+ of FP16 precision.

AWQ

AWQ (Activation-aware Weight Quantization, MIT 2023) observed: **not all weights are equally important — weights corresponding to large activation magnitudes are more important**.

Traditional quantization treats all weights equally; quantization error for "important weights" is amplified (large activation × quantization error = large loss). AWQ's fix: *pre-scale* important channels to reduce their relative quantization error.

Specifically, for each column of the weight matrix (corresponding to one activation channel):

> $$W' = W \cdot s, \quad \hat{W} = \text{Quantize}(W')$$

During inference, multiply activations by s: $\mathbf{x}' = \mathbf{x} / s$. Mathematically equivalent but quantization error distribution changes — important weights have smaller error; unimportant weights have larger error (doesn't matter).

How to choose s? Measure activation magnitudes using calibration data; important channels use $s > 1$ (weight amplified; smaller relative quantization error); unimportant channels use $s \le 1$.

GPTQ vs AWQ

GPTQ	AWQ

Approach	Column-by-column quantization with error compensation	Channel scaling to distribute error
Compute	High (Hessian)	Medium
Quality	Slightly better, especially INT3	Good; INT4 sufficient
Inference speed	Same (both INT4 GEMM)	Same
Ecosystem	AutoGPTQ, ExLlamaV2	vLLM, TGI native support

In practice AWQ is more commonly chosen — simpler engineering, same inference speed, negligible precision difference.

GGUF and llama.cpp

The quantization format used by llama.cpp is GGUF, with its own quantization schemes (Q4_K_M, Q5_K_S, Q8_0, etc.). Similar to AWQ/GPTQ but optimized to the extreme on both CPU and GPU. First choice for edge deployment (personal computers, phones).

Ultra-Low Bit

The 2024 frontier is **INT2/INT1**. BitNet (Microsoft) trains with ternary (-1, 0, +1) weights but requires training from scratch.

More practical are **QuIP** and **AQLM** — first apply an orthogonal transform to "smooth" weights, then quantize to 2-3 bits; precision approaches INT4.

General production deployment remains INT4 mainstream — precision starts degrading noticeably below INT3.

Practical Quantization Workflow

```
# quantize LLaMA-3-70B with AutoAWQ
from awq import AutoAWQForCausalLM
from transformers import AutoTokenizer

model_path = "meta-llama/Meta-Llama-3-70B-Instruct"
```

```
quant_path = "Meta-Llama-3-70B-Instruct-AWQ"

model = AutoAWQForCausalLM.from_pretrained(model_path)
tokenizer = AutoTokenizer.from_pretrained(model_path)

quant_config = {
    "zero_point": True,
    "q_group_size": 128,
    "w_bit": 4,
    "version": "GEMM",
}
model.quantize(tokenizer, quant_config=quant_config,
calib_data="pile_calib.jsonl")
model.save_quantized(quant_path)

# load directly with vLLM:
# vllm serve Meta-Llama-3-70B-Instruct-AWQ --quantization awq
```

4.9.2 KV Cache Quantization and FP8

> *Position: Beyond weights — KV Cache and activations also need compression to further free memory.*

KV Cache Quantization

INT4 weight quantization is now standard, but KV Cache remains FP16 — at long contexts it can be larger than the model weights.

INT8 KV Cache: simple and effective. K, V per token quantized by channel. Memory halved; precision almost unchanged. Supported by vLLM and TensorRT-LLM.

INT4 KV Cache: memory halved again, but precision loss starts to become noticeable. Requires careful group quantization (one scale per 64 or 128 tokens) to maintain quality.

KIVI (2024): Key quantized by channel, Value quantized by token — because Key and Value have different distribution characteristics. Can maintain precision at INT2.

KV Cache quantization offers maximum benefit in long-context scenarios. A 100K token conversation's KV Cache drops from 30GB to 7.5GB (INT4), completely changing deployment possibilities.

FP8 and Hopper

NVIDIA H100 Hopper architecture natively supports FP8 — two formats:

E4M3: 4-bit exponent, 3-bit mantissa. Smaller dynamic range, relatively higher precision. Stores weights and activations.
E5M2: 5-bit exponent, 2-bit mantissa. Larger dynamic range, lower precision. Stores gradients and other quantities where outliers must be preserved.

FP8 compute doubles compared to FP16 on H100 Tensor Cores. Importantly, FP8 can also be used for training — unlike INT8 which is inference-only.

Next generation: NVIDIA Blackwell. In 2024, NVIDIA released the Blackwell architecture (B200/GB200), with single-chip FP4 compute reaching 20 PFLOPS and 192GB HBM3e — the next-generation flagship training/inference chip after the H100. Blackwell supports FP4 precision natively, further reducing inference cost; NVLink 5 provides 1.8 TB/s inter-chip bandwidth. From 2025, major cloud providers began deploying GB200 clusters, which have become the new standard for large-scale LLM training.

FP8 Training

DeepSeek-V3, LLaMA-3, and Qwen 2.5 have all tried FP8 training. Key technical points: forward activations use FP8; backward gradients use FP8; weight master copies remain FP32 (for optimizer updates); per-tensor or per-token scaling prevents outlier overflow; some sensitive operations (normalization, softmax) retain FP16/FP32.

DeepSeek-V3 measured FP8 as 1.5-2× faster than BF16 with almost no precision loss. One of the major 2024 training advances.

FP8 Inference

FP8's compute advantage in inference + smaller memory footprint than FP16. But toolchain maturity currently lags INT4 — the main challenge is **dynamic range**: INT4 relies on group quantize calibration; FP8's per-tensor scale requires more precision.

TensorRT-LLM has the best FP8 inference support (being NVIDIA's product). vLLM and SGLang are following suit.

KV Cache + Weight Dual Quantization

Extreme compression route: INT4 weights + INT4 KV Cache. LLaMA-3-70B on a single H100 (80G):

- Weights: 140GB FP16 → 35GB INT4
- KV Cache: 2.5GB per sample at 8K context → 0.6GB INT4
- Concurrency: from under 10 to 50+

This is the key to serving open-source 70B models at scale on a single H100.

PART TEN

4.10 RAG

Giving LLMs access to what they don't know — enterprise knowledge, real-time data, and private documents all flow through this gateway.

4.10.1 Basic RAG Pipeline

> *Orientation: the complete naive RAG workflow — design decisions at every step.*

Why RAG?

An LLM's parameters encode knowledge from training time. Three fundamental problems:

1. **Staleness**: training data has a cutoff; the model knows nothing after that point
2. **Hallucination**: when the model doesn't know, it fabricates
3. **Private data**: enterprise-internal documents were never seen during training

Solution A is fine-tuning — expensive, and must be repeated every time new knowledge arrives. Solution B is RAG (Retrieval-Augmented Generation): **at query time, retrieve relevant documents from a knowledge base, splice them into the prompt, and let the model answer based on the retrieved context**.

Naive RAG Pipeline

```
flowchart LR subgraph Offline DOC[Documents] --> CHUNK[Chunking]
  CHUNK --> EMB[Embedding] EMB --> VDB[Vector DB] end subgraph
    Online Q[Query] --> QE[Query Embedding] QE --> SEARCH[ANN
 Search] VDB --> SEARCH SEARCH --> CTX[Retrieved Chunks] CTX -->
PROMPT[Prompt Construction] Q --> PROMPT PROMPT --> LLM[LLM] LLM
                        --> A[Answer] end
```

Chunking

Documents are split into small chunks, each embedded independently. Why? A vector must represent the full chunk content — chunks that are too large can't be captured in a single vector; the LLM's context is limited, so overly large chunks leave room for fewer of them; and retrieval granularity matters — user queries typically match a specific passage, not an entire document.

Splitting strategies:

1. **Fixed size**: one chunk per 500 tokens, the simplest approach. Drawback: sentences get cut mid-way.
2. **Recursive splitting**: prefer paragraph boundaries, fall back to sentence boundaries, then fixed size. This is the default in LangChain's RecursiveCharacterTextSplitter.

3. **Semantic splitting**: split at points where consecutive sentence similarity drops sharply.
4. **Structure-aware**: split Markdown at headings, code at function/class boundaries.

Practical chunk sizes: 512–1024 tokens for technical docs; 256–512 tokens for Q&A/conversational content; by function or class for code. Use **overlap** of 10–20% between adjacent chunks to prevent information loss at boundaries.

Embedding and Indexing

Each chunk passes through an embedding model (Section 4.4.3), yielding 768/1024/4096-dimensional vectors that are stored in a vector database (covered in Section 3.4.4).

Vector database selection criteria: scale (pgvector/Chroma sufficient for millions; Milvus/Qdrant for hundreds of millions); filtering (Qdrant and Milvus are strong on metadata filtering); deployment (LanceDB/Chroma for embedded; Milvus/Qdrant for server deployments); hybrid search (Weaviate and Qdrant support natively).

Query and Retrieval

The user query passes through the same embedding model and searches the vector store for the top-K (typically 3–10) nearest neighbors. The raw text and metadata are returned.

Prompt Construction

Typical template:

```
Answer the question based on the reference materials below. If
the materials do not contain relevant information, say you don't
know.

[Reference Materials]
[1] {chunk_1 text}
Source: {chunk_1 source}

[2] {chunk_2 text}
```

```
Source: {chunk_2 source}

...

[Question]
{user_query}

[Requirements]
- Cite relevant reference numbers when answering, e.g. [1]
- Answer only based on the reference materials; do not fabricate
```

Once assembled, the prompt is sent to the LLM to generate an answer.

Limitations of Naive RAG

It looks simple but there are many pitfalls in practice: low retrieval recall (the gap between query phrasing and document phrasing means vector similarity misses relevant content); poor ranking (the top-K includes relevant chunks but not at the top); lost cross-chunk context (related content spans multiple chunks, each of which is incomplete in isolation); hallucinated citations (the LLM invents references that don't exist); misinterpreted query intent (ambiguous queries and multi-intent queries are handled poorly).

Advanced RAG in the next section addresses each of these problems.

4.10.2 Advanced RAG Techniques

> *Orientation: from naive RAG to production RAG — query rewriting, hybrid retrieval, reranking, and context expansion.*

Query-Side Optimization

Query Expansion / Rewriting: use an LLM to first rewrite or expand the user's query into multiple variants, retrieve for each, then merge the results.

```
def rewrite_query(query, llm):
    prompt = f"""Rewrite the user's question below into 3
different versions,
covering possible phrasings and angles. Output only the
rewritten questions, one per line.

Original question: {query}

Rewrites:"""
    return llm(prompt).strip().split('\n')

# retrieve with all variants, merge and deduplicate results
all_chunks = []
for q in rewrite_query(user_query, llm):
    all_chunks.extend(vector_db.search(q, top_k=5))
```

HyDE (Hypothetical Document Embedding): have the LLM first *fabricate* a hypothetical answer, then use that answer for retrieval — this matches more accurately than using the raw question (answers and documents share the same "answer space").

Query Decomposition: split a complex, multi-intent query into sub-queries, retrieve for each independently, then synthesize the results.

Hybrid Retrieval

Vector retrieval captures semantic similarity, but struggles with **technical terms, product model numbers, and proper nouns** where exact matching matters. Keyword search (BM25) excels here.

Hybrid Search: run both retrieval methods simultaneously and merge results with **Reciprocal Rank Fusion (RRF)**:

> \text{score}_{\text{RRF}}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}

R is the set of rankings from different retrieval methods, \text{rank}_r(d) is document d's rank under method r, and k is typically 60. Simple and blunt but empirically effective — no

model training required. Weaviate, Qdrant, and Elasticsearch all support hybrid search natively.

Reranking

Stage 1 recalls top-50 (vector + BM25); Stage 2 uses a more precise model to rerank down to top-5.

Cross-encoder reranker: unlike the dual-tower design of embedding models, a reranker takes the (query, chunk) pair together and directly outputs a relevance score. 100× slower but far more accurate.

Open-source rerankers:

BGE-Reranker-v2 (BAAI): strong in both Chinese and English
Cohere Rerank: commercial, API-based
Jina Reranker: open-source, long-context
Qwen3-Reranker: new in 2024, top of leaderboards

In production RAG, **reranking is the single most impactful improvement** — adding a reranker typically raises retrieval accuracy from 60% to 85%+. The cost is one extra network round-trip and tens of milliseconds of latency.

Contextual Compression

Retrieved chunks can be large and wasteful to stuff into a prompt. **Contextual Compression**: use a small model or LLM to compress each chunk down to "the parts relevant to the query," discarding irrelevant sentences. LangChain's ContextualCompressionRetriever implements this.

Parent-Child Document Strategy

Embedding-friendly chunks (small, sentence-level) and LLM-friendly chunks (large, paragraph-level) are in tension. The solution is a two-level structure — use small chunks for retrieval, then return the large parent chunk to the LLM once a match is found.

```
chunks = {}   # chunk_id  → {"text": ..., "parent_id": ...}
parents = {}  # parent_id → large_chunk_text

# at retrieval time
top_chunks = vector_db.search(query, top_k=10)
parent_ids = set(c.parent_id for c in top_chunks)
context = [parents[pid] for pid in parent_ids]   # parent chunks
are sent to the LLM
```

Sentence Window

A variant: index one vector per sentence, but when a sentence is retrieved, return *N sentences before and after it* together to the LLM. This achieves both retrieval precision and contextual completeness.

Self-Correction: Corrective RAG

CRAG (2024): after retrieval, an evaluator model judges whether the retrieved results are truly relevant. If not → trigger broader retrieval or fall back to the LLM's own knowledge.

Self-RAG

More aggressive: let the LLM itself decide when to retrieve, what to retrieve, and how to use the results — special tokens (Retrieve / Relevant / Support) give the model a form of "metacognition." Requires dedicated fine-tuning.

Evaluation

RAG is hard to evaluate. The popular framework **RAGAS** scores across several dimensions:

Faithfulness: whether the answer is grounded in the given context (no hallucination)
Answer Relevancy: whether the answer actually addresses the question
Context Precision: whether the retrieved chunks are relevant
Context Recall: whether all the information truly needed is present in the chunks

Each dimension is evaluated automatically using LLM-as-judge. Not perfectly reliable, but far better than tuning blind.

4.10.3 GraphRAG and Agentic RAG

> *Orientation: two frontier directions for RAG — knowledge-graph augmentation and agentification.*

GraphRAG

Naive RAG retrieves by "semantic similarity" and performs poorly on two query types:

1. **Multi-hop questions**: "What is the relationship between A and B?" — requires chaining multiple entities
2. **Global questions**: "What are the core themes in this document set?" — requires synthesis across all documents

Microsoft's GraphRAG approach (2024):

1. **Entity extraction**: use an LLM to extract entities and relations from each chunk, building a knowledge graph
2. **Community detection**: apply a community-discovery algorithm (Leiden) to partition the graph into thematic communities
3. **Community summarization**: generate a summary for each community (LLM summarizes all entities and relations within it)
4. **Querying**: depending on question type, retrieve a combination of relevant chunks / entities / community summaries to feed the LLM

Benefits: multi-hop reasoning, global synthesis, explainability (can point to "reasoning based on this edge in the knowledge graph").

Cost: high construction overhead — for a document corpus of hundreds of thousands of chunks, entity extraction alone can cost thousands of dollars in LLM calls. Only suitable for scenarios where the document base is relatively stable and

queries are high-value (financial research reports, medical literature, legal compliance).

LightRAG and Variants

Microsoft GraphRAG is too heavy. LightRAG (2024) takes a lighter approach: dual-layer indexing (entity-level and relation-level), incremental updates, and graph traversal only at query time. It has gained popularity in the open-source community.

Neo4j + LlamaIndex also offers a graph RAG integration at a higher level of maturity.

Agentic RAG

Naive RAG is "one retrieval, one answer." Agentic RAG: **the LLM itself decides when, how, and how many times to retrieve**.

```
flowchart TB Q[Query] --> LLM[LLM Agent] LLM --> DEC{Decide} DEC
-->|answer directly| ANS[Answer] DEC -->|retrieve| RET[Retrieve]
  RET --> CTX[Context] CTX --> LLM DEC -->|multiple subtasks|
     PLAN[Plan Subtasks] PLAN --> SUB[Subtask] SUB --> LLM
```

Typical patterns:

ReAct RAG: the ReAct loop from Section 4.11.1 applied to RAG — the LLM outputs "Search: xxx", executes it, gets the results, and continues
Self-Ask: the LLM breaks a complex question into sub-questions, retrieves and answers each independently, then synthesizes
Adaptive RAG: answer simple questions directly; use multi-round retrieval for complex ones

Combining HyDE + Agentic

Advanced production RAG systems typically blend multiple techniques: query rewriting → multi-path recall (vector + BM25) → reranking → if confidence is low, the LLM triggers a second retrieval → final generation. LangChain and LlamaIndex provide composable tooling, or you can wire it yourself.

When Not to Use RAG

RAG is not a universal key. These scenarios see limited benefit or outright degradation:

- **Reasoning tasks** (math, code): knowledge is not the bottleneck — reasoning is
- **Creative tasks**: free expression is required; retrieval constraints actually hinder it
- **Highly dynamic / personalized**: a knowledge base is irrelevant; fine-tuning is more appropriate
- **Short factual Q&A** where precise sourcing is unnecessary: the model's own knowledge is usually sufficient

PART ELEVEN

4.11 Agent

LLMs calling external tools and planning autonomously — the shift from "generating text" to "getting things done."

4.11.1 From Prompt to ReAct

Orientation: the intellectual roots of agents — from CoT to ReAct to the modern agent loop.

Chain-of-Thought

CoT (Wei et al. 2022) is the first step toward the agent paradigm. The core finding: prompting LLMs to "think step by step" (adding "Let's think step by step") improves accuracy over

directly giving answers, especially on math and logical reasoning tasks.

The intuition: each token generation in an LLM is a fixed-cost forward pass. Complex problems require multi-step reasoning; asking for a direct answer means completing that reasoning in a single forward pass — insufficient compute. Letting the model output its reasoning process means one forward pass per step, with total compute scaling linearly with reasoning length.

CoT provides little benefit in small models — the model must be large enough (~60B+) for significant gains. This is part of the emergence phenomenon.

Self-Consistency

CoT + sample multiple reasoning chains + majority vote. The model generates several reasoning paths for the same question; the most consistent answer wins. This corrects errors in individual chains. The cost is N× inference.

Tree of Thoughts / Graph of Thoughts

A step further — instead of a linear reasoning chain, reasoning is expanded as a tree or graph, with each node as an intermediate reasoning state, explored via search algorithms (BFS, DFS, beam search).

Suitable for planning problems (Game of 24, word puzzles). High engineering cost, rarely used in industry — CoT + self-consistency is usually sufficient.

ReAct

Reasoning + Acting (Yao et al. 2022). The key insight: make the LLM's "reasoning" more than just text — have it **alternate between thinking and acting**, where actions are calls to external tools.

The ReAct loop:

```
    flowchart LR T[Thought: What information do I need?] -->
  A[Action: call tool] A --> O[Observation: tool result] O -->
T2[Thought: next step based on result] T2 --> A2[Action: ...] A2
                --> FIN[Action: finish + Answer]
```

Example prompt format:

```
Question: Who won the 2024 F1 World Championship? How old is he?

Thought: I need to find the 2024 F1 World Champion.
Action: search("2024 F1 world champion")
Observation: Max Verstappen won the 2024 F1 World Championship.

Thought: Now I need to find Max Verstappen's birth date.
Action: search("Max Verstappen birth date")
Observation: Max Verstappen was born on 30 September 1997.

Thought: He was 27 in 2024. I have my answer.
Action: finish("Max Verstappen, age 27")
```

ReAct Implementation

```
def run_react(question, tools, llm, max_steps=10):
    transcript = f"Question: {question}\n\n"
    for step in range(max_steps):
        # prompt the LLM to generate the next Thought and Action
        response = llm(transcript + "Thought:", stop=
["Observation:"])
        transcript += "Thought:" + response
        # parse the Action
        action = parse_action(response)   # e.g. "search('xxx')"
        if action.name == "finish":
            return action.args
        # execute the tool
        result = tools[action.name](*action.args)
        transcript += f"Observation: {result}\n"
    return "Max steps reached"
```

Limitations of ReAct

ReAct is a classic early agent framework, but it has problems: fragile prompt formatting (the model occasionally produces output that doesn't match the format, causing parse failures); every step requires re-sending the full transcript (high token cost, slow); error accumulation (a single mistake in a long loop corrupts all subsequent steps); no structured tool interface (tools described in plain text are error-prone).

Modern Agents

ReAct has been largely superseded by **Function Calling** (covered in the next section) — once models natively support structured tool calls, prompt-convention workarounds are no longer needed.

However, ReAct's core idea (Thought → Action → Observation → loop) lives on — the modern agent loop is essentially this, just with an upgraded mechanism for expressing tool calls.

4.11.2 Function Calling / MCP / Multi-Agent

> *Orientation: modern agent infrastructure — native tool calling, standard protocols, and multi-agent collaboration.*

Function Calling

OpenAI introduced Function Calling in June 2023 (later unified as the Tools API): models are trained to output structured JSON function calls as part of their native format. No more relying on prompt conventions — this is API-level support.

Developers declare available tools to the model using JSON Schema:

```
{
  "type": "function",
  "function": {
```

```
    "name": "get_weather",
    "description": "Get current weather for a location",
    "parameters": {
      "type": "object",
      "properties": {
        "location": {"type": "string", "description": "City
name"},
        "unit":     {"type": "string", "enum": ["celsius",
"fahrenheit"]}
      },
      "required": ["location"]
    }
  }
}
```

The model's output may be ordinary text, or it may be:

```
{
  "tool_calls": [{
    "id": "call_123",
    "type": "function",
    "function": {
      "name": "get_weather",
      "arguments": "{\"location\": \"Paris\", \"unit\":
\"celsius\"}"
    }
  }]
}
```

The application executes the function and returns the result to the model as a `tool_message`, continuing the loop.

Function Calling in Open-Source Models

Closed-source models (OpenAI, Anthropic) led early on. After 2024, open-source models caught up — LLaMA-3.1, Qwen2.5, DeepSeek, and Mistral all support function calling natively.

Training approach: large amounts of function-calling data (human-annotated or synthetic) are added during SFT, teaching the model to recognize when to call a tool and how to fill in parameters.

Benchmark: the Berkeley Function Calling Leaderboard (BFCL) is the standard — a set of tool-calling tasks that evaluates different models. Qwen2.5-72B, DeepSeek-V3, and others approach GPT-4o on BFCL.

Structured Output

Function Calling is fundamentally structured output. In recent years, **Constrained Decoding** has matured — enforcing compliance with a JSON Schema / regex / CFG directly at generation time.

Notable implementations:

Outlines: Python library enforcing JSON / regex / Pydantic schema
xgrammar: C++, integrated with vLLM/SGLang, fast
LMQL: query-language-style prompting
OpenAI Structured Output: API-level support

How it works: at each token position, the schema is used to dynamically compute the set of "valid next tokens," which are masked before the softmax. This guarantees 100% format compliance with almost no performance overhead.

MCP

Model Context Protocol (proposed by Anthropic in November 2024). The problem it solves: every AI application has to write its own tool-integration code — duplicated effort, hard to reuse.

MCP defines a standard protocol (JSON-RPC based), split into server and client:

MCP Server: a process that exposes tools, resources, and prompt templates. Can represent a file system, database, GitHub, Slack, and more
MCP Client: an LLM application (Claude Desktop, Cursor, etc.) that connects to one or more MCP servers and automatically gains access to their tools

Benefit: a standardized tool ecosystem — write an MCP server once, and every MCP-compatible LLM application can use it.

In 2025, the MCP ecosystem expanded explosively — from a single-vendor proposal by Anthropic to a de facto industry standard. Claude Desktop, Cursor, VS Code LLM plugins, Cline, Windsurf, Continue, and Zed all integrated. Open-source models like Qwen3 and DeepSeek V3.1 support the MCP format natively at the tokenizer level. Major SaaS platforms including GitHub, Slack, Notion, and Linear launched official MCP servers. From mid-2025 onward, "MCP-ready" became a baseline requirement for AI developer tools.

This is one of the rare "standardization success stories" in the LLM ecosystem — its impact runs deeper than OpenAI's plugins (now discontinued) or ChatGPT's GPT marketplace, because MCP is a model-agnostic protocol not tied to any single vendor.

Multi-Agent Systems

A single LLM doing everything vs. multiple LLMs dividing the work. Several patterns:

Supervisor-Worker: a primary LLM delegates subtasks to specialized worker LLMs (one for research, one for coding, one for writing) and aggregates the results. LangGraph and CrewAI champion this pattern.

Conversational Multi-Agent: multiple agents collaborate through dialogue. AutoGen (Microsoft) is the representative, where two or more agents exchange messages to complete a task (one writes code, one reviews, one tests).

Agent Mesh: loosely coupled, each agent has independent capabilities and collaborates on demand.

Multi-Agent Pitfalls

Lessons from building these in practice:

1. **Cost explosion**: N agents communicating with each other multiplies token consumption
2. **Error amplification**: one agent's error flows to the next, causing cascading failures

3. **Hard to debug**: fault isolation requires tracing messages back and forth across agents
4. **Not necessarily better than a single powerful LLM**: giving a single LLM more time to "think" (test-time compute) often outperforms multi-agent collaboration

Most production agent systems are **Supervisor + a small number of specialized tools**, not dozens of conversing agents. OpenAI Swarm and LangGraph's StateGraph both favor simple state-machine patterns.

Memory and Planning

Agents in long-running conversations need memory. Typical architecture:

Short-term: current conversation history (within the context window)
Mid-term: session summaries (long conversations are rolled and compressed into summaries)
Long-term: important facts stored in a vector store (fused with RAG)

On the planning side, ReAct is "reactive" — it decides at each step based on current state. **Plan-then-Execute** produces a complete plan first, then executes it — better suited for tasks with clear structure.

The Limits of Agents

Agents are heavily hyped, but as of early 2026, reliably deployable commercial agent scenarios remain relatively narrow:

Coding assistance: Cursor, Windsurf, Claude Code, Cline, OpenCode (open-source Claude Code alternative, 2025) — the most mature agent category
Data analysis: OpenAI's Data Analysis and Code Interpreter
Browser / screen operation: Anthropic Computer Use, Claude for Chrome (released late 2025) — giving LLMs real control over graphical interfaces marks a qualitative shift in agent capability

Vertical automation: financial analysis, legal compliance, customer service ticketing

"General-purpose agents" (personal assistants, all-purpose secretaries) remain far from practical in 2025–2026 — the main obstacles are error accumulation in long-horizon tasks and insufficient self-correction. In the second half of 2025, investment in this area accelerated rapidly: Google launched the Gemini Agent + Antigravity environment, and Anthropic continued polishing Claude's agent reliability. Reasoning models (R1, V3.2, Claude 4.5) offer strong long-horizon planning, forming the foundation for agent reliability.

A parallel trend is the flourishing of **agent programming frameworks**: LangChain and LlamaIndex keep evolving, and 2025 saw more "code-first" frameworks emerge (DSPy, Pydantic AI, Mastra, smolagents), turning agent orchestration into a more explicit engineering problem rather than relying solely on prompt conventions.

4.11.3 Agent Framework Ecosystem

> *Orientation: A panoramic view of agent frameworks in 2025–2026 — from orchestration engines to multi-agent collaboration platforms.*

Tier 1: Orchestration Frameworks

LangChain / LangGraph: LangChain burst onto the scene in late 2022 and rapidly became the de facto standard for LLM application development. In 2025, LangChain 1.0 and LangGraph 1.0 were released as stable versions simultaneously. LangGraph is its agent orchestration layer, with **persistent state** as the core selling point — agent execution can be paused, resumed, and rolled back, with support for human-in-the-loop approval nodes. Companies such as Uber, LinkedIn, and Klarna are already using LangGraph in production.

CrewAI: A multi-agent framework centered on "role-playing." Each agent has an explicit role, goal, and backstory, and collaborates via task assignment. Supports three orchestration modes: sequential, hierarchical, and consensual. Over 45,000 GitHub stars, with more than 12 million agent invocations per day. Natively supports MCP and Agent-to-Agent (A2A) communication. For small and mid-sized teams, it offers the lowest barrier to entry for multi-agent development.

AutoGen → Microsoft Agent Framework: AutoGen, released by Microsoft in 2023, was a pioneer in multi-agent conversation research. In early 2025, v0.4 was rewritten as an async event-driven architecture, and the project subsequently evolved into the closed-source Microsoft Agent Framework, with AutoGen entering maintenance mode. Its core contribution was demonstrating that the "multi-agent conversation" paradigm is viable.

Tier 2: Platform-Level Products

Dify: An open-source low-code agent platform with 60,000+ GitHub stars. It provides a visual workflow editor, RAG pipelines, 50+ built-in tools (search, DALL-E, Wolfram, etc.), model management, and monitoring. Its positioning is "enabling non-engineers to build agent applications." The company raised US$30 million in a Pre-A funding round.

Coze (ByteDance): In July 2025, ByteDance open-sourced the core components Coze Studio and Coze Loop. Version 2.0 was released in January 2026, transforming the product from a chat tool into an "intelligent work partner" — supporting long-running Agent Plans (spanning multiple days) and Agent Skills (a customizable skill marketplace). It competes head-to-head with Dify in Asian markets.

Tier 3: Vendor SDKs

SDK	Vendor	Released	Core Design

Agents SDK	OpenAI	2025.3	Four primitives: Agent, Handoff, Guardrail, Tracing. Minimalist design, model-agnostic
Claude Agent SDK	Anthropic	2025.9	Shares toolchain with Claude Code (file read/write, command execution, code editing); supports sub-agents and hooks
ADK	Google	2025	Code-first, model-agnostic, deployment-agnostic. Supports Python, Go, Java, and TypeScript

Trend: All three major vendors have released their own agent SDKs, but with distinct design philosophies — OpenAI pursues minimalism (4 concepts are enough), Anthropic pursues pragmatism (giving you direct access to the filesystem and terminal), and Google pursues universality (cross-language, cross-platform).

Framework Selection Decision Tree

- Single agent + tool calling only → **OpenAI Agents SDK** or native Function Calling
- Stateful, interruptible agent workflows → **LangGraph**
- Multi-agent role specialization → **CrewAI**
- Non-engineers building agents → **Dify** or **Coze**
- Building a coding agent → **Claude Agent SDK**
- Enterprise-grade, multi-cloud deployment → **Google ADK**

4.11.4 Coding Agent: The Most Mature Agent Vertical

> *Orientation: Programming is the most successful real-world application of agents — the toolchain is closed-loop, feedback is immediate, and evaluation is objective.*

Why Coding Agents Matured First

Agents remain unreliable in general-purpose scenarios, but the programming domain got there first: code can be compiled and tested, giving an objective correctness signal at every step; the toolchain of file read/write, terminal execution, and Git operations has been standardized via MCP; benchmarks like SWE-bench enable quantitative comparison; and developer hourly rates are high, so the automation ROI is immediate.

Leading Coding Agents (2025–2026)

United States

Product	Type	Key Characteristics
Claude Code (Anthropic)	Terminal CLI + IDE extension	Released 2025.2. A terminal-first agentic coding tool that reads entire codebases, performs multi-file editing, integrates Git workflows, and runs parallel sub-agents. VS Code extension and desktop app launched in 2025.9. The majority of Anthropic's internal code is now generated by it.
Cursor (Anysphere)	AI-native IDE	Built on VS Code. Composer 1.5 (in-house reasoning model with adaptive inference time), Visual Editor (drag-and-drop UI generation). Series D in 2025.11 valued the company at US29.3B; annualized revenue exceeded US2B in early 2026.
Devin (Cognition)	Fully autonomous agent	v2.0 released 2025.4; pricing dropped from US500/month to US20/month + US2.25/ACU. Autonomously completes coding tasks from Jira/Linear tickets. Goldman Sachs deployed it as a "new-hire"-level pilot in 2025.7. Acquired Windsurf in 2025 (~US250M), merging IDE + autonomous agent capabilities.
GitHub Copilot (Microsoft)	IDE plugin + async agent	Agent Mode: a self-iterating plan → edit → test → fix loop, now stable in VS Code. Coding Agent: executes asynchronously in GitHub Actions. Project Padawan: the next generation,

		capable of being assigned issues directly to generate PRs.
OpenAI Codex CLI	Open-source terminal agent	Implemented in Rust, open-source. Uses a dedicated codex-mini model, supports parallel sub-agents and MCP integration. Lightweight and community-driven.

China

Product	Type	Key Characteristics
Trae (ByteDance)	Free AI IDE	Released 2025.3 with the tagline "The Real AI Engineer." VS Code-based, offering unlimited free access to DeepSeek R1 and Claude 3.7 Sonnet. Features Builder Mode (preview global changes), MCP support, and Figma integration. MAU surpassed one million in 2025.5; over 80% of ByteDance engineers use it internally.
Qwen3-Coder + Qoder (Alibaba)	Model + platform	Qwen3-Coder ranks second globally on the OpenRouter Coder leaderboard (behind only Claude Sonnet 4). Qoder is the coding platform launched in 2025.8.

Open-Source Agent Ecosystem

In the second half of 2025, a wave of open-source activity erupted around agents:

- **OpenClaw** (2025.11): an open-source personal AI agent created by Peter Steinberger. Not limited to coding — it can browse the web, fill out forms, read and write files, execute shell commands, and even create new skills automatically by writing code. Supports over ten platforms including WhatsApp, Telegram, Slack, and Discord. By 2026.4, eleven major Chinese internet companies had launched products based on OpenClaw.
- **ZeroClaw**: a Rust rewrite of OpenClaw, built for extreme lightness — 400× faster startup, 99% less memory, an 8.8 MB binary, and under 5 MB runtime memory. Trait-based architecture with pluggable model, channel, memory, and tool layers.

- **Hermes Agent** (Nous Research): an open-source self-improving agent. Its defining feature is a **learning loop**: when a tool is called more than five times, it automatically abstracts the operation into a reusable "skill." Cross-session persistent memory, 40+ built-in tools, and six terminal backends (local / Docker / SSH / Daytona / Singularity / Modal). Supports any model backend.
- **Aider**: an early open-source terminal pair-programming tool supporting multiple models; remains actively developed.
- **Continue**: an open-source VS Code/JetBrains plugin supporting local models (Ollama) or cloud APIs, positioned as the open-source Copilot.

SWE-bench: The Standard Exam for Coding Agents

SWE-bench (Princeton 2024) draws programming tasks from real GitHub issues, requiring agents to fix bugs and add features on complete codebases. SWE-bench Verified is a human-validated subset.

In early 2024, the best systems could solve only around 5% of problems. By April 2026:

Benchmark	Top System	Score
SWE-bench Verified	Claude Mythos Preview	93.9%
SWE-bench Verified (public models)	Claude Opus 4.5	80.9%
SWE-bench Pro (harder, multilingual)	GPT-5.3 Codex	56.8%

Worth noting: the large gap between Verified and Pro scores (80% → 57%) suggests Verified may be subject to data contamination. Pro is the more reliable measure.

Limitations of Coding Agents

- **Weak creative design**: excels at well-defined tasks (fixing bugs, writing tests, migrations); struggles with ambiguous ones (designing architecture from scratch)

- **Instability on long tasks**: for complex tasks exceeding 30 minutes, error accumulation remains a problem
- **Security risks**: autonomous code execution introduces risks such as prompt injection and credential leakage; sandbox isolation is required
- **Human-agent collaboration is optimal**: the current best practice is not "full autonomy" but rather agent drafts + human review and confirmation

PART TWELVE

4.12 Multimodal

LLMs that see and hear — the expansion from "language model" to "foundation model."

4.12.1 VLM Architecture

Orientation: Vision-Language Models — the main architectural approaches for connecting images to LLMs.

Three Architectural Approaches

The core problem in VLMs (Vision-Language Models): how do you feed image input to an LLM? Three main approaches:

1. Projection Adapter: the image passes through a standalone visual encoder (e.g., CLIP ViT) to produce image feature vectors, which are then mapped into the LLM's token space via a lightweight projection layer (MLP or Q-Former) and fed in alongside text tokens. Representatives: LLaVA, MiniGPT, BLIP-2. Advantage: cheap to train (the LLM is frozen; only the

projection is trained). Disadvantage: information loss from compressing visual features.

2. Cross-Attention: visual features are not converted into tokens; instead they serve as the K and V of additional cross-attention layers added at each LLM layer, letting text tokens "look at" the image via cross-attention. Representatives: Flamingo, IDEFICS. Advantage: visual features retain high resolution. Disadvantage: modifies the LLM architecture, making training more complex.

3. Unified Transformer: image patches and text tokens are mixed together from the start and processed jointly by a single Transformer with self-attention throughout — no separate "visual encoder." Representatives: Fuyu (Adept), native multimodal Gemini, Chameleon (Meta). Advantage: no information loss, scales well. Disadvantage: requires training from scratch with large data and compute.

LLaVA Architecture in Detail

LLaVA (Haotian Liu 2023) is the canonical open-source multimodal model, with a clean, simple architecture:

```
flowchart LR IMG[Image 336x336] --> VE[CLIP ViT-L/14] VE --> IF[Image Features 576 tokens] IF --> PROJ[MLP Projector] PROJ --> VT[Visual Tokens] TXT[Text tokens] --> LLM[LLaMA / Vicuna] VT --> LLM LLM --> OUT[Text output]
```

Training has two stages:

1. **Feature alignment**: freeze the ViT and the LLM; train only the projector. Uses image-text pair data (image + caption). The goal is to teach the projector to map visual features into a space the LLM can "understand."
2. **Instruction fine-tuning**: unfreeze the LLM and train the full pipeline on multimodal instruction data (image + question + answer).

Total training cost is only a few thousand dollars — far cheaper than training a vision model from scratch.

Improved Variants

LLaVA-1.5: better data, MLP replacing the single linear layer. Significant improvement.

LLaVA-NeXT: the image is split into multiple 336×336 patches that are encoded separately, enabling high-resolution support.

Qwen-VL / Qwen2-VL: dynamic resolution (no need to scale to a fixed size), with video support. Qwen2-VL topped the open-source VLM leaderboards in 2024.

InternVL2: open-sourced by SenseTime, strong capabilities, available in multiple sizes.

High-Resolution Challenges

At low resolution (336×336), VLMs cannot read text or see fine details. Two approaches to high resolution:

1. **Patch splitting**: slice the large image into sub-images, encode each independently, then concatenate the features. LLaVA-NeXT's approach.
2. **Dynamic resolution**: the visual encoder accepts arbitrary input sizes. Qwen2-VL's Naive Dynamic Resolution achieves this.

Cost: more visual tokens (a 2K image can produce 2000+ tokens) crowds the context window and slows inference.

What VLMs Can Do

Task	Examples
Image description	Captioning, VQA (visual question answering)
Document understanding	PDF extraction, table recognition, chart parsing
OCR integration	Directly reading text within images
UI understanding	Screenshot-based operation (WebAgent, Anthropic Computer Use)

Multi-image reasoning	Comparing differences between two images
Video understanding	Key-frame extraction + per-frame analysis

Closed-Source Frontier

At the commercial API level: GPT-4o, Claude 3.5/3.7 Sonnet, and Gemini 2.0 all have strong multimodal capabilities. GPT-4o excels at real-time image + voice conversation; Claude is strong at UI operation and document understanding; Gemini leads on video (1.5 Pro can process a one-hour video).

Open-source is catching up fast — Qwen2-VL-72B, InternVL2-Pro, and LLaVA-OneVision approach closed-source performance on many tasks.

4.12.2 Audio, Video, and Unified Multimodal

> *Orientation: beyond images and text — speech, video, and the trend toward "one model for all modalities."*

Speech Understanding

Traditional ASR (automatic speech recognition) is a standalone task: a dedicated model (Whisper is the canonical example) converts speech to text, which is then sent to the LLM. This "cascade" has two problems: ASR errors propagate downstream; and information like emotion and tone in the speech is lost.

The new approach is to **connect audio directly to the LLM**. Similar to the VLM architecture:

1. Audio passes through an acoustic encoder (e.g., Whisper encoder) to produce features
2. A projection layer maps features to the LLM's token space
3. Features are fed into the LLM alongside text tokens

Notable open-source implementations: **Qwen2-Audio**, **LLaMA-Omni**, **SALMONN**, **Whisper + Vicuna** combinations.

Speech Synthesis and Full-Duplex

Traditional speech *generation* (TTS) is also a standalone task (Tacotron, VITS, XTTS). The emerging trend: LLMs that directly output audio tokens.

Neural audio codecs like SNAC, EnCodec, and SoundStream compress audio into discrete token sequences (roughly 50–100 tokens per second). An LLM can generate these tokens just as it generates text, and a decoder reconstructs the waveform.

Representatives: **OpenAI GPT-4o** (voice mode, claimed to be end-to-end), **Fish Speech**, **LLaMA-Omni**, **Moshi** (Kyutai, open-source end-to-end voice conversation).

The critical metric is **latency** — the traditional ASR + LLM + TTS cascade introduces 1–3 seconds of delay; end-to-end LLMs can achieve under 300ms, which is what natural conversation actually requires.

Video Understanding

Video = image sequence + audio. The challenge is the temporal dimension — a 1-minute video at 30fps has 1800 frames; sending all of them is prohibitively expensive.

Mainstream strategies:

1. **Uniform sampling**: take one frame every N frames. Crude but sufficient for most scenarios.
2. **Key-frame extraction**: use traditional CV (scene-cut detection) or an LLM to select frames with "high information density."
3. **Spatiotemporal modeling**: dedicated video encoders (VideoMAE, Video-LLaMA) encode adjacent frames together.
4. **Long context windows**: Gemini 1.5's 1M-token window can accommodate large numbers of frames.

Open-source representatives: Video-LLaMA, VideoChat, LLaVA-Video, Qwen2-VL (with video input support).

Image Generation

Strictly speaking this is outside the "LLM" category, but it is routinely discussed alongside LLMs in modern multimodal frameworks. Mainstream image generation architectures:

Diffusion models (dominant): Stable Diffusion, SDXL, SD3, FLUX, DALL-E 3. See Chapter 3, Section 4.4.4.
Autoregressive: images are split into patch tokens and generated like text. Parti, LLaMA-Gen, Emu3, and Chameleon are examples. Naturally integrates with LLMs, but quality currently lags behind diffusion.
Hybrid: an LLM generates a latent; a diffusion decoder renders it. The approach taken by Transfusion (Meta) and Janus (DeepSeek).

A clear trend: **understanding and generation unified** — a single model that can both read images and draw them. DeepSeek Janus-Pro and OmniGen are the open-source representatives.

Video Generation

Major breakthrough in 2024: OpenAI Sora, Google Veo, Runway Gen-3, Kling — capable of generating high-quality videos several minutes long. The underlying technology remains diffusion models + DiT (Diffusion Transformer) architecture, treating video as spatiotemporal patches.

Open-source is catching up: **CogVideoX** (Tsinghua + Zhipu), **Open-Sora** (version 2.0 reached commercial-grade quality at just US$200K training cost), **HunyuanVideo** (Tencent), **Mochi**, **LTX-Video**. Quality is approaching closed-source, but the compute barrier remains high (a single training run takes hundreds of GPU-days).

A significant pivot: on 2026.3.24, OpenAI announced the shutdown of the entire Sora product line, redirecting resources toward robotic world simulation. Sora 2 (released 2025.12) had

supported 25-second generation and native audio sync, but commercial adoption fell short of expectations. The torch passed to **Kling 3.0** (native 4K 60fps broadcast-grade quality), **Google Veo 3.1**, and **Seedance 1.5 Pro** as the leading video generation platforms.

Unified Multimodal

The frontier direction: **Any-to-Any** — a single model that supports understanding and generation across text, images, audio, and video.

Representatives:

Gemini 2.0 (closed-source): claims native multimodality
GPT-4o (closed-source): unified text / speech / image
Chameleon (Meta, open-source): unified text + image token space
Emu3 (BAAI, open-source): a single model for multimodal generation
AnyGPT: four modalities (text, image, audio, video) end-to-end

The technical core is "converting all modalities into discrete tokens" — images use VQ-VAE, audio uses neural codecs, video uses quantized spatiotemporal patches. A single Transformer then models all of them together.

Evaluating Multimodal Models

Multimodal evaluation is harder than single-modality — there are no standard labels. Common benchmarks:

MMBench, MMMU, MMLU-Pro: overall multimodal capability
ChartQA, DocVQA: document and chart understanding
MVBench, VideoMME: video understanding
GenEval, T2I-CompBench: image generation
VBench: video generation

These benchmarks are also evolving rapidly — model capabilities grow so fast that old benchmarks saturate within months.

The Future of Multimodal

Three directions worth watching:

1. **Real-time performance**: end-to-end voice interaction, real-time visual understanding (AR glasses on phones) — latency must drop to the 100ms range
2. **Long video**: understanding full-length films, hours-long meetings — requires efficient video encoding and long context
3. **Embodied intelligence**: multimodal + physical-world action — robotics, autonomous driving. VLA (Vision-Language-Action) models are the new paradigm

From 2025 onward, the boundaries of "LLM" are increasingly blurred as these models become the core of "foundation models" — with language as just one modality among many.

Chapter 5: Paper Deep Dives

26 Selected Papers

Ch 2 Related · Systems

P01 HNSW: The Retrieval Engine of Vector Databases

Corresponds to main text: Chapter 2, Section 2.4.4 — Vector Databases

Efficient and robust approximate nearest neighbor using Hierarchical Navigable Small World graphs

Authors Yu. A. Malkov, D. A. Yashunin

Affiliation Yandex (independent research)

Published IEEE TPAMI 2020 (preprint 2016/2018)

Citations 3,500+

Link arXiv:1603.09320

The Problem: The Curse of High-Dimensional Nearest Neighbor Search

Given a 768-dimensional vector (say, the output embedding of BERT), find the top-10 most similar vectors among one million candidates — this is the Approximate Nearest Neighbor (ANN) problem. Exact methods require linear scan, and O(N) is unacceptable. Traditional approaches like KD-trees degrade to linear time in high dimensions; LSH (Locality-Sensitive Hashing) demands large memory for multiple hash tables and offers a poor recall-speed tradeoff.

HNSW's starting point: organize vectors into a **multi-layer graph** and perform greedy search on that graph. The graph construction guarantees the "small-world" property — any two nodes are reachable in a small number of hops.

Core Method: Hierarchical Navigable Small World Graphs

The Intuition Behind NSW (Single Layer)

Start with the single-layer Navigable Small World graph: each vector is a node, and each node is connected to several "neighbors." Search starts from an entry point and greedily jumps to the neighbor closest to the query until no closer neighbor can be found — this is greedy search. The key insight of NSW: if the graph simultaneously contains **short-range links** (local precision) and **long-range links** (spanning large distances), the average number of hops in greedy search is $O(\log N)$.

Layered Structure (What the "H" Means)

HNSW adds a hierarchical structure on top of NSW, analogous to a skip list: the top layer holds only a small number of nodes connected by long-range edges for rapid coarse localization; each successive layer down adds more nodes with shorter edges for increasingly fine-grained approximation of the target.

```
flowchart TB subgraph Layer2["Layer 2 (sparsest)"] A2((v_3)) ---
   |long-range edge| B2((v_89)) end subgraph Layer1["Layer 1"]
A1((v_3)) --- C1((v_17)) C1 --- D1((v_45)) D1 --- B1((v_89)) end
 subgraph Layer0["Layer 0 (all nodes)"] A0((v_3)) --- E0((v_5))
  E0 --- F0((v_12)) F0 --- C0((v_17)) C0 --- G0((v_23)) G0 ---
 D0((v_45)) D0 --- H0((v_67)) H0 --- B0((v_89)) end A2 -.-> A1
        -.-> A0 B2 -.-> B1 -.-> B0 C1 -.-> C0 D1 -.-> D0
```

Insertion Algorithm

To insert a new vector q:

1. Randomly draw a layer l = \lfloor -\ln(\text{uniform}(0,1)) \cdot m_L \rfloor, where m_L = 1/\ln(M). This ensures an exponentially decaying distribution of layer assignments.
2. Starting from the entry point at the highest layer, greedily search down to the closest node to q, descending layer by layer until reaching layer l.
3. From layer l down to layer 0: at each layer, find the ef_{\text{Construction}} nearest neighbors of q, select the best M of them as q's neighbors, and add bidirectional edges.

Search Algorithm Pseudocode

```
def hnsw_search(query, entry_point, top_k, ef):
    """HNSW approximate nearest neighbor search"""
    current_nearest = entry_point

    # Phase 1: greedy descent from the top layer down to layer 1
    for layer in range(top_layer, 0, -1):
        changed = True
        while changed:
            changed = False
            for neighbor in get_neighbors(current_nearest,
layer):
                if dist(neighbor, query) < dist(current_nearest,
query):
                    current_nearest = neighbor
                    changed = True

    # Phase 2: beam search on layer 0 (ef controls beam width)
    candidates = MinHeap([current_nearest])   # candidate set
```

```
    visited = set([current_nearest])
    result = MaxHeap([current_nearest])       # result set,
ordered by descending distance

    while len(candidates) > 0:
        c = candidates.pop_min()
        f = result.peek_max()  # furthest node currently in
result set
        if dist(c, query) > dist(f, query):
            break  # candidate is farther than the worst result,
stop

        for neighbor in get_neighbors(c, layer=0):
            if neighbor not in visited:
                visited.add(neighbor)
                if dist(neighbor, query) < dist(f, query) or
len(result) < ef:
                    candidates.push(neighbor)
                    result.push(neighbor)
                    if len(result) > ef:
                        result.pop_max()

    return result.top(top_k)
```

Complexity Analysis

\lt td>O(1) greedy steps per layer, O(\log N) layers total \lt td>M edges per node

Operation	Time Complexity	Notes
Search	O(\log N)	
Insert	O(\log N)	Search + edge creation
Space	O(N \cdot M)	

Key Parameters

Parameter	Meaning	Typical Value
M	Maximum number of neighbors per node	16–64

ef_{\text{Construction}}	Beam width during index construction	100–200
ef_{\text{Search}}	Beam width during search (higher = more accurate but slower)	50–500

ef_{\text{Search}} is tunable at runtime — this gives HNSW a "precision-speed dial" that users can adjust per use case on the fly.

Experimental Results

On ANN-Benchmarks (128-dimensional SIFT 1M dataset), HNSW achieves a recall@10 of 0.99 while exceeding all contemporary methods (Annoy, FLANN, IVF-PQ) by an order of magnitude in QPS. The advantage is even more pronounced in higher dimensions (768-dim GloVe), because quantization-based methods (PQ) suffer severe precision degradation in high dimensions while HNSW's graph search is not directly affected by dimensionality.

Impact

HNSW is the **default index in every mainstream vector database today**:

Milvus: HNSW is the recommended in-memory index type
Qdrant: entirely built on HNSW under the hood
FAISS: HNSW used both as a standalone index and as the coarse quantizer in IVF
Pinecone, Weaviate, Chroma: all support HNSW
PostgreSQL pgvector: HNSW index added in version 0.5+

In a RAG pipeline (main text, Section 4.10), the "retrieval" step of "embedding + vector retrieval" runs on HNSW under the hood. Without this paper, the vector database boom of 2023–2025 would not have happened.

Critical Perspective

HNSW's main drawback is **large memory footprint** — the entire graph must reside in memory. One billion vectors × 768 dimensions × float32 ≈ 3 TB, and that does not even count the

graph edge storage. In large-scale settings, HNSW is therefore typically combined with quantization (PQ/SQ) or sharding. Additionally, HNSW does not support dynamic deletion (vectors can only be soft-deleted), so long-running indices must eventually be rebuilt. DiskANN (Microsoft, 2019) is an alternative specifically designed for ANN on disk.

> **Why You Must Read This:** If you are building RAG, semantic search, or recommendation systems, the vector queries in your stack are almost certainly running on HNSW under the hood. Understanding its graph construction and search logic is what lets you tune parameters correctly (M, ef) and decide when to switch to IVF-PQ or DiskANN.

P02 Raft: The Understandable Consensus Algorithm

> *Corresponds to main text: Chapter 2 — Distributed Systems Foundations*

In Search of an Understandable Consensus Algorithm

Authors	Diego Ongaro, John Ousterhout
Affiliation	Stanford University
Published	USENIX ATC 2014
Citations	6,000+
Link	raft.github.io

The Problem: Paxos Is Too Hard to Understand

Distributed consensus — getting multiple machines to agree on a single, ordered log — is the foundational problem of distributed systems. Lamport's Paxos (1989/1998) is the theoretical gold standard, but is notorious for being "extremely difficult to understand": the original paper uses a Greek parliament as a metaphor, real-world engineering requires numerous patches

(Multi-Paxos), and no two Paxos implementations are quite the same.

The goal of Ongaro's doctoral dissertation is explicit: **design a consensus algorithm equivalent to Paxos but far easier to understand and implement**.

Core Method: Leader Election + Log Replication + Safety

Raft decomposes consensus into three relatively independent subproblems:

1. Leader Election: The cluster always has exactly one Leader. Each node is in one of three states: Follower, Candidate, or Leader. A Follower that times out without receiving a heartbeat becomes a Candidate and initiates an election; it becomes Leader upon receiving a majority of votes. Terms (monotonically increasing integers) ensure that an old Leader automatically steps down when it sees a higher term.

2. Log Replication: All write requests are forwarded to the Leader, which appends the log entry locally and then sends it in parallel to all Followers. Once a majority of nodes acknowledge the entry, it is "committed" and the Leader notifies the client.

3. Safety: The core constraint is the Election Restriction: only a node that already holds all committed log entries can be elected Leader. This guarantees that a new Leader will never overwrite committed data.

```
stateDiagram-v2 [*] --> Follower Follower --> Candidate: Election timeout Candidate --> Leader: Receives majority votes Candidate --> Follower: Discovers higher term Candidate --> Candidate: Election timeout (retry) Leader --> Follower: Discovers higher term
```

Impact

Raft's industrial impact far exceeds its academic reception — it is the standard consensus layer in today's distributed

infrastructure:

etcd: Kubernetes' metadata store, built on Raft
TiKV: TiDB's storage engine, using Multi-Raft (one Raft Group per Region)
CockroachDB: distributed SQL database with Raft consensus
Consul: HashiCorp's service discovery, using Raft consensus
RabbitMQ Quorum Queues: durable queues based on Raft

The success of the Raft paper validates an important principle in systems research: **understandability is itself a form of value**. An algorithm that is "good enough" but easy to implement correctly is more valuable in engineering practice than one that is "theoretically optimal" but easy to implement incorrectly.

> **Why You Must Read This:** Distributed systems are the substrate of AI infrastructure — training clusters, inference services, vector databases, and configuration centers all depend on consensus. Understanding Raft's leader election and log replication is the entry point for understanding the reliability guarantees of all these systems.

PART SIX

Chapter 3 Related · Classic Model Supplements

The three major generative models (GAN, VAE, diffusion) and word vectors — the intellectual origins of modern generative AI.

P03 ResNet: Making Networks Truly "Deep"

Corresponds to main text: Chapter 3, §3.3.1 CNN · Chapter 4, §4.2.4 (residual connection concept)

Deep Residual Learning for Image Recognition

Authors	Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun
Affiliation	Microsoft Research Asia
Published	CVPR 2016 (Best Paper Award)
Citations	250,000+ (all-time highest in CS)
Link	arXiv:1512.03385

The Problem: The Depth Paradox

In 2014, VGG-19 demonstrated that a 19-layer CNN was substantially better than the 8-layer AlexNet. Intuitively, going even deeper should yield further gains. But experiments revealed a striking anomaly: a 20-layer network had a lower training loss than a 56-layer network—not due to overfitting (overfitting means good training performance but poor test performance), but because the **training itself could not converge**. Vanishing/exploding gradients are a surface-level explanation; the deeper cause is that stacking many nonlinearities makes it very difficult for the optimizer to find good parameters.

An intuitive counterargument: theoretically, a deep network should never perform worse than a shallower one—it only needs to learn identity mappings f(x)=x in the extra layers. But in practice, SGD struggles to learn a true "identity mapping" layer

(it requires learning exactly the right weights). This is precisely the insight that Kaiming He's team seized upon.

Core Method: The Residual Block

Instead of learning H(x), learn F(x) = H(x) - x, and output H(x) = F(x) + x.

```
flowchart TB X[x] --> C1[Conv 3x3 + BN] C1 --> R1[ReLU] R1 --> C2[Conv 3x3 + BN] X -. identity .-> ADD((+)) C2 --> ADD ADD --> R2[ReLU] R2 --> OUT[output]
```

If "learning nothing" is optimal, driving F \to 0 is far easier than coaxing the entire composite function H toward the identity. This is the simplest yet most crucial reframing.

Key Formula and Initialization

A single residual block:

> \mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}

During backpropagation, the gradient of loss L with respect to input \mathbf{x}:

> \frac{\partial L}{\partial \mathbf{x}} = \frac{\partial L}{\partial \mathbf{y}} \cdot \left(1 + \frac{\partial \mathcal{F}}{\partial \mathbf{x}}\right)

The "+1" is the key—it guarantees that the gradient *cannot decay to zero*, even when \partial \mathcal{F} / \partial \mathbf{x} is very small. This directly alleviates the vanishing gradient problem.

Another important detail is **He initialization** (from the same first author, a 2015 companion paper). Under ReLU, weights are initialized as \mathcal{N}(0, 2/n_{in}), keeping forward-pass activation variance stable. Without this initialization, residual networks cannot be trained either.

PyTorch Implementation

```
import torch.nn as nn

class BasicBlock(nn.Module):
    def __init__(self, in_ch, out_ch, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(in_ch, out_ch, 3, stride, 1,
bias=False)
        self.bn1 = nn.BatchNorm2d(out_ch)
        self.conv2 = nn.Conv2d(out_ch, out_ch, 3, 1, 1,
bias=False)
        self.bn2 = nn.BatchNorm2d(out_ch)
        # When stride != 1 or channels change, use 1x1 conv to
adjust dimensions
        self.shortcut = nn.Sequential()
        if stride != 1 or in_ch != out_ch:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_ch, out_ch, 1, stride, bias=False),
                nn.BatchNorm2d(out_ch),
            )

    def forward(self, x):
        out = torch.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out = out + self.shortcut(x)   # residual connection
        return torch.relu(out)
```

The Bottleneck Variant

Deep ResNets (50, 101, 152 layers) use a "bottleneck" block to reduce computation: a 1×1 conv to reduce dimensionality → a 3×3 conv → a 1×1 conv to restore dimensionality. Three layers replace two, but with fewer total parameters and less computation. ResNet-50 uses this design, and virtually all subsequent CNNs followed suit.

Experimental Results

- ImageNet: the 34-layer ResNet achieved 7.5% top-5 error rate, vs. 8.4% for VGG-19, and was significantly faster
- The 152-layer ResNet reached 5.7%—the first model to surpass the human-level benchmark of 5.1%

- On CIFAR-10, a **1202-layer** network was trained (purely to prove "it can really go that deep"); although it overfit, training loss continued to decrease—demonstrating that the residual idea is not bounded by depth

Impact

ResNet's residual concept has long transcended CNNs—it has become a **foundational component of all deep networks**:

1. **Transformer**: every attention and FFN module is wrapped with a residual connection (main text §4.2.1). Without residuals, deep training of BERT/GPT would not be possible.
2. **DenseNet, U-Net**: variants of the residual idea—dense connections, skip connections
3. **ControlNet, LoRA**: both embody the architectural philosophy of "bypass + residual"
4. **Neural ODE**: treats the residual network as a discrete approximation of an ODE: $\mathbf{y} = \mathbf{x} + \mathcal{F}(\mathbf{x}, t)$

Critical Perspective

After 2020, some researchers questioned whether ResNet's success *might* be primarily due to an "ensemble effect"—residual paths cause the network to behave like an ensemble of 2^N sub-networks of varying depths, rather than truly "learning residuals." But regardless of the explanation, the value of this architectural decision is beyond doubt.

> **Why You Must Read This:** Without reading ResNet, you will never truly understand why every sublayer in the Transformer is wrapped in a residual connection—it is not decorative; it is a prerequisite for deep networks to train at all. With this single paper, He Kaiming essentially defined the canonical form of a "trainable deep network" in practice.

P04 LSTM: The Classic Gatekeeper of Sequence Modeling

> *Corresponds to main text: Chapter 3, §3.3.2 RNN/LSTM/GRU*

Long Short-Term Memory

Authors Sepp Hochreiter, Jürgen Schmidhuber

Affiliation IDSIA / TU Munich

Published Neural Computation 1997

Citations 90,000+

Link doi:10.1162/neco.1997.9.8.1735

The Problem: Vanishing Gradients in Vanilla RNNs

Hochreiter's 1991 master's thesis had already quantitatively proven that in a standard RNN, the gradient multiplies by a weight matrix W at each timestep during backpropagation through time. If the spectral radius of W is less than 1, the gradient decays exponentially; if greater than 1, it explodes. In either case, the RNN cannot learn dependencies "10 steps back." Six years later, he and Schmidhuber proposed LSTM as the solution.

Core Method: Three Gates + Cell State

A "highway"—the cell state \mathbf{c}_t—is introduced, with three gates controlling information flow:

> \mathbf{f}_t = \sigma(W_f \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad \text{(forget gate)}

> \mathbf{i}_t = \sigma(W_i \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad \text{(input gate)}

> \tilde{\mathbf{c}}_t = \tanh(W_c \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] +

> \mathbf{b}_c) \quad \text{(candidate)}

> \mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t

> \mathbf{o}_t = \sigma(W_o \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad \text{(output gate)}

> \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)

The key to the gradient pathway: \mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \ldots is an addition (not a matrix multiply). When the forget gate \mathbf{f}_t \approx 1, \mathbf{c}_t retains all history and gradients pass through time with virtually no decay. This is the mathematical reason LSTM can learn long-range dependencies.

Code

```
class LSTMCell(nn.Module):
    def __init__(self, input_size, hidden_size):
        super().__init__()
        # Merge all 4 gates into one large matrix for compute
efficiency
        self.W = nn.Linear(input_size + hidden_size, 4 *
hidden_size)
        self.hidden_size = hidden_size

    def forward(self, x, state):
        h_prev, c_prev = state
        gates = self.W(torch.cat([h_prev, x], dim=-1))
        i, f, g, o = gates.chunk(4, dim=-1)
        i, f, g, o = torch.sigmoid(i), torch.sigmoid(f),
torch.tanh(g), torch.sigmoid(o)
        c = f * c_prev + i * g
        h = o * torch.tanh(c)
        return h, (h, c)
```

Impact and Succession

LSTM dominated sequence modeling for the 20-year span of 1997–2017: speech recognition (Graves's deep LSTM, 2013), machine translation (Google NMT 2016), and image captioning (Show-and-Tell 2014) were all built on LSTMs. *The "attention" mechanism (Bahdanau 2014) was originally designed for encoder-decoder LSTMs*—it was only later discovered that attention alone sufficed, without LSTM as the foundation.

In 2017, the Transformer displaced LSTM from the mainstream, relegating it to "small-model scenarios." But in embedded time-series and certain specialized tasks, LSTM remains a first choice. The resurgence of SSMs like Mamba in 2023–2024 is, in a sense, a modern revival of LSTM's core ideas.

An Interesting Historical Note

The 1997 original paper by Hochreiter and Schmidhuber actually contains a few details that are now considered "suboptimal"—for instance, it lacks the forget gate (which was added by Gers in 1999). What we call "LSTM" today is the "Gers-LSTM," not the original version. This is a reminder for anyone reading the original paper: widely-used tools are often the product of accumulated evolution, not something finalized in a single paper.

> **Why You Must Read This:** Understanding "gating + additive memory channel" is the conceptual origin of all modern "selective memory" mechanisms—Mamba's selective state, Transformer's attention gating. This is the root from which they grew.

P05 BatchNorm: Making Deep Network Training Simple

> *Corresponds to main text: Chapter 3, §3.2.5 Normalization*

Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift

Authors Sergey Ioffe, Christian Szegedy

Affiliation Google

Published ICML 2015

Citations 60,000+

Link arXiv:1502.03167

The Problem: Why Deep Network Training Is Exquisitely Sensitive to Learning Rate

Ioffe observed that the input distribution of each layer shifts dramatically as the parameters of earlier layers are updated. This is called "internal covariate shift." The consequence is that the next layer must constantly "chase" the shifting distribution, making training unstable; moreover, different layers have optimal learning rates differing by orders of magnitude, making a single global learning rate a painful choice.

Core Method

For each feature channel within a mini-batch, apply standardization:

> $\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta$

μ_B, σ_B^2 are the batch statistics; γ, β are learnable scale and shift parameters—allowing the network to "restore" the pre-normalization distribution, but without constraining the upstream layers' parameters.

Training vs. Inference: during training, batch statistics are used; during inference, "running mean/variance" (maintained via exponential averaging during training) is used. This distinction

is a common source of subtle BN bugs—it is a common reason why a model performs well on the training set but poorly at inference.

The Debate Over "Why BN Works"

The original paper attributed the effect to "reducing internal covariate shift." But a 2018 paper from MIT by Santurkar et al., "How Does Batch Normalization Help Optimization?", experimentally demonstrated that *covariate shift has little to do with BN's effectiveness*. The true reason is more likely that BN smooths the loss landscape, preventing gradient explosion even with large learning rates.

This is a classic pattern in deep learning research: **a method is shown to work, but the reason it works turns out to be different from what the authors claimed**.

Impact

BN became standard equipment for CNNs after 2015. It made training 100+ layer networks possible and, together with ResNet, supported the deep era of computer vision.

However, BN does not work well for RNNs or Transformers—batch-dimension statistics carry little meaning for sequential tasks. Transformers use LayerNorm throughout, which later evolved into RMSNorm. This is why, in main text §4.2.4, you see "Pre-Norm + RMSNorm" as the standard for LLMs. BN's story is largely confined to the CV domain.

> **Why You Must Read This:** Understanding that "normalization is not data preprocessing—it is a structural component of deep networks"—this shift in perspective is BatchNorm's true contribution. Reading this paper will give you a foundational understanding of why LayerNorm, RMSNorm, and GroupNorm exist in the main text.

P06 Adam: The Default Optimizer for Deep Learning

> *Corresponds to main text: Chapter 1, §1.7.4 · Chapter 3, §3.5.1*

Adam: A Method for Stochastic Optimization

Authors Diederik P. Kingma, Jimmy Ba

Affiliation University of Amsterdam, University of Toronto

Published ICLR 2015

Citations 200,000+

Link arXiv:1412.6980

The Problem: SGD's Learning Rate Is Hard to Tune

Pure SGD requires careful learning rate scheduling—different learning rates for different parameters and different training phases. **AdaGrad** (Duchi 2011) adaptively adjusts the learning rate per parameter based on the accumulated sum of squared gradients—parameters seen frequently get a smaller lr, new parameters get a larger lr. Problem: the accumulation only grows, so in the later stages of training all learning rates converge to zero and training stops. **RMSProp** (Hinton 2012 course notes) replaces accumulation with an exponential moving average, solving the "permanent decay" problem.

Adam = RMSProp + momentum, plus a bias correction.

Core Algorithm

Two states are maintained: a first-moment estimate (momentum) $\mathbf{m}_t$ and a second-moment estimate (exponential moving average of squared gradients) $\mathbf{v}_t$.

> $$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t$$

> $$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2$$

Bias correction—initially $\mathbf{m}_0 = \mathbf{v}_0 = 0$, so the exponential average is biased low; dividing by $(1 - \beta^t)$ compensates:

> $$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}$$

Final parameter update:

> $$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \frac{\alpha}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \hat{\mathbf{m}}_t$$

Typical hyperparameters: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, $\alpha = 10^{-3}$. These numbers are the defaults in virtually every PyTorch/TensorFlow codebase.

AdamW

Loshchilov (2017) pointed out that "weight decay" regularization in Adam is incorrectly coupled with the learning rate. **AdamW** decouples them and is now the default choice for LLM training. The "AdamW" in LLaMA's training recipe in main text §4.7.4 refers to this.

Impact and Controversy

Adam is the most widely used optimizer in deep learning—present by default like air. But it has not been without controversy:

- **Convergence**: Reddi et al. (2018), "On the Convergence of Adam," proved that the original Adam can fail to converge on certain simple convex problems. AMSGrad was proposed as a fix (but is rarely used in practice)
- **Generalization**: On CV tasks, SGD+Momentum often generalizes better. But for Transformer training, Adam/AdamW has no viable alternative
- **Memory**: Adam requires storing $\mathbf{m}$ and $\mathbf{v}$—twice the parameter footprint. In the era of large models, this is a GPU memory killer. ZeRO (main text §4.7.2) was created specifically to shard this state

> **Why You Must Read This:** Every LLM training run uses AdamW under the hood. This is the power of defaults—but knowing the math behind the defaults lets you diagnose training instabilities when they arise.

P07 XGBoost: The De Facto Standard for Tabular Data

> *Corresponds to main text: Chapter 3, §3.1.2 Tree Models*

XGBoost: A Scalable Tree Boosting System

Authors	Tianqi Chen, Carlos Guestrin
Affiliation	University of Washington
Published	KDD 2016
Citations	30,000+
Link	arXiv:1603.02754

The Problem: Two Bottlenecks of Traditional GBDT

Traditional GBDT (Friedman 2001) fits each tree to the residuals from the previous step (equivalent to the negative gradient for squared loss). Two problems:

1. **Only first-order gradients**: limited optimization direction information, slow convergence
2. **Engineering-level slowness**: single-machine training on large datasets takes hours or more; distributed implementations were absent

Core Method: Second-Order Taylor Expansion + Regularization + Parallelism

Tianqi Chen applied a second-order Taylor expansion of the objective function at the current model:

> $$\mathcal{L}^{(t)} \approx \sum_i \left[g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t(\mathbf{x}_i)^2 \right] + \Omega(f_t)$$

where $g_i = \partial L / \partial F$ (first-order), $h_i = \partial^2 L / \partial F^2$ (second-order), and $\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_j w_j^2$ is the regularization term (T is the number of leaves, w_j are leaf weights).

For a tree with a fixed structure, the optimal weight of each leaf has a closed-form solution:

> $$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}$$

The split gain also has a closed form:

> $$\text{Gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\right] - \gamma$$

With this formula, a greedy search picks the split with the highest gain until gain < 0 (or another stopping criterion). A detailed derivation is in main text §3.1.2.

Engineering Innovations

Approximate split algorithm: for large datasets, rather than enumerating all split points, discretize by quantiles (LightGBM later pushed this to the extreme with its histogram algorithm)
Sparsity-aware: missing values are automatically routed to a "default direction," requiring no manual imputation
Column-block storage: features are stored in compressed column format, sorted once and reused across rounds
Parallelism: feature-level parallelism (each feature independently searches for its split point)
Cache-aware access: prefetching data into CPU cache to improve hit rates

Python Usage

```
import xgboost as xgb
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

X, y = load_breast_cancer(return_X_y=True)
Xt, Xv, yt, yv = train_test_split(X, y, test_size=0.2)

model = xgb.XGBClassifier(
    n_estimators=500, max_depth=6, learning_rate=0.05,
    reg_lambda=1.0, reg_alpha=0.1,
    early_stopping_rounds=50, eval_metric='logloss',
)
model.fit(Xt, yt, eval_set=[(Xv, yv)])
print("Accuracy:", model.score(Xv, yv))
```

Impact

From 2015 to 2020, **90%+ of winning Kaggle tabular competition solutions** included XGBoost, LightGBM, or CatBoost. These three tools are all essentially variants of the XGBoost approach:

LightGBM (Microsoft 2017): histogram-based acceleration, leaf-wise growth instead of level-wise
CatBoost (Yandex 2018): native categorical feature handling, ordered boosting

In industry, **XGBoost remains the first choice for tabular data**. The reasons: fast training and inference (millions of samples in minutes), no GPU required, no feature normalization needed, and naturally interpretable feature importance. Deep learning has long had no advantage on tabular data—TabNet and TabTransformer never became mainstream.

> **Why You Must Read This:** Even in the LLM era, this classic paper remains important—because in financial risk management, healthcare, recommendation systems, and network traffic classification, XGBoost comprehensively outperforms deep learning on latency and interpretability. Knowing when to use it and when not to is a basic judgment call for any AI engineer.

PART TWO

Ch. 4 · Transformer & Pretraining

The foundational pillars of the LLM era — Attention, BERT/GPT, Scaling Laws, and RoPE.

P08 GAN: The Birth of Adversarial Training

Corresponds to main text: Chapter 3, Section 3.3.4 — Generative Models

Generative Adversarial Nets

Authors Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, Yoshua Bengio

Affiliation Universite de Montreal

Published NeurIPS 2014

Citations 70,000+

Link arXiv:1406.2661

The Problem: Why Generative Models Were So Hard

Before 2014, the dominant generative models were: Restricted Boltzmann Machines (RBMs), Variational Autoencoders (VAEs, introduced the same year), and autoregressive models (PixelRNN would not arrive for another two years). Their shared limitation: either they required explicitly defining a likelihood function and performing complex inference, or they produced blurry outputs. Goodfellow, while discussing the problem with a friend at a bar, conceived a way to circumvent

the likelihood function entirely — have two networks engage in **adversarial competition**.

Core Method: Two-Player Zero-Sum Game

Two networks are trained simultaneously:

Generator G : generates fake samples $G(z)$ from noise $z \sim p_z(z)$, with the goal of "fooling the discriminator"
Discriminator D: judges whether an input is a real or generated sample, with the goal of "not being fooled"

Objective Function

> $$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))]$$

The game-theoretic meaning of this minimax: D wants to maximize its discriminative ability (pulling V up), while G wants to minimize D's discriminative ability (pushing V down).

Theoretical Results

Goodfellow proves two key theorems:

1. **Optimal discriminator**: given G, the optimal $D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}$
2. **Global optimum**: when $p_g = p_{\text{data}}$, $D^* = 1/2$ and V reaches its global minimum of $-\log 4$. That is, at equilibrium the generated distribution perfectly matches the data distribution.

Substituting D^* into V reveals that V is equivalent to a transformation of the Jensen-Shannon divergence between the data distribution and the generated distribution:

> $$V(D^*, G) = -\log 4 + 2 \cdot \text{JSD}(p_{\text{data}} \| p_g)$$

Training Loop

```
flowchart LR Z["Noise z ~ N(0,I)"] --> G["Generator G"] G --> FakeImg["Fake samples G(z)"] RealImg["Real samples x"] --> D["Discriminator D"] FakeImg --> D D --> Loss["D: max log D(x) + log(1-D(G(z))) G: min log(1-D(G(z)))"]
```

```
import torch
import torch.nn as nn

class Generator(nn.Module):
    def __init__(self, z_dim=100, img_dim=784):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(z_dim, 256), nn.LeakyReLU(0.2),
            nn.Linear(256, 512), nn.LeakyReLU(0.2),
            nn.Linear(512, img_dim), nn.Tanh(),
        )

    def forward(self, z):
        return self.net(z)

class Discriminator(nn.Module):
    def __init__(self, img_dim=784):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(img_dim, 512), nn.LeakyReLU(0.2),
            nn.Linear(512, 256), nn.LeakyReLU(0.2),
            nn.Linear(256, 1), nn.Sigmoid(),
        )

    def forward(self, x):
        return self.net(x)

# Training loop
z_dim = 100
G = Generator(z_dim)
D = Discriminator()
opt_G = torch.optim.Adam(G.parameters(), lr=2e-4, betas=(0.5,
0.999))
opt_D = torch.optim.Adam(D.parameters(), lr=2e-4, betas=(0.5,
0.999))
bce = nn.BCELoss()

for epoch in range(num_epochs):
    for real_batch, _ in dataloader:
```

```
        batch_size = real_batch.size(0)
        real_batch = real_batch.view(batch_size, -1)

        # --- Train discriminator ---
        z = torch.randn(batch_size, z_dim)
        fake = G(z).detach()
        loss_D = bce(D(real_batch), torch.ones(batch_size, 1)) +
\
                 bce(D(fake), torch.zeros(batch_size, 1))
        opt_D.zero_grad(); loss_D.backward(); opt_D.step()

        # --- Train generator ---
        z = torch.randn(batch_size, z_dim)
        fake = G(z)
        # Trick: use -log D(G(z)) instead of log(1 - D(G(z)))
for stronger gradients
        loss_G = bce(D(fake), torch.ones(batch_size, 1))
        opt_G.zero_grad(); loss_G.backward(); opt_G.step()
```

Training Instability: GAN's "Original Sin"

GAN's theory is elegant, but training in practice is extremely difficult:

Mode collapse: G learns to produce only a few types of samples that are sufficient to fool D, sacrificing diversity
Training oscillation: G and D must stay balanced in capability — if one becomes too strong, the other receives no useful learning signal
Vanishing gradients: when D is too accurate, \log(1 - D(G(z))) \to 0 and G receives no gradient

A large body of subsequent work addressed these issues: WGAN (replacing JSD with Wasserstein distance), SNGAN (spectral normalization), Progressive GAN (progressively increasing resolution), StyleGAN (disentangling style and content).

Impact and Decline

From 2014 to 2021, GANs were the undisputed leaders in image generation. StyleGAN2 (2019) produced synthetic faces that remain the benchmark for "fake faces." But in 2022, diffusion

models (DDPM / Stable Diffusion) surpassed GANs comprehensively in both quality and diversity, and GAN's dominant position rapidly eroded. By 2025, GANs are primarily used for:

1. Real-time applications (inference requires only one forward pass, roughly 100× faster than diffusion models)
2. Specific tasks such as super-resolution and style transfer
3. Accelerating diffusion models (e.g., consistency distillation uses adversarial training)

Critical Perspective

The "adversarial training" idea outlasts the GAN architecture itself. The reward model in RLHF that discriminates between "good" and "bad" responses is, in a sense, a discriminator; adversarial example attacks and defenses (adversarial robustness) draw directly from GAN's game-theoretic framework. Goodfellow himself later shifted toward adversarial security research.

> **Why You Must Read This:** "Adversarial training" is one of the most important paradigms in deep learning. Even though GANs have ceded ground in image generation to diffusion models, their game-theoretic perspective has influenced RLHF, adversarial robustness, and data augmentation, among other areas. Understanding GANs is what lets you understand why RLHF requires a reward model.

P09 VAE: Variational Inference Meets Deep Learning

> *Corresponds to main text: Chapter 3, Section 3.3.4 — Generative Models*

Auto-Encoding Variational Bayes

Authors Diederik P. Kingma, Max Welling

Affiliation University of Amsterdam

Published	ICLR 2014
Citations	45,000+
Link	arXiv:1312.6114

The Problem: A Missing Probabilistic Framework for Deep Generative Models

GANs sidestep the likelihood function, but at the cost of training instability and an inability to compute probabilities. Traditional Bayesian methods support probabilistic inference but cannot handle high-dimensional data (images, text). Kingma and Welling's question: can we **use neural networks for variational inference while preserving a probabilistic framework**?

Core Method: The Variational Lower Bound (ELBO)

Generative Model Assumption

Assume data x is generated by latent variable z: first sample $z \sim p_\theta(z)$ (prior, typically a standard Gaussian), then decode $x \sim p_\theta(x|z)$ using a neural network. We want to maximize the marginal likelihood of the data:

> $$\log p_\theta(x) = \log \int p_\theta(x|z) p_\theta(z) \, dz$$

This integral is intractable for high-dimensional z.

Variational Inference

Introduce an "encoder" $q_\phi(z|x)$ to approximate the true posterior $p_\theta(z|x)$. Applying Jensen's inequality yields the ELBO (Evidence Lower BOund):

> $$\log p_\theta(x) \geq \underbrace{\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)]}_{\text{reconstruction term}} - \underbrace{D_{\text{KL}}(q_\phi(z|x) \| p_\theta(z))}_{\text{regularization term}} = \text{ELBO}$$

The intuition behind each term:

Reconstruction term: how well x can be recovered from z (maximize)
KL divergence term: prevents the encoder's output distribution from straying too far from the prior (minimize; encourages a structured latent space)

The Reparameterization Trick

The key technical innovation: sampling from $q_\phi(z|x) = \mathcal{N}(\mu_\phi(x), \sigma^2_\phi(x))$ does not allow gradients to flow back through the sampling operation (which is non-differentiable). Kingma's solution: factor out the randomness:

> $$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

Now z is a deterministic function of ϕ (with external noise ϵ), and gradients can backpropagate normally.

```
flowchart LR X["Input x"] --> Enc["Encoder q_phi"] Enc --> Mu["mu"] Enc --> Sigma["sigma"] Eps["epsilon ~ N(0,I)"] --> RP["z = mu + sigma * epsilon"] Mu --> RP Sigma --> RP RP --> Dec["Decoder p_theta"] Dec --> Xhat["Reconstruction x'"]
```

PyTorch Implementation

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class VAE(nn.Module):
    def __init__(self, input_dim=784, z_dim=20):
        super().__init__()
        # Encoder
        self.fc1 = nn.Linear(input_dim, 400)
        self.fc_mu = nn.Linear(400, z_dim)
        self.fc_logvar = nn.Linear(400, z_dim)
        # Decoder
        self.fc3 = nn.Linear(z_dim, 400)
        self.fc4 = nn.Linear(400, input_dim)
```

```
    def encode(self, x):
        h = F.relu(self.fc1(x))
        return self.fc_mu(h), self.fc_logvar(h)

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)       # external noise
        return mu + std * eps             # reparameterization

    def decode(self, z):
        h = F.relu(self.fc3(z))
        return torch.sigmoid(self.fc4(h))

    def forward(self, x):
        mu, logvar = self.encode(x.view(-1, 784))
        z = self.reparameterize(mu, logvar)
        return self.decode(z), mu, logvar

def vae_loss(recon_x, x, mu, logvar):
    # Reconstruction loss (binary cross-entropy)
    BCE = F.binary_cross_entropy(recon_x, x.view(-1, 784),
reduction='sum')
    # KL divergence: -0.5 * sum(1 + log(sigma^2) - mu^2 -
sigma^2)
    KLD = -0.5 * torch.sum(1 + logvar - mu.pow(2) -
logvar.exp())
    return BCE + KLD
```

VAE's "Blurriness" Problem and Subsequent Developments

VAE's generation quality (especially for images) has always lagged behind GANs — outputs tend to be blurry. The cause: pixel-level reconstruction loss (MSE/BCE) encourages "averaging" over all plausible outputs. Subsequent improvements include:

VQ-VAE (van den Oord, 2017): discrete latent space replacing the continuous Gaussian, yielding sharper generations
VQ-VAE-2 (Razavi, 2019): hierarchical discrete codes, quality approaching BigGAN
Stable Diffusion: diffusion performed in VQ-VAE's latent space — the VAE handles compression, diffusion handles generation. This is the standard image generation pipeline of 2023–2025.

Impact

VAE's influence extends well beyond the application of image generation:

1. **Reparameterization trick**: has become the standard method for any setting that requires backpropagating gradients through sampling operations (reinforcement learning, Gumbel-Softmax, etc.)
2. **Latent space learning**: VAE defines the "encoder–latent space–decoder" paradigm, which directly shaped the Latent Diffusion architecture of Stable Diffusion
3. **Variational inference + deep learning**: opened the door to probabilistic deep learning, spawning variational sequence models, variational graph neural networks, and more

> **Why You Must Read This:** VAE is a prerequisite for understanding the Stable Diffusion architecture — the "Latent" in Latent Diffusion is exactly the VAE's latent space. The reparameterization trick is also widely used in the LLM domain (e.g., Gumbel-Softmax for discrete selection). Kingma is the author of both Adam and VAE; together, those two papers may constitute the highest-citation combination in all of deep learning.

P10 DDPM: Diffusion Models, the Foundation of Modern Image Generation

---

> *Corresponds to main text: Chapter 3, Section 3.3.4 — Generative Models*

Denoising Diffusion Probabilistic Models

Authors Jonathan Ho, Ajay Jain, Pieter Abbeel

Affiliation UC Berkeley

Published NeurIPS 2020

Citations 15,000+

Link	arXiv:2006.11239

The Problem: Where Is the Alternative to GANs?

In 2020, GANs (particularly StyleGAN2) held the highest image generation quality, but training instability and mode collapse continued to frustrate the community. The theoretical framework of diffusion models had been proposed as early as 2015 by Sohl-Dickstein, but generation quality remained far below GANs. Ho et al.'s contribution was **the first time a diffusion model matched GAN quality** — and with dramatically simpler training.

Core Method: Forward Noising + Reverse Denoising

Forward Process (Diffusion)

Starting from a real image x_0, Gaussian noise is added incrementally over T steps until the result is pure noise:

> $$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} \, x_{t-1}, \beta_t I)$$

Here β_t is a predefined noise schedule that increases from small to large (e.g., $\beta_1 = 10^{-4}$ to $\beta_T = 0.02$). A key property: one can jump directly from x_0 to any x_t without sequential computation:

> $$q(x_t \mid x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} \, x_0, (1 - \bar{\alpha}_t) I)$$

where $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$. For sufficiently large T, $x_T \approx \mathcal{N}(0, I)$.

Reverse Process (Denoising)

If we can learn to reverse each step — predicting x_{t-1} from x_t — we can generate images starting from pure noise. The reverse process is also Gaussian:

> $p_\theta(x_{t-1} | x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 I)$

Ho et al.'s key simplification: instead of directly predicting μ, the network **predicts the noise ϵ**. The network $\epsilon_\theta(x_t, t)$ takes a noisy image and a timestep as input and outputs the predicted noise.

```
flowchart LR subgraph Forward["Forward Process (fixed)"] X0["x_0
   (original)"] -->|"add noise"| X1["x_1"] X1 -->|"add noise"|
   X2["x_2"] X2 -->|"..."| XT["x_T (pure noise)"] end subgraph
   Reverse["Reverse Process (learned)"] XT2["x_T ~ N(0,I)"] --
>|"denoise"| XTm1["x_{T-1}"] XTm1 -->|"denoise"| XTm2["x_{T-2}"]
           XTm2 -->|"..."| X02["x_0 (generated)"] end
```

Training Objective: Radical Simplification

DDPM's training loss is the simplest of any deep generative model:

> $L_{\text{simple}} = \mathbb{E}_{t, x_0, \epsilon}\left[\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t} \epsilon, \, t)\|^2\right]$

In plain terms: randomly sample a timestep t, randomly sample noise ϵ, add the noise to x_0 to obtain x_t, have the network predict the added noise, and train with MSE. That is all.

Training and Sampling Code

```
import torch
import torch.nn as nn

# Noise schedule
def linear_schedule(T, beta_start=1e-4, beta_end=0.02):
    return torch.linspace(beta_start, beta_end, T)

T = 1000
betas = linear_schedule(T)
```

```
alphas = 1.0 - betas
alpha_bars = torch.cumprod(alphas, dim=0)

# Forward noising: jump directly from x_0 to x_t
def q_sample(x_0, t, noise=None):
    if noise is None:
        noise = torch.randn_like(x_0)
    sqrt_alpha_bar = alpha_bars[t].sqrt().view(-1, 1, 1, 1)
    sqrt_one_minus = (1 - alpha_bars[t]).sqrt().view(-1, 1, 1,
1)
    return sqrt_alpha_bar * x_0 + sqrt_one_minus * noise, noise

# Training loop
model = UNet(...)  # U-Net that predicts noise
optimizer = torch.optim.Adam(model.parameters(), lr=2e-4)

for batch in dataloader:
    x_0 = batch['image']
    t = torch.randint(0, T, (x_0.size(0),))       # random
timestep
    x_t, noise = q_sample(x_0, t)                   # add noise
    pred_noise = model(x_t, t)                       # predict
noise
    loss = nn.functional.mse_loss(pred_noise, noise) # just MSE
    optimizer.zero_grad(); loss.backward(); optimizer.step()

# Sampling: iteratively denoise from pure noise
@torch.no_grad()
def sample(model, shape):
    x = torch.randn(shape)  # x_T ~ N(0, I)
    for t in reversed(range(T)):
        t_batch = torch.full((shape[0],), t, dtype=torch.long)
        pred_noise = model(x, t_batch)
        # Compute mean of x_{t-1}
        alpha = alphas[t]; alpha_bar = alpha_bars[t]
        x = (1 / alpha.sqrt()) * (x - (1 - alpha) / (1 -
alpha_bar).sqrt() * pred_noise)
        if t > 0:
            x = x + betas[t].sqrt() * torch.randn_like(x)  #
reintroduce stochasticity
    return x
```

Why Diffusion Models Surpassed GANs

Dimension	GAN	DDPM
Training stability	Poor (minimax game)	Good (simple MSE)
Mode coverage	Prone to mode collapse	Naturally covers all modes (iterative denoising)
Generation diversity	Limited	Very high
Inference speed	One forward pass (fast)	Requires T steps (slow)
Conditional generation	Requires specialized architecture	Classifier-free guidance suffices

Downstream Development Chain

1. **DDIM** (Song, 2020): deterministic sampling — 50 steps is enough, no need for 1000
2. **Classifier-Free Guidance** (Ho & Salimans, 2022): conditional generation without a separate classifier, dramatically improving quality
3. **Latent Diffusion / Stable Diffusion** (Rombach, 2022): diffusion in VAE latent space; 64× compression makes the process tractable
4. **DALL-E 2/3, Midjourney, Imagen**: text-to-image generation, all based on diffusion
5. **Sora** (OpenAI, 2024): video generation using diffusion + Transformer (DiT)

Critical Perspective

The inference speed problem of diffusion models is not fully solved — compressing 1000 steps to 4–8 (consistency distillation) incurs quality degradation. Additionally, the theoretical framework of diffusion models limits their applicability to discrete data (text) — text generation remains the domain of autoregressive models (GPT). There are some 2024–2025 attempts at discrete diffusion models (e.g., MDLM), but no substantial breakthrough has emerged yet.

> **Why You Must Read This:** Stable Diffusion, DALL-E, Midjourney, Sora — every stunning image and video generation result from 2022–2025 builds on DDPM's ideas. Once you understand the mathematics of forward noising and reverse denoising, you can grasp why text-to-image generation requires so many inference steps, and why Latent Diffusion moves the diffusion process into latent space.

P11 Word2Vec: The Starting Point of Word Embeddings

> *Corresponds to main text: Chapter 3, Section 3.5.2 — Embeddings*

Efficient Estimation of Word Representations in Vector Space

Authors	Tomas Mikolov, Kai Chen, Greg Corrado, Jeffrey Dean
Affiliation	Google
Published	ICLR 2013 Workshop
Citations	40,000+
Link	arXiv:1301.3781

The Problem: A Mathematical Representation of Words

The first challenge in NLP: how do we turn words into numbers? Before 2013, the mainstream approach was one-hot vectors (a V-dimensional vector with a single 1, where V is the vocabulary size) or TF-IDF. The problems: one-hot vectors are extremely high-dimensional (vocabularies of tens to hundreds of thousands), and the distance between any two words is identical (zero semantic information).

Mikolov's goal: train dense, low-dimensional (100–300 dimensions) vectors such that semantically similar words are close together in the vector space.

Core Method: Two Architectures

CBOW (Continuous Bag of Words): predict the center word from the words in a context window. For example, given "the cat sat on the ___", use {the, cat, sat, on, the} to predict "mat."

Skip-gram: the reverse — predict the context from the center word. Given "sat", predict {cat, on}, etc. In practice, Skip-gram performs better (especially for rare words) and became the standard.

> $$P(w_o \mid w_c) = \frac{\exp(\mathbf{v}_{w_o}' \cdot \mathbf{v}_{w_c})}{\sum_{w \in V} \exp(\mathbf{v}_w' \cdot \mathbf{v}_{w_c})}$$

Summing the denominator over the entire vocabulary is too slow. Two speedup strategies were introduced: Hierarchical Softmax (using a Huffman tree) and **Negative Sampling** (randomly sample a few "negatives" from the vocabulary and update only those). Negative Sampling is extremely simple and efficient and has become the de facto standard.

A Surprising Property: Vector Arithmetic

The most famous phenomenon discovered by Word2Vec:

> $$\text{king} - \text{man} + \text{woman} \approx \text{queen}$$

Semantic relationships are encoded as directions in vector space. This finding directly shaped the design objectives of every subsequent embedding method.

Impact

Word2Vec ushered in the era of "everything can be embedded":

GloVe (Stanford, 2014): replaces the local window with a global co-occurrence matrix for a more elegant mathematical framework
FastText (Facebook, 2016): subword-level embeddings that handle out-of-vocabulary words
ELMo (Peters, 2018): contextual word vectors (the same word has different vectors in different sentences)

BERT/GPT: essentially upgrading "static embeddings" to "dynamic contextual embeddings"

By 2025, no one uses Word2Vec for NLP tasks — BERT/GPT contextual embeddings have replaced them entirely. Yet Word2Vec's ideas live on: Item2Vec in recommendation systems, Node2Vec in graph embeddings, Code2Vec in code embeddings — all are extensions of Word2Vec's paradigm of "mapping discrete objects into continuous space."

> **Why You Must Read This:** Word2Vec is the best entry point for understanding the concept of "embedding." The token embeddings in BERT, the positional encodings in Transformers, the vector retrieval in RAG — all of these trace their intellectual origins to "representing discrete symbols with low-dimensional dense vectors." Mikolov's paper transformed this idea from an academic concept into an engineering standard.

PART SEVEN

Chapter 4 Related · LLM Supplements

Challengers to the Transformer, retrieval-augmented generation, and the reasoning framework for Agents.

Ch 4 Related · Transformer & Pretraining

P12 Attention Is All You Need: The Birth of the Transformer

> *Corresponds to main text: Chapter 4, §4.2 (entire section)*

Attention Is All You Need

Authors Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin

Affiliation Google Brain, Google Research, University of Toronto

Published NeurIPS 2017

Citations 140,000+ (second highest in CS history)

Link arXiv:1706.03762

The Problem: The Fundamental Limitation of RNNs

In 2017, the mainstream approach to machine translation was seq2seq + attention (Bahdanau 2014, Luong 2015)—the encoder used an LSTM to encode the source sentence into a sequence of vectors, the decoder used an LSTM to autoregressively generate output, and attention was used to consult the encoder's output. The results were good, but **LSTM's sequential nature across timesteps made training impossible to parallelize**—the advantages of GPU acceleration could not be exploited, limiting model scale.

Vaswani et al.'s question: can we remove recurrence entirely and rely on attention alone?

Core Method: Scaled Dot-Product Attention

Each position in the sequence is represented as three vectors—Query, Key, and Value—and the output is computed by weighting V according to the similarity between Q and all Ks:

> $$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Key detail: the **\sqrt{d_k}** scaling. The variance of the dot product of d-dimensional vectors is d; dividing by \sqrt{d} normalizes the variance and prevents the softmax from becoming oversaturated (main text §4.2.2 contains a detailed derivation).

Multi-Head Attention

Q, K, and V are projected into H subspaces; attention is computed independently in each subspace, and the results are concatenated and projected:

> \text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_H) W^O

> \text{head}_i = \text{Attention}(Q W_i^Q, K W_i^K, V W_i^V)

Multiple heads let the model simultaneously learn multiple "attention patterns"—some heads focus on syntactic proximity, others on coreference, others on global topic.

Complete Transformer Block

```
flowchart TB X[Input x] --> N1[LayerNorm] N1 --> MHA[Multi-Head Attention] MHA --> A1((+)) X --> A1 A1 --> N2[LayerNorm] N2 --> FFN[FFN d_model to 4d to d_model] FFN --> A2((+)) A1 --> A2 A2 --> OUT[Output]
```

(The diagram shows the Pre-Norm version; the original paper used Post-Norm, but most modern implementations have switched to Pre-Norm. See main text §4.2.4.)

Positional Encoding

Attention itself is a "set operation"—permuting the input order leaves the output unchanged. To give the model a sense of position, positional encodings are added:

> \text{PE}(\text{pos}, 2i) = \sin(\text{pos}/10000^{2i/d})

> \text{PE}(\text{pos}, 2i+1) = \cos(\text{pos}/10000^{2i/d})

Position information is encoded using sinusoids of different frequencies. This was later superseded by RoPE (see P09).

Complete PyTorch Implementation

```
import torch
import torch.nn as nn
import math

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_heads):
        super().__init__()
        assert d_model % n_heads == 0
        self.H = n_heads
        self.d_head = d_model // n_heads
        self.Wq = nn.Linear(d_model, d_model, bias=False)
        self.Wk = nn.Linear(d_model, d_model, bias=False)
        self.Wv = nn.Linear(d_model, d_model, bias=False)
        self.Wo = nn.Linear(d_model, d_model, bias=False)

    def forward(self, x, mask=None):
        B, L, D = x.shape
        Q = self.Wq(x).view(B, L, self.H,
self.d_head).transpose(1, 2)
        K = self.Wk(x).view(B, L, self.H,
self.d_head).transpose(1, 2)
        V = self.Wv(x).view(B, L, self.H,
self.d_head).transpose(1, 2)

        scores = Q @ K.transpose(-2, -1) /
math.sqrt(self.d_head)  # [B, H, L, L]
        if mask is not None:
            scores = scores.masked_fill(mask, float('-inf'))
        attn = torch.softmax(scores, dim=-1)
        out = attn @ V                              # [B, H, L,
d_head]
        out = out.transpose(1, 2).reshape(B, L, D)
        return self.Wo(out)


class FFN(nn.Module):
```

```
    def __init__(self, d_model, d_ff=None):
        super().__init__()
        d_ff = d_ff or 4 * d_model
        self.fc1 = nn.Linear(d_model, d_ff)
        self.fc2 = nn.Linear(d_ff, d_model)

    def forward(self, x):
        return self.fc2(torch.relu(self.fc1(x)))


class TransformerBlock(nn.Module):
    """Pre-Norm version — standard for modern LLMs"""
    def __init__(self, d_model, n_heads):
        super().__init__()
        self.ln1 = nn.LayerNorm(d_model)
        self.attn = MultiHeadAttention(d_model, n_heads)
        self.ln2 = nn.LayerNorm(d_model)
        self.ffn = FFN(d_model)

    def forward(self, x, mask=None):
        x = x + self.attn(self.ln1(x), mask)
        x = x + self.ffn(self.ln2(x))
        return x


class Transformer(nn.Module):
    def __init__(self, vocab_size, d_model=512, n_heads=8,
n_layers=6, max_len=5000):
        super().__init__()
        self.tok_emb = nn.Embedding(vocab_size, d_model)
        self.pos_emb = self._build_pos_enc(max_len, d_model)
        self.blocks = nn.ModuleList([
            TransformerBlock(d_model, n_heads) for _ in
range(n_layers)
        ])
        self.ln_f = nn.LayerNorm(d_model)
        self.lm_head = nn.Linear(d_model, vocab_size,
bias=False)

    def _build_pos_enc(self, max_len, d):
        pe = torch.zeros(max_len, d)
        pos = torch.arange(max_len).unsqueeze(1).float()
        div = torch.exp(torch.arange(0, d, 2).float() * -
(math.log(10000.0)/d))
        pe[:, 0::2] = torch.sin(pos * div)
        pe[:, 1::2] = torch.cos(pos * div)
```

```
        return nn.Parameter(pe.unsqueeze(0),
requires_grad=False)

    def forward(self, idx):
        B, L = idx.shape
        x = self.tok_emb(idx) + self.pos_emb[:, :L]
        # Causal mask: upper triangular masking
        mask = torch.triu(torch.ones(L, L, dtype=torch.bool,
device=idx.device),
                          diagonal=1)
        for block in self.blocks:
            x = block(x, mask)
        x = self.ln_f(x)
        return self.lm_head(x)                    # [B, L,
vocab]

# Usage
model = Transformer(vocab_size=10000)
x = torch.randint(0, 10000, (2, 50))   # batch 2, seq 50
logits = model(x)                       # [2, 50, 10000]
```

These roughly 70 lines implement the core of the Transformer. Real production implementations (LLaMA, Qwen) add a dozen engineering refinements on top: RoPE replacing sinusoidal positional encoding, RMSNorm replacing LayerNorm, SwiGLU replacing the ReLU FFN, GQA replacing MHA, and so on—but the **skeleton has not changed in 7 years**.

Experimental Results

In the original paper on WMT 2014 English-to-German translation: BLEU 28.4, surpassing Google NMT (LSTM-based) at 27.3 by 1.1 points, *while requiring only 1/10 of the training time*. That is the key—it wasn't just a marginal improvement in quality; it was *fast enough to train much larger models*.

Impact

"Attention Is All You Need" is one of the most important papers in the history of AI. Seven years later (2024), only ResNet surpasses it in citation count in the CS field.

Yet its *direct* contribution was only a 1.1 BLEU improvement in machine translation—what truly changed the world was the *possibilities it unlocked*:

1. **BERT** (2018) and **GPT** (2018): using the Transformer for pretraining
2. **GPT-3** (2020): proving that the Transformer can scale to tens of billions of parameters and produce emergent capabilities
3. **ViT** (2020): bringing the Transformer to computer vision
4. **ChatGPT** (2022): Transformer + RLHF as a product
5. **All modern LLMs** (2023+): LLaMA, Claude, Qwen, DeepSeek are all Transformer variants

An Interesting Footnote on an Engineering Detail

Section 3.2.2 of the original paper contains the statement: "We found that the Transformer with sinusoidal position encodings performs similarly to learned position embeddings." This claim later proved overly optimistic—learned positional embeddings fail completely when the sequence length exceeds training length, and sinusoidal encodings also extrapolate poorly. This detail was truly resolved by RoPE (P09), five years later.

This is also the value of reading papers: even original authors can be wrong. Understanding these subtleties gives you a clearer picture of how the field evolves.

> **Why 3 pages instead of 1:** If only one paper could be read to understand modern AI, this is it. Mastering the attention mechanism is the entry ticket to all subsequent work. Recommended companion resources: The Annotated Transformer (Harvard NLP) and Andrej Karpathy's "Let's build GPT" video.

P13 BERT and GPT: The Twin Stars of the Pretraining Era

> *Corresponds to main text: Chapter 4, §4.2.5 (Encoder/Decoder divide) · Chapter 3, §3.5.2 (self-supervised pretraining)*

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Authors Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova (Google)

Published NAACL 2019 · arXiv:1810.04805 · 110,000+ citations

Language Models are Few-Shot Learners (GPT-3)

Authors Tom Brown et al. (OpenAI)

Published NeurIPS 2020 · arXiv:2005.14165 · 35,000+ citations

BERT and GPT-3 belong to different technical lineages, but together they defined the "pretraining era." Comparing them side by side is the clearest way to understand both.

BERT: Bidirectional Encoder + MLM

Core innovation: Masked Language Modeling (MLM).

Traditional language models are unidirectional (predicting the next word from left to right). BERT randomly masks 15% of tokens and trains the model to predict the masked content based on *bidirectional* context. Every position's representation can "see" the entire sentence.

```
flowchart LR I["The [MASK] sat on the mat"] --> ENC[Transformer Encoder
bidirectional self-attention] ENC --> P["[MASK] = cat (prediction)"]
```

Second training objective: NSP (Next Sentence Prediction): predicting whether two sentences are consecutive. Subsequent work (RoBERTa 2019) showed that NSP was actually not useful, but MLM has persisted.

The BERT Paradigm

BERT inaugurated the "pretrain + fine-tune" pattern:

1. Perform MLM pretraining on massive unlabeled text (Wikipedia + BookCorpus, ~3.3B tokens)
2. Fine-tune for downstream tasks: classification, QA, NER, etc.—only an output head needs to be added

This paradigm reset the state of the art across virtually all NLP tasks in one sweep. BERT-Base (110M parameters), BERT-Large (340M)—large models by the standards of the time, small by today's.

GPT-3: Unidirectional Decoder + Pure Few-Shot

GPT-3 took the opposite approach:

- Only uses a Decoder (causal language model, predicting the next token left to right)
- **No fine-tuning**—task instructions and a few examples are written into the prompt, and the model learns "in context"
- Dramatic scale increase: 175B parameters, 500× the size of BERT-Large

GPT-3's Key Discovery: In-Context Learning

The true contribution of the GPT-3 paper is not architectural innovation—the architecture is nearly identical to GPT-2. The contribution is demonstrating that: **when a model scales to a sufficient size, the ability to "learn a new task from a few examples" emerges**.

Example:

```
Translate English to French:
sea otter => loutre de mer
plush giraffe => girafe peluche
cheese => ???
```

After seeing just 2 examples, GPT-3 correctly outputs "fromage"—and can generalize to math, reasoning, and coding tasks. *This is*

the starting point at which LLMs transformed from "language models" into "general-purpose reasoning engines."

Comparison of the Two Lineages

	BERT (Encoder)	GPT (Decoder)
Attention direction	Bidirectional attention	Unidirectional causal
Training objective	MLM + NSP	Next-token prediction
Task adaptation	Pretrain + fine-tune	Prompting / in-context learning
Typical tasks	Classification, NER, QA, sentence embeddings	Text generation, dialogue, code
Representative scale	BERT-Large 340M	GPT-3 175B
Post-2024 descendants	RoBERTa, DeBERTa, still used for embeddings	LLaMA, Claude, Qwen, DeepSeek — mainstream

Why Decoder-Only Won in the End

Main text §4.2.5 analyzes the key factors:

1. Decoder-only training is simpler—a single objective, and all text is training data
2. In-context learning makes "one model for all tasks" a reality
3. Scaling properties are more favorable for decoder-only—this became consensus after GPT-3
4. Generation capability naturally encompasses understanding (to write a good answer, you must first understand the question)

Today, BERT-family encoder models are primarily used as "text encoders"—embedding models (BGE, E5), rerankers, and so on. The retrieval stage in most RAG systems still uses encoder models.

Impact

BERT inaugurated the **pretraining paradigm**—all LLMs today inherit this idea. GPT-3 demonstrated **the power of scaling**—directly leading to ChatGPT and GPT-4. These two papers are the twin stars of LLM history.

> **Why You Must Read This:** BERT helps you understand the "pretrain-then-fine-tune" paradigm. GPT-3 helps you understand "emergent capabilities" and "in-context learning"—two concepts that are the core assumptions underlying the entire LLM era that followed.

P14 Scaling Laws and Chinchilla: The Economics of Training Large Models

> *Corresponds to main text: Chapter 4, §4.1.1 · Chapter 3, §3.5.2*

Scaling Laws for Neural Language Models

Authors Jared Kaplan et al. (OpenAI)

Published arXiv:2001.08361 (2020)

Training Compute-Optimal Large Language Models (Chinchilla)

Authors Jordan Hoffmann et al. (DeepMind)

Published NeurIPS 2022 · arXiv:2203.15556

The Question: Given a Fixed Compute Budget, How Should It Be Split Between Parameters and Data?

Training a large model incurs a total compute cost of roughly $C \approx 6 \cdot N \cdot D$ (N = number of parameters, D = number of training tokens). Suppose you have a fixed C (say, 1e25

FLOPs): is it better to train a large model on a small dataset, or a smaller model on a large dataset?

Kaplan (2020): Loss Follows a Power Law

OpenAI ran hundreds of models of varying sizes and found that loss decreases as a power law with both parameters and data:

> $$L(N, D) \approx E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}$$

E is the irreducible entropy, $\alpha \approx 0.07$, $\beta \approx 0.28$.

Kaplan's key conclusion (later proven incorrect): under a fixed compute budget, the optimal allocation grows the parameter count N much faster than the data D. This inspired the design of GPT-3 at 175B—trained on only 300B tokens (near-optimal by Kaplan's prediction).

Chinchilla (2022): Kaplan Was Wrong

DeepMind redid the experiments with a wider range of N, D combinations. New fitted formula:

> $$L(N, D) = E + \frac{A}{N^{0.34}} + \frac{B}{D^{0.28}}$$

Key: α changed from 0.07 to 0.34—this means parameters and data should scale in **roughly equal proportion**.

Specific conclusion: the optimal *tokens-to-parameters ratio is approximately 20:1*.

Model	Parameters	Training tokens	Actual ratio	Chinchilla-optimal?
GPT-3	175B	300B	1.7:1	❌ Severely data-starved
Chinchilla	70B	1.4T	20:1	✅ Optimal

LLaMA-1 65B	65B	1.4T	21.5:1	✅ Near-optimal
LLaMA-3 70B	70B	15T	214:1	Inference-optimized, far exceeds Chinchilla

Experimental Proof of Chinchilla

Chinchilla (70B parameters, 1.4T training tokens) vs. Gopher (280B parameters, 300B tokens)—under equivalent compute, Chinchilla comprehensively outperformed the 4× larger Gopher on benchmarks like MMLU. **"Smaller but better-trained" beats "larger but data-starved"**—a significant correction in Scaling Law research.

Training-Optimal vs. Inference-Optimal

Chinchilla optimizes for "lowest training loss." But in practice, models must be run for inference millions of times, so *inference efficiency* is also an optimization target.

The LLaMA-2/3 series recognized this: deliberately training *beyond* Chinchilla-optimal—LLaMA-3-70B was trained on 15T tokens (214:1), far past "optimal" in training terms, but yielding a model that is "smaller in parameters yet stronger in capability," making inference cheaper. This is known as "over-training for inference."

Impact

These two papers fundamentally changed decision-making in LLM training:

1. The "just pile on parameters" approach after GPT-3 was corrected
2. Data collection and cleaning became equally important as model architecture
3. The LLaMA series' training data volumes (1.4T → 2T → 15T) stem directly from these insights

> **Why You Must Read This:** If you want to understand the "parameters vs. data" discussion that accompanies every major model release since 2022, these two papers are required reading. They are also the starting point for understanding "why data is so expensive now"—Chinchilla's conclusions set off a data arms race among all major AI companies.

P15 RoPE: Rotary Position Embedding

> *Corresponds to main text: Chapter 4, §4.3.2 Deep Dive on RoPE*

RoFormer: Enhanced Transformer with Rotary Position Embedding

Authors Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, Yunfeng Liu

Affiliation Zhuiyi Technology

Published Neurocomputing 2024 · arXiv:2104.09864 (2021)

Citations ~4,500+ (rapidly growing)

The Problem: Absolute Positional Encodings Cannot Extrapolate

The original Transformer used sinusoidal absolute positional encodings; GPT-2 used learned absolute positional encodings. Both share the same problem—**if positions 0–2047 were seen during training, the model fails for positions 2048+**. Learned encodings are worse (they break entirely out of range); sinusoidal encodings are slightly better but still don't work. Relative positional encodings (T5 bias, ALiBi) improved matters but had limited expressiveness.

Core Idea: Rotation Instead of Addition

RoPE does not add a positional vector to the input; instead, it *rotates* Q and K. The key insight: when two vectors are each

rotated independently and then dot-producted, the result depends only on their *relative angle*.

> $$\mathbf{q}_m^\top \mathbf{k}_n = (R_m \mathbf{q})^\top (R_n \mathbf{k}) = \mathbf{q}^\top R_m^\top R_n \mathbf{k} = \mathbf{q}^\top R_{n-m} \mathbf{k}$$

(Because rotation matrices satisfy $R_m^\top R_n = R_{n-m}$)

So the attention score naturally depends only on the relative position $n - m$. This is exactly the property that relative positional encodings aim for, achieved here in a more elegant way.

2D Rotation Matrix

A 2D rotation is applied to each pair of adjacent dimensions:

> $$R_m^{(i)} = \begin{pmatrix} \cos m\theta_i & -\sin m\theta_i \\ \sin m\theta_i & \cos m\theta_i \end{pmatrix}, \quad \theta_i = 10000^{-2(i-1)/d}$$

Different dimensions use different frequencies θ_i: low dimensions use high frequencies, high dimensions use low frequencies—each dimension captures positional differences at a different scale.

Complex Number Perspective (More Elegant)

Treating a 2D vector as a complex number $q = q_1 + iq_2$, rotation by $m\theta$ is equivalent to multiplying by $e^{im\theta}$:

> $$q_m = q \cdot e^{im\theta}, \quad k_n = k \cdot e^{in\theta}$$

> $$\langle q_m, k_n \rangle = \text{Re}(q \cdot e^{im\theta} \cdot \overline{k \cdot e^{in\theta}}) = \text{Re}(q \bar{k} \cdot e^{i(m-n)\theta})$$

This depends only on $m - n$. This is the most mathematically elegant expression of RoPE.

Implementation

```
def build_rope_cache(d_head, max_len, base=10000,
device='cuda'):
    theta = 1.0 / (base ** (torch.arange(0, d_head, 2).float() /
d_head))
    t = torch.arange(max_len, device=device).float()
    freqs = torch.outer(t, theta.to(device))            #
[max_len, d_head/2]
    return torch.polar(torch.ones_like(freqs), freqs)   #
complex form

def apply_rope(x, freqs_cis):
    # x: [B, H, L, d_head]
    x_complex = torch.view_as_complex(
        x.float().reshape(*x.shape[:-1], -1, 2)
    )
    x_rotated = x_complex * freqs_cis[:x.shape[-2]]
    return torch.view_as_real(x_rotated).flatten(-2).type_as(x)

# Usage: apply to Q and K inside attention (not V)
Q = apply_rope(Q, freqs_cis)
K = apply_rope(K, freqs_cis)
# V does not get RoPE — it carries content, not position
```

Derivative Techniques Built on RoPE

RoPE's continuous trigonometric nature gives it "extrapolation potential"—the following techniques extend RoPE-trained models far beyond their training length:

PI (Position Interpolation, Meta 2023): compress positions into the training range, enabling 2–4× extrapolation

NTK-aware: compress only high-frequency dimensions while preserving low-frequency ones—enables extrapolation without any fine-tuning

YaRN (2023): per-frequency-band refinement + attention temperature adjustment; 1B tokens of fine-tuning enables 32× extrapolation

LongRoPE (2024): independently searches for the optimal scaling factor per dimension; feasible up to 2M token contexts

Impact

RoPE is today the de facto standard for open-source LLMs: LLaMA, Qwen, DeepSeek, Mistral, and ChatGLM all use RoPE. The reasons:

1. Relative position property emerges naturally from the math
2. Zero additional parameters
3. Extrapolation-friendly (in combination with YaRN, etc.)
4. Simple implementation (<30 lines of code)

Interestingly, RoPE's author Jianlin Su (Zhuiyi Technology) was not a researcher at a major lab; the paper was originally published on his personal blog. This is a microcosm of the open-source community's influence—good ideas don't have to come from big companies.

> **Why You Must Read This:** Long context (128K, 1M tokens) is one of the main battlegrounds for LLMs in 2024–2026, and RoPE + YaRN is the underlying technology for that direction. Understanding RoPE is what lets you understand why today's models can support such long contexts.

> **Ch 4 Related · Efficiency & Inference**

P16 FlashAttention: IO-Aware Exact Attention

> *Main text reference: Chapter 3, §3.6.3 Operator Optimization · Chapter 4, §4.2.2 / §4.8*

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

Authors Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, Christopher Ré

Affiliation	Stanford University
Published	NeurIPS 2022 · arXiv:2205.14135 · 5,000+ citations
Follow-up	FlashAttention-2 (2023), FlashAttention-3 (2024, FP8 + H100 optimizations)

The Problem: Attention's Real Bottleneck Is Not FLOPs

Self-attention has time complexity $O(L^2 d)$ for long sequences, often labeled the "compute bottleneck." But Tri Dao observed that **on GPUs, attention is actually a memory-bandwidth bottleneck, not a compute bottleneck**.

The numbers: an A100 GPU delivers 312 TFLOPs for FP16 GEMM, but its HBM bandwidth is only 1.5 TB/s. The ratio (arithmetic intensity) needs to reach ~208 FLOP/byte to saturate compute — and standard attention falls far below this. Most of the time, the GPU is stalled waiting for data to move from HBM.

The Problem with Standard Attention

A standard PyTorch implementation does four things:

1. Compute $S = QK^\top \in \mathbb{R}^{L \times L}$, write it to HBM
2. Read S from HBM, compute $P = \text{softmax}(S)$, write it back to HBM
3. Read P from HBM, multiply by V to get O, write it back to HBM
4. (Possible dropout and masking operations add further read/write passes)

At $L = 4096$, the S matrix alone is 64 MB. Each read/write is a 128 MB round-trip to HBM. As L grows, this cost scales linearly while compute utilization stays extremely low.

Core Method: Tiling + Online Softmax

FlashAttention splits Q, K, V into tiles sized to fit in the GPU's on-chip SRAM. The outer loop iterates over Q tiles Q_i; the inner loop iterates over K, V tiles (K_j, V_j). **The full S or P matrix is never materialized** — all intermediate results stay in SRAM.

```
flowchart TB HBM["HBM — large, slow"] -. "read Q_i, K_j, V_j" .-
                > SRAM SRAM["SRAM — small, fast
tile-level computation"] -. "write O_i" .-> HBM SRAM --> S["S_ij
 = Q_i K_j^T (in SRAM)"] S --> SM["online softmax: update m, l,
             O"] SM --> NEXT["next K_j, V_j tile"]
```

The Key Challenge: Online Softmax

Softmax requires normalization — \text{softmax}(x_i) = \exp(x_i) / \sum_j \exp(x_j). It seems like you must see all x_j before computing the denominator. FlashAttention uses an **incremental update** instead: each time a new S_{ij} tile arrives, it maintains a running max and running sum, producing a result that is mathematically equivalent to the full softmax.

Say we have processed up to tile j-1, with state (m_{j-1}, l_{j-1}, O_{j-1}) — the current running max m, running denominator l, and current weighted output O. A new tile (K_j, V_j) arrives:

Compute S_{ij} = Q_i K_j^\top. Let \tilde{m}_j = \max(S_{ij}). New max:

> m_j = \max(m_{j-1}, \tilde{m}_j)

Rescale both the new and old exponential terms using the new max:

> l_j = e^{m_{j-1} - m_j} \cdot l_{j-1} + \sum_k e^{S_{ij,k} - m_j}

New output (old contribution is rescaled to the new max):

> O_j = \frac{1}{l_j}\left[l_{j-1} e^{m_{j-1} - m_j} O_{j-1} + \sum_k e^{S_{ij,k} - m_j} V_{j,k} \right]

After scanning all K/V tiles, O_J is exactly equivalent to the full softmax result. It is numerically identical to standard attention — this is not an approximation.

Pseudocode

```
def flash_attention(Q, K, V, block_size=64):
    """Q, K, V: [L, d].  Returns O: [L, d], never materializing
the LxL matrix S"""
    L, d = Q.shape
    O = torch.zeros_like(Q)
    m = torch.full((L, 1), -float('inf'))  # running max per row
    l = torch.zeros((L, 1))                 # running sum per
row

    for j in range(0, L, block_size):
        K_j = K[j:j+block_size]              # load tile into
SRAM
        V_j = V[j:j+block_size]
        S_ij = Q @ K_j.T / math.sqrt(d)      # computed in SRAM,
[L, block]

        # Causal mask (if needed)
        mask = torch.arange(j, j+K_j.shape[0]).unsqueeze(0) > \
               torch.arange(L).unsqueeze(1)
        S_ij = S_ij.masked_fill(mask, -float('inf'))

        m_new = torch.maximum(m, S_ij.max(dim=-1,
keepdim=True).values)
        P_ij = torch.exp(S_ij - m_new)
        l_new = torch.exp(m - m_new) * l + P_ij.sum(dim=-1,
keepdim=True)

        # Rescale old O to new max, add current tile's
contribution
        O = (l / l_new) * torch.exp(m - m_new) * O + (P_ij @
V_j) / l_new
        m, l = m_new, l_new

    return O
```

The actual CUDA implementation involves substantial engineering — warp-level shuffling, register reuse, pipelining — but the

core algorithm is exactly what is shown above.

Performance Gains

- **HBM accesses**: reduced from O(L^2 d) to O(L^2 d^2 / M) (M = SRAM size) — several orders of magnitude fewer
- **Speed**: 2–4× faster than standard attention on A100, with greater gains at longer sequences
- **Memory**: intermediate S and P are never materialized, reducing peak memory from O(L^2) to O(L) — this is what makes 128K+ context windows feasible

FlashAttention-2 and -3

FlashAttention-2 (2023): Refined outer loop ordering, reduced non-matmul operations, improved warp partitioning. Another 2× speedup, totaling 4–8× over standard attention.

FlashAttention-3 (2024): Targets the H100 Hopper architecture, leveraging FP8 Tensor Cores, asynchronous execution, and more granular pipelining. Achieves ~75% of peak throughput on H100.

These three generations represent the most important attention optimization work from 2022 to 2024, and directly influenced the implementations of PyTorch SDPA, vLLM, xFormers, and TensorRT-LLM. Modern LLM training and inference almost universally defaults to FlashAttention.

Key Insight

FlashAttention's impact extends well beyond attention itself — it is the exemplar of **IO-aware algorithm design**. Many GPU kernels can be restructured similarly: keep intermediate results in SRAM, and use mathematically equivalent incremental forms to avoid large intermediate buffers. FlashDecoding and RingAttention are direct successors of this idea.

> **Why You Must Read This:** The online softmax algorithm is something you need to work through by hand to truly understand. Once you grasp it, you will recognize

> the same technique appearing in many other places — PagedAttention, StreamingLLM, and FlashDecoding all rely on the same mathematical structure.

P17 LoRA / QLoRA: Making Fine-Tuning Affordable

> *Main text reference: Chapter 3, §3.5.3 Fine-Tuning · Chapter 4, §4.4.1 Low-Rank Ideas*

LoRA: Low-Rank Adaptation of Large Language Models

Authors Edward Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen

Affiliation Microsoft

Published ICLR 2022 · arXiv:2106.09685 · 8,000+ citations

QLoRA: Efficient Finetuning of Quantized LLMs

Authors Tim Dettmers et al. (University of Washington)

Published NeurIPS 2023 · arXiv:2305.14314

The Problem: Full Fine-Tuning a Large Model Is Too Expensive

Fine-tuning a 70B model requires storing: parameters at 140 GB (FP16) + gradients at 140 GB + Adam optimizer state at 560 GB ≈ 840 GB total, plus activations. Even a multi-GPU single node isn't enough — you need ZeRO-3-level distributed training.

Yet many use cases only need light customization — adding a domain-specific style, or making the model follow a particular output format. Is it really necessary to update every parameter?

LoRA's Core Hypothesis

Hu et al. hypothesize that **when a model adapts to a downstream task, the weight update \Delta W has a low-rank structure**.

That is, \Delta W \in \mathbb{R}^{d \times k} can be decomposed as \Delta W = BA, where B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k}, and r \ll \min(d, k).

During training, the original weights W_0 are frozen; only A and B are trained:

> W = W_0 + \Delta W = W_0 + BA, \quad h = W_0 x + BA x

flowchart LR X[x] --> W0["W_0 (frozen, d x k)"] W0 --> Y1[W_0 x] X --> A["A (r x k, trainable)"] A --> B["B (d x r, trainable)"] B --> Y2["BA x"] Y1 --> ADD((+)) Y2 --> ADD ADD --> H[h]

Initialization: A is initialized with Gaussian noise; B is initialized to zero — so the starting point BA = 0, matching full fine-tuning's initial state.

Parameter Count Comparison

For GPT-3 175B, consider a layer where W \in \mathbb{R}^{12288 \times 12288}, with 150M parameters. With r = 8: LoRA introduces A \in \mathbb{R}^{8 \times 12288} and B \in \mathbb{R}^{12288 \times 8}, for a total of 200K parameters — a **750× reduction**.

Across the full model, LoRA trains approximately 0.01% of parameters. Memory requirements are dramatically reduced: frozen weights require no Adam state or gradient storage — only A and B need gradients — yielding *10–20× memory savings over full fine-tuning*.

Implementation (PEFT)

```
from peft import LoraConfig, get_peft_model
from transformers import AutoModelForCausalLM

base = AutoModelForCausalLM.from_pretrained("meta-llama/Llama-3-8B")
```

```
config = LoraConfig(
    r=16,                          # rank
    lora_alpha=32,                 # scaling factor; effective W
+= (alpha/r) BA
    target_modules=["q_proj", "v_proj"],  # apply LoRA only to Q
and V projections
    lora_dropout=0.05,
    task_type="CAUSAL_LM",
)
model = get_peft_model(base, config)
model.print_trainable_parameters()
# trainable params: 4,194,304 || all params: 8,034,455,552 ||
trainable%: 0.05%
```

Merging at Inference Time

At inference, BA can be merged back into W_0: W_{\text{new}} = W_0 + BA, producing a standard weight matrix. **Zero additional inference overhead** — this is a major advantage over Adapters, which insert small modules between layers and increase latency at inference time.

QLoRA: Another 4× Memory Reduction

QLoRA (2023) quantizes the base model to 4-bit while keeping the LoRA parameters in FP16. Memory breakdown:

- Frozen weights: 140 GB (FP16) → 35 GB (INT4), 4× saving
- LoRA parameters and gradients: tens of MB, negligible
- Total: **a 65B model fits on a single 48 GB GPU**

QLoRA introduces several technical innovations:

1. **NF4** (NormalFloat 4-bit): a 4-bit data type designed for normally distributed weights, achieving higher precision than standard INT4
2. **Double quantization**: quantizes the quantization scales themselves, further reducing memory
3. **Paged optimizers**: uses NVIDIA Unified Memory to swap optimizer state to CPU when GPU memory is insufficient

QLoRA made "fine-tuning a 65B model on a consumer GPU" a reality — it was the inflection point that opened large-scale fine-

tuning to the open-source community. Projects like Alpaca and Vicuna all relied on LoRA/QLoRA.

Variants

LoRA+: uses different learning rates for A and B (B should be larger), a small change that yields 1–2% quality improvement
DoRA (Weight-Decomposed LoRA): decomposes weights into magnitude and direction, adapting each separately
AdaLoRA: dynamically allocates rank per layer
rsLoRA: improved scaling rules to stabilize training at larger ranks

Impact

LoRA is **the most important fine-tuning work since 2022**. Nearly every open-source LLM fine-tune today uses LoRA or one of its variants — HuggingFace's PEFT library was built around it. LoRA also inspired architectural thinking: DeepSeek's MLA uses low-rank decomposition to compress the KV Cache (main text §4.4.1), a direct conceptual descendant.

> **Why You Must Read This:** LoRA is the most important "lever" in the open-source community — it lets individuals and small teams participate in frontier LLM adaptation. QLoRA's NF4 design is worth reading carefully; it offers excellent intuition about data type design tailored to a distribution.

P18 PagedAttention / vLLM: Treating KV Cache Like Virtual Memory

> *Main text reference: Chapter 4, §4.8.3*

Efficient Memory Management for Large Language Model Serving with PagedAttention

Authors Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, Ion Stoica

Affiliation UC Berkeley, Stanford, UCSD, CMU

Published SOSP 2023 · arXiv:2309.06180

The Problem: Memory Fragmentation in KV Cache

LLM inference requires contiguous KV Cache space per request. A naive implementation pre-allocates by `max_seq_len`: a request reserving space for 4096 tokens might only use 500, *wasting 87%*. There are four categories of waste at the system level:

1. **Reservation waste**: space reserved for sequences that might grow longer
2. **Internal fragmentation**: each request has a different actual length, so reserved space goes unused
3. **External fragmentation**: small gaps left by completed short requests can't fit new larger ones
4. **No sharing**: requests with identical prefixes each store their own copy (pure waste)

Core Method: Bringing OS Virtual Memory to KV Cache

PagedAttention directly mirrors OS virtual memory:

GPU memory is divided into fixed-size blocks (KV data for 16 tokens per block)
Each request maintains a block table mapping logical token positions to physical blocks
Blocks are allocated on demand and grow dynamically with token generation — no pre-reservation
The attention kernel accesses KV data indirectly via the block table

flowchart TB subgraph GPU["GPU Memory Pool (blocks of 16 tokens KV)"] B0[Block 0: free] B3[Block 3: R1 tokens 16-19] B5[Block 5: R2 tokens 0-14] B7[Block 7: R1 tokens 0-15] B8[Block 8: free] end R1["Request 1 (20 tokens)

```
block_table: [7, 3]"] --> B7 R1 --> B3 R2["Request 2 (15 tokens)
                    block_table: [5]"] --> B5
```

Modified Attention Kernel

The standard kernel assumes K and V are contiguous. PagedAttention's kernel uses the block table for indirect lookup:

```
// Simplified pseudocode
for each query q_i:
    output = 0
    max_score = -inf; sum_exp = 0
    for each logical token j in query's block_table:
        block_id  = block_table[j / BLOCK_SIZE]
        inner_idx = j % BLOCK_SIZE
        k_j = K_pool[block_id * BLOCK_SIZE + inner_idx]
        v_j = V_pool[block_id * BLOCK_SIZE + inner_idx]
        score = q_i @ k_j / sqrt(d)
        // online softmax update (same as FlashAttention)
        ...
```

The one extra indirection (`block_table[...]`) adds a small overhead, but the memory savings far outweigh this cost.

Sharing and Copy-on-Write

When multiple requests share the same prefix (e.g., a system prompt or few-shot examples), their block tables can point to the same physical blocks. A copy is made only when divergence occurs — when a request starts generating different tokens. This makes **prefix caching** a natural optimization, directly benefiting high-concurrency LLM serving.

Beam search and parallel sampling benefit similarly: multiple candidates share the prefix KV Cache.

Measured Gains

Paper results (LLaMA-13B):

- Memory utilization: ~40% → ~96%

- Concurrent requests per GPU: 4–10×
- Throughput: 2–4× improvement

vLLM

The paper authors simultaneously open-sourced vLLM — a production-grade inference engine combining PagedAttention, continuous batching, and speculative decoding. Since its mid-2023 release:

- It became the de facto standard for open-source LLM serving
- In 2024, it ranked alongside SGLang as one of the top two inference frameworks by OpenRank
- A major v1 architectural overhaul shipped in 2025
- Adopted by virtually every major cloud provider and LLM service

vLLM's success exceeds the citation value of the PagedAttention paper itself — it is a textbook example of "research result becoming industrial infrastructure."

Key Insight

This is a classic example of cross-domain borrowing. The virtual memory concept was mature by 1961 (the Atlas computer) — over 60 years later, it was transplanted into LLM inference to solve a new problem. A programmer who studied operating systems can read this paper and understand it almost instantly. That is the power of foundational knowledge.

> **Why You Must Read This:** Without this paper, you won't understand why vLLM can be 5–10× faster than a naive implementation. For systems engineers, it is also an excellent model of cross-domain knowledge transfer.

P19 Switch Transformer & Mixtral: Making MoE Practical

> *Main text reference: Chapter 4, §4.6*

Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity

Authors William Fedus, Barret Zoph, Noam Shazeer (Google)

Published JMLR 2022 · arXiv:2101.03961

Mixtral of Experts

Authors Mistral AI team

Published arXiv:2401.04088 (2024)

MoE Is an Old Idea

The MoE concept traces back to Jacobs et al.'s 1991 "Adaptive Mixtures of Local Experts." Shazeer et al. first applied it to deep learning in 2017 (Sparsely-Gated MoE). But it was Switch Transformer and Mixtral that truly demonstrated MoE's practical value.

Switch Transformer: Top-1 Routing

Switch's core simplification: **each token is routed to exactly 1 expert** (prior work typically used top-2). This drastically simplifies the gating mechanism and reduces communication complexity.

> $$\text{expert}(x) = \arg\max_i g_i(x), \quad g(x) = \text{softmax}(W_r x)$$

If expert i is selected, the output is:

> $$y = g_i(x) \cdot \text{Expert}_i(x)$$

The key engineering challenge is **load balancing**. If routing is uneven — some experts overloaded, others idle — GPU utilization suffers. Switch adds an auxiliary loss:

> $\mathcal{L}_{\text{aux}} = \alpha \cdot N \sum_{i=1}^N f_i \cdot P_i$

Here f_i is the fraction of tokens routed to expert i, and P_i is the average gate probability for expert i across all tokens. At the optimum, $f_i = P_i = 1/N$ and $\mathcal{L}_{\text{aux}} = 1$.

Switch Transformer trained a **1.6 trillion parameter** model — the first publicly disclosed "trillion-parameter LLM." It shocked the field at the time. However, its quality advantage over dense models of the same compute budget was modest, and the community remained skeptical about whether MoE was truly worthwhile.

Mixtral 8x7B: MoE Proves Itself

Mistral AI's late-2023 release of Mixtral 8x7B changed the status of MoE entirely:

- 8 experts, each roughly the size of a 7B parameter model
- Top-2 routing (rather than Switch's Top-1)
- Total parameters: 47B (with shared attention); activated parameters per token: ~13B
- Quality comparable to LLaMA-2 70B (dense), at the inference speed of a 13B model

```
flowchart TB X[Token x] --> R[Router: softmax W_r x] R -->|top-2
gate| G["g_i=0.6
g_j=0.4"] G --> E1[Expert 1 FFN] G --> E2[Expert 2 FFN] E1 --
>|w=0.6| SUM((+)) E2 -->|w=0.4| SUM SUM --> Y[Output] E3[Expert
3 FFN] -.-> SUM E4[Expert 4 FFN] -.-> SUM E5[...] -.-> SUM
```

Mixtral matched LLaMA-2 70B on MMLU, math, and coding benchmarks, while being significantly faster to run. This was the first clear demonstration that **"MoE achieves the same capability at lower cost."**

Engineering Challenges of MoE

MoE looks appealing on paper but is difficult in practice. Key engineering challenges:

1. **Memory footprint**: all experts must be loaded at inference time (you don't know routing in advance). Mixtral 8x7B requires 80 GB+ of GPU memory — it is not as lean as the "13B activated" framing suggests
2. **Load imbalance**: even with auxiliary loss, some experts still get overloaded. A "capacity factor" of 1.25 (allowing 25% overflow) reduces token dropping
3. **Small-batch inefficiency**: with small batches, each expert receives very few tokens, and GPU utilization is low. MoE works best at large batch sizes
4. **All-to-all communication**: in distributed MoE, tokens must be routed to different devices per layer — two all-to-all operations per layer, with significant communication overhead

Subsequent Evolution

DeepSeek-V2/V3 (2024) pushed MoE further:

Fine-grained experts: moved from 8 large experts to 256 small experts, enabling far more diverse combinations
Shared experts: 1–2 experts that every token passes through (handling general knowledge), with other experts gated
Multi-token prediction, **MLA attention compression**, and **FP8 training** combined with MoE to further improve efficiency

Qwen3-Next, Kimi K2, GLM-4.5, and LLaMA 4 all follow the fine-grained MoE path. By late 2025, nearly every frontier open-source model is a MoE.

> **Why You Must Read This:** MoE is the core technology enabling large-scale model scaling. Reading both Switch and Mixtral gives you a full picture of "sparse activation" as an engineering principle — one idea (compute only part of the parameters) reshaped the entire open-source LLM landscape from 2024 to 2026.

> **PART FOUR**

Chapter 4 · Alignment & Applications

RLHF, DPO, CoT, DeepSeek-V3 — 4 key papers that took LLMs from "can talk" to "can do things."

P20 Mamba: Linear-Time Sequence Modeling

Corresponds to main text: Chapter 4, Section 4.4.3 — Non-Transformer Architectures

Mamba: Linear-Time Sequence Modeling with Selective State Spaces

Authors	Albert Gu, Tri Dao
Affiliation	Carnegie Mellon University, Princeton University
Published	COLM 2024 (preprint 2023)
Citations	3,000+
Link	arXiv:2312.00752

The Problem: Transformer's Quadratic Complexity

Transformer's self-attention is O(N^2) in both time and memory with respect to sequence length N — a fundamental limitation. FlashAttention (P10) reduces the constant factor through hardware-level optimization, but the quadratic asymptotic complexity remains. For ultra-long sequences at the million-token scale (genomes, long videos, code repositories), Transformers remain impractical.

From 2020 to 2023, Gu explored **State Space Models (SSMs)** — a sequence modeling framework rooted in control theory with O(N) complexity. Prior work S4 (2021) and H3 (2022) demonstrated

feasibility, but these models consistently fell short of Transformers in language modeling. Mamba is the breakthrough in this line of research.

Background: Structured State Space Models (S4)

The continuous-time SSM is defined as:

> $$h'(t) = Ah(t) + Bx(t)$$

> $$y(t) = Ch(t) + Dx(t)$$

After discretization (using zero-order hold or bilinear transform):

> $$h_t = \bar{A} h_{t-1} + \bar{B} x_t$$

> $$y_t = C h_t + D x_t$$

This is a linear recurrence — similar to an RNN, but with fixed parameters $\bar{A}$, $\bar{B}$, C that do not depend on the input. Fixed parameters mean: the recurrence can be unrolled into a convolution and accelerated with FFT ($O(N \log N)$ training), while also supporting $O(1)$ per-step inference via the recurrence form.

S4's limitation: because A, B, C do not depend on the input, the model **cannot selectively remember or forget information based on content**. This is precisely the core strength of Transformer attention (attention weights are entirely determined by the input).

Core Innovation: Selective State Spaces

Mamba's key insight: make B, C, and the discretization step size Δ **input-dependent**:

> $$B_t = \text{Linear}(x_t), \quad C_t = \text{Linear}(x_t), \quad \Delta_t = \text{softplus}(\text{Linear}(x_t))$$

This allows the model to decide "how much to remember" and "what to output" based on the content of the current token. \Delta_t controls the discretization step size — when \Delta is large, \bar{A} approaches 0 (forgetting history); when \Delta is small, \bar{A} approaches 1 (retaining history).

Input dependence means FFT can no longer be used (the convolution kernel is no longer fixed). Gu and Dao (also the author of FlashAttention) designed a **hardware-aware selective scan algorithm** — essentially replacing the naive recurrence with parallel prefix sums (parallel scan) for efficient GPU execution.

```
flowchart TB X["Input x_t"] --> Lin1["Linear -> B_t"] X --> Lin2["Linear -> C_t"] X --> Lin3["Linear -> Delta_t"] Lin3 --> Disc["Discretize A, B"] Lin1 --> Disc Disc --> Scan["Selective scan (parallel scan)"] H["h_{t-1}"] --> Scan Scan --> H2["h_t"] H2 --> Out["y_t = C_t * h_t"] Lin2 --> Out
```

Mamba Block Architecture

The full Mamba block is more than just the SSM — it also wraps a structure similar to the Transformer FFN:

```
class MambaBlock(nn.Module):
    """Simplified Mamba block"""
    def __init__(self, d_model, d_state=16, d_conv=4, expand=2):
        super().__init__()
        d_inner = d_model * expand
        self.in_proj = nn.Linear(d_model, d_inner * 2,
bias=False)  # split into two branches
        self.conv1d = nn.Conv1d(d_inner, d_inner, d_conv,
padding=d_conv-1,
                                groups=d_inner)  # local
convolution
        self.act = nn.SiLU()

        # SSM parameters (input-dependent)
        self.x_proj = nn.Linear(d_inner, d_state * 2 + 1,
bias=False)  # B, C, delta
        self.A_log = nn.Parameter(torch.log(torch.arange(1,
d_state+1).float()))
```

```
        self.D = nn.Parameter(torch.ones(d_inner))

        self.out_proj = nn.Linear(d_inner, d_model, bias=False)

    def forward(self, x):
        # x: (batch, seq_len, d_model)
        xz = self.in_proj(x)
        x_branch, z = xz.chunk(2, dim=-1)

        # Local convolution (captures nearby context)
        x_branch = self.conv1d(x_branch.transpose(1, 2))[:, :,
 :x.size(1)]
        x_branch = x_branch.transpose(1, 2)
        x_branch = self.act(x_branch)

        # Selective SSM
        y = self.selective_ssm(x_branch)
        y = y + self.D.unsqueeze(0).unsqueeze(0) * x_branch  #
 skip connection

        # Gated output
        output = y * self.act(z)
        return self.out_proj(output)

    def selective_ssm(self, x):
        # Compute B, C, delta from input
        proj = self.x_proj(x)
        # ... discretization + parallel scan (in practice uses a
 Triton kernel)
        pass  # full implementation requires selective_scan_cuda
```

Complexity Comparison

\lt td>O(N d) (KV cache) \lt td>O(d) \lt td>O(d) (fixed state)

Model	Training Complexity	Inference (per token)	Long-Sequence Support
Transformer	O(N^2 d)	Limited by quadratic complexity	
S4 (fixed SSM)	O(N \log N \cdot d)	Good, but lacks selectivity	

Mamba	O(N d)	Theoretically unbounded

Experimental Results

- Language modeling: Mamba-3B matches same-parameter-count Transformer++ perplexity on The Pile
- Inference throughput: at sequence length 64K, Mamba is 5× faster than Transformer
- DNA modeling: substantially surpasses Transformers on the HG38 human genome (natural advantage on ultra-long sequences)
- Audio: leads on SC09 speech classification

Impact and Controversy

Mamba sparked a wave of interest in "Transformer alternatives":

Mamba-2 (Gu & Dao, 2024): unifies SSMs into a matrix multiplication framework for further speedup
Jamba (AI21, 2024): hybrid Mamba + Transformer architecture
RWKV (Peng, 2023): another RNN revival approach, complementary in spirit to Mamba
Zamba (Zyphra, 2024): Mamba backbone with a small number of attention layers

Yet the controversy is significant: as of 2025, **no Mamba-only model has genuinely surpassed a same-scale Transformer in large-scale (>10B) language modeling**. The mainstream direction has shifted toward hybrid architectures (a few attention layers combined with many SSM layers). Mamba's value is more likely as a "complementary component" to Transformers than as a "replacement."

> **Why You Must Read This:** Understanding "why Transformer's quadratic complexity is a problem" and "where the tradeoffs in linear alternatives lie" is foundational for evaluating the trajectory of LLM architecture evolution. Mamba's selective state space idea — letting the model decide its memory strategy based on content — is the modern 2024 revival of LSTM's gating concept.

P21 InstructGPT / RLHF: The Starting Point of Alignment

Main text reference: Chapter 3, §3.5.3 Fine-Tuning & Alignment · Chapter 4, §4.1.1

Training language models to follow instructions with human feedback

Authors Long Ouyang, Jeff Wu, Xu Jiang, et al. (20+ authors)

Affiliation OpenAI

Published NeurIPS 2022 · arXiv:2203.02155 · 12,000+ citations

The Problem: The Gap Between a Pretrained Model and a "Helpful Assistant"

GPT-3's pretraining objective is "predict the next token" — the model can continue text, but doesn't necessarily *answer*. Ask it "How do I make a cake?" and it might continue with "? This is a common question..." rather than actually teaching you how to bake.

InstructGPT addresses the problem of teaching a pretrained model to respond "as humans expect." It is the direct predecessor of ChatGPT, and the foundational work of the entire alignment field.

Three-Stage Pipeline

```
flowchart LR PT[Pretrained GPT-3] --> SFT[Stage 1 SFT
human instruction–response pairs] SFT --> RM[Stage 2 RM
human preference data] RM --> PPO[Stage 3 PPO
RL optimization] PPO --> FINAL[InstructGPT]
```

Stage 1: SFT (Supervised Fine-Tuning)

Human annotators write 13K "prompt + ideal response" pairs, and GPT-3 is fine-tuned on these with standard supervised learning. Goal: teach the model the basic format of "respond to instructions."

Stage 2: Reward Model

For each prompt, the SFT model generates multiple candidate responses, and human annotators rank them. A scoring model $r_\phi(x, y)$ is trained on this preference data:

> $$\mathcal{L}_{\text{RM}} = -\log \sigma(r_\phi(x, y_{\text{win}}) - r_\phi(x, y_{\text{lose}}))$$

This is maximum likelihood estimation under the Bradley-Terry preference model — the RM learns to assign higher scores to responses humans prefer.

Stage 3: PPO Reinforcement Learning

The reward model serves as the reward function; PPO trains the policy model to generate responses with higher RM scores. A KL penalty prevents the policy from drifting too far from the SFT model (preventing reward hacking):

> $$\mathcal{L}_{\text{PPO}} = -\mathbb{E}[r_\phi(x, y) - \beta \cdot D_{\text{KL}}(\pi_\theta(y|x) \| \pi_{\text{SFT}}(y|x))]$$

Training requires four models loaded simultaneously: the policy (being trained), the reference (frozen SFT, for KL computation), the reward model, and the critic (value function). This is memory-intensive and training is unstable.

Results

InstructGPT at 1.3B parameters **outperforms GPT-3 at 175B** on human preference evaluations. 100× smaller, yet better — the value of alignment was quantified for the first time.

ChatGPT launched a few months later, essentially the productized version of InstructGPT.

InstructGPT's Legacy

1. The three-stage pipeline (SFT → RM → RLHF) became the **standard alignment recipe**, adopted by Claude, Gemini, LLaMA, Qwen, and virtually every major LLM
2. Preference data became the new "gold" — expensive to label, but immensely valuable
3. KL penalty as the standard "alignment tax" mechanism
4. The Bradley-Terry preference model was widely adopted

Critical Perspectives

- **Reward hacking**: the policy can learn to fool the RM rather than genuinely improve
- **RM bias**: annotator preferences don't represent all users
- **"Alignment tax"**: RLHF can cause regression on certain capabilities (particularly on factual benchmarks)
- **Engineering complexity**: four models, unstable PPO training

These problems motivated a wave of alternatives after 2023 — DPO (next paper), KTO, ORPO, RLAIF, and others — all attempting to simplify or improve upon RLHF.

> **Why You Must Read This:** Why is ChatGPT an "assistant" rather than a text completion engine? The answer is in this paper. The details of the three-stage RLHF pipeline must be read in the original to be truly understood.

P22 DPO: Turning RLHF into a Classification Loss

> *Main text reference: Chapter 3, §3.5.3*

Direct Preference Optimization: Your Language Model is Secretly a Reward Model

Authors Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, Chelsea Finn

Affiliations Stanford University

Published NeurIPS 2023 (Outstanding Paper Award) · arXiv:2305.18290

The Problem: RLHF's Engineering Complexity Is Too High

RLHF requires training two models (RM + policy), running PPO reinforcement learning, loading four model copies simultaneously, tuning a large number of hyperparameters, and worrying about reward hacking. A full RLHF pipeline is practically infeasible for most small and medium teams.

DPO's contribution is showing that *the entire process can be reduced to a single supervised learning loss*.

Core Mathematical Derivation

RLHF's objective is reward maximization with a KL constraint:

> $$\max_{\pi} \mathbb{E}_{x, y \sim \pi}[r(x,y)] - \beta D_{\text{KL}}(\pi(y|x) \| \pi_{\text{ref}}(y|x))$$

This problem has a closed-form optimal solution:

> $$\pi^*(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)$$

Z(x) is the normalization constant. Inverting for r:

> $$r(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$$

Key observation: when this r is substituted into the Bradley-Terry loss, $\beta \log Z(x)$ cancels out (since the loss takes

r(y_w) - r(y_l), the two \log Z(x) terms subtract to zero). This yields:

> $$\mathcal{L}_{\text{DPO}} = -\log \sigma\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right)$$

This is an ordinary binary classification loss.

Result: No RM Needed, No RL Needed

- No reward model to train
- No PPO, no critic
- Only 2 models loaded (policy + frozen reference)
- Standard SGD, no reward hacking

Implementation

```
def dpo_loss(policy, ref, batch, beta=0.1):
    """batch contains (prompt, chosen, rejected) triples"""
    # compute log-probabilities of chosen / rejected under
policy and reference
    pi_logps_chosen = compute_logps(policy,  batch.chosen,
batch.prompt)
    pi_logps_reject = compute_logps(policy,  batch.rejected,
batch.prompt)
    ref_logps_chosen = compute_logps(ref,    batch.chosen,
batch.prompt)
    ref_logps_reject = compute_logps(ref,    batch.rejected,
batch.prompt)

    pi_logratio  = pi_logps_chosen  - pi_logps_reject
    ref_logratio = ref_logps_chosen - ref_logps_reject

    loss = -F.logsigmoid(beta * (pi_logratio -
ref_logratio)).mean()
    return loss
```

An entire DPO training run looks just like a standard fine-tuning job — an order of magnitude simpler than PPO.

Impact

DPO rapidly became the dominant alignment method in late 2023 and throughout 2024. Open-source models including Zephyr (HuggingFace), Tulu (AI2), Nous-Hermes, Qwen, and DeepSeek all use DPO or variants.

But DPO is not a silver bullet:

- Sensitive to noisy preference data — IPO (2024) improves on this
- Requires paired data — KTO (2024) enables single-point feedback
- Variants merging SFT and DPO — ORPO and SimPO continue to evolve

The alignment field saw explosive diversity in 2024–2025, but the theoretical underpinnings all trace back to DPO. Recent reasoning model training (DeepSeek R1's GRPO) returned to RL-based methods — alignment techniques have not settled, and they continue to evolve.

> **Why You Must Read This:** "PPO + RM + KL = one classification loss" is one of the most elegant reframings in the alignment field. Reading this will show you why everyone was "escaping RL" — when a problem can be solved with supervised learning, why introduce RL at all.

P23 Chain-of-Thought: Letting the Model "Think" Before Answering

> *Main text reference: Chapter 4, §4.11.1 · §4.1.1 Test-Time Compute*

Chain-of-Thought Prompting Elicits Reasoning in Large Language Models

Authors Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, Denny Zhou

Affiliation	Google Research
Published	NeurIPS 2022 · arXiv:2201.11903 · 10,000+ citations

The Problem: LLMs Are Badly Wrong on Multi-Step Reasoning

In 2021, GPT-3 was strong at many tasks, but had a high error rate on math word problems at the elementary school level. A typical example:

```
Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls.
   Each can has 3 tennis balls. How many tennis balls does he have now?
Model answers directly: 27  (wrong)
```

The model has the "computational ability" (can compute 5 + 2×3 = 11), but when asked to answer directly, it skips steps and gets it wrong.

Core Finding: Let the Model "Write Out Its Reasoning"

What Wei et al. did sounds almost trivially simple: change the few-shot examples from "question + answer" to "question + reasoning steps + answer."

```
Q: Roger has 5 tennis balls. He buys 2 more cans. Each can has 3.
   How many balls does he have now?
A: Roger started with 5 balls. 2 cans of 3 balls each is 6 balls.
   5 + 6 = 11. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and
   bought 6 more, how many apples do they have?
A: [let the model continue]
```

The model now outputs something like: "The cafeteria had 23 apples. They used 20, so they had 23 - 20 = 3. Then they bought 6 more, so 3 + 6 = 9. The answer is 9." — correct.

An Emergent Phenomenon

CoT has a key characteristic: **it has little or no effect on small models, but produces dramatic improvements on large ones**. Experimental data from Wei et al. (GSM8K math benchmark):

Model	Direct Answer	CoT
GPT-3 0.4B	4.5%	5.6%
GPT-3 5.7B	5.1%	11.6%
GPT-3 175B	15.6%	46.9% (3×)
PaLM 540B	17.9%	56.9% (3×)

This phenomenon — *a capability appearing suddenly once a model reaches sufficient scale* — is called "emergence." CoT is one of the earliest clearly documented emergent abilities.

Why CoT Works: A Compute Perspective

Each LLM forward pass has a fixed compute budget (O(N), where N is the number of parameters). Direct answering = completing multi-step reasoning in one forward pass, which is compute-insufficient. CoT = one forward pass per reasoning step, with total compute scaling linearly with reasoning length — effectively **trading token count for compute**.

This framing opened the door to "test-time compute" — OpenAI o1 and DeepSeek R1's extended reasoning are essentially CoT taken to its logical extreme.

Zero-Shot CoT

Kojima et al. (2022) ("Let's think step by step") found that even without few-shot examples, simply appending "Let's think step by step" to a prompt causes large models to produce CoT

output automatically. This absurdly simple prompt briefly became a sensation in the field.

CoT's Subsequent Evolution

Self-Consistency (2022): CoT + multiple samples + majority voting — improves error correction
Tree of Thoughts (2023): non-linear reasoning via a search over multiple reasoning paths
ReAct (2022): CoT interleaved with "reasoning" and "tool calling" — the foundation of agents
o1 / R1 (2024): uses RL to push CoT length to extremes, generating tens of thousands of reasoning tokens before producing an answer

Impact

CoT is a key unlock for LLM capability — it transformed "reasoning tasks" from "nearly impossible" to "doable." Today all math, coding, and reasoning benchmarks default to CoT evaluation. The "thinking mode" in modern LLMs (DeepSeek V3.1, Qwen3, Claude thinking, Gemini Deep Think) are all productized forms of CoT.

> **Why You Must Read This:** CoT is the starting point for understanding "test-time scaling." The progression from CoT → Self-Consistency → ToT → o1 is one of the two major threads of LLM capability improvement from 2022 to 2026 (running in parallel with scaling laws).

P24 DeepSeek-V3: The Culmination of Open-Source MoE

> *Main text reference: Chapter 4, §4.4.1 MLA · §4.6 MoE · §4.9.2 FP8*

DeepSeek-V3 Technical Report

Authors DeepSeek-AI team

Published arXiv:2412.19437 (2024.12)

Follow-up DeepSeek-V3.1 (2025.8, hybrid thinking) · V3.2 (2025.10, DSA sparse attention)

Why This Paper Is the Capstone

DeepSeek-V3 is not the first MoE, the first FP8 training run, or the first hybrid reasoning model — but it is the first open-source model to *integrate all of these techniques together and execute each to its limits*. 671B total parameters, 37B activated per token, trained in just 2.788M H800 GPU-hours (approximately US$5.6M), reaching GPT-4o-level capability.

It is a case study in "engineering integration" winning over "single-point breakthroughs," and a decisive moment in open-source catching up to closed-source.

Architecture Overview

```
flowchart TB IN[Input] --> EMB[Embedding] EMB --> L1[Layer 1 to
61] L1 --> ATT["MLA Attention
low-rank KV compression"] ATT --> MOE["Fine-grained MoE
256 routed + 1 shared
top-8 activated"] MOE --> MTP["Multi-Token Prediction
auxiliary prediction of future tokens"] MTP --> NEXT[Next Layer]
L1 --> LM[LM Head]
```

Innovation 1: MLA (Multi-head Latent Attention)

Covered in depth in main text §4.4.1. Core idea: use low-rank decomposition to compress the KV Cache.

> $$\mathbf{c}_{\text{kv}} = W_{DKV} \mathbf{x}, \quad \mathbf{k} = W_{UK} \mathbf{c}_{\text{kv}}, \quad \mathbf{v} = W_{UV} \mathbf{c}_{\text{kv}}$$

With $d_c \ll d_{model}$, the KV Cache stores only $\mathbf{c}_{\text{kv}}$. At inference, W_{UK} can be absorbed into the Q projection, adding zero extra computation.

Compared to equivalent MHA, KV Cache is **reduced by 90%+** — the core advantage enabling long-context inference.

Innovation 2: Fine-Grained MoE + Shared Expert

Each layer in V3 has 256 routed experts + 1 shared expert, with top-8 routing:

- **Fine-grained**: 256 small experts yield $C_{256}^8 = 4 \times 10^{14}$ possible combinations — vastly more diverse than Mixtral's 28
- **Shared expert**: handles general knowledge; every token passes through it
- **Load balancing without auxiliary loss**: V3 innovates by adding learnable bias terms to routing scores, dynamically adjusting to equalize expert load — eliminating the traditional auxiliary loss

Innovation 3: Multi-Token Prediction (MTP)

During training, V3 predicts not just the next token but also the next D tokens (D = 1 in V3, i.e., one extra token predicted). Each prediction position has a dedicated MTP module (with its own Transformer layer). Benefits:

- Denser training signal: the model learns more from the same number of tokens
- At inference, the MTP module can directly serve as a speculative decoding draft head, eliminating the need for a separate draft model

Innovation 4: FP8 Training at Scale

V3 is the first model at the 671B scale to train fully in FP8. Core techniques:

1. **Fine-grained quantization**: per-tensor granularity is insufficient; V3 uses per-block (groups of 1×128 elements

share a scale)

2. **High-precision accumulation**: FP8 × FP8 → BF16 accumulation, reducing numerical error
3. **Selective FP8**: numerically sensitive operations (softmax in attention, layer normalization) retain BF16
4. **Pipeline optimization**: overlapping all-to-all communication with computation, hiding cross-node latency

In practice, FP8 achieves 1.5–2× training speedup over BF16, with negligible loss degradation.

Training Economics

Metric	Value
Total parameters / Activated	671B / 37B
Training tokens	14.8T
Pretraining GPU-hours	2.664M H800 (~US$5.3M)
Total training GPU-hours (incl. SFT + RLHF)	2.788M
Estimated comparison: GPT-4	~50–100M GPU-hours
Capability level	Comparable to GPT-4o / Claude 3.5 Sonnet

Approximately US$5.6M to train a GPT-4-level model — over 10× cheaper than closed-source estimates. This is the economic foundation of the "DeepSeek moment" (February 2025) — a cost low enough to shock the global industry.

Subsequent Evolution

DeepSeek-R1 (2025.1): built on V3, trained as a reasoning model using pure RL (GRPO algorithm). No human-labeled CoT data needed — reasoning ability emerged from RL alone

V3-0324 (2025.3): post-training improvements incorporating lessons from R1's RL, with major reasoning gains

R1-0528 (2025.5): upgraded R1, surpassing GPT-4o on the AIME math benchmark at that time
V3.1 (2025.8): hybrid thinking mode — a single model supporting switchable thinking / non-thinking, merging V3 and R1
V3.2 (2025.10): introduces DSA (DeepSeek Sparse Attention), reducing long-context inference cost by another 50%

Significance

V3 proved three things:

1. **Open-source can match closed-source**: not in 6 months, but on competitive terms
2. **Compounding returns from architectural innovation**: MLA + MoE + MTP + FP8 combined yield efficiency far beyond any single component
3. **Engineering excellence from Chinese AI labs**: V3's training efficiency and architectural details received broad international recognition

The open-source ecosystem moved quickly to follow in 2025 — Qwen3, Kimi K2, GLM-4.5, and LLaMA 4 all adopt similar architectural philosophy (fine-grained MoE + long-context optimization + FP8 training).

Critical Perspectives

- The training cost figures are debated — US$5.6M may count only GPU time, excluding data, infrastructure, and labor
- "GPT-4 level" varies across benchmarks — V3 leads on some and trails on others
- MLA's engineering complexity is high; small teams may find it difficult to reproduce

Whatever the caveats, V3 is one of the most important milestones in the history of open-source LLMs.

> **Why You Must Read This:** The V3 technical report is the most worthwhile LLM engineering paper of 2024 — it brings together all the technologies covered in P06–P13 (Transformer, MoE, LoRA ideas, FP8, inference optimization) and

> shows what they can achieve in combination. After reading this paper, the relationships among the previous 16 papers will become strikingly clear.

— Chapter 4 Part B End —

Part B: 8 papers · Combined chapter total: 17 papers

P25 RAG: Giving LLMs Access to External Knowledge

> *Corresponds to main text: Chapter 4, Section 4.10 — RAG*

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Authors Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, Douwe Kiela

Affiliation Facebook AI Research, University College London

Published NeurIPS 2020

Citations ≈4,000+

Link arXiv:2005.11401

The Problem: The Closed-Knowledge Nature of LLMs

Pretrained language models "bake" knowledge into their parameters. The problems this creates:

1. **Hallucination**: knowledge not seen during training gets fabricated
2. **Staleness**: information after the training cutoff is unknown
3. **Traceability**: answers cannot cite their sources

Lewis et al. propose a simple but far-reaching paradigm: retrieve relevant documents first, then feed those documents into the generative model's input.

Core Method: Retrieve-then-Generate

The RAG model consists of two components:

Retriever: given a query q, use DPR (Dense Passage Retrieval) to retrieve top-k documents $\{d_1, \ldots, d_k\}$ from a knowledge base
Generator: concatenate the query with the retrieved documents and use a seq2seq model (the paper uses BART) to generate the answer

The paper defines two marginalization strategies:

> $$p_{\text{RAG-Sequence}}(y|x) = \sum_{z \in \text{top-}k} p_\eta(z|x) \, p_\theta(y|x, z)$$

> $$p_{\text{RAG-Token}}(y|x) = \prod_i \sum_{z \in \text{top-}k} p_\eta(z|x) \, p_\theta(y_i|x, z, y_{1:i-1})$$

RAG-Sequence: each document independently generates a complete answer, then results are weighted and combined. RAG-Token: each token can attend to a different document. In engineering practice, most RAG systems use a further simplified version — concatenating retrieved documents directly into the prompt without any probabilistic marginalization.

Impact: From Paper to Industry Paradigm

From 2023 to 2025, RAG evolved from an academic paper into the **standard architecture for LLM applications**:

Enterprise knowledge Q&A: embed company documents into a vector store and use RAG to answer employee questions
Code assistants: retrieve relevant files from a code repository to assist code generation (Cursor, GitHub Copilot)

Search engines: Perplexity and Bing Chat are both "search + generation" RAG variants
Vector database boom: the rise of Milvus, Pinecone, Qdrant, and others was directly driven by RAG demand (connecting to P18 HNSW)

In engineering practice, the main challenges of RAG are not the probabilistic framework discussed in the original paper, but rather: chunking granularity (how large should chunks be), retrieval quality (semantic search vs. keyword search), context window limitations (too many retrieved chunks won't fit), and the "lost-in-the-middle" problem of retrieving context that the model fails to use effectively.

> **Why You Must Read This:** RAG is the dominant paradigm for deploying LLMs in production from 2023 to 2025 — almost every enterprise AI application starts with "build a RAG." Understanding the retrieve-then-generate framework from the original paper, and the gap between that framework and current engineering practice, is a fundamental skill for any AI engineer.

P26 ReAct: Synergizing Reasoning and Acting in LLMs

> *Corresponds to main text: Chapter 4, Section 4.11 — Agents*

ReAct: Synergizing Reasoning and Acting in Language Models

Authors Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, Yuan Cao

Affiliation Princeton University, Google Research

Published ICLR 2023

Citations 3,000+

Link arXiv:2210.03629

The Problem: The Disconnect Between Reasoning and Acting

Chain-of-Thought (P16) taught LLMs to "think" — but only as internal reasoning, with no ability to interact with the external world. On the other hand, "tool use" (having LLMs call APIs) provides action without reasoning — the model offers no explanation for why it is invoking a particular API. Yao et al.'s observation: when humans solve problems, reasoning and action **alternate** — "I think I should look this up (reasoning) → search (action) → the search results show... (observation) → so the answer is... (reasoning)."

Core Method: Thought-Action-Observation Loop

ReAct has the LLM interleave three types of content within the prompt:

Thought: reason about the current state and decide the next step ("I need to find X's birth year")

Action: invoke an external tool (search, calculator, database query, etc.)

Observation: the result returned by the tool (search results, computed value, etc.)

A typical ReAct trace:

```
Question: Which was founded earlier — Obama's alma mater or
Trump's alma mater?

Thought 1: I need to find Obama's alma mater first.
Action 1: Search[Barack Obama education]
Observation 1: Barack Obama attended Columbia University and
Harvard Law School.

Thought 2: Obama's alma mater is Columbia University and Harvard
Law School. Now find Trump's.
Action 2: Search[Donald Trump education]
Observation 2: Donald Trump attended the Wharton School at the
University of Pennsylvania.
```

```
Thought 3: Trump's alma mater is Penn. Now compare founding
dates. Columbia was founded in 1754, Penn in 1740.
Action 3: Lookup[University of Pennsylvania founding year]
Observation 3: The University of Pennsylvania was founded in
1740.

Thought 4: Penn (1740) was founded earlier than Columbia (1754).
Action 4: Finish[Trump's alma mater, the University of
Pennsylvania (1740), was founded earlier than Obama's alma
mater, Columbia University (1754).]
```

Comparison with CoT

Method	Reasoning	Action	Knowledge Source	Verifiability
Standard prompting	None	None	Parameters	Low
CoT (P16)	Yes	None	Parameters	Medium (reasoning visible)
Act-only	None	Yes	External tools	Medium (tool calls visible)
ReAct	Yes	Yes	Parameters + external	High (reasoning + tools both visible)

Impact: The Starting Point of the Agent Paradigm

ReAct directly defined the basic pattern of LLM Agents from 2023 to 2025:

LangChain Agent: the default agent type is the ReAct pattern
AutoGPT, BabyAGI: essentially automated versions of the ReAct loop (with added task decomposition and memory)
OpenAI Function Calling: formats ReAct's Action from free text into structured JSON calls
Claude Code, Cursor Agent: ReAct in a programming context — "analyze the problem → read files → modify code → run tests"

The essence of ReAct is not any specific technology, but a **paradigm shift**: an LLM is not just "a model that answers questions" but "an Agent capable of reasoning and acting." This shift in perspective gave rise to the entire AI Agent ecosystem.

> **Why You Must Read This:** If you are building LLM Agents — whether customer service bots, code assistants, or automated workflows — the architecture underneath is almost certainly a variant of the ReAct pattern. Understanding this simple but powerful "alternating reasoning and action" framework is the first step toward designing reliable Agents.

References

Bibliography

Core Papers (P01–P26)

[P01] Yu. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Using Hierarchical Navigable Small World Graphs," *IEEE Trans. PAMI*, vol. 42, no. 4, pp. 824–836, 2020. arXiv:1603.09320.

[P02] D. Ongaro and J. Ousterhout, "In Search of an Understandable Consensus Algorithm," in *Proc. USENIX ATC*, 2014, pp. 305–320.

[P03] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. IEEE CVPR*, 2016. arXiv:1512.03385.

[P04] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, 1997.

[P05] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training," in *Proc. ICML*, 2015. arXiv:1502.03167.

[P06] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *Proc. ICLR*, 2015. arXiv:1412.6980.

[P07] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. ACM SIGKDD*, 2016. arXiv:1603.02754.

[P08] I. J. Goodfellow *et al.*, "Generative Adversarial Nets," in *Advances in NeurIPS*, 2014. arXiv:1406.2661.

[P09] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," in *Proc. ICLR*, 2014. arXiv:1312.6114.

[P10] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in *Advances in NeurIPS*, 2020. arXiv:2006.11239.

[P11] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient Estimation of Word Representations in Vector Space," in *Proc. ICLR Workshop*, 2013. arXiv:1301.3781.

[P12] A. Vaswani *et al.*, "Attention Is All You Need," in *Advances in NeurIPS*, 2017. arXiv:1706.03762.

[P13a] J. Devlin *et al.*, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL-HLT*, 2019. arXiv:1810.04805.

[P13b] T. Brown *et al.*, "Language Models are Few-Shot Learners," in *Advances in NeurIPS*, 2020. arXiv:2005.14165.

[P14a] J. Kaplan *et al.*, "Scaling Laws for Neural Language Models," arXiv:2001.08361, 2020.

[P14b] J. Hoffmann *et al.*, "Training Compute-Optimal Large Language Models (Chinchilla)," in *Advances in NeurIPS*, 2022. arXiv:2203.15556.

[P15] J. Su *et al.*, "RoFormer: Enhanced Transformer with Rotary Position Embedding," *Neurocomputing*, 2024. arXiv:2104.09864.

[P16] T. Dao *et al.*, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in *Advances in NeurIPS*, 2022. arXiv:2205.14135.

[P17a] E. J. Hu *et al.*, "LoRA: Low-Rank Adaptation of Large Language Models," in *Proc. ICLR*, 2022. arXiv:2106.09685.

[P17b] T. Dettmers *et al.*, "QLoRA: Efficient Finetuning of Quantized LLMs," in *Advances in NeurIPS*, 2023. arXiv:2305.14314.

[P18] W. Kwon *et al.*, "Efficient Memory Management for LLM Serving with PagedAttention," in *Proc. ACM SOSP*, 2023. arXiv:2309.06180.

[P19a] W. Fedus, B. Zoph, and N. Shazeer, "Switch Transformers: Scaling to Trillion Parameter Models," *JMLR*, vol. 23, 2022. arXiv:2101.03961.

[P19b] Mistral AI, "Mixtral of Experts," arXiv:2401.04088, 2024.

[P20] A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," in *Proc. COLM*, 2024. arXiv:2312.00752.

[P21] L. Ouyang *et al.*, "Training Language Models to Follow Instructions with Human Feedback," in *Advances in NeurIPS*, 2022. arXiv:2203.02155.

[P22] R. Rafailov *et al.*, "Direct Preference Optimization," in *Advances in NeurIPS*, 2023. arXiv:2305.18290.

[P23] J. Wei *et al.*, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *Advances in NeurIPS*, 2022. arXiv:2201.11903.

[P24] DeepSeek-AI, "DeepSeek-V3 Technical Report," arXiv:2412.19437, 2024.

[P25] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in NeurIPS*, 2020. arXiv:2005.11401.

[P26] S. Yao *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proc. ICLR*, 2023. arXiv:2210.03629.

Additional References

[27] A. Krizhevsky et al., "ImageNet Classification with Deep CNNs (AlexNet)," NeurIPS, 2012.

[28] K. Simonyan and A. Zisserman, "Very Deep CNNs for Large-Scale Image Recognition (VGG)," ICLR, 2015.

[29] C. Szegedy et al., "Going Deeper with Convolutions (GoogLeNet)," CVPR, 2015.

[30] D. Bahdanau et al., "Neural Machine Translation by Jointly Learning to Align and Translate," ICLR, 2015.

[31] J. Pennington et al., "GloVe: Global Vectors for Word Representation," EMNLP, 2014.

[32] M. E. Peters et al., "Deep Contextualized Word Representations (ELMo)," NAACL, 2018.

[33] C. Raffel et al., "Exploring the Limits of Transfer Learning with a Unified T2T Transformer (T5)," JMLR, 2020.

[34] G. Hinton et al., "Distilling the Knowledge in a Neural Network," NeurIPS Workshop, 2015.

[35] M. Arjovsky et al., "Wasserstein GAN," ICML, 2017.

[36] T. Karras et al., "A Style-Based Generator Architecture for GANs (StyleGAN)," CVPR, 2019.

[37] J. Sohl-Dickstein et al., "Deep Unsupervised Learning Using Nonequilibrium Thermodynamics," ICML, 2015.

[38] J. Schulman et al., "Proximal Policy Optimization Algorithms (PPO)," arXiv:1707.06347, 2017.

[39] E. Frantar et al., "GPTQ: Accurate Post-Training Quantization," ICLR, 2023.

[40] J. Lin et al., "AWQ: Activation-aware Weight Quantization," MLSys, 2024.

[41] C. Leviathan et al., "Fast Inference from Transformers via Speculative Decoding," ICML, 2023.

[42] S. Yao et al., "Tree of Thoughts: Deliberate Problem Solving with LLMs," NeurIPS, 2023.

[43] X. Wang et al., "Self-Consistency Improves Chain of Thought Reasoning," ICLR, 2023.

[44] T. Kojima et al., "Large Language Models are Zero-Shot Reasoners," NeurIPS, 2022.

[45] A. Gu et al., "Efficiently Modeling Long Sequences with Structured State Spaces (S4)," ICLR, 2022.

[46] T. Dao and A. Gu, "FlashAttention-2," ICLR, 2024.

[47] DeepSeek-AI, "DeepSeek-R1," arXiv:2501.12948, 2025.

[48] A. Dosovitskiy et al., "An Image is Worth 16x16 Words (ViT)," ICLR, 2021.